Privacy-Preserving Machine Learning for Practitioners

The results without proofs, and workflows that run real libraries on public data

Manuel V. Loureiro

Version 2026-11-04

Table of contents

Preface **1**
How this book is organized . . . 1

I. Prerequisites **3**
What each chapter is for . . . 4
Notation . . . 4
How to use the chapters . . . 4

1. Divergences **5**
1.1. What it is . . . 5
1.2. When you meet it . . . 5
1.3. How to use it . . . 6
1.4. What goes wrong . . . 7

2. Hypothesis testing **8**
2.1. Why this matters . . . 8
2.2. Tests and their two errors . . . 8
2.3. The Neyman–Pearson lemma . . . 8
2.4. Trade-off curves . . . 9
2.5. The Gaussian trade-off curve . . . 9
2.6. The Laplace trade-off curve . . . 9
2.7. Best advantage and total variation . . . 9
2.8. ROC summaries: why AUC hides the low-FPR regime . . . 10
2.9. The Bayesian view . . . 10
2.10. Measuring rates from finite trials . . . 11
2.11. Evaluating a detector at a low false positive rate . . . 11
2.12. Cheat sheet . . . 12
2.13. References . . . 13

II. Threat models and attacks **14**

3. Threat models and privacy games **16**
3.1. Why this matters . . . 16
3.2. Writing down the threat model . . . 16
3.3. Scoring an attack against its baseline . . . 17
3.4. Reporting: TPR at a low FPR . . . 18
3.5. Average case and worst case . . . 19
3.6. Cheat sheet . . . 20
3.7. References . . . 20

4. Membership inference **21**
4.1. Why this matters . . . 21
4.2. Membership as a test about one record . . . 21
4.3. The quick check: the generalization gap . . . 21
4.4. LiRA: a likelihood-ratio test per record . . . 22
4.5. Running LiRA on your model . . . 23
4.6. Reading the result . . . 24
4.7. Who is exposed, and what helps . . . 24
4.8. Cheat sheet . . . 25
4.9. References . . . 25

5. Attribute inference and model inversion **26**
5.1. What it is . . . 26
5.2. When you meet it . . . 26
5.3. How to use it . . . 26
5.4. What goes wrong . . . 28
5.5. References . . . 28

6. Property inference **29**
6.1. What it is . . . 29
6.2. When you meet it . . . 29
6.3. How to use it . . . 29
6.4. What goes wrong . . . 30
6.5. References . . . 30

7. Memorization and training-data extraction **31**
7.1. Why this matters . . . 31
7.2. Which memorization to measure . . . 31
7.3. What gets memorized, and the first defences . . . 32
7.4. Auditing with canaries and exposure . . . 32
7.5. Extraction without the prefix: generate, rank, verify . . . 33
7.6. Cheat sheet . . . 34
7.7. References . . . 34

8. Gradient inversion **35**
8.1. Why this matters . . . 35
8.2. The attacker and what it sees . . . 35
8.3. Exact recovery from a fully connected layer . . . 35
8.4. Label recovery from the last layer . . . 36
8.5. Deep Leakage from Gradients . . . 36
8.6. Batches and averaging . . . 37
8.7. What to do as a deployer . . . 38
8.8. Cheat sheet . . . 38
8.9. References . . . 39

9. Linkage and re-identification **40**
9.1. Why this matters . . . 40
9.2. Quasi-identifiers and the linkage attack . . . 40

9.3. Pseudonyms and hashed identifiers . . . 41
9.4. How many people are unique? . . . 42
9.5. Sparse data: the Netflix attack . . . 42
9.6. Does releasing a sample protect? . . . 43
9.7. A pre-release risk check . . . 43
9.8. Cheat sheet . . . 43
9.9. References . . . 44

10. Syntactic anonymity and how it breaks **45**
10.1. Why this matters . . . 45
10.2. What the adversary believes about a class . . . 45
10.3. k-anonymity and what it costs . . . 45
10.4. Homogeneity, background knowledge and l-diversity . . . 46
10.5. Measuring what a release teaches . . . 47
10.6. Skewness, similarity and t-closeness . . . 47
10.7. The composition attack . . . 48
10.8. Choosing and running an anonymization . . . 48
10.9. Cheat sheet . . . 48
10.10. References . . . 49

11. Reconstruction attacks **50**
11.1. What it is . . . 50
11.2. When you meet it . . . 50
11.3. How to use it . . . 51
11.4. What goes wrong . . . 51
11.5. References . . . 52

12. Pre-release privacy risk assessment **53**
12.1. Why this matters . . . 53
12.2. Fix the threat model and the thresholds first . . . 53
12.3. Classifier: the free check . . . 55
12.4. Classifier: LiRA with shadow models . . . 55
12.5. Fine-tuned language model: canaries and extraction . . . 55
12.6. Table: re-identification on quasi-identifiers . . . 56
12.7. Write the decision . . . 56
12.8. Checklist . . . 57
12.9. References . . . 58

III. Differential privacy foundations **59**

13. The definition of differential privacy **61**
13.1. Why this matters . . . 61
13.2. Neighbouring datasets . . . 61
13.3. The definition . . . 61
13.4. The privacy loss and δ . . . 62
13.5. Randomized response . . . 62
13.6. What ε means for an attacker . . . 63

13.7. How small δ must be . . . 64
13.8. What differential privacy does not protect . . . 64
13.9. Writing the guarantee down . . . 65
13.10. Code . . . 65
13.11. Cheat sheet . . . 65
13.12. References . . . 66

14. Post-processing, group privacy and composition **67**
14.1. Why this matters . . . 67
14.2. Post-processing . . . 67
14.3. Group privacy . . . 67
14.4. Sequential composition . . . 68
14.5. Parallel composition . . . 69
14.6. Keeping a budget . . . 69
14.7. Cheat sheet . . . 71
14.8. References . . . 71

15. The Laplace and Gaussian mechanisms **72**
15.1. Why this matters . . . 72
15.2. Sensitivity . . . 72
15.3. The Laplace mechanism . . . 73
15.4. The Gaussian mechanism . . . 74
15.5. Laplace or Gaussian? . . . 74
15.6. Calibrating a release . . . 75
15.7. Code . . . 75
15.8. Cheat sheet . . . 76
15.9. References . . . 76

16. Advanced and optimal composition **78**
16.1. What it is . . . 78
16.2. When you meet it . . . 78
16.3. How to use it . . . 79
16.4. What goes wrong . . . 80

17. Selection: exponential mechanism, report-noisy-max, sparse vector **81**
17.1. Why this matters . . . 81
17.2. Which mechanism to use . . . 81
17.3. The exponential mechanism . . . 81
17.4. Accuracy, and the cost of many candidates . . . 82
17.5. A private quantile or clipping norm . . . 82
17.6. Report-noisy-max and Gumbel noise . . . 83
17.7. The sparse vector technique . . . 84
17.8. When the sparse vector technique goes wrong . . . 84
17.9. Cheat sheet . . . 85
17.10. References . . . 86

18. Query release and private multiplicative weights **87**
18.1. Why this matters . . . 87

18.2. Which method to use . . . 87
18.3. Noise on every cell . . . 88
18.4. Releasing a marginal with OpenDP . . . 88
18.5. Synthetic data . . . 89
18.6. Private multiplicative weights . . . 90
18.7. Cheat sheet . . . 90
18.8. References . . . 91

19. Beyond global sensitivity **92**
19.1. What it is . . . 92
19.2. When you meet it . . . 92
19.3. How to use it . . . 93
19.4. What goes wrong . . . 93

20. Discrete and floating-point-safe mechanisms **94**
20.1. What it is . . . 94
20.2. When you meet it . . . 94
20.3. How to use it . . . 95
20.4. What goes wrong . . . 95

21. Local, shuffle and distributed trust models **97**
21.1. Why this matters . . . 97
21.2. Choosing a trust model . . . 97
21.3. Central and local differential privacy . . . 97
21.4. Frequency estimation: k-ary randomized response and unary encoding . . . 99
21.5. Numbers in one bit: local mean estimation . . . 99
21.6. Collecting every day: memoization . . . 99
21.7. The shuffle model and amplification by shuffling . . . 100
21.8. Distributed DP: splitting the noise . . . 100
21.9. Cheat sheet . . . 101
21.10. References . . . 101

22. Lower bounds: what privacy costs **102**
22.1. What it is . . . 102
22.2. When you meet it . . . 102
22.3. How to use it . . . 102
22.4. What goes wrong . . . 103

23. Private mean estimation **104**
23.1. Why this matters . . . 104
23.2. Bounded data: privacy adds a lower-order term . . . 104
23.3. The cost of an unknown range . . . 105
23.4. Many dimensions: clip the norm . . . 106
23.5. Heavy tails: clip into the data . . . 106
23.6. Code . . . 107
23.7. Cheat sheet . . . 108
23.8. References . . . 108

24. Release statistics with a library **109**
24.1. Why this matters . . . 109
24.2. Choose the library . . . 109
24.3. Decide the unit and the budget . . . 110
24.4. Bound each customer's contribution . . . 110
24.5. Set up the context . . . 111
24.6. Release overall statistics with error bars . . . 112
24.7. Group by a public key list . . . 113
24.8. Group by unknown keys with partition selection . . . 114
24.9. Check the release before publishing . . . 115
24.10. Write the guarantee down . . . 115
24.11. Checklist . . . 116
24.12. References . . . 116

IV. Privacy accounting **118**

25. Choosing an accountant **120**
25.1. Why this matters . . . 120
25.2. Which accountant . . . 120
25.3. The inverse problem: noise for a target ε . . . 121
25.4. Choosing δ . . . 122
25.5. Matching the accountant to the code . . . 122
25.6. Checking an accountant . . . 123
25.7. Cheat sheet . . . 124
25.8. References . . . 124

26. Calibrate and account a DP-SGD run **125**
26.1. Why this matters . . . 125
26.2. Fix the privacy unit, n and δ . . . 125
26.3. Choose the sampler . . . 126
26.4. Plan the noise . . . 126
26.5. Check ε with a second accountant . . . 128
26.6. If the loader is not Poisson . . . 129
26.7. Train, stop early and resume . . . 129
26.8. Write the statement . . . 130
26.9. Checklist . . . 131
26.10. References . . . 132

27. Amplification by subsampling **133**
27.1. Why this matters . . . 133
27.2. One step under Poisson sampling . . . 133
27.3. Match the sampler to the accountant . . . 134
27.4. Accounting each sampler . . . 135
27.5. Batch size, epochs and noise . . . 136
27.6. Cheat sheet . . . 136
27.7. References . . . 137

28. Rényi differential privacy **138**
28.1. Why this matters . . . 138
28.2. The RDP curve . . . 138
28.3. RDP curves of the basic mechanisms . . . 138
28.4. Composition adds curves . . . 139
28.5. Post-processing and group privacy . . . 140
28.6. Compose, then convert once . . . 140
28.7. Check the order grid . . . 140
28.8. Accounting with a library . . . 141
28.9. Cheat sheet . . . 141
28.10. References . . . 142

29. From Rényi DP to (ε, δ) and to tests **143**
29.1. What it is . . . 143
29.2. When you meet it . . . 143
29.3. How to use it . . . 144
29.4. What goes wrong . . . 144
29.5. References . . . 144

30. Numerical accounting with privacy loss distributions **145**
30.1. What it is . . . 145
30.2. When you meet it . . . 145
30.3. How to use it . . . 146
30.4. What goes wrong . . . 146
30.5. References . . . 147

31. f-DP and Gaussian differential privacy **148**
31.1. What it is . . . 148
31.2. When you meet it . . . 148
31.3. How to use it . . . 149
31.4. What goes wrong . . . 149
31.5. References . . . 150

32. Adaptive composition: filters, odometers and individual accounting **151**
32.1. What it is . . . 151
32.2. When you meet it . . . 151
32.3. How to use it . . . 151
32.4. What goes wrong . . . 152
32.5. References . . . 152

33. Concentrated differential privacy **154**
33.1. What it is . . . 154
33.2. When you meet it . . . 154
33.3. How to use it . . . 154
33.4. What goes wrong . . . 155
33.5. References . . . 156

34. Amplification by shuffling and by iteration **157**
34.1. What it is . . . 157

34.2. When you meet it . . . 157
34.3. How to use it . . . 157
34.4. What goes wrong . . . 158
34.5. References . . . 159

V. Differential privacy in machine learning 160

35. Private empirical risk minimization 162
35.1. What it is . . . 162
35.2. When you meet it . . . 162
35.3. How to use it . . . 163
35.4. What goes wrong . . . 164

36. DP-SGD 165
36.1. Why this matters . . . 165
36.2. The algorithm and its guarantee . . . 165
36.3. The implementation checklist . . . 166
36.4. Clipping and the learning rate . . . 167
36.5. Batch size at a fixed budget . . . 167
36.6. Making it fast . . . 167
36.7. What privacy costs in accuracy . . . 168
36.8. Cheat sheet . . . 168
36.9. References . . . 169

37. Train a model with DP-SGD 170
37.1. Why this matters . . . 170
37.2. Fix the privacy unit, n and δ . . . 170
37.3. Preprocess from public facts . . . 170
37.4. Train the baselines . . . 171
37.5. Check the model . . . 171
37.6. Choose the clipping norm, batch and epochs . . . 172
37.7. Calibrate the noise . . . 172
37.8. Train . . . 173
37.9. Cross-check ε . . . 173
37.10. Tune honestly . . . 174
37.11. Measure utility and groups . . . 174
37.12. When it goes wrong . . . 175
37.13. Record the run . . . 176
37.14. Checklist . . . 176
37.15. References . . . 177

38. Hyperparameter tuning and private selection 178
38.1. Why this matters . . . 178
38.2. The procedure and the report . . . 178
38.3. Paying for every run . . . 179
38.4. A random number of runs . . . 179
38.5. Selecting by a noisy validation score . . . 180

38.6. Tuning on Adult . . . 180
38.7. Cheat sheet . . . 181
38.8. References . . . 182

39. User-level differential privacy **183**
39.1. Why this matters . . . 183
39.2. The privacy unit and δ . . . 183
39.3. What an example-level model gives a user . . . 184
39.4. User-level DP-SGD . . . 184
39.5. Contribution bounding . . . 185
39.6. Heterogeneous users . . . 185
39.7. Which strategy . . . 186
39.8. Code . . . 186
39.9. Cheat sheet . . . 187
39.10. References . . . 187

40. Choosing ε **188**
40.1. Why this matters . . . 188
40.2. Justifying a choice . . . 188
40.3. The pair and the whole curve . . . 189
40.4. What an attacker can believe . . . 190
40.5. How small δ must be . . . 191
40.6. Comparing with deployed values . . . 191
40.7. Cheat sheet . . . 192
40.8. References . . . 192

41. Public data and private fine-tuning **193**
41.1. Why this matters . . . 193
41.2. How many weights to train . . . 193
41.3. Linear probing first . . . 194
41.4. Low-rank adapters and full fine-tuning . . . 194
41.5. Which method at which budget . . . 195
41.6. What public data does not buy . . . 196
41.7. Cheat sheet . . . 196
41.8. References . . . 196

42. Fine-tune a language model privately with LoRA **198**
42.1. Why this matters . . . 198
42.2. Fix the privacy unit, n and δ . . . 198
42.3. Plant canaries . . . 199
42.4. Load a public model and tokenizer . . . 199
42.5. Choose what to train . . . 200
42.6. Make the model Opacus-compatible . . . 200
42.7. Calibrate and train . . . 201
42.8. Evaluate . . . 202
42.9. Check memorization . . . 202
42.10. Variants . . . 204
42.11. Report . . . 204

42.12. Checklist . . . 205
42.13. References . . . 206

43. Private prediction: PATE and in-context learning **207**
43.1. What it is . . . 207
43.2. When you meet it . . . 207
43.3. How to use it . . . 208
43.4. What goes wrong . . . 208
43.5. References . . . 209

44. DP-FTRL and matrix factorization **210**
44.1. What it is . . . 210
44.2. When you meet it . . . 210
44.3. How to use it . . . 211
44.4. What goes wrong . . . 211
44.5. References . . . 212

45. Disparate impact **213**
45.1. Why this matters . . . 213
45.2. Evaluating a private model per group . . . 213
45.3. Disparate impact, measured . . . 214
45.4. Why small groups pay: clipping and noise . . . 215
45.5. Mitigations and what they cost . . . 216
45.6. Cheat sheet . . . 217
45.7. References . . . 218

46. Auditing private training **219**
46.1. Why this matters . . . 219
46.2. Known bugs, and which audit catches them . . . 219
46.3. A CI test for the noise and the clipping . . . 219
46.4. From an attack to a lower bound . . . 221
46.5. A one-run audit of the trained model . . . 222
46.6. Cheat sheet . . . 223
46.7. References . . . 223

47. Adaptive data analysis **224**
47.1. What it is . . . 224
47.2. When you meet it . . . 224
47.3. How to use it . . . 225
47.4. What goes wrong . . . 225
47.5. References . . . 225

48. Release and governance **227**
48.1. Why this matters . . . 227
48.2. Assemble the run record . . . 227
48.3. Recompute the guarantee . . . 228
48.4. Attach the evidence . . . 229
48.5. Write the statement . . . 230
48.6. Map the claim to regulation . . . 230

48.7. Review someone else's claim 232
48.8. After release 233
48.9. Checklist 233
48.10. References 234

Appendices **235**

A. Glossary **235**

B. Index **238**

Preface

This is the practitioner edition of *Privacy-Preserving Machine Learning*. It is for engineers, data scientists and auditors who build or review machine-learning systems that handle personal data, and it tells them what each result says, when it applies, how to use it and what goes wrong, without proofs.

It keeps the parts and the chapter files of the full edition, so that every page here has a twin there that derives its results step by step, but it is organized around the work rather than the theory. Three kinds of page do that. A **workflow chapter** does one job end to end with real libraries on a real public dataset: assessing a model's privacy risk before release, releasing statistics with OpenDP and diffprivlib, calibrating and reporting a run of differentially private stochastic gradient descent, training and fine-tuning with Opacus, and governing a release. Each part reads in the order of the work, so a workflow comes early in its part and the pages it relies on follow. A **full chapter** covers a topic a practitioner computes with: the result and its formula, why it matters, a recipe with a worked example, what goes wrong, and a cheat sheet. A **reference page** covers a theory-heavy topic a practitioner meets but rarely computes with, in a few hundred words: what it is, when you meet it, how to use it and what goes wrong. Every page ends with a link to the full edition's chapter.

You need some Python and NumPy, basic probability, and enough machine learning to know what a loss and stochastic gradient descent are.

How this book is organized

1. **Prerequisites** covers hypothesis testing and divergences, the two pieces of mathematics a later chapter cites most. The rest of the mathematics stays in the full edition's Part 1.
2. **Threat models and attacks** shows how models and data leak, and assesses a model's risk before release.
3. **Differential privacy foundations** defines the guarantee, builds its mechanisms, and releases statistics with them.
4. **Privacy accounting** tracks the guarantee through private training, and calibrates and reports a run.
5. **Differential privacy in machine learning** trains, fine-tunes, audits and releases private models.

You do not need to read Part 1 first. With a background in statistics or machine learning, start at Part 2 or Part 3 and follow the links back when you need a fact.

Every piece of code on a page has been run: a snippet runs on its own, and an excerpt is a marked region of a workflow script in the repository's `code/` directory, which a test runs

and checks against the numbers the page quotes. To rerun a workflow, run `uv run python code/<part>/practitioner_<topic>.py`.

Part I.

Prerequisites

This part introduces two mathematical tools used throughout the book. Later chapters link to each result where they use it, so you can read this part as needed.

What each chapter is for

Both chapters use the **likelihood ratio**, the relative probability of an observation under two explanations. Later chapters use it to compare privacy guarantees and attacks.

Hypothesis testing shows how to read a detector's trade-off curve at a low false positive rate and compute an interval for a measured rate. Attacks, audits and the Gaussian view of accounting use these results. Divergences is a short reference page on measures that compare two distributions; the definitions in Parts 3 and 4 use them.

The rest of the mathematics is in the full edition's Part 1, which a page here links to where it uses a fact: probability distributions in the full edition, concentration inequalities in the full edition, information theory in the full edition, optimization in the full edition, algorithmic stability and generalization in the full edition and cryptography basics in the full edition.

Notation

Table 1.: Notation used throughout the book.

Symbol	Meaning
Pr, $\mathbb{E}$, Var	Probability, expectation, variance
$\mathcal{N}(\mu, \sigma^2)$, Lap($b$)	Normal and Laplace distributions
ln, $\log_2$	Natural logarithm (nats) and base-2 logarithm (bits)
$\|x\|$	Euclidean length of a vector x
D, θ, $\ell(\theta; z)$	A dataset, the parameters of a model, and its loss on one record z

How to use the chapters

Each chapter states its results, works an example with real numbers and links to the derivations in the full edition. A snippet or a seeded script in the full edition checks every number, so each run reproduces it.

1. Divergences

1.1. What it is

A **divergence** measures the difference between two distributions. It is zero when they are equal and increases as they become easier to distinguish. For distributions P and Q, let $p(x)$ and $q(x)$ denote their probabilities of outcome x. Divergences compare the **likelihood ratio** $p(x)/q(x)$.

The **Kullback–Leibler (KL) divergence** is the average log ratio when the data come from P:

$$D_{\mathrm{KL}}(P\|Q) = \sum_x p(x) \ln \frac{p(x)}{q(x)}. \tag{1.1}$$

It is not symmetric, and it adds up across independent observations.

The **Rényi divergence** is a dial set by an order α above 1:

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \sum_x p(x)^\alpha q(x)^{1-\alpha}. \tag{1.2}$$

It equals the KL divergence as the order goes to 1. Higher orders give more weight to outcomes with large ratios, up to the **max-divergence** D_∞, the largest log ratio of all outcomes.

The **total variation (TV)** is the mass of P that does not overlap with Q:

$$\mathrm{TV}(P, Q) = \frac{1}{2} \sum_x |p(x) - q(x)|. \tag{1.3}$$

The **hockey-stick divergence** scales q up by a factor γ of at least 1 and measures the mass of p that still sticks out above it. Writing $(y)_+$ for the positive part of y, it is

$$E_\gamma(P\|Q) = \sum_x \big(p(x) - \gamma q(x)\big)_+. \tag{1.4}$$

At $\gamma = 1$ it is the total variation, and it reaches zero once γ exceeds the largest likelihood ratio.

1.2. When you meet it

Use divergences to quantify feature drift between training and production or to assess how a test-data shift may affect classifier accuracy. Papers may state a result using a different divergence from the one needed for an application.

1.3. How to use it

To measure drift, put the two samples in bins with the same edges and call `scipy.stats.entropy(p, q)`. It returns natural-log units (nats), or bits with `base=2`. Take a feature whose last bin was empty last month and holds a tenth of this month's data. With last month as P, the divergence is 0.1116 nats, but the reverse is infinite, because this month has mass where last month had none.

```
import math

from scipy import stats

before, after = [0.5, 0.3, 0.2, 0.0], [0.4, 0.3, 0.2, 0.1]
print(f"{stats.entropy(before, after):.4f} {stats.entropy(before, after, base=2):.4f}")
 ↪  # 0.1116 0.1610
print(stats.entropy(after, before))  # inf
print(math.ceil(2 * 0.9**2 / 0.005), math.ceil((2 * stats.norm.ppf(0.95) / 0.1) ** 2))
 ↪  # 324 1083
```

For noise and a copy shifted by s, closed forms replace histograms. For normal noise, write σ for the standard deviation and $\mu = s/\sigma$ for the shift in standard deviations. The divergences are

$$D_{\mathrm{KL}} = \frac{\mu^2}{2}, \qquad D_\alpha = \frac{\alpha\mu^2}{2}, \qquad D_\infty = \infty, \tag{1.5}$$

the last because the ratio of normal densities is unbounded. The shift is squared, so a drift of 0.1 standard deviations gives a KL divergence of only $0.1^2/2 = 0.005$. For Laplace noise of scale b, the log ratio is bounded, and the max-divergence is the shift in scales, s/b.

The TV bounds how well a test distinguishes P from Q. With equal prior probabilities, the best test's **advantage**—its true-positive rate minus its false-positive rate—equals the TV. Its accuracy is $\frac{1}{2} + \frac{1}{2}\mathrm{TV}$. For normal distributions one standard deviation apart, the TV is 0.3829, so the best classifier's accuracy is 0.6915. A shift in the test distribution changes a fixed classifier's accuracy by at most the TV.

For the same normal pair, the hockey-stick divergence is

$$E_\gamma = \Phi\Big(\frac{\mu}{2} - \frac{\ln\gamma}{\mu}\Big) - \gamma\,\Phi\Big(-\frac{\mu}{2} - \frac{\ln\gamma}{\mu}\Big), \tag{1.6}$$

where Φ is the standard normal distribution function. At $\mu = 1$ and $\gamma = 2$ it is 0.1906.

Pinsker's inequality bounds the TV using the KL divergence:

$$\mathrm{TV}(P, Q) \le \sqrt{\tfrac{1}{2} D_{\mathrm{KL}}(P\|Q)}. \tag{1.7}$$

The bound is convenient but loose. Detecting a drift of 0.1 standard deviations with 95% power requires a TV of 0.9. KL divergence adds across independent samples, so a sample of size n has KL divergence $0.005n$. Pinsker's inequality gives a sample-size requirement of 324; the exact requirement is 1,083 (the snippet's last call).

1.4. What goes wrong

KL divergence is directional; report its direction and do not mix nats with bits. An empty bin makes it infinite. Add half a count per bin or merge bins, and report which adjustment you use. Estimates depend on the bins and sample size, so compare estimates only when these settings match.

The **data processing inequality** says that processing an observation cannot make two distributions easier to distinguish, though it may make them harder to distinguish. For normal distributions one standard deviation apart, a midpoint threshold reduces KL divergence from 0.5 to 0.3090. It preserves the TV because the threshold implements the optimal test.

A small KL divergence is an average, so it can hide rare outputs that are strong evidence. When every output matters, bound the worst case with the max-divergence, a high Rényi order or the hockey-stick divergence.

Derivations. Every result here is derived step by step in the full edition's chapter.

2. Hypothesis testing

2.1. Why this matters

A fraud detector classifies each transaction as normal or fraudulent, testing which of two distributions generated its score. Every binary classifier is a hypothesis test. Evaluate both error rates at the required low false-alarm rate; accuracy does not report that rate.

2.2. Tests and their two errors

The default assumption, the **null hypothesis** H_0, is that the observation comes from a distribution P, such as the fraud model's scores for normal transactions. The **alternative hypothesis** H_1 is that it comes from a distribution Q, such as its scores for fraudulent ones. A **test** decides whether to reject the null, which here means to flag the transaction.

A test can make two errors. A false positive flags a normal case; its probability is the **false positive rate (FPR)**, written α. A false negative misses a real case; its probability is the **false negative rate (FNR)**, written β. The **true positive rate (TPR)** is $1 - \beta$. A lower threshold raises both TPR and FPR. First set the FPR budget, or **level**, then find the test with the highest TPR, or **power**, at that level.

2.3. The Neyman–Pearson lemma

Write $p(x)$ and $q(x)$ for the probabilities, or densities, of an observation x under the two hypotheses. Their quotient is the **likelihood ratio**

$$\Lambda(x) = \frac{q(x)}{p(x)}. \tag{2.1}$$

A **likelihood ratio (LR) test** rejects wherever this ratio exceeds a threshold τ, which you set so that the FPR of the test equals the budget. The **Neyman–Pearson lemma** says that no other test at that level catches more real cases.

The optimal test thresholds the likelihood ratio. Thresholding a raw score is equivalent only if higher scores always imply higher ratios. Estimate the ratio from labelled scores by fitting logistic regression of the label on the score; its log-odds minus the log prior odds estimates $\ln \Lambda$. If scores shift across known groups, the likelihood ratio also depends on group, and the optimal test uses group-specific thresholds.

2.4. Trade-off curves

The best FNR at every level says how hard P is to tell from Q. This curve is the **trade-off function** $T(P, Q)$, which you get when you move the LR threshold across its range. The **receiver operating characteristic (ROC)** curve holds the same information upside down, as TPR against FPR. Each real detector has its own ROC curve, on or below the best one.

2.5. The Gaussian trade-off curve

A common model has normal scores under both hypotheses, with equal standard deviations. Scale the normal-case scores to mean 0 and standard deviation 1. Let μ be the distance between the means in standard deviations, and let Φ be the standard normal cumulative distribution function. The optimal threshold is $\Phi^{-1}(1-\alpha)$, and the FNR at level α is the Gaussian trade-off function

$$G_\mu(\alpha) = \Phi(\Phi^{-1}(1-\alpha) - \mu). \tag{2.2}$$

In SciPy this is `norm.cdf(norm.isf(alpha) - mu)`. Use `isf`, because `norm.ppf(1 - alpha)` loses the digits that matter at a very small α. The single number μ fixes the best TPR at every FPR. At $\mu = 2$, for example, the best TPR is 0.6388 at an FPR of 5%, and only 0.1378 at an FPR of 10^{-3}. In the other direction, a measured pair gives $\mu = \Phi^{-1}(1-\alpha) + \Phi^{-1}(\text{TPR})$, or `norm.isf(alpha) + norm.ppf(tpr)`. If that estimate changes with α, the scores do not have this shape.

2.6. The Laplace trade-off curve

Take a Laplace distribution and a copy shifted by κ times its scale. Their likelihood ratio stays between $e^{-\kappa}$ and e^{κ} (Divergences). So each outcome that a test rejects is at most e^{κ} times more likely under Q than under P, and every test has

$$\text{TPR} \le e^{\kappa}\,\text{FPR}. \tag{2.3}$$

At a low FPR the best test reaches this cap. At $\kappa = 1$ and an FPR of 0.05, its TPR is $e \times 0.05 = 0.1359$.

This cap distinguishes the Laplace pair from the normal pair. A Laplace pair at $\kappa = 1$ and a normal pair at $\mu = 1.0301$ have the same maximum advantage, the largest possible TPR minus FPR (Section 2.7). At a level of 10^{-6}, however, the normal pair permits 36 times the TPR. Its likelihood ratio is unbounded, allowing a test to detect rare, high-evidence observations. Compare distributions at the FPR relevant to the application.

2.7. Best advantage and total variation

The **advantage** of a test is its TPR minus its FPR. The best advantage over all tests is the **total variation (TV)** distance of Divergences:

$$\max(\text{TPR} - \text{FPR}) = \text{TV}(P, Q) = \frac{1}{2}\sum_x |q(x) - p(x)|. \tag{2.4}$$

For the normal pair, TV is $2\Phi(\mu/2) - 1$, or 0.3829 at a shift of one standard deviation. Accuracy on a balanced set is one half plus half the advantage. Thus, accuracy describes one point on the curve and does not determine performance at low FPR.

2.8. ROC summaries: why AUC hides the low-FPR regime

The **area under the curve (AUC)** is the probability that a random positive scores higher than a random negative. It weights every FPR equally from 0 to 1, so performance at very low FPR contributes little. For example, consider two detectors with standard normal scores on negatives. Detector A's positive scores have mean 1 and standard deviation 1; detector B's have mean 1.5811 and standard deviation 2. Both have an AUC of 0.7602.

But at an FPR of 10^{-3}, A catches 0.0183 of the positives and B about 12 times as many (Figure 2.1). A few positives of B score far above all negatives, and a strict threshold catches exactly those.

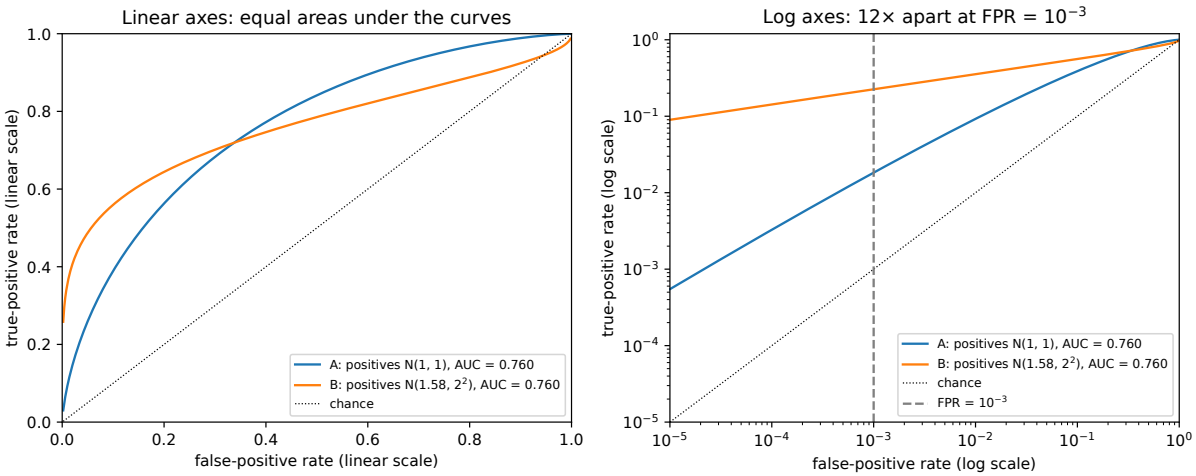


Figure 2.1.: Two detectors with the same AUC of 0.760, on linear axes (left) and log axes (right).

Plot the ROC curve on log–log axes and report TPR at fixed low FPRs. Drop the first point of `roc_curve`, whose FPR is 0. Pass scores, not `predict` labels, to `roc_curve(y_true, y_score)`. At a target FPR, use the highest TPR at or below it; `np.interp` may report a value no threshold achieves. Do not report TPR below an FPR of one divided by the number of negatives, where the curve represents at most one false positive.

2.9. The Bayesian view

Often you know a base rate π, for example that one transaction in a thousand is fraud. Bayes' rule says that the odds of H_1 after you see x are the odds before, multiplied by the likelihood ratio:

$$\frac{\Pr[H_1 \mid x]}{\Pr[H_0 \mid x]} = \frac{\pi}{1-\pi} \times \Lambda(x). \tag{2.5}$$

The decision with the fewest errors says H_1 when these odds exceed 1, so it is again an LR test.

The same calculation applies to an alarm, whose likelihood ratio is TPR divided by FPR. Consider a test with $\mu = 2$ in a population where 1% of cases are positive. At FPR 0.05, a flagged case is positive with probability 0.114, so most alarms are false. At FPR 10^{-3}, that probability rises to 0.582. When positives are rare, the FPR strongly affects the probability that an alarm is correct.

2.10. Measuring rates from finite trials

Estimate a detector's FPR by counting false positives among n negative test cases. If each case is independently flagged with the same probability, the count k is binomial, with standard deviation approximately $\sqrt{k}$ when the rate is small.

The **Clopper–Pearson interval** gives rates consistent with the observed count. It is exact because it uses the binomial distribution, and conservative because a 95% interval covers the true rate at least 95% of the time. In SciPy, compute it with `binomtest(k, n).proportion_ci(method="exact")`. To show that an FPR is below a target, report only the upper bound using `alternative="less"`. With no events, the one-sided 95% upper bound is about $3/n$, the **rule of three**. An exact computation requires 2,995 negative cases with no false positives to bound the FPR below 10^{-3}.

These intervals can be wide. Ten false positives in 10,000 negatives give the interval $[0.00048, 0.00184]$, from about half to nearly twice the observed rate. About 100 events estimate a rate within about 20%, so an FPR of p requires about $100/p$ negative cases. The **Wald interval** is too narrow and gives $[0, 0]$ when there are no events.

2.11. Evaluating a detector at a low false positive rate

1. Fix the target FPRs, such as 10^{-2} and 10^{-3}.
2. Plan the trials with about $100/p$ negatives for a target FPR p.
3. Choose the threshold on the negatives of one split, and measure the rates on another. A threshold tuned on the evaluation set looks better there than it is. The calibration split needs as many negatives, because below one over their number, `np.quantile` only interpolates between the top scores.
4. Report the TPR and FPR measured at that threshold, with Clopper–Pearson intervals on the two, and the full ROC curve on log–log axes. A TPR read off the ROC curve of the test split is too high, because it picks its threshold on the test data.
5. Compare two detectors at the same FPR, never by AUC or accuracy alone. On a shared evaluation set, check the gap with McNemar's test, `binomtest(m1, m1 + m2, 0.5)`. Here m_1 and m_2 count the positives that only one of the two detectors catches.

The snippet applies steps 3 and 4 to scikit-learn's breast cancer dataset. It uses logistic regression to detect malignant tumours and reserves 30% of the data for calibration and 30% for testing. For a target FPR of 1%, it sets the threshold at the 99th percentile of benign calibration scores.

```
import numpy as np
from scipy.stats import binomtest
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
y = 1 - y  # make malignant (coded 0) the positive class
X_tr, X_ev, y_tr, y_ev = train_test_split(X, y, test_size=0.6, stratify=y,
 ↪  random_state=0)
X_cal, X_te, y_cal, y_te = train_test_split(X_ev, y_ev, test_size=0.5, stratify=y_ev,
 ↪  random_state=0)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(X_tr,
 ↪  y_tr)
threshold = np.quantile(model.decision_function(X_cal[y_cal == 0]), 0.99)  # target FPR
 ↪  1%, set on calibration
s = model.decision_function(X_te)
pos, neg = s[y_te == 1] > threshold, s[y_te == 0] > threshold  # flagged positives,
 ↪  negatives
tpr = binomtest(int(pos.sum()), pos.size).proportion_ci(method="exact")
fpr_upper = binomtest(int(neg.sum()), neg.size,
 ↪  alternative="less").proportion_ci(method="exact").high
print(pos.sum(), pos.size, f"{pos.mean():.3f} [{tpr.low:.3f}, {tpr.high:.3f}]")  # 56
 ↪  63 0.889 [0.784, 0.954]
print(neg.sum(), neg.size, f"{neg.mean():.4f} <= {fpr_upper:.4f}")  # 2 108 0.0185 <=
 ↪  0.0571
```

The detector catches 56 of 63 malignant test cases: TPR 0.889, with a 95% interval from 0.784 to 0.954. The threshold set for 1% FPR flags 2 of 108 benign test cases. These counts give a one-sided 95% upper bound of 0.0571 on the FPR. Step 2 requires about 10,000 negative cases for this target, so this dataset is too small to establish the target rate; report the interval instead.

2.12. Cheat sheet

Table 2.1.: The questions of this chapter and their answers.

Question	Answer
What is the best test?	Threshold the likelihood ratio, per group if groups score differently (Neyman–Pearson)
How good can any test be?	The trade-off curve; normal pair $G_\mu(\alpha) = \Phi(\Phi^{-1}(1-\alpha) - \mu)$
What does a capped ratio allow?	$\text{TPR} \le e^\kappa \,\text{FPR}$
What do accuracy and advantage measure?	Best advantage = TV, one point of the curve
Why is AUC not enough?	Equal AUC can hide a 12-fold gap in TPR at an FPR of 10^{-3}

Question	Answer
What does a base rate do?	Posterior odds = prior odds × TPR/FPR
How sure is a measured rate?	Clopper–Pearson; zero events give about $3/n$ at 95%

Rules of thumb

- Plan for $100/p$ negatives, and use Clopper–Pearson, never Wald.
- Report TPR at fixed low FPRs on log–log axes, never AUC alone.

2.13. References

- Fawcett, "An introduction to ROC analysis" (Pattern Recognition Letters, 2006): ROC curves and their pitfalls.
- Brown, Cai & DasGupta, "Interval Estimation for a Binomial Proportion" (Statistical Science, 2001): why Wald fails and what to use.

Derivations. Every result here is derived step by step in the full edition's chapter.

Part II.

Threat models and attacks

This part describes attacks for assessing models and data releases before release. These tests complement differential privacy: they can reveal implementation errors, but cannot prove a privacy guarantee. Each attack is framed as a game in which an adversary guesses a secret from a model or release. Its score measures improvement over a baseline that does not use the model.

Start with Threat models and privacy games, then read Membership inference. Choose subsequent chapters by the secret at risk. Memorization and extraction tests whether a trained model reproduces text; Gradient inversion tests whether a federated-learning update reveals a client's input. Two reference pages cover less common attacks: attribute inference targets a person's sensitive field, and property inference targets a fact about the training population.

For released data, read the final three chapters in order. Linkage can identify records without names. Syntactic anonymity can reveal sensitive values even when records appear in groups. Reconstruction attacks can recover a dataset from accurate aggregates. Removing names or releasing only aggregates does not guarantee anonymization. Part 3 introduces differential privacy, which gives a formal guarantee for releases.

For each membership audit, report the true positive rate at a low false positive rate, such as one in a thousand. Accuracy and area under the curve can obscure risk in this region. Risk often concentrates in outliers, duplicates and people with unique attribute combinations, so an average can hide exposed records. A failed attack provides only a lower bound on leakage.

The workflow Pre-release privacy risk assessment sets blocking thresholds, compares audit results with them and records the release decision. By the end of the part, you can specify a threat model, run a membership audit, test a table for unique records and assess release risk for a model or dataset.

3. Threat models and privacy games

3.1. Why this matters

A hospital trains a model on patient records and serves it to doctors. Privacy risk depends on the adversary's access and knowledge. One adversary may see only predicted labels; another may hold the model weights, know other patients' records or add crafted records to training data. A **threat model** specifies these assumptions. An attack can then be framed as a **game** in which an adversary guesses a hidden **secret**, such as whether a patient's record was used.

3.2. Writing down the threat model

Specify the threat model before running attacks. It determines which tests to run and which results block release. The first six rows of Table 3.1 define the threat model; the final two rows specify the tests and decision rule.

Table 3.1.: The threat-model worksheet.

Question	What to write down	Default when unsure
Goal: what is the secret?	Membership, an attribute of a known record, the records themselves, or a property of the whole dataset	Membership, the goal that differential privacy bounds directly
Access: what does the adversary see?	Labels only, scores, the weights, or the training transcript	The most anyone outside the training team can get
Behaviour: does it follow the rules?	Honest-but-curious, or malicious (poisoning, tampered models)	Malicious, if it runs any part of the system
Trust: who sees the raw data?	Central curator, local (data protected on each device before collection), or distributed	Central, only if the data holder is not an adversary
Side information: what does it know?	The population, public datasets, other records, the target's other fields	Every record except the target's
Unit: what is one record?	A row, a user, a household	The person, whenever one person has several rows

Question	What to write down	Default when unsure
Attack and metric	Membership: the strongest attack's true positive rate (TPR) at a low false positive rate (FPR). A table: unique share and smallest class. A language model: canary exposure and extraction	The row of the goal and artefact in the risk assessment's thresholds
Acceptable result	The number that blocks the release	Fixed before any attack is run

Access ranges from labels to scores, weights and the **transcript**, which includes everything exposed during training, such as per-step gradients. Each level can produce the preceding one. Therefore, the best attack cannot become weaker as access increases; this ordering is the **access chain**. It compares optimal attacks, not only the attacks tested, so a failed test does not establish safety (Membership inference).

Determine access from the serving code and release plan. A service that returns probability vectors gives black-box access; a checkpoint shared with a partner gives white-box access. If a model has several deployments, assess the most exposed one. An **honest-but-curious** adversary follows the protocol and analyses its observations. A **malicious** adversary may add crafted records to training data (**poisoning**) or send tampered models. If the data collector may be an adversary, protect data before collection (Local, shuffle and distributed trust models).

Assume the adversary knows every record except the target's. The privacy unit determines the guarantee's scope: a guarantee per row covers each of a user's 200 rows individually, not the group of 200 rows.

For the hospital classifier, the API returns class probabilities, so the goal is membership inference and access is black-box scores. The doctor is honest-but-curious and may query adaptively. The side information is a public sample from the same population, and the privacy unit is the patient. Block release if the lower end of the TPR interval at FPR 10^{-3} exceeds ten times that FPR. Set this threshold before running the attack to avoid tuning it to the result.

3.3. Scoring an attack against its baseline

In the membership game, a challenger flips a coin to decide whether to train the model with or without the target's record. The adversary then guesses whether the target is a **member**, meaning that the model was trained on its record.

The **true positive rate** is the fraction of members that the attack flags. The **false positive rate** is the fraction of non-members that it flags by mistake. Write π_1 for the chance that the coin makes the target a member, the prior, and $\pi_0 = 1 - \pi_1$ for the opposite. The attack wins with probability

$$\Pr[\text{win}] = \pi_1 \,\text{TPR} + \pi_0 \,(1 - \text{FPR}). \tag{3.1}$$

The two rates do not depend on the prior, but accuracy does. If one target in ten is a member, guessing "not a member" wins $\pi_0 = 90\%$ of games without using the model. Compare each attack with a **baseline**, the best score from a guess that does not use the model.

With a fair prior, $\pi_1 = \pi_0 = 1/2$, the baseline is one half. The attack's **advantage** is how far its TPR exceeds its FPR. With the fair prior in the win rate,

$$\Pr[\text{win}] = \frac{1}{2} + \frac{1}{2}(\text{TPR} - \text{FPR}). \tag{3.2}$$

An attack with 55% accuracy at a fair prior therefore has advantage 0.1. Unlike accuracy, advantage does not depend on the prior. Differential privacy bounds every attack's advantage through the total variation distance described in Divergences.

3.4. Reporting: TPR at a low FPR

An attack assigns each target a score and flags targets above a threshold. Its **receiver operating characteristic (ROC)** curve plots TPR against FPR as the threshold varies; advantage is the curve's maximum height above the diagonal. Neither advantage nor area under the curve shows performance where false positives are rare. An attack that flags 1% of members with no false positives has small overall advantage but identifies those flagged members with certainty. Report membership attacks by TPR at a low FPR, such as 10^{-3}, following Carlini et al. (2022).

A flagged person cares about the chance that the verdict is right, the **precision**,

$$\Pr[\text{member} \mid \text{flagged}] = \frac{\pi_1 \,\text{TPR}}{\pi_1 \,\text{TPR} + \pi_0 \,\text{FPR}}. \tag{3.3}$$

When members are rare, π_0 is large, and even a small FPR among non-members can exceed the number of true positives. For example, release a count with Gaussian noise of standard deviation 1. A member adds 1 to the count, so an adversary who knows all other records flags the target when the count is high. At FPR 10^{-3}, the TPR is 0.0183. With a fair prior, precision is $0.0183/(0.0183 + 0.001) = 0.948$. If one target in a hundred is a member, precision falls to 0.156. Report precision with its prior.

Each rate is estimated from a finite target set and needs a confidence interval. The Hypothesis testing procedure plans about $100/\alpha$ non-members for target FPR α, yielding about 100 false positives. It sets the threshold on one split, evaluates on another and computes a Clopper–Pearson interval for each rate.

The snippet below applies a short form of the recipe to the black-box membership attack of the Adversarial Robustness Toolbox (ART). The target is a random forest trained on half of scikit-learn's 569 breast cancer records. The attacker trains its attack on every other record, whose membership it knows, and reads the TPR on the rest at an FPR of at most 0.01. It sets the threshold on those same scores.

```
import numpy as np
from art.attacks.inference.membership_inference import MembershipInferenceBlackBox
from art.estimators.classification import SklearnClassifier
from scipy.stats import binomtest
```

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import auc, roc_curve

np.random.seed(0)  # ART's attack model draws from NumPy's global generator
x, y = load_breast_cancer(return_X_y=True)
member = np.random.default_rng(0).random(len(y)) < 0.5  # the target's training records
target = SklearnClassifier(RandomForestClassifier(random_state=0).fit(x[member],
 ↪  y[member]))
known = np.arange(len(y)) % 2 == 0  # records whose membership the attacker knows, to
 ↪  train its attack
attack = MembershipInferenceBlackBox(target, attack_model_type="rf")
attack.fit(x[known & member], y[known & member], x[known & ~member], y[known &
 ↪  ~member])
fpr, tpr, _ = roc_curve(member[~known], attack.infer(x[~known], y[~known],
 ↪  probabilities=True)[:, 0])
i = np.flatnonzero(fpr <= 0.01)[-1]  # the best TPR whose FPR is at most 0.01
n_in, n_out = int(member[~known].sum()), int((~member[~known]).sum())
tp, fp = round(tpr[i] * n_in), round(fpr[i] * n_out)
print(f"AUC={auc(fpr, tpr):.3f}  TP={tp} of {n_in}  FP={fp} of {n_out}")  # AUC=0.552
 ↪  TP=3 of 133  FP=1 of 151
t, f = (binomtest(k, n).proportion_ci(method="exact") for k, n in ((tp, n_in), (fp,
 ↪  n_out)))
print(f"TPR 95% CI [{t.low:.4f}, {t.high:.4f}]  FPR 95% CI [{f.low:.4f},
 ↪  {f.high:.4f}]")  # TPR 95% CI [0.0047, 0.0645]  FPR 95% CI [0.0002, 0.0363]
```

With only 151 non-members, the smallest FPR above zero is one in 151, far above one in a thousand. So the FPR's interval runs from 0.0002 to 0.0363, and it overlaps the TPR's interval, from 0.0047 to 0.0645. So this test cannot show that the attack beats chance at a low FPR, and its area under the ROC curve, 0.552, adds little. At 10^{-3} the recipe asks for about 100,000 non-members, which is why audits retrain many models on many records (Membership inference).

So report the TPR at a stated low FPR, with its counts, an interval on each rate and the ROC curve on log axes. Also give the worksheet's threat model, the baseline, the prior behind any precision, and whether the targets were random or chosen worst cases.

3.5. Average case and worst case

An evaluation that draws the data and the target from the population measures the **average case**. If the adversary knows or picks the other records and the target, the evaluation measures the **worst case**. Audits of real models measure the first, a guarantee must control the second, and the gap can be large. In the noisy count, the adversary who knows the other records has an advantage of 0.3829. If 100 of those records are fair coins unknown to the adversary, their sum blurs the count like extra noise, and the advantage falls to 0.0779. But an adversary who planted the other records knows them, so a defence that relies on the crowd fails against the adversary it most needs to stop.

An average over targets also hides the worst-treated person, because a few exposed outliers barely move it. So treat an average-case number as a lower bound on the risk. Audits close the gap with

canaries, records built to be as exposed as possible (Memorization and training-data extraction). Differential privacy bounds the worst case directly.

3.6. Cheat sheet

Table 3.2.: The results of this chapter.

Quantity	Formula	Use it for
Advantage	$\mathrm{TPR} - \mathrm{FPR}$	Comparing attacks at any prior; 0 means no attack
Precision	$\pi_1\mathrm{TPR}/(\pi_1\mathrm{TPR} + \pi_0\mathrm{FPR})$	What a flagged person faces; state the prior

Rules of thumb

- Fill in the worksheet, including the blocking threshold, before running or reading any attack.
- Assume the adversary knows the other records, and audit with worst-case canaries.

3.7. References

- Salem et al., "SoK: Let the Privacy Games Begin! A Unified Treatment of Data Inference Privacy in Machine Learning" (IEEE S&P 2023): every inference attack written as a game.
- Carlini, Chien, Nasr, Song, Terzis & Tramèr, "Membership Inference Attacks From First Principles" (IEEE S&P 2022): the case for reporting TPR at a low FPR on log-scaled ROC curves.

Derivations. Every result here is derived step by step in the full edition's chapter.

4. Membership inference

4.1. Why this matters

A hospital publishes a disease-prediction model. A patient may ask whether their record was in the training set; if so, the model's training data included a record of their disease. **Membership inference** tests whether a model used a specified record. It is a standard measure of leakage because a model that reveals record membership may reveal other information about that record. An audit therefore compares each record with the model's behaviour on that record.

4.2. Membership as a test about one record

In the game described in Threat models and privacy games, the attacker receives either a training member or a fresh record and classifies it as a member or non-member. Its **true positive rate (TPR)** is the fraction of members it flags; its **false positive rate (FPR)** is the fraction of non-members it flags.

Fix a target record z and measure the model's output on it, such as its loss. Across retrainings on fresh data samples, outputs have an **IN** distribution when the training set includes z and an **OUT** distribution when it excludes z. The optimal membership test compares their likelihoods at the observed output (the Neyman–Pearson lemma); it performs best where the distributions differ most.

The IN and OUT distributions for a typical record often overlap because other records predict it. For an outlier, they may differ more because the model predicts it well only after training on it. As a result, low-FPR attacks often identify outliers.

The numbers in this chapter come from practitioner_lira.py. It trains 64 logistic-regression classifiers, each on a random half of scikit-learn's 1,797 images of handwritten digits. Each model in turn is the target, and the other 63 are the attacker's **shadow models**, copies trained the same way on images the attacker knows.

4.3. The quick check: the generalization gap

Before running an attack, compare training and test errors. Because models tend to predict training records better than fresh records, Yeom et al. (2018) proposed the rule "member if the label is correct." When members and non-members come from the same population, the attack's advantage (TPR minus FPR) equals the difference between test and training errors:

$$\text{Adv} = \text{test error} - \text{training error}. \tag{4.1}$$

On the digits, averaged over the 64 models, the gap is 0.0422.

The gap is a low-cost indicator of leakage. However, **a small gap does not establish privacy**: it averages across records and can hide exposed individuals. For example, a loss-threshold attack flags records below a single loss threshold. On the digits data, its TPR is 0.0009 at FPR 10^{-3}, no better than random guessing at that rate.

The attack of Shokri et al. (2017) also learns one rule for all records from shadow models. The script runs it with four shadow models using the Adversarial Robustness Toolbox (ART).

```
# ART copies the target's recipe and trains each shadow on a random half of the images.
shadows = ShadowModels(models.first, num_shadow_models=N_ART_SHADOWS,
 ↪  random_state=seed)
(x_in, y_in, pred_in), (x_out, y_out, pred_out) = shadows.generate_shadow_dataset(x,
 ↪  onehot)
# One attack model learns "member" from a shadow's probabilities and the true label.
attack = MembershipInferenceBlackBox(models.first, attack_model_type="rf")
attack.fit(x_in, y_in, x_out, y_out, pred=pred_in, test_pred=pred_out)
# The same attack model scores every target's probabilities on every image.
scores = [attack.infer(x, onehot, pred=p, probabilities=True)[:, 0] for p in
 ↪  models.probs]
```

This global rule also performs poorly at low FPR: it detects 0.0014 of members at FPR 10^{-3}.

4.4. LiRA: a likelihood-ratio test per record

The likelihood ratio attack (LiRA) of Carlini et al. (2022) uses shadow models to estimate each record's IN and OUT distributions. Its statistic is the logit of the model's confidence p_y in the true label y:

$$\phi = \ln \frac{p_y}{1 - p_y}. \tag{4.2}$$

The logit is approximately normal across retrainings, while confidence values can cluster near 1. For each record, the attacker fits a normal distribution to its logits from shadow models that included the record, with mean μ_{in} and standard deviation σ_{in}. A second fit to models that excluded it has mean μ_{out} and standard deviation σ_{out}.

Online LiRA compares the target model's logit ϕ_z with the IN and OUT fits. If $\mathcal{N}(\phi; \mu, \sigma^2)$ denotes a normal density, the score is

$$\ln \Lambda_z = \ln \mathcal{N}(\phi_z;\, \mu_{\text{in}}, \sigma_{\text{in}}^2) - \ln \mathcal{N}(\phi_z;\, \mu_{\text{out}}, \sigma_{\text{out}}^2). \tag{4.3}$$

Offline LiRA uses only OUT models, which exclude the target record and can be reused for new records. Its score is the logit's distance above the OUT mean in standard deviations:

$$\frac{\phi_z - \mu_{\text{out}}}{\sigma_{\text{out}}}. \tag{4.4}$$

Each score already compares a record with its own fits, so one threshold works for all records. In the script, ART's `SklearnClassifier` wraps the scikit-learn recipe, trains every model and returns its probabilities.

```
# Each image is in each model's training set with probability 1/2.
member = rng.random((n_models, n)) < 0.5
for k in range(n_models):
    model = recipe()  # a fresh, untrained copy of the target's recipe
    model.fit(x[member[k]], onehot[member[k]])
    probs[k] = model.predict(x)  # the probabilities the attacker sees, for every image
    p = np.clip(probs[k, np.arange(n), y], 1e-15, 1 - 1e-15)  # confidence on the true
↪   label
    phi[k] = np.log(p) - np.log(1 - p)  # its logit, close to Gaussian over retrainings
    confidence[k] = p
```

No library installed with this book computes the LiRA scores, so `lira_scores` in practitioner_privacy_tests.py fits the two normals per image and returns Equation 4.3 and Equation 4.4. The command `uv run python code/part02-attacks/practitioner_lira.py` runs in seconds. It compares the attacks by their TPR at two low FPRs and by their area under the curve (AUC) of the TPR against the FPR.

```
  loss attack: AUC=0.518  TPR@0.001=0.0009  TPR@0.01=0.0105
  ART shadow-model attack: AUC=0.570  TPR@0.001=0.0014  TPR@0.01=0.0146
  LiRA offline: AUC=0.577  TPR@0.001=0.0087  TPR@0.01=0.0345
  LiRA online: AUC=0.623  TPR@0.001=0.0318  TPR@0.01=0.0790
  LiRA online, global variance: AUC=0.614  TPR@0.001=0.0250  TPR@0.01=0.0737
```

The attacks have similar AUC values, and ART nearly matches offline LiRA. At FPR 10^{-3}, their TPRs differ: online LiRA detects 0.0318 of members, compared with 0.0009 for the loss attack. All attacks use the same model outputs; LiRA's improvement comes from comparing each record's output with its own IN and OUT fits.

Noisy fits are a practical limitation because only about half the shadow models estimate each record's variance. An underestimated variance makes ordinary outputs appear extreme. With few shadow models, Carlini et al. recommend one **global variance** across records. This can work with 16 shadows; record-specific variances require about 64.

4.5. Running LiRA on your model

For any model, train the target and shadow models on random halves of the same data pool using the target's exact training procedure. A different procedure, including a different augmentation, estimates different IN and OUT distributions. Fit the normal distributions to each record's logits, then treat each model in turn as the target and score every model-record pair. Pre-release privacy risk assessment compares these results with its release rule.

For a confident model, p_y may be so close to 1 that $1 - p_y$ rounds to 0, making the logit infinite. Compute the logit from class scores f before softmax, using $f_y - \ln \sum_{y' \neq y} e^{f_{y'}}$, or use `log_softmax`.

Do not clip probabilities to avoid this problem. On a nearly separable model, many confidences reach the clipping value, leaving no variation for the fits. The affected pairs are then dropped,

potentially including the most exposed records. Treat dropped pairs as a failed audit. The script clips only because the digits models' confidences remain well within the clipping range.

Draw members and non-members from the same pool and split it at random. If non-members come from another period, site or preprocessing pipeline, the attack may measure that distribution shift instead of membership.

Returning only labels does not prevent membership inference. A label-only attack measures how much an input must change before its predicted label flips (Choquette-Choo et al. 2021). ML Privacy Meter implements LiRA. TensorFlow Privacy's `run_attacks` and ART's `MembershipInferenceBlackBox` apply the same attack rule to all records.

Training changes can affect leakage, so rerun LiRA on each release candidate. Train 16 models with the candidate's recipe and score online LiRA using a global variance. On the digits, `practitioner_lira.py --models 16` runs in seconds and blocks the model under the next section's rule, as does the 64-model run. Compare candidates using their intervals; point estimates vary with the random seed.

4.6. Reading the result

Report the TPR at an FPR of 10^{-3} or lower, with the receiver operating characteristic (ROC) curve on log axes. Put a Clopper–Pearson interval on each rate, as in Part 1's recipe for evaluating a detector at a low FPR. That recipe also sets the thresholds on a separate split, which the script, like the literature, does not do.

```
 LiRA offline: TP=502/57704  TPR=0.0087 [0.0080, 0.0095]  FP=56/57304  FPR=0.00098
↪  [0.00074, 0.00127]  ln(TPR/FPR)=2.19  pass
 LiRA online: TP=1833/57704  TPR=0.0318 [0.0303, 0.0332]  FP=57/57304  FPR=0.00099
↪  [0.00075, 0.00129]  ln(TPR/FPR)=3.46  block
```

The final word on each line reports the risk assessment's blocking rule. It blocks release when the lower end of the TPR's 95% interval at FPR 10^{-3} exceeds ten times that FPR. Online LiRA's lower end, 0.0303, exceeds 0.01 and blocks this model; offline LiRA alone would pass it. For online LiRA, $\ln(\text{TPR}/\text{FPR}) = 3.46$ estimates the differential privacy level ε (Part 3) needed to permit this attack. It is not a certified lower bound because pairs share models and are not independent trials, unlike canaries in Auditing.

4.7. Who is exposed, and what helps

The highest-scoring records are often outliers, members of rare subgroups or mislabeled points that other data do not predict well. On the digits data, the most exposed image is a 9. Models assign it confidence 0.812 when trained on it and 0.003 when not trained on it. Online LiRA therefore flags this 9 at FPR 10^{-3} in all 33 target models that included it.

Removing exposed records can expose other records in turn, the "onion effect" described by Carlini et al. (2022b). Reducing the average generalization gap is also insufficient because the low-FPR risk depends on the most exposed records. Differential privacy provides a guarantee:

Private training limits how much one record can change the model and thus limits the difference between its IN and OUT distributions. This may reduce utility when accurate prediction requires learning rare subgroups (Feldman 2020).

4.8. Cheat sheet

Table 4.1.: Which attack or check to use.

Situation	Use	Notes
A first look, no attack budget	Test error − training error	A lower bound on the leak; a small gap proves nothing
A release decision	Online LiRA, Equation 4.3, 64 shadows	Report TPR at FPR 10^{-3} with intervals
Few shadow models	Online LiRA with a global variance	16 shadows are usable this way

Rules of thumb

- Audit with LiRA. A failed weak attack, such as a global loss threshold, is not evidence of safety.

4.9. References

- Carlini, Chien, Nasr, Song, Terzis & Tramèr, "Membership Inference Attacks From First Principles" (IEEE S&P 2022): LiRA, the global variance, and the audit protocol.
- Yeom, Giacomelli, Fredrikson & Jha, "Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting" (IEEE CSF 2018): the loss attack and its link to the generalization gap.

Derivations. Every result here is derived step by step in the full edition's chapter.

5. Attribute inference and model inversion

5.1. What it is

An **attribute inference** attack uses a trained model and known fields of a person's record to infer a sensitive field, such as smoking status. For example, an insurer with a person's age and diagnosis could query a hospital model that predicts disease from age and smoking. A public study may support the same inference, so attack success alone does not establish a privacy failure. The relevant measure is how much more the model reveals because it trained on the person's record.

Yeom et al. (2018) turned this question into a game (Threat models and privacy games). A fair coin decides whether the target is a training record, a **member**, or a fresh draw from the same population. The attacker knows the target's public features and label, queries the model and guesses the sensitive value. Its score is the **attribute advantage**,

$$\mathrm{Adv}^{\mathrm{attr}} = \Pr[\text{guess right} \mid \text{member}] - \Pr[\text{guess right} \mid \text{non-member}]. \tag{5.1}$$

A guess based only on the population, called **population inference**, helps members and non-members equally because both come from the same population. It cancels in the difference, leaving the model's additional information about individuals.

5.2. When you meet it

Evaluate attribute inference when assessing a claim that a model reveals a sensitive field or when a model omits that field. Only additional success on members is a privacy leak. Inference supported by population-level correlations does not require the person's record and is primarily a policy and fairness concern.

5.3. How to use it

First measure the **imputation baseline**: the best prediction of the sensitive value from public features and the label, without querying the model. Many published attacks do no better (Jayaraman & Evans 2022). Estimate the baseline by fitting a classifier to population records that were not used to train the target model.

Next run the attack. **Model inversion** (Fredrikson et al. 2014) scores each candidate sensitive value s using two quantities. The first, $p(s \mid x)$, is the estimated probability of s given the target's

public features x. The second, $f_y(x, s)$, is the model's confidence in the known label y for input (x, s). The attack chooses the value that maximizes their product:

$$\hat{s} = \arg\max_s \; p(s \mid x)\, f_y(x, s). \tag{5.2}$$

Give the attacker every column it could know. A near-copy of the sensitive field, such as relationship status alongside marital status, can raise the baseline. Run the attack on training members and as many held-out non-members as possible. Report the baseline, both success rates and their confidence intervals. Also report an interval for the advantage; Pre-release privacy risk assessment blocks a release when that interval lies above zero.

The script practitioner_attribute.py, run with `uv run python code/part02-attacks/practitioner_attribut` applies the Adversarial Robustness Toolbox (ART) to Adult, a census dataset with 45,222 records. A random forest predicts whether income exceeds $50K using all columns; sex is the sensitive field. The attacker has 4,000 auxiliary records and knows each target's other columns and income.

```
def label(rows: IntArray) -> Array:  # the label the model returns for each record, as
 ↪  a column
    return np.argmax(target.predict(x[rows]), axis=1).reshape(-1, 1).astype(np.float64)

np.random.seed(seed)  # ART builds its random forests unseeded, from NumPy's global
 ↪  generator
attack = AttributeInferenceBlackBox(target, attack_model_type="rf",
 ↪  attack_feature=SENSITIVE)
attack.fit(x[attacker])  # the attacker's own records, sensitive field included
baseline = AttributeInferenceBaselineTrueLabel(attack_model_type="rf",
 ↪  attack_feature=SENSITIVE)
baseline.fit(x[attacker], y[attacker])  # the same records and the true label, without
 ↪  the model
hits: dict[str, list[int]] = {}
for name, rows in (("members", members), ("non-members", others)):
    known = np.delete(x[rows], SENSITIVE, axis=1)  # everything but the sensitive field
    guess = attack.infer(known, pred=label(rows), values=[0.0, 1.0])
    imputed = baseline.infer(known, y=y[rows], values=[0.0, 1.0])
    hits[name] = [int(np.count_nonzero(np.equal(g.ravel(), female[rows]))) for g in
 ↪  (guess, imputed)]
```

```
ART black-box attack: right on members 0.8300 [0.8180, 0.8415]  on non-members 0.8423
 ↪  [0.8306, 0.8534]
ART black-box attack: advantage -0.0123 [-0.0285, +0.0040]
imputation baseline: right on members 0.8307 [0.8188, 0.8423]  on non-members 0.8417
 ↪  [0.8301, 0.8529]
```

The attack is correct for 0.8300 of members; the baseline is correct for 0.8307. The result is consistent with population inference, and the advantage interval includes zero, so it passes the rule. On a synthetic population in the full edition, inversion is correct for 0.7832 of members and 0.7262 of non-members, for an advantage of 0.0569.

5.4. What goes wrong

The most common reporting error is to give only the member success rate, such as 0.7832, without the baseline and non-member rate. This overstates leakage. Reporting only the rate on fresh records understates it because models are more confident on members (Membership inference).

A model without the sensitive column may still support population inference through correlated fields. **Confidence masking**, which rounds confidence scores or returns only the top label, does not bound the gap between members and non-members; it can increase that gap. In the full edition's experiment, returning only labels raised advantage from 0.0569 to 0.0650 because success fell more on non-members. Reducing overfitting can narrow the gap. Differential privacy bounds it for every attacker and output format.

5.5. References

- Fredrikson, Jha & Ristenpart, "Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures" (CCS 2015): inversion from confidences, and rounding as a countermeasure.
- Jayaraman & Evans, "Are Attribute Inference Attacks Just Imputation?" (CCS 2022): the imputation baseline, and what it takes to beat it.

Derivations. Every result here is derived step by step in the full edition's chapter.

6. Property inference

6.1. What it is

A **property** describes a training set as a whole, such as the share of a hospital's patients over age 65. **Property inference** attempts to recover that property from a trained model. In the game described in Threat models and privacy games, a fair coin selects one of two subgroup shares. The owner trains on data with the selected share, and an attacker who knows both shares and the training method guesses which was used. Random guessing succeeds half the time.

A property may leak even when no individual record does. Let n be the number of training records and g the gap between the two shares. Changing the share changes about ng records, so its effect on the model can exceed that of changing one record. The property leak can therefore grow with n even as the risk to an individual decreases.

Even a release of only the exact count of subgroup records shows this gap. With 10,000 records, the best guess of whether one patient is in the data is right with probability 0.502. The best guess of whether the share was 0.45 or 0.55 is right with probability 1.0000.

6.2. When you meet it

Assess property inference before sharing or publishing a model trained on organizational data. First determine whether the property is secret—whether it describes the *data owner* rather than the general population. A hospital's patient mix may be secret; the association between smoking and lung cancer may be the intended finding of a medical study. If the property is not secret, document that in the threat model of Pre-release privacy risk assessment.

Differential privacy (DP) does not protect a secret property of the whole dataset. It protects one record at a time; for a group, the bound scales with the group's size (Post-processing, group privacy and composition). For a hospital with 1,000 patients and subgroup shares that differ by 0.1, the datasets differ in $ng = 100$ patients. The group guarantee is therefore 100ε, which is not useful for typical values of ε.

6.3. How to use it

Measure risk with the shadow-model attack of Ateniese et al. (2015), which treats models as data points. Draw many datasets of the training set's size from the same population, with each of the two shares. Train a **shadow model** on each dataset using the target architecture and training procedure. Record each model's outputs on a fixed set of probe inputs.

Fit a linear **meta-classifier**, such as scikit-learn's `LogisticRegression`, to distinguish the two shares from the recorded outputs. Its accuracy on held-out shadow models measures the attack. If the property is secret, block release when the confidence interval for accuracy lies above one half (Pre-release privacy risk assessment).

Use probe outputs rather than raw weights. Permuting hidden units or flipping the signs of tanh units changes the weights without changing the network function. In the full edition's small tanh-network experiment, each raw weight averages to zero under both shares. A linear meta-classifier scores 0.497 from raw weights, near chance, and 1.000 from probe outputs. Because probes require only query access, this attack also applies to models behind an application programming interface.

6.4. What goes wrong

Do not assume that DP or a passed membership audit protects a dataset property. A weak measured attack also does not establish safety: a meta-classifier trained on noisy shadow models may fail to detect a small shift.

Removing the group column does not remove its correlates. Noise large enough to hide ng records may destroy model utility. If the property is secret, do not release a model trained on that data mix. If release is necessary, report the attack accuracy and state that DP protects individuals, not dataset properties.

6.5. References

- Ganju, Wang, Yang, Gunter & Borisov, "Property Inference Attacks on Fully Connected Neural Networks using Permutation Invariant Representations" (CCS 2018): the practical attack on neural networks, and why raw weights fail.
- Zhang, Tople & Ohrimenko, "Leakage of Dataset Properties in Multi-Party Machine Learning" (USENIX Security 2021): property inference through queries alone.

Derivations. Every result here is derived step by step in the full edition's chapter.

7. Memorization and training-data extraction

7.1. Why this matters

A language model trained on scraped text or support tickets may learn phone numbers, addresses, keys and private messages. Given the prompt "Anna Smith's phone number is", it may generate the real number. This is **training-data extraction**. Membership inference tests whether the model used a specified record; extraction recovers the record itself.

7.2. Which memorization to measure

The memorization definitions in Table 7.1 produce rates that differ by orders of magnitude. Reports must specify which definition they use. Each definition uses greedy decoding, which selects the most likely next token at each step.

Table 7.1.: The notions of memorization and when to use each. The first three are verbatim, needing an exact match, and each implies approximate. Discoverable implies extractable.

Definition	The string counts as memorized if	Use it for
Discoverable (Carlini et al. 2023)	The text before it in the training data makes greedy decoding produce it	Measuring memorization on a model you own
Extractable (Carlini et al. 2021)	Some prompt, any at all, makes greedy decoding produce it	The real risk, which attacks find but no test measures
k-eidetic	It is extractable and appears in at most k documents	Flagging the dangerous case, $k = 1$
Approximate (Ippolito et al. 2023)	Greedy decoding comes within a set edit distance of it	Near copies, such as a number with one digit changed

Discoverable memorization is measurable because the auditor has the training text used as a prompt. A verbatim rate is a lower bound on leakage because near copies may also reveal information. A filter that blocks only verbatim text can be bypassed (Ippolito et al. 2023).

Deployed systems sample rather than decode greedily, so extraction probability depends on the attacker's sample count. At temperature 1, let p be the model's probability of generating the

string after the prompt and let β be the attacker's tolerated failure probability. At least m independent samples are sufficient when

$$m \geq \frac{\ln(1/\beta)}{p}. \tag{7.1}$$

For a string with probability 10^{-3} and $\beta = 0.01$, the requirement is about 4,600 samples. Report the sampling budget with the greedy rate and use the deployed decoding settings.

7.3. What gets memorized, and the first defences

Memorization increases with the number of copies of a string. In open models, the largest reproduced at least 1% of the training data under the discoverable definition (Carlini et al. 2023). A sequence repeated ten times is generated about a thousand times more often than one seen once (Kandpal, Wallace & Raffel 2022).

The lowest-cost defence is **deduplication**, which reduces repeated copies. Models trained on deduplicated data emit memorized text about ten times less often at the same quality (Lee et al. 2022). Deduplicate before inserting canaries so their copy counts are controlled.

The second defence removes personal data before training. A personally identifiable information (PII) detector, such as Microsoft's Presidio, identifies fields such as names and phone numbers for masking, but can miss patterns it does not cover. Neither deduplication nor detection protects a secret that appears once in a large model. Fine-tuning with differential privacy limits how much one training sequence can change the model and provides a guarantee for such singletons.

7.4. Auditing with canaries and exposure

Discoverable memorization is binary, but an audit needs a graded measure. Carlini et al. (2019) insert **canaries**, random secrets with a fixed format, such as "my pin is" followed by five random digits. After training, rank all possible secret values by model likelihood. The planted value's **rank** is the number of guesses required if an attacker tests them in order.

Write $\hat{r}$ for the planted value and $|\mathcal{R}|$ for the number of values in the set $\mathcal{R}$ it could have taken. The **exposure** is the number of bits of search the model saves an attacker,

$$\text{exposure}(\hat{r}) = \log_2 |\mathcal{R}| - \log_2 \text{rank}(\hat{r}). \tag{7.2}$$

Each exposure bit halves the number of required guesses. Rank 1 gives maximum exposure $\log_2 |\mathcal{R}|$ and allows extraction. For five digits, there are 10^5 values and maximum exposure is 16.61 bits.

Exposure supports an audit because its null behaviour is known. A uniformly random canary that the model never saw has a uniformly random rank. Therefore, for any threshold of t bits,

$$\Pr\left[\text{exposure}(\hat{r}) \geq t\right] \leq 2^{-t}. \tag{7.3}$$

Thus, $2^{-\text{exposure}}$ bounds the p-value: the probability that an uninserted canary appears at least this memorized. A uniform rank gives an uninserted canary mean exposure $\log_2 e$, about 1.44 bits; controls should be near this value.

Draw canaries in the format of secrets present in the data; an unrelated format tests only how the model treats unfamiliar text. Insert each canary a controlled number of times, retain uninserted controls and fine-tune normally. Score each canary and many random values in the same format using their model log-probability after the prefix. The fraction of random values with at least as high a score estimates the **rank fraction**, the rank divided by $|\mathcal{R}|$. Its negative base-2 logarithm estimates exposure.

Scoring errors can invalidate the audit. Candidate log-probability is the sum over tokens, whereas Hugging Face models return mean loss divided by token count. Scores for candidates with different token lengths are therefore not comparable. Use summed log-probabilities and a format that gives every candidate the same number of tokens. For example, write a pin as "the pin of Alice is 8 6 5 2 3", with each digit represented by one token.

Fine-tune a language model privately with LoRA runs this audit on Pythia-14m. It inserts three canaries at each copy count from 1 to 64 into 1,024 Tiny Shakespeare sequences, keeps three controls and ranks each value among 2,000 random candidates. Exposure increases with copy count. Controls have 1.4 bits of exposure, near the expected 1.44. Greedy decoding extracts only the three canaries repeated 64 times, while two canaries repeated four times reach 6.9 and 8.2 bits. Exposure therefore detects risk that extraction tests miss.

The 2,000 candidates cap the measured exposure. If a canary outranks all candidates, its rank fraction is only known to be below about 1 in 2,000, so measured exposure is capped at $\log_2 2000 = 10.97$ bits. To estimate exposure beyond this cap, fit a skew-normal distribution to candidate scores and estimate the canary's rank from its tail, as Carlini et al. (2019) do. Ties also hide memorization: if all candidates have the same score, each has exposure 0.

Use a canary repeated 16 or more times as a positive control. If it does not reach high exposure without privacy, the run is too short to assess memorization. In the cited run, canaries occupy one quarter of the training set, allowing this small model to memorize them. Pre-release privacy risk assessment specifies the exposure threshold for blocking release.

7.5. Extraction without the prefix: generate, rank, verify

A real attacker has access only to the model. Carlini et al. (2021) extracted data from GPT-2 in three steps: they generated many samples, ranked them by likelihood and checked the highest-ranked candidates for memorization. Ranking is difficult because a model may assign high probability to a string either because it is ordinary text or because it memorized it. A **reference model**, trained on similar data but not on the target's training set, estimates ordinary-text likelihood. Write $p_\theta(x)$ for the target model's probability of a string x, and $p_{\text{ref}}(x)$ for the reference model's probability. Rank by the log-likelihood ratio

$$\Lambda(x) = \ln p_\theta(x) - \ln p_{\text{ref}}(x). \tag{7.4}$$

The ratio discounts strings that any model finds likely, such as software licenses and boilerplate. Raw likelihood ranks these strings highly even though they reveal no memorization. A smaller

model from the same family can serve as the reference. For a fine-tuned model, use its base model.

Without the prefix, attackers extract less than the discoverable rate suggests, but alignment does not prevent extraction. Nasr et al. (2023) prompted ChatGPT to repeat one word until it stopped following its aligned behaviour and generated memorized text. They extracted over 10,000 records using about 200 US dollars of queries.

7.6. Cheat sheet

Table 7.2.: The results of this chapter.

Question	Answer
Exposure of a canary	$\log_2\lvert\mathcal{R}\rvert - \log_2$ rank, at most 16.61 bits for five digits; p-value at most $2^{-\text{exposure}}$; controls average 1.44 bits
How to score a candidate	Summed log-probability, in a format where all candidates have the same token count
Ranking generations	$\ln p_\theta(x) - \ln p_{\text{ref}}(x)$, never raw likelihood
Defences, in order	Deduplicate (`deduplicate-text-datasets`, `datasketch`), scrub PII, then differential privacy for singletons

Rules of thumb

- A verbatim rate is a lower bound on the leak, not a measure of it.

7.7. References

- Carlini, Liu, Erlingsson, Kos & Song, "The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks" (USENIX Security 2019): canaries, exposure and how to estimate it.
- Carlini, Ippolito, Jagielski, Lee, Tramèr & Zhang, "Quantifying Memorization Across Neural Language Models" (ICLR 2023): how memorization scales with duplication, model size and prompt length.

Derivations. Every result here is derived step by step in the full edition's chapter.

8. Gradient inversion

8.1. Why this matters

In **federated learning (FL)**, a server sends the current model to each client. Each client computes the loss gradient on its records and returns the gradient; the server averages the gradients and updates the model. This arrangement is sometimes described as private because raw data remain on the device.

However, a client's gradient may reveal its data because many gradient components are simple functions of the input. Averaging over a batch provides limited protection (Section 8.6). Treat each client update as sensitive as the underlying data.

8.2. The attacker and what it sees

The server knows the model parameters and sees each client's update. Its access is the training **transcript** defined in Threat models and privacy games. First assume the server is **honest-but-curious**: it follows the protocol but analyses all received information. A client with a batch of B records sends the mean gradient. Let ℓ_i be the loss on record i, which depends on model parameters θ. The client sends

$$g = \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \ell_i. \tag{8.1}$$

8.3. Exact recovery from a fully connected layer

For a fully connected first layer with a bias, one division recovers the input. Let W_1 and b_1 be the weights and bias. Unit j multiplies input x by its weight row and adds its bias. Let u_j be the loss derivative with respect to the unit's output. The chain rule gives the weight and bias gradients

$$\nabla_{W_1} \ell = u\, x^\top, \qquad \nabla_{b_1} \ell = u. \tag{8.2}$$

Thus, row j of the weight gradient is x scaled by u_j, and unit j's bias gradient is u_j. For one record, their ratio recovers the input exactly whenever the bias gradient is nonzero:

$$x = \frac{(\nabla_{W_1} \ell)_{j,:}}{(\nabla_{b_1} \ell)_j}. \tag{8.3}$$

The network depth, loss and label affect u_j, which cancels in the ratio. The attack therefore requires neither optimization nor prior knowledge of the input. The snippet simulates one client using an image from scikit-learn's digits dataset in PyTorch. Its final line recovers the label, as described in Section 8.4.

```
import torch
import torch.nn.functional as F
from sklearn.datasets import load_digits

torch.manual_seed(0)
images, labels = load_digits(return_X_y=True)
x = torch.tensor(images[[7]] / 16, dtype=torch.float32)  # one client's image, pixels
 ↪  in [0, 1]
y = torch.tensor(labels[[7]])
first, last = torch.nn.Linear(64, 32), torch.nn.Linear(32, 10)
F.cross_entropy(last(torch.relu(first(x))), y).backward()  # the gradient the client
 ↪  sends
# Row j of the weight gradient is the input times the bias gradient of unit j: divide
 ↪  them.
j = int(first.bias.grad.abs().argmax())
recovered = first.weight.grad[j] / first.bias.grad[j]
print(f"image recovered: {torch.allclose(recovered, x[0], atol=1e-6)}")  # image
 ↪  recovered: True
# Only the true label's logit has a negative gradient (softmax minus one-hot).
print(f"recovered label {int(last.bias.grad.argmin())}, true label {int(y)}")  #
 ↪  recovered label 7, true label 7
```

The snippet uses the unit with the largest bias gradient because an inactive ReLU unit has zero gradient; one active unit suffices. A convolutional first layer does not permit this division. It reuses each filter across image positions, so its gradient sums products from multiple patches. The attacker must then use a search method (Section 8.5).

8.4. Label recovery from the last layer

The label, such as a diagnosis, may be the most sensitive part of a record. It can be recovered from the last layer without search (Zhao, Mopuri & Bilen 2020) when that layer has a bias and feeds a softmax with cross-entropy loss. Let p be the predicted probabilities and e_y the vector with 1 at the true label y. The last-layer bias gradient b_2 is $p - e_y$. Since each entry of p is between 0 and 1, only the true-label entry is negative. For one record,

$$y = \arg\min_k (\nabla_{b_2} \ell)_k. \tag{8.4}$$

This is the snippet's `argmin`. If the model is so confident that p_y rounds to 1, the true-label gradient becomes zero and the rule no longer identifies a negative entry.

A batch hides the label of each record but may reveal each class count. Each bias-gradient entry equals that class's average predicted probability minus its fraction of the batch. A server that estimates the average prediction can recover the class counts. In the full edition's experiment, all counts were exact for every batch of 4 and for 52.5% of batches of 64.

8.5. Deep Leakage from Gradients

When division does not apply, the attacker can use **Deep Leakage from Gradients (DLG)** (Zhu, Liu & Han 2019). DLG guesses inputs, computes their gradients and adjusts the inputs

until the average gradient matches the observed gradient g. Let x'_i be the guessed inputs, y_i their labels (recovered from the last layer), and $\ell(\theta; x', y)$ the loss of a guess. The attack minimizes the **matching loss** R, the squared distance between the guessed mean gradient and g:

$$\hat{x}_1, \dots, \hat{x}_B = \arg\min_{x'_1,\dots,x'_B} R(x'), \qquad R(x') = \left\| \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \ell(\theta; x'_i, y_i) - g \right\|^2. \tag{8.5}$$

In PyTorch, `create_graph=True` in `torch.autograd.grad` makes the guessed gradient differentiable. An optimizer such as Adam can then follow the gradient of R with respect to the input.

Two conditions affect the search. First, the activation's slope must vary because the search differentiates a gradient. ReLU has constant slope on either side of zero, so Zhu et al. used sigmoid. Second, R is nonconvex and the search may get stuck. Stop when R no longer decreases, then compare it with the observed gradient's squared norm $\|g\|^2$. In double precision, a successful run in the full edition reaches about 10^{-27}, while a stuck run reaches about 10^{-5}. Because other inputs may also give small R, use several random restarts and accept a reconstruction only when the results agree.

8.6. Batches and averaging

Clients usually send a batch average or a parameter change after several on-device training steps (**local steps**). Averaging prevents the direct division and slows search, but preserves structure. Each record adds an outer product to the first-layer gradient, so each gradient row is a combination of the B inputs. The server therefore learns their **span**, the set of their linear combinations, which contains every record.

The division still recovers a record exactly if a ReLU unit activates only for that record; other records then contribute nothing to its gradient row. Such a unit **isolates** the record. Isolation is more common than a count of random units predicts because similar images may activate the same units. In the full edition's network with 1,024 hidden units, a naive estimate predicts exposure for 1.6% of records in a batch of 16; the measured share is 69.1% (Table 8.1). Larger batches reduce this share, while wider first layers increase it. Measure the risk at the deployed width and batch size, and treat any isolated record as exposed.

Table 8.1.: Isolation attack on synthetic images, from the full edition.

Hidden units	Batch size	Share of the batch recovered exactly
64	4	0.838
64	8	0.550
64	16	0.144
1,024	16	0.691

Batches make DLG less reliable: in the full edition's sigmoid-network experiment, the median search on a batch of 4 gets stuck. However, the isolation attack still recovered all four images in such a batch. Local training steps also fail to protect a single record. At each step, the first-layer gradient is an outer product with the same input, so the ratio still recovers that input from the

total parameter change. A **malicious** server that chooses the weights can plant units that activate only on rare inputs. Each unit can then isolate one record, allowing recovery of most records in a batch of any size (Boenisch et al. 2023, Fowl et al. 2022).

8.7. What to do as a deployer

The risk of an FL deployment depends on who can see each client's update (Table 8.2).

Table 8.2.: Gradient leakage by FL setting, and the response.

Setting	What leaks to the server	What to do
One record per update, fully connected first layer with bias	The input exactly, and the label	Never deploy without protection
Batches of any size, or several local steps	Label counts, the span of the batch, isolated records, and the rest by search in small batches	Treat each update as raw data
Server not trusted to choose weights	Most records of any batch	Batch size is no defence; treat each update as raw data
Updates protected by differential privacy (DP)	Bounded per client by the DP guarantee	The only protection with a guarantee

Secure aggregation lets the server see only the sum of updates from multiple clients, preventing the single-client access required by the attacks above. A sum from a very small cohort may still reveal an individual update. Set a minimum cohort size and prevent client dropouts from reducing the sum below it.

Secure aggregation alone does not provide a formal guarantee. The aggregate remains a batch average that may be vulnerable to the attacks above, and a malicious server may still choose the model weights. Add DP at the client level for a guarantee: clip each client's update to a fixed maximum norm and add noise to the sum. This bounds the effect of any one client's data (User-level privacy). Flower and TensorFlow Federated offer both protections. Gradient pruning and similar defences may hinder attacks but provide no guarantee.

Assess whether the server sees individual updates or only aggregates, and identify the minimum cohort size. Run the attacks using the deployed architecture, batch size and number of local steps. Geiping's open-source PyTorch framework `breaching` implements these attacks. If the server can see an individual update, block deployment until secure aggregation or client-level DP is in place. Record the decision in Pre-release privacy risk assessment.

8.8. Cheat sheet

Table 8.3.: The results of this chapter.

Attack	Formula	Needs	Fails when
Exact recovery, one record	$x = (\nabla_{W_1}\ell)_{j,:}/(\nabla_{b_1}\ell)_j$	Fully connected first layer with bias	Convolutional first layer; batches, in part
Label, one record	$y = \arg\min_k(\nabla_{b_2}\ell)_k$	Softmax and cross-entropy	p_y rounds to 1
Deep Leakage from Gradients	Minimize the matching loss R	An activation whose slope varies; the labels	Search stuck (final $R \gg 0$); larger batches
Isolation, batch	Isolated records recovered exactly	Fully connected first layer, ReLU units	Rarely on real data

Rules of thumb

- Secure aggregation needs a minimum cohort, and only client-level DP bounds what any update reveals.

8.9. References

- Zhu, Liu & Han, "Deep Leakage from Gradients" (NeurIPS 2019): the gradient-matching attack.
- Geiping, Bauermeister, Dröge & Moeller, "Inverting Gradients — How easy is it to break privacy in federated learning?" (NeurIPS 2020): cosine matching with an image prior, on trained networks, local steps and batches.

Derivations. Every result here is derived step by step in the full edition's chapter.

9. Linkage and re-identification

9.1. Why this matters

A hospital removes names from discharge records but retains postcode, date of birth, sex and diagnosis. A voter roll lists names with the same quasi-identifiers. Joining the tables can identify many records. This is a **linkage attack**; identifying a person in a supposedly de-identified record is **re-identification**.

Estimate linkage risk by measuring identifying information in bits. Each bit halves the candidate set, so $\log_2 n$ bits can distinguish one person among n. For the 1990 United States, this is 27.9 bits. When known attributes carry slightly more information, many people have unique combinations that can be linked.

9.2. Quasi-identifiers and the linkage attack

Direct identifiers, such as names or email addresses, identify a person on their own. **Sensitive attributes**, such as diagnoses, are the subject of the release. **Quasi-identifiers** do not identify people individually but can do so in combination; they also appear beside names in other datasets.

To assess a release, group its n records into **equivalence classes**, each containing records with identical quasi-identifiers. Let J be the number of classes. Suppose an adversary knows the target's quasi-identifiers and knows that the target is in the release. If the target's class contains k records, the adversary identifies the target with probability $1/k$. Each class contributes one expected correct identification across its k records. Thus, across all n targets, the success probability is

$$\Pr[\text{win}] = \frac{J}{n}, \tag{9.1}$$

No adversary using only quasi-identifiers can do better. This is the **prosecutor model**, a worst-case assumption in which the adversary knows that the target is in the release.

The script practitioner_table_risk.py measures uniqueness on Adult, a dataset of 48,842 people from the 1994 US Current Population Survey in the UCI repository. Because Adult lacks ZIP codes and birth dates, the script starts with age, sex and race, then adds quasi-identifiers one at a time. For each set, it prints the unique-record share, the share in classes smaller than 5, the minimum class size and J/n.

```
n = df.height
for qis in QI_SETS:
    # One row per equivalence class (records that agree on every quasi-identifier),
     ↪  with its size.
    sizes = df.group_by(qis).len()
    unique = sizes.filter(pl.col("len") == 1).height / n  # share of records alone in
 ↪  their class
    small = sizes.filter(pl.col("len") < 5)["len"].sum() / n  # share in classes
 ↪  smaller than 5
    min_k = sizes["len"].min()  # the table is k-anonymous for this k
    print(
        f"{' + '.join(qis)}: unique={unique:.2%}  in classes <5={small:.2%}  "
        f"min k={min_k}  J/n={sizes.height / n:.3f}"
    )
```

Run it with `uv run python code/part02-attacks/practitioner_table_risk.py`, which prints:

```
age + sex + race: unique=0.12%  in classes <5=0.75%  min k=1  J/n=0.012
age + sex + race + native-country: unique=3.07%  in classes <5=7.83%  min k=1
 ↪  J/n=0.060
age + sex + race + native-country + marital-status: unique=5.88%  in classes <5=11.99%
 ↪  min k=1  J/n=0.100
age + sex + race + native-country + marital-status + education: unique=14.64%  in
 ↪  classes <5=26.83%  min k=1  J/n=0.227
age + sex + race + native-country + marital-status + education + hours-per-week:
 ↪  unique=36.55%  in classes <5=57.05%  min k=1  J/n=0.480
```

Three columns leave 0.12% of records unique; six leave 14.64%. Adding hours worked per week, which is not usually treated as an identifier, raises the unique share to 36.55%. Test every column an adversary could know, adding them one at a time. Every set here contains single-record classes, despite the common minimum class-size rule of 5.

9.3. Pseudonyms and hashed identifiers

Many releases replace identifiers with **pseudonyms**, codes that remain constant across one person's records. These codes can fail in two ways. First, an adversary can reverse an unkeyed hash such as MD5 when the possible identifiers can be enumerated: hash each candidate and look for a match. New York City used this approach for taxi-driver licenses in its 2013 trip release. The set contained about 2 million possible licenses; in 2014, Vijay Pandurangan hashed them all and identified the drivers.

The second risk remains even with a key. A pseudonym links one person's records, so identifying one record reveals the others. In the taxi data, a passenger who knows a trip's pickup time and place can identify the driver; the driver's pseudonym then links all their trips. The General Data Protection Regulation (GDPR) therefore still treats pseudonymized data as personal data.

If a join key is necessary, generate it with a keyed hash, such as a hash-based message authentication code (HMAC), and store the key separately from the data.

```
import hashlib
import hmac
import secrets

licence = "482913"  # a six-digit licence number, the identifier being pseudonymized
pseudonym = hashlib.md5(licence.encode()).hexdigest()  # what an unkeyed release prints

# Dictionary attack: hash all 10**6 possible licences once, then look the pseudonym up.
lookup = {hashlib.md5(f"{i:06d}".encode()).hexdigest(): f"{i:06d}" for i in
 ↪  range(10**6)}
print(lookup[pseudonym])  # 482913

# A keyed hash still gives one code per person, but without the key nobody can build
 ↪  the table.
key = secrets.token_bytes(32)  # store it apart from the data; draw a new one for each
 ↪  release
keyed = hmac.new(key, licence.encode(), hashlib.sha256).hexdigest()
print(keyed in lookup)  # False
```

The key prevents dictionary attacks but does not prevent linkage through a person's record history. Reusing the key across releases enables cross-release linkage, so generate a new key for each release.

9.4. How many people are unique?

Before the table exists, estimate uniqueness by summing information across attributes. An attribute with v equally likely values carries $\log_2 v$ bits; independent attributes' bits add. Attributes with b total bits divide the population into 2^b cells. If each person is assigned uniformly at random to a cell, the expected fraction of n people alone in their cell is approximately

$$\frac{\text{unique people}}{n} \approx \exp\left(-2^{\log_2 n - b}\right). \tag{9.2}$$

The fraction depends on the number of bits above $\log_2 n$ and increases rapidly with that excess. Half the people are unique at 0.53 bits above $\log_2 n$.

ZIP code, birth date and sex carry 31.1 bits, slightly more than the 27.9 bits needed to single out one American. The formula predicts that 0.899 of Americans are unique, close to Latanya Sweeney's 87% estimate from the 1990 census. Real attributes are skewed and correlated, which usually lowers uniqueness; the uniform-cell estimate is therefore cautious.

9.5. Sparse data: the Netflix attack

Ratings, purchases and locations can identify people without quasi-identifier columns. In 2006, Netflix released about 100 million movie ratings with random identifiers instead of subscriber names. Narayanan and Shmatikov (2008) matched these records to public reviews. Their score sums agreement with known items, weighting rare items more heavily than popular ones. Eight

ratings with dates known within 14 days identified 99% of records, even when two ratings were incorrect.

The highest score always identifies some record, even when the target is absent. The **eccentricity test** declares a match only when the highest score exceeds the second-highest by a specified margin, measured in standard deviations of all scores. Calibrate this margin using people known to be outside the release. In the full edition's simulated ratings, the original attack's margin also matched 26.3% of outsiders. Report attack success at a specified false positive rate (Hypothesis testing).

9.6. Does releasing a sample protect?

Agencies often release samples, as with Adult, because a record unique in the sample may match someone outside it. Let q be the **sampling rate**, the fraction of the population in the release. Suppose the adversary finds exactly one matching record, and let λ be the expected number of other people in the population who share the target's cell. A look-alike can account for the match only if the target was not sampled, with probability $1 - q$. For a rare cell, the probability κ that the match is correct is approximately

$$\kappa \approx \frac{1}{1 + \lambda(1 - q)}. \tag{9.3}$$

For a cell with $\lambda = 0.1$ expected other people, a match is correct with probability 0.910 even in a 1% sample. Since $1 - q$ cannot exceed 1, the probability of a correct match is at least $1/(1 + \lambda)$. Reducing the sample size therefore offers little protection for rare cells. Sampling mainly helps when cells are already crowded.

The Adult figures above measure **sample uniqueness**, which is exact under the assumption that the target is in the release. **Population uniqueness** is lower but remains close for rare cells. Estimate it with a model; ARX and sdcMicro provide such estimates, and Rocher, Hendrickx and de Montjoye (2019) model attribute correlations.

9.7. A pre-release risk check

List every field an adversary could find beside a name in another source, such as a date, location or rare event. No tool can identify all available auxiliary data. Reduce risk by first coarsening geography and dates, which often carry many bits, then suppressing records that remain unique (Syntactic anonymity and how it breaks). For analyses that require fine-grained or sparse data, release differentially private outputs instead (Release statistics with a library). Pre-release privacy risk assessment applies thresholds set before examining the counts.

9.8. Cheat sheet

Table 9.1.: The results of this chapter.

Question	Result	Use it for
Bits to name one person	$\log_2 n$	27.9 for the US of 1990
Risk of a release you hold	Success J/n; at most $1/k$ if every class has at least k records	Prosecutor model
Share unique, uniform cells	$\exp(-2^{\log_2 n - b})$	50% at $\log_2 n + 0.53$ bits, 99% at $\log_2 n + 6.64$
Releasing a sample	$\kappa \approx 1/(1 + \lambda(1 - q))$	0.910 at $\lambda = 0.1$ in a 1% sample

Rules of thumb

- Add quasi-identifiers one at a time, because one innocent column can more than double the unique share.
- Estimate uniqueness for the population, not just the sample.
- Hashing an identifier is not anonymization. Key the hash, and use a new key for each release.

9.9. References

- Sweeney, "Simple Demographics Often Identify People Uniquely" (2000): the 87% estimate and its variants with coarser geography.
- Narayanan & Shmatikov, "Robust De-anonymization of Large Sparse Datasets" (2008): the Netflix attack.
- Rocher, Hendrickx & de Montjoye (2019), in *Nature Communications*: population uniqueness from a sample.

Derivations. Every result here is derived step by step in the full edition's chapter.

10. Syntactic anonymity and how it breaks

10.1. Why this matters

Linkage and re-identification showed that removing names does not make a table anonymous. Combinations of age, postcode and sex can still identify people. A common response is to generalize the table until each person belongs to a group. These definitions are **syntactic** because they can be checked directly from the released table. Statistics offices and tools such as ARX and sdcMicro still use them. Each definition applies to one table and assumes fixed adversarial knowledge; additional information or another release can defeat it.

10.2. What the adversary believes about a class

To anonymize a table, the custodian—the organization holding the data—applies **generalization**, such as replacing an age with a range or a postcode with a prefix. Records with identical generalized quasi-identifiers form an **equivalence class**. An adversary who knows the target's quasi-identifiers can identify the target's class E but cannot distinguish its records. The adversary's **posterior** for a sensitive value is therefore the fraction of the class with that value. Let $|E|$ be the class size and $n_E(v)$ the number of records in it with sensitive value v. The posterior that the target's sensitive value S_{target} equals v is

$$\Pr[S_{\text{target}} = v \mid \text{release}] = \frac{n_E(v)}{|E|}. \tag{10.1}$$

An adversary can use background knowledge to eliminate values, then recompute the posterior over the remaining records. The attacks in this chapter use this calculation.

10.3. k-anonymity and what it costs

k-anonymity (Sweeney and Samarati, 1998) requires each class to contain at least k records, where the custodian chooses k. Generalization and **suppression**—deleting records that remain distinguishable—can enforce the requirement. In the previous chapter, an adversary who knows the target is in the release links it with probability J/n, where J is the number of classes and n the number of records. If each class contains at least k records, there are at most n/k classes, so

$$\Pr[\text{win}] = \frac{J}{n} \leq \frac{1}{k}. \tag{10.2}$$

This bound concerns linkage and assumes the adversary knows only the quasi-identifiers. The choice of k is a policy decision; 5 is a common minimum.

Anonymizers such as **Mondrian** search for a low-cost generalization. Alternatively, generalize the table manually and recheck it. The script practitioner_table_risk.py uses the Adult census extract and six quasi-identifiers from the previous chapter. It groups age into 10-year bands, native country into US or other, and education into four levels. It then counts records alone in their class and the distinct occupations in each class, treating occupation as the sensitive attribute. The script uses Polars because no maintained Python package computes k-anonymity or distinct ℓ.

```
general = df.with_columns(
    (pl.col("age") // 10 * 10).alias("age"),  # 10-year bands: 30 stands for 30-39
    pl.when(pl.col("native-country") == "United-States")
    .then(pl.lit("US"))
    .otherwise(pl.lit("other"))
    .alias("native-country"),
    # education-num runs from 1 (preschool) to 16 (doctorate); 9 is high-school
     ↪  graduate.
    pl.col("education-num")
    .cut([8, 9, 12], labels=["no diploma", "HS-grad", "some college", "bachelor+"])
    .alias("education"),
)
# Recheck, and count the occupations in each class (distinct l).
classes = general.group_by(QIS).agg(pl.len(),
 ↪  pl.col("occupation").n_unique().alias("l"))
unique = classes.filter(pl.col("len") == 1).height / general.height
one_job = classes.filter(pl.col("l") == 1)["len"].sum() / general.height
print(f"after generalizing: unique={unique:.2%}  in a class with one
 ↪  occupation={one_job:.2%}")
```

Run it with `uv run python code/part02-attacks/practitioner_table_risk.py`. Its first two output lines use the six generalized quasi-identifiers, and the third adds the raw hours worked per week.

```
after generalizing: unique=0.94%  in a class with one occupation=1.20%
age + sex + race + native-country + marital-status + education: unique=0.94%  in
 ↪  classes <5=3.31%  min k=1  J/n=0.032
age + sex + race + native-country + marital-status + education + hours-per-week:
 ↪  unique=10.09%  in classes <5=21.78%  min k=1  J/n=0.175
  records in classes of size >=5 with l=1: 0.03%
```

Generalization reduced the unique share from 14.64% to 0.94%, but the smallest class still contains one record. Suppress the 3.31% of records in classes smaller than 5 or use broader bands. The result remains fragile: an adversary who also knows hours worked finds 10.09% of records unique.

10.4. Homogeneity, background knowledge and l-diversity

k-anonymity obscures record identity but not sensitive values (Machanavajjhala et al., 2006). In a **homogeneity attack**, all three records in Alice's class report heart disease, so her posterior probability of heart disease is 1. In a **background knowledge attack**, Bob's class contains two

cancer records and one flu record. A neighbour who knows Bob did not have flu can eliminate that record and infer cancer.

l-diversity requires each class to contain enough distinct sensitive values. Its distinct version requires at least ℓ values but does not limit the most common value's share; a class can be 3-diverse even if one value dominates. The recursive version also caps that share using a custodian-selected constant c. If an adversary rules out $\ell - 2$ values, the posterior is below $c/(1 + c)$. At $c = 1$, this bound is one half, so the guarantee prevents certainty but not a high posterior.

Suppressing small classes also removes most homogeneous classes. In Adult, 1.20% of generalized records are in a class where every record has the same occupation; only 0.03% are in such a class of at least 5 records. In the full edition's synthetic hospital table, distinct 3-diversity removed certain disclosures at little cost. The recursive version required some classes to grow to 641 patients.

10.5. Measuring what a release teaches

Compare each posterior with the **imputation baseline** from Attribute inference and model inversion, the best population-only prediction of a sensitive value. Only improvement over this baseline reflects information about an individual. On the hospital table, age alone predicts each patient's diagnosis correctly for 53.6% of patients; a 5-anonymous release raises this to 59.3%. Most of the increase reflects the age-diagnosis association that the study reports. The additional 5.7 percentage points are leakage.

10.6. Skewness, similarity and t-closeness

l-diversity does not prevent two attacks (Li, Li and Venkatasubramanian, 2007). In a **skewness attack**, a class that is half HIV-positive is 2-diverse, but if 1% of the full table is HIV-positive, membership in the class raises the posterior fifty-fold. In a **similarity attack**, a class containing three distinct stomach diseases is 3-diverse but still reveals that the person has a stomach disease.

t-closeness addresses both attacks by requiring the sensitive-value distribution in every class E to be within distance t of the full table's distribution. Let $P_E(A)$ and $P(A)$ be the probabilities that a value drawn from the class or full table lies in set A. Under **total variation (TV)** distance, no such probability—and therefore no posterior belief—changes by more than t:

$$|P_E(A) - P(A)| \leq t. \tag{10.3}$$

The bound controls an absolute difference, not a ratio, so it gives weak protection for rare values. At $t = 0.15$, a value present in 1% of the table may reach 16% in a class. To address similarity attacks, Li and colleagues used earth mover's distance, which accounts for distances between values.

t-closeness reduces utility. At $t = 0.15$, a guess of diagnoses from the hospital release was correct for 22.2% of patients, below the 53.6% accuracy from age alone. The release therefore provided less information than age alone.

10.7. The composition attack

Ganta, Kasiviswanathan and Smith (2008) asked what happens when two hospitals each publish a k-anonymous or l-diverse table. A patient treated at both sits in a class E_A of one release and a class E_B of the other. The adversary keeps only the diagnoses that appear in both classes,

$$\{v : n_{E_A}(v) > 0\} \cap \{v : n_{E_B}(v) > 0\}. \tag{10.4}$$

When one candidate is left, the diagnosis is **pinned down**.

In the full edition's simulation, two hospitals share 6,649 patients. Together, their two 5-anonymous releases pin down the diagnosis of 9.9% of them, over three times what either pins alone. With distinct 3-diversity, neither release pins anyone, yet together they pin 18.

A single-table definition cannot prevent this attack because neither custodian checks the intersection of two releases. A robust definition must compose across releases, hold under arbitrary adversarial knowledge and bound belief ratios to protect rare values. The definition of differential privacy has these properties; statistical agencies such as the US Census Bureau have adopted it (Reconstruction attacks).

10.8. Choosing and running an anonymization

Start with the quasi-identifiers described in Linkage and re-identification; an incomplete list is a common failure. Use k-anonymity to limit linkage. Add distinct l-diversity when a class containing one sensitive value is a disclosure. Use t-closeness when the sensitive attribute is skewed or contains similar values, because it can reduce utility substantially.

Audit the anonymizer's output in Polars as the excerpt does, or use pycanon, which reports k, ℓ and t for a pandas table. Pre-release privacy risk assessment uses these results to inform the release decision. Use differential privacy when other releases of the same people exist, the sensitive attribute is rare or the data are sparse. For counts and sums, Release statistics with a library provides an end-to-end workflow.

10.9. Cheat sheet

Table 10.1.: The syntactic definitions and the attacks on them.

Definition or attack	Condition	What it gives, and where it ends
k-anonymity	Every class has at least k records	Linkage at most $1/k$; says nothing about attributes
Distinct, entropy l-diversity	At least ℓ values, or $\log_2 \ell$ bits of entropy	No cap on the top value's share
Recursive (c, ℓ)-diversity	Top value below c times the values ranked ℓ-th and lower	Posterior below $c/(1+c)$ after ruling out $\ell - 2$ values

Definition or attack	Condition	What it gives, and where it ends
t-closeness (TV)	Each class within t of the table	A 1% value may reach $1\% + t$
Composition	Intersect the target's classes in two releases	No single-table definition prevents it

Rules of thumb

- k-anonymity limits linkage but does not protect sensitive attributes; inspect the values in each class.
- Check for other releases of the same records; a guarantee for one table does not protect a person across releases.

10.10. References

- Machanavajjhala and colleagues, "l-Diversity" (2006), and Li, Li & Venkatasubramanian, "t-Closeness" (2007): the attacks and the definitions that answer them.
- Ganta, Kasiviswanathan & Smith, "Composition Attacks and Auxiliary Information in Data Privacy" (2008): the intersection attack.

Derivations. Every result here is derived step by step in the full edition's chapter.

11. Reconstruction attacks

11.1. What it is

A **reconstruction attack** recovers records from published aggregates because each count is a linear equation in the values of the people counted. Suppose each of n people has a secret bit, represented in vector d. A **subset-sum query** counts the people in a set whose bit is 1. The attacker's m queries form a binary matrix A, one query per row. The **curator** publishes the true answers plus an error vector e:

$$a = Ad + e. \tag{11.1}$$

Least squares finds real-valued records whose query answers are closest to the published answers, then rounds each value to 0 or 1. Suppose each query includes each person with probability one half and the noise is normal with standard deviation σ. Least squares then recovers approximately the following fraction

$$\Phi\left(\frac{\sqrt{m-n}}{4\sigma}\right) \tag{11.2}$$

of the bits, where Φ is the standard normal distribution function. The baseline is the **majority guess**, which assigns the more common value to every person and is correct on half of balanced bits. With several times more queries than people, noise well below $\sqrt{n}$ can reveal the data. If the curator answers all 2^n subset sums, only error of order n prevents reconstruction (Dinur and Nissim 2003).

These results form the **Fundamental Law of Information Recovery**: too many accurate answers can destroy privacy. Preventing reconstruction alone does not establish privacy. A curator could answer queries using a secret random half of the population, preventing reconstruction while revealing that half exactly. Membership inference also requires far fewer statistics (Lower bounds).

11.2. When you meet it

Every published count, cross-tabulation, dashboard cell and query answer contributes an equation. The Census Bureau attacked its 2010 publications and reconstructed a record for every person. About 46% of records matched a real person exactly on block, sex, age, race and ethnicity. Commercial data confirmed the identities of about 17% of the population, motivating the 2020 Census to adopt differential privacy. Query systems are also vulnerable: because Diffix noise did not increase with query count, Cohen and Nissim (2020) averaged it away.

11.3. How to use it

To audit an aggregate release, count its equations. If the cells or allowed queries outnumber the people, protection depends on the noise. Substitute the noise level and query count into Equation 11.2. If predicted recovery exceeds the majority baseline substantially, run the attack. The snippet uses OpenDP's Gaussian mechanism and least squares. Because OpenDP noise cannot be seeded, the snippet checks whether the recovered share is within tolerance of the prediction rather than printing that share.

```
from statistics import NormalDist

import numpy as np
import opendp.prelude as dp

dp.enable_features("contrib")
rng = np.random.default_rng(0)
n, m = 500, 2000  # people with a secret bit each, and random subset-sum queries
sigma = 0.25 * np.sqrt(n)  # noise sd added to each answer
secret = rng.integers(0, 2, n)
queries = rng.integers(0, 2, (m, n))
space = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.l2_distance(T=float)
gaussian = dp.m.make_gaussian(*space, scale=sigma)  # OpenDP's Gaussian mechanism on
 ↪  all m answers
answers = np.array(gaussian((queries @ secret).astype(float).tolist()))
estimate = np.linalg.lstsq(queries, answers, rcond=None)[0]  # the textbook attack:
 ↪  least squares
recovered = np.mean(np.clip(estimate, 0, 1).round() == secret)
predicted = NormalDist().cdf(np.sqrt(m - n) / (4 * sigma))
print(f"predicted {predicted:.4f}")  # predicted 0.9584
print(f"recovered within 0.04 of it: {abs(recovered - predicted) < 0.04}")  # recovered
 ↪  within 0.04 of it: True
```

Query count matters as much as noise. In the full edition's simulation, noise of twice $\sqrt{n}$ and four queries per person recovered 0.5875 of the bits. At the same noise level, Equation 11.2 predicts that 106.1 queries per person recover 90%. Fixed noise therefore becomes insufficient as the query count grows; differential privacy scales noise with the number of answers.

For nonrandom queries, use Dinur and Nissim's linear-programming decoder, which constrains unknowns to $[0, 1]$ with `scipy.optimize.linprog`. For published tables, express each cell as a constraint on integer unknowns and solve with `scipy.optimize.milp`. Count the feasible solutions: one solution is enough to reconstruct records. The release fails if either decoder outperforms the majority baseline.

11.4. What goes wrong

Rounding, household swapping and small-cell suppression do not provide noise. Rounding hides nothing because an attacker can round candidate answers in the same way. Swapping leaves most constraints exact. A suppressed cell adds a constraint that its count is below the threshold. Sticky

noise—returning the same answer to a repeated query—prevents only duplicate queries, which reconstruction does not require.

Test the union of all releases, not each table separately. Least squares alone is insufficient because adversarially chosen errors affect it more than random noise. Before publication, apply both decoders to the release and record the results in Pre-release privacy risk assessment.

11.5. References

- Cohen & Nissim, "Linear Program Reconstruction in Practice" (Journal of Privacy and Confidentiality, 2020): the attack on Diffix.
- Garfinkel, Abowd & Martindale, "Understanding Database Reconstruction Attacks on Public Data" (2019): a census block rebuilt with a solver.

Derivations. Every result here is derived step by step in the full edition's chapter.

12. Pre-release privacy risk assessment

12.1. Why this matters

Before release, assess whether a model or table exposes information about individuals. This chapter evaluates three artefacts: a classifier behind a scoring API, a language model whose weights are shared with a partner, and a census extract. It compares attack results from earlier chapters with fixed thresholds and records the decision.

Specify blocking thresholds before running attacks to prevent choosing thresholds that fit the results. A failed attack establishes only that the tested attack failed, so report its strength as well.

The three scripts below produce the results on this page. The first two run in seconds on public data. The third is from Fine-tune a language model privately with LoRA and takes a few minutes on an idle CPU.

```
uv run python code/part02-attacks/practitioner_lira.py
uv run python code/part02-attacks/practitioner_table_risk.py
uv run python code/part05-dp-ml/practitioner_lora.py
```

12.2. Fix the threat model and the thresholds first

Complete the worksheet in Threat models and privacy games for each artefact because releases can expose the same data differently. By default, use the person as the privacy unit and assume the adversary sees model scores, as many services return them.

Record the blocking rules in Table 12.1. The membership rule uses two rates: the true positive rate (TPR), the fraction of members flagged, and the false positive rate (FPR), the fraction of non-members incorrectly flagged. At FPR 10^{-3}, block release if the lower end of the TPR's 95% confidence interval exceeds ten times the FPR. Using the lower bound reduces sensitivity to a lucky run. The factor of ten is a policy choice; it allows some advantage over chance.

Several remedies use differential privacy (DP), which bounds information any attack can learn about one person at privacy level ε (Part 3). One method is stochastic gradient descent (SGD) with clipped per-record gradients and added noise, called DP-SGD.

Table 12.1.: The blocking thresholds, written before any attack runs.

Artefact	The adversary sees	Attack	Blocks the release when	Then
Classifier behind a scoring API	Confidences for any input	Online LiRA, 64 models	Lower end of the TPR's 95% interval at FPR 10^{-3} is above 0.01	Regularize, or train with DP-SGD
Fine-tuned language model shared with a partner	The weights	Canaries, exposure, greedy extraction	Any secret extracted, or a one-copy canary's extrapolated exposure within 3 bits of the maximum	Deduplicate, or fine-tune with DP
Record-level table	The table and a register with the quasi-identifiers	Uniqueness on the agreed quasi-identifiers	Any record unique, or in a class smaller than 5 (11 for counts from US Medicare and Medicaid data)	Generalize and suppress, else release DP statistics
Model, sensitive field	The target's other fields and label	Model inversion	Attribute advantage's 95% interval above 0	Regularize, or train with DP
Model whose training mix is secret	Probe outputs	Shadow models, meta-classifier	Accuracy's 95% interval above one half	Do not release that mix
Published counts or queries	Every answer	Least squares, linear programming	Either beats the majority guess at the planned noise and query count	DP noise scaled to all answers
Federated training	Each client's update	Division and search	Any single update visible	Secure aggregation, client-level DP
Any model claimed to be DP	Depends on the claim	Audit with canaries	Certified lower bound on ε above the claimed ε	Block: the code or the accounting has a bug

A canary within 3 bits of maximum exposure ranks among the 8 most likely possible secrets. The rule uses one-copy canaries because a deduplicated corpus contains each secret once. For a corpus that is not deduplicated, apply it at the largest observed repeat count.

The four middle rows block release when an attack exceeds its baseline. Attribute inference, property inference, reconstruction attacks and gradient inversion specify these baselines. For the final row, independent audit trials provide a certified lower bound on a DP model's ε. A bound above the claim indicates an implementation or accounting error (Auditing private training).

12.3. Classifier: the free check

First compare model accuracy on training and held-out data. The classifier is logistic regression trained on scikit-learn's 1,797 handwritten-digit images, as in Membership inference. Across 64 models, the mean gap is 0.0422. The rule "member if correctly classified" therefore has advantage 0.0422, a lower bound on leakage.

This check cannot clear a model because the average gap does not identify exposed records. For example, the loss attack flags records with low loss. At FPR 10^{-3}, its TPR is 0.0009, no better than random guessing. Failure of a weak attack provides little evidence. A large gap motivates regularization; a small gap requires further testing.

12.4. Classifier: LiRA with shadow models

The release decision uses the likelihood ratio attack (LiRA) from Membership inference. It compares each target output with those of **shadow models**, trained with and without that record. Fix attack strength before testing; a weaker attack may pass a model that a stronger attack blocks. Follow the defaults in Running LiRA on your model: use online LiRA with 64 shadows when feasible. With at most 16 shadows, fit one variance for all records. Include each record in each model by a fair coin, and evaluate each model in turn as the target.

With these defaults, the digits experiment has 57,704 member pairs and 57,304 non-member pairs of a model and an image. Reading the result reports the attack's hits. Online LiRA has TPR 0.0318 at FPR 0.00099. Its 95% interval has lower bound 0.0303, above the 0.01 threshold, so the classifier fails Table 12.1. Offline LiRA and the Adversarial Robustness Toolbox shadow-model attack would pass it. Base the decision on the strongest affordable attack.

Report the FPR interval as well. At 10^{-3}, the non-member pairs yield about 57 false positives, giving an interval from 0.00075 to 0.00129. The script sets and evaluates thresholds on the same pairs, which slightly inflates attack performance. Inspect the ten highest-scoring records; they are often rare or mislabeled.

12.5. Fine-tuned language model: canaries and extraction

A language model may reproduce a secret, which membership tests do not measure. This assessment inserts **canaries**, random secrets with a fixed format, and measures how strongly the trained model memorizes them. Fine-tune a language model privately with LoRA runs this audit using the scoring method in Memorization and training-data extraction. It adapts Pythia-14m to 1,024 Tiny Shakespeare sequences, once without privacy and once with DP at $\varepsilon = 8$ per sequence. Canaries with 1 to 64 copies and uninserted controls receive two scores each. Exposure, in bits,

ranks the secret by the sum of its tokens' log-probabilities, not their mean. Extraction checks whether greedy decoding with the partner's settings outputs the secret.

Apply the exposure rule to the extrapolated values, not the measured column. Ranking against 2,000 random candidates caps measured exposure at 10.97 bits, while the full secret space allows 16.61 bits. The measured value therefore cannot approach within 3 bits of the maximum. First check the controls: a random rank has mean exposure $\log_2 e = 1.44$ bits, and the controls' 1.4 bits are consistent with that value.

The non-private model fails Table 12.1. Its one-copy canaries have exposure of 2.0 bits, below the threshold, but greedy decoding outputs all three 64-copy secrets. Do not share this model. The DP run extracts no canaries, but its most repeated canaries still average about 4 bits, compared with 2.2 bits for its controls. The guarantee applies to one sequence, while a repeated secret spans multiple sequences. Deduplicate the training data before training.

12.6. Table: re-identification on quasi-identifiers

Quasi-identifiers are the columns an adversary can find elsewhere, such as age, sex and country in a register. Records that agree on all of them form an **equivalence class**, and a record alone in its class can be linked to a name. The table is Adult, a census extract of 48,842 records, and the agreed quasi-identifiers are six columns, from age to education.

Linkage and re-identification measures uniqueness. The agreed six columns make 14.64% of records unique, so the table fails. Adding hours per week raises this to 36.55%. List every column an adversary might know, and test uncertain columns as well.

Syntactic anonymity and how it breaks generalizes age, country and education, then repeats the check. Generalization reduces the unique share to 0.94%, but the smallest class still has one record, so the table fails. Suppressing classes smaller than 5 removes 3.31% of records. This result applies only to the tested columns: adding raw hours per week makes 10.09% of records unique. In addition, 1.20% of records are in a class with one occupation, a distinct ℓ of 1 (Syntactic anonymity). Such a class reveals occupation even without identifying a record.

These shares measure uniqueness within the sample, matching the prosecutor model in which the adversary knows the target is in the table. Otherwise, estimate population uniqueness (Linkage and re-identification). A table that passes after suppression may still be vulnerable to intersection with another release of the same people (Syntactic anonymity). If the table cannot remain useful after anonymization, release differentially private statistics as in Release statistics with a library. Accurate aggregates are not a safe fallback; they can enable reconstruction (Reconstruction attacks).

12.7. Write the decision

Compare each result with its threshold and record it in the release memo (Table 12.2). Include an interval for each estimated rate so that a reviewer can verify the decision.

Table 12.2.: A release memo for the three artefacts.

Memo entry	This assessment
Threat model	Scores from an API; weights shared with a partner; a table linked with a register. Unit: the person
Attacks and their strength	Loss attack, offline and online LiRA with 64 models; 12 canaries and 3 controls with exposure and greedy extraction; uniqueness on six quasi-identifiers plus one
Results	Online LiRA TPR 0.0318 [0.0303, 0.0332] at FPR 0.00099; all three 64-copy secrets extracted; 14.64% unique, 0.94% after generalizing
Verdict	All three blocked
Mitigation	Classifier: regularize or train with DP-SGD, then rerun. Language model: deduplicate and fine-tune with DP. Table: release DP statistics
Residual risk	Not assessed: attribute inference, property inference and reconstruction from aggregates. Not applicable, with no federated training: gradient leakage

The residual-risk entry lists attacks that were not run. Include it even when all tested attacks fail: an untested attack may still reveal information. A DP guarantee bounds all attacks. For a DP model, include the guarantee and supporting evidence in the memo; Release and governance describes the release package.

12.8. Checklist

Table 12.3.: The steps of a pre-release risk assessment, their defaults and their checks.

Step	Default	Check
Threat model and thresholds	Person as unit; adversary sees scores; rules of Table 12.1	Written and dated before any attack runs
Free check	Train and test accuracy; loss attack	Gap and loss TPR at FPR 10^{-3} reported; a pass clears nothing
LiRA	Online, 64 models, fair-coin inclusion, target role rotated	TPR's interval at FPR 10^{-3} against 0.01; enough non-members; top records inspected

Step	Default	Check
Canaries	Secret format, 1 to many copies, uninserted controls	Controls near 1.44 bits; summed log-probability; deployed decoding
Table	Every column an adversary may hold; classes of at least 5	Unique share and smallest class after generalizing; distinct ℓ on the sensitive column
Decision	Memo with intervals, verdict, mitigation and residual risk	Every attack not run is named

12.9. References

- Carlini et al., "Membership Inference Attacks From First Principles" (IEEE S&P 2022): LiRA, and why attacks are reported by their TPR at a low FPR.
- Carlini et al., "The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks" (USENIX Security 2019): canaries, exposure and the skew-normal extrapolation.
- Rocher, Hendrickx & de Montjoye, "Estimating the success of re-identifications in incomplete datasets using generative models" (Nature Communications 2019): population uniqueness from a sample.

Derivations. This workflow has no counterpart in the full edition. It draws on the full edition's chapters on threat models in the full edition, membership inference in the full edition, memorization and extraction in the full edition, linkage and re-identification in the full edition and syntactic anonymity in the full edition.

Part III.

Differential privacy foundations

Part 2 showed that anonymization and access control do not prevent leakage. This part introduces **differential privacy (DP)**, a property of a release process rather than a dataset. If one person's data changes, the probability of each output changes by at most a factor e^{ε}, with additive slack δ. A DP claim must state ε, δ and the unit of privacy, such as a record or a user. This guarantee limits the attacks covered in Part 2.

Table 12.4.: Notation used throughout Part 3.

Symbol	Meaning
ε, δ	Privacy level and slack
$\Delta_1 f$, $\Delta_2 f$	Sensitivity, the most one record moves the answer
$\mathrm{Lap}(b)$, $\mathcal{N}(0, \sigma^2)$	Laplace noise of scale b, normal noise of standard deviation σ

The workflow has three steps. First, specify the unit of privacy and neighbouring relation (the definition of differential privacy). Next, bound each person's contribution and calibrate a mechanism to that bound (The Laplace and Gaussian mechanisms). Finally, account for every release; only post-processing is free (Post-processing, group privacy and composition). Release statistics with a library applies these steps to a shop's invoices using OpenDP.

The full chapters cover common tasks. Selection chooses the best of several options or a private median; Query release publishes multiple counts. Trust models compares settings where no central party can hold raw data, and Private mean estimation calculates the sample size required for a private mean. Four reference pages cover advanced composition and sensitivity, discrete noise mechanisms and accuracy lower bounds.

Part 4 then replaces hand composition with accountants, and Part 5 uses all of it to train models. At the end of the part, you can release a count, sum, median or table with error bars and a privacy guarantee.

13. The definition of differential privacy

13.1. Why this matters

Attacks in Threat models and attacks compare outputs from datasets that differ in one person. **Differential privacy (DP)** bounds how much the output distribution changes between these datasets, including for tests not yet developed. DP is a property of the algorithm, not the data, so it can be checked before data access.

13.2. Neighbouring datasets

The **neighbouring relation** defines what it means for datasets to differ in one record. Under **add/remove**, neighbours differ by one record, so the guarantee hides whether a person is included. Under **replace**, neighbours have the same size but one record has a different value, so the guarantee hides the record's value. A replacement can be represented as a removal followed by an addition. Each step costs privacy level ε from Section 13.3, so

$$\varepsilon\text{-DP under add/remove} \implies 2\varepsilon\text{-DP under replace.} \tag{13.1}$$

The reverse fails, because replace never compares datasets of different sizes.

Specify the **unit of privacy**, the entity represented by one record. At the **example level**, the unit is one training record or event. At the **user level**, it is all data from one person. A user with 200 messages is a group of 200 records under example-level DP, with a guarantee that scales to 200ε (Post-processing, group privacy and composition). Confirm that the stated unit matches the intended protection. The guarantee must hold for every record allowed by the data format, including an age of 900 that could strongly affect an average. Private algorithms therefore clip each input to a fixed range before adding noise.

13.3. The definition

A **mechanism** M is a randomized algorithm that takes a dataset and returns an output. Write D for a dataset and D' for one of its neighbours. The definition asks that no set of outputs becomes more than a factor e^{ε} more likely when one record changes, up to a small additive slack δ. Here ε is not negative. A mechanism is (ε, δ)**-differentially private** if every pair of neighbours and every set S of outputs satisfy

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{13.2}$$

The probabilities come only from the randomness of the mechanism, and the data stay fixed.

When $\delta = 0$, the guarantee is **pure DP**; otherwise, it is **approximate DP**. A deterministic mechanism that returns different answers on neighbouring datasets assigns probability 1 to an answer on one dataset and 0 on the other. No finite factor e^{ε} can bound this change. Rounding, aggregation and suppression of small cells therefore provide no guarantee by themselves.

13.4. The privacy loss and δ

For output y, let $p_{M(D)}(y)$ be its probability or density when the mechanism runs on D. The **privacy loss** of y is the log-likelihood ratio between D and D':

$$L(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}. \tag{13.3}$$

Independent outputs multiply their likelihood ratios, so their privacy losses add. Part 4's accountants use this property. Pure ε-DP holds exactly when every output has privacy loss between $-\varepsilon$ and ε. Laplace noise meets this condition; Gaussian noise has unbounded privacy loss in the tail (The Laplace and Gaussian mechanisms).

Under approximate DP, δ bounds the probability mass where privacy loss exceeds ε. It counts only the probability on D above e^{ε} times the corresponding probability on D'. Thus, δ is not the probability that privacy loss exceeds ε; the exact quantity is the **hockey-stick divergence** (Divergences). Each ε has a smallest corresponding δ, and the function $\delta(\varepsilon)$ is the mechanism's **privacy curve**. Report the noise parameters with the guarantee so that others can reproduce the curve (Numerical accounting).

13.5. Randomized response

In **randomized response**, each survey respondent tells the truth with probability t and otherwise lies. A respondent who answers truthfully on the first coin toss and uses a second toss on tails answers truthfully with probability $\frac{3}{4}$. The true answer is then three times as likely as the false answer, giving pure DP at $\varepsilon = \ln 3$. In general, randomized response at level ε tells the truth with probability

$$t = \frac{e^{\varepsilon}}{1 + e^{\varepsilon}}. \tag{13.4}$$

With independent randomization, the guarantee is ε per person for any number of respondents. It holds only under replace: under add/remove, the number of reports reveals who participated. Let $\bar{y}$ be the fraction of "yes" reports among n people. Correcting for false answers gives an estimate of the true fraction π and its standard deviation:

$$\hat{\pi} = \frac{\bar{y} - (1 - t)}{2t - 1}, \qquad \mathrm{sd}(\hat{\pi}) = \frac{e^{\varepsilon/2}}{(e^{\varepsilon} - 1)\sqrt{n}}. \tag{13.5}$$

For 1,000 people, the two-coin scheme has a standard deviation of 0.0274. When the true fraction is 0.3, that is about twice the sampling error of 0.0145 that the survey already has. This increase is the cost of the **local** model, in which no party sees a true answer (Local, shuffle and distributed trust models).

OpenDP builds the mechanism from its truth probability, 0.75 for the two coins, and `map` turns one changed answer into its ε, $\ln 3$. The `contrib` feature turns on mechanisms that are not yet formally vetted.

```
import opendp.prelude as dp

dp.enable_features("contrib")
rr = dp.m.make_randomized_response_bool(prob=0.75)
print(f"{rr.map(1):.4f}")  # 1.0986
```

13.6. What ε means for an attacker

A membership attacker observes the output and predicts whether the target was included in the data (Hypothesis testing). Its **false positive rate (FPR)** is the fraction of non-members it flags; its **true positive rate (TPR)** is the fraction of members it flags. The flagged outputs form a set bounded by the definition, so under add/remove every attack satisfies

$$\mathrm{TPR} \le e^{\varepsilon}\,\mathrm{FPR} + \delta. \tag{13.6}$$

This form matches Part 2's evaluation of attacks by TPR at low FPR. For $\varepsilon = 1$ and $\delta = 10^{-5}$, the default in Section 13.7 for 10,000 people, an attack at FPR 10^{-3} has TPR at most $e \cdot 10^{-3} + 10^{-5} = 0.002728$. As FPR approaches 0, the bound depends almost entirely on δ.

Rearranging the cap gives a lower bound on ε from a measured attack:

$$\varepsilon \ge \ln \frac{\mathrm{TPR} - \delta}{\mathrm{FPR}}. \tag{13.7}$$

Suppose a model is claimed to be $(2, 10^{-5})$-DP, and an attack has TPR 0.05 at FPR 0.001. If these rates were exact, then ε would be at least $\ln 50 = 3.91$, contradicting the claim. A real audit uses confidence bounds on the rates and a **canary**, a record designed to be easy to detect (Auditing).

Two summaries aid interpretation. The **membership advantage** is TPR minus FPR, the attack's improvement over random guessing. Under (ε, δ)-DP, it is at most

$$\mathrm{TPR} - \mathrm{FPR} \le \frac{e^{\varepsilon} - 1 + 2\delta}{e^{\varepsilon} + 1}. \tag{13.8}$$

The second summary bounds posterior belief. Under pure DP, one output can multiply the adversary's odds between two neighbouring datasets by at most e^{ε}. A prior probability p that the data are D can therefore rise at most to

$$\Pr[D \mid y] \le \frac{e^{\varepsilon} p}{e^{\varepsilon} p + 1 - p}. \tag{13.9}$$

Table 13.1 turns a claimed ε into both numbers.

Table 13.1.: What a pure ε-DP guarantee promises against the strongest attacker.

ε	Advantage bound	Posterior from a prior of 0.01	Posterior from a prior of 0.5
0.1	0.0500	0.0110	0.5250
1	0.4621	0.0267	0.7311
3	0.9051	0.1687	0.9526
8	0.9993	0.9679	0.9997

An ε at most 1 gives a strong bound for any mechanism: a 1% prior rises to less than 3%. For ε between 1 and 10, justify the claim with the mechanism's privacy curve. Above 10, the bounds provide little protection, so include an audit (Choosing ε). For statistics, specify acceptable error and solve for ε (The Laplace and Gaussian mechanisms). The table assumes pure DP; for an (ε, δ) guarantee, report both the advantage bound and TPR cap, which include δ.

13.7. How small δ must be

A small δ has little effect on the advantage bound, but an average can hide severe harm to a few people. Consider a mechanism that independently publishes each record in full with probability δ. It is $(0, \delta)$-DP under add/remove because a record changes the output only when its own random draw publishes it. Across n records, the probability of publishing at least one is

$$1 - (1 - \delta)^n \approx 1 - e^{-n\delta}. \tag{13.10}$$

With 10,000 records and $\delta = 1/n$, that is $1 - e^{-1} = 0.6321$.

Treat δ as a per-person budget for catastrophic failures such as full record publication. Following Ponomareva et al. (2023), this book's default keeps $n\delta$, the expected number of people exposed this way, well below 1. Let n count people, not records, and choose

$$\delta \leq \frac{1}{n^{1.1}}, \tag{13.11}$$

rounded down to a power of ten. For 10,000 people, the rule gives 10^{-5}. The publishing mechanism then has $n\delta = 0.1$, and it exposes someone with probability only 0.0952.

13.8. What differential privacy does not protect

DP compares datasets with and without a record, so it does not prevent conclusions that hold in both datasets. A study that never used Alice's record may still show that smokers have a high cancer rate. This can affect beliefs about Alice, but the study is 0-DP with respect to her. A harm violates DP only if the inference would materially change without her record. Correlated records create a second limitation: a household of five whose members' records always change together is a group of five unless the household is the privacy unit. Implementation errors create a third limitation (Discrete and floating-point-safe mechanisms).

13.9. Writing the guarantee down

A reviewer can check only a fully specified claim. State the privacy unit and neighbouring relation, and report ε and δ for the whole release, not per query. Also name the mechanism, library version, trust model and data treated as public; DP does not protect public data. This example uses the **central model**, in which a trusted curator holds the raw data and adds noise:

> This release satisfies $(1, 10^{-5})$-DP for adding or removing one customer and all their invoice lines, at most 100 per customer, under the central model, with Gaussian noise from OpenDP 0.16.0. The market list was fixed in advance.

Release statistics with a library and Release and governance print such statements from the code that made the release.

13.10. Code

This snippet checks two numbers quoted above.

```
from math import exp, log

print(f"{(exp(3) - 1) / (exp(3) + 1):.4f}")  # 0.9051
print(f"{log((0.05 - 1e-5) / 1e-3):.2f}")  # 3.91
```

13.11. Cheat sheet

Table 13.2.: The results of this chapter.

Result	Formula	Notes
(ε, δ)-DP	$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta$	All neighbours, all sets S
Attack cap	$\mathrm{TPR} \leq e^{\varepsilon}\,\mathrm{FPR} + \delta$	Audit: $\varepsilon \geq \ln((\mathrm{TPR} - \delta)/\mathrm{FPR})$
Choosing δ	$\delta \leq 1/n^{1.1}$, rounded down to a power of ten	n counts people, not records

Rules of thumb

- When people contribute many rows, the unit of privacy is the person.
- An ε is never a probability of re-identification. Under pure DP it bounds how far any output can shift an attacker's odds.

13.12. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), ch. 2: the definition and what DP promises.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015): the hypothesis-testing reading behind the attack cap.

Derivations. Every result here is derived step by step in the full edition's chapter.

14. Post-processing, group privacy and composition

14.1. Why this matters

A private mechanism is one component of a system. Its output may be rounded or used to fit a model; one person may contribute multiple records; and an analyst may ask multiple questions. Three properties of differential privacy (DP) address these cases: post-processing, group privacy and composition. They allow separate analysis of each mechanism and tracking of the total privacy budget.

14.2. Post-processing

Let M be a mechanism that is DP with privacy level ε and slack δ. Apply a function f to its output. The function may use additional randomness but must not access the data. The processed release remains equally private:

$$f(M(D)) \text{ is } (\varepsilon, \delta)\text{-DP.} \tag{14.1}$$

Rounding a noisy count, clamping it at zero or reconciling noisy subtotals with their total therefore costs no additional privacy. A system can add noise to counts or gradients, then treat subsequent computations as post-processing. The rule also applies to attacks: membership inference is a function of the output, so the definition's limit on attack success applies to future attacks.

The condition is that f never accesses the data. For example, selecting the noisy top category and then reporting its exact count violates post-processing because the second step is a new, non-private mechanism.

14.3. Group privacy

The definition protects one record, but an online-shop customer may have dozens of invoice lines. For a person with k records, the guarantee must cover datasets that differ in k records. Transforming one dataset into the other requires k single-record changes, each costing ε. Under pure DP, where $\delta = 0$, per-record ε-DP therefore becomes group-level $k\varepsilon$-DP. Under approximate DP, the slack can grow faster than k. For a group of k records, an (ε, δ)-DP mechanism is

$$\Big(k\varepsilon,\ \delta_k\Big)\text{-DP}, \qquad \delta_k = \delta\,\frac{e^{k\varepsilon} - 1}{e^{\varepsilon} - 1} \le k\, e^{(k-1)\varepsilon}\delta. \tag{14.2}$$

The right-hand form is simpler but looser. At $\varepsilon = 0.5$ and $\delta = 10^{-5}$ per record, a person with 10 records has slack 2.272×10^{-3}. This exceeds the book's default, which sets δ far below one divided by the number of people (How small δ must be).

OpenDP calculates group guarantees from the mechanism itself. A measurement's `map(d_in=k)` gives its guarantee when neighbouring datasets differ in k records, here for a Laplace count and a Gaussian count.

```
count = dp.t.make_count(dp.vector_domain(dp.atom_domain(T=int)),
 ↪  dp.symmetric_distance())
laplace = count >> dp.m.then_laplace(scale=2.0)  # epsilon = 0.5 per record
print(f"Laplace count: epsilon {laplace.map(d_in=1):.1f} per record,
 ↪  {laplace.map(d_in=10):.1f} for 10")

def gaussian(scale: float) -> dp.Measurement:  # a Gaussian count, as a privacy curve
    return dp.c.make_zCDP_to_approxDP(count >> dp.m.then_gaussian(scale=scale))

def at_1e5(scale: float) -> dp.Measurement:
    return dp.c.make_fix_delta(gaussian(scale), delta=1e-5)

scale = dp.binary_search_param(at_1e5, d_in=1, d_out=(0.5, 1e-5))  # (0.5, 1e-5) per
 ↪  record
curve = gaussian(scale).map(d_in=10)  # the privacy curve for a group of 10 records
print(f"Gaussian count, group of 10: epsilon {curve.epsilon(1e-5):.2f} at delta 1e-5")
# delta_k is the slack that the group-privacy formula gives the group
print(f"Gaussian count, group of 10: epsilon {curve.epsilon(delta_k):.2f} at delta
 ↪  {delta_k:.3e}")
```

```
Laplace count: epsilon 0.5 per record, 5.0 for 10
Gaussian count, group of 10: epsilon 6.43 at delta 1e-5
Gaussian count, group of 10: epsilon 4.57 at delta 2.272e-03
```

For the Laplace count, `map(d_in=10)` gives $k\varepsilon = 5.0$, matching the formula. The Gaussian count, calibrated to $(0.5, 10^{-5})$ per record, gives a tighter group bound because OpenDP uses the mechanism's full privacy curve. At the formula's slack of 2.272×10^{-3}, the group has $\varepsilon = 4.57$, below the formula's 5.0. Use the library's group mapping when available; otherwise use the formula.

Prefer the person as the privacy unit, so neighbouring datasets differ in all of one person's records. Bound each person's contribution before adding noise, as in Release statistics with a library. Use group privacy when a person-level guarantee is unavailable or when records from different people reveal information about each other, such as shared household address and income.

14.4. Sequential composition

Suppose an analyst asks several queries of the same data, each with fresh noise, and an adversary sees all answers. For k mechanisms with budgets $(\varepsilon_i, \delta_i)$, the release of all outputs is

$$\Big(\sum_{i=1}^{k} \varepsilon_i,\ \sum_{i=1}^{k} \delta_i\Big)\text{-DP}. \tag{14.3}$$

This is **basic composition**. It needs every mechanism to draw fresh noise and to use the same neighbouring relation, add/remove or replace.

The rule also covers **adaptive** composition, where each query depends on earlier answers, provided every query's budget is fixed in advance. If the budget depends on the answers—for example, continuing until an estimate meets a target—a privacy filter must determine when to stop (Adaptive composition).

For many releases, basic composition can be loose. Ten Gaussian counts at $(0.5, 10^{-5})$ each cost $(5, 10^{-4})$ under basic composition, but their exact ε at that slack is 1.504. For multiple releases, use an accountant that composes the mechanisms themselves (Advanced and optimal composition, Part 4).

14.5. Parallel composition

Statistics by region use disjoint data subsets. If each person has one record, each record belongs to one region, so each region can use the full budget. The partition must follow a public rule fixed in advance that assigns every possible record to exactly one part. Run a mechanism M_i with independent noise on each part. If each M_i is $(\varepsilon_i, \delta_i)$-DP under add/remove, the full release is

$$(\max_i \varepsilon_i,\ \max_i \delta_i)\text{-DP under add/remove.} \tag{14.4}$$

A histogram with Laplace noise of scale $1/\varepsilon$ on each bin is therefore ε-DP for any number of bins. Under replace, a record can move between two bins, doubling the cost to 2ε. The rule also fails when a person contributes records to multiple bins. Charge each bin as a separate query or bound the number of bins each person can affect.

The fixed-partition condition is easy to violate. A split into "the top 10% of earners and the rest" changes its boundary when one high earner is added, so both parts change. A partition based on observed keys also fails: a key present in only one person's record identifies that person. Use fixed bins or a public key list. If the keys are not public, release only keys whose noisy counts exceed a threshold, using **partition selection** (Release statistics with a library).

14.6. Keeping a budget

Set a total budget before the first release and charge every release against it. In diffprivlib, a `BudgetAccountant` tracks the total, and each function charges its ε and δ. A call that would exceed the total raises `BudgetError` and returns no answer. The script practitioner_adult_statistics.py uses it on Adult, a dataset with 48,842 people and one row per person. The default slack is 10^{-6}. The script spends the budget on two queries, then attempts a third count.

```
age = adult["age"].to_numpy()
acc = BudgetAccountant(epsilon=1.0, delta=1e-6)  # the total, fixed in advance
mean_age = tools.mean(age, epsilon=0.5, bounds=(17, 90), accountant=acc,
 ↪  random_state=seed)
over_50 = tools.count_nonzero(age > 50, epsilon=0.5, accountant=acc, random_state=seed)
print(f"private mean age: {mean_age:.2f}  private count over 50: {over_50:,}")
print(f"spent: {acc.total()}")
try:
    tools.count_nonzero(age > 60, epsilon=0.1, accountant=acc, random_state=seed)
```

```
except BudgetError as err:
    print(f"refused: {err}")
```

Run it with `uv run python code/part03-dp-foundations/practitioner_adult_statistics.py`. With `random_state=0` it prints the following.

```
private mean age: 38.64  private count over 50: 9,813
spent: (epsilon=1.0, delta=0.0)
refused: Privacy spend of (0.1,0) not permissible; will exceed remaining privacy
 ↪  budget. Use BudgetAccountant.remaining() to check remaining budget.
```

The true values are 38.64 and 9,808. Laplace noise uses no δ, leaving 10^{-6} available for a Gaussian query. The ledger has two limitations. First, it applies basic composition to each call, overcharging calls on disjoint data subsets. Second, it trusts each call's guarantee. diffprivlib's mean divides by the exact row count, which is public only when neighbouring datasets have equal size. The ledger therefore holds only under replace neighbours.

Most composition failures violate a condition of one of these properties, often invalidating the guarantee (Table 14.1).

Table 14.1.: What composes and what doesn't.

Situation	Composes?	What to do
A function of the output, with its own coins	Yes, for free	Process as much as you like
Another query on the same data, fresh noise	Yes, budgets add	Charge it to the ledger
A query chosen from earlier answers	Yes, budgets add	Fix its budget in advance
Two releases that share one noise draw	No	Draw fresh noise for every query
"Post-processing" that reads the data again	No	Charge it as a new mechanism
A DP release beside an exact statistic of the same data	No, the exact one has no guarantee	Release the statistic privately too
A split whose boundaries depend on the data	No, not in parallel	Fix the bins in advance
Guarantees under add/remove and replace in one ledger	Only after conversion	Convert to replace: 2ε and $(1 + e^{\varepsilon})\delta$

Shared noise violates the fresh-randomness condition. Suppose a release gives the number of smokers and the number of smokers aged 40 or more, using the same Laplace draw of scale 1 for both. Each release alone is 1-DP, but subtracting them cancels the noise and reveals the exact number of smokers under 40. The pair is not DP. Code can reuse noise through a shared random seed, cached value or shard-level draw. A fixed seed such as `random_state=0` makes examples reproducible, but in production it makes the noise known and removable. Releasing the same noisy answer to an identical query is safe because it is post-processing of one release.

14.7. Cheat sheet

Table 14.2.: The results of this chapter.

Property	Rule	Condition
Post-processing	$f(M(D))$ is (ε, δ)-DP	f never reads the data
Group privacy	$(k\varepsilon,\ \delta\frac{e^{k\varepsilon}-1}{e^{\varepsilon}-1})$ for k records	Prefer the person as the unit, with bounded contributions
Basic composition	$(\sum \varepsilon_i, \sum \delta_i)$	Fresh noise, one neighbouring relation, budgets fixed in advance
Parallel composition	$(\max \varepsilon_i, \max \delta_i)$	Add/remove, parts fixed in advance; 2ε under replace

Rules of thumb

- Add noise once, as early as possible. All that you calculate from the noisy output is then free.
- Fix the total budget before the first release, and let one accountant refuse what goes above it.
- Do not share a random seed between mechanisms, and do not fix one in production.

14.8. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014): post-processing, group privacy and basic composition, the standard reference.
- McSherry, "Privacy Integrated Queries" (SIGMOD 2009): parallel composition, and a query system built on post-processing and composition as its only rules.

Derivations. Every result here is derived step by step in the full edition's chapter.

15. The Laplace and Gaussian mechanisms

15.1. Why this matters

Suppose a hospital wants to publish the number of patients with diabetes under differential privacy (DP). Most private releases use one of the two mechanisms in this chapter. Private training also uses them to add noise to gradients. Each mechanism computes a query f on dataset D, adds fresh noise Z and publishes

$$M(D) = f(D) + Z. \tag{15.1}$$

Three choices determine the mechanism. Calibrate noise to the query's **sensitivity**, the largest change one record can cause. Choose Laplace noise for pure DP or Gaussian noise with slack δ. Then apply a calibration formula to obtain the noise required for target ε. The formulas follow from privacy loss in the definition of differential privacy.

15.2. Sensitivity

Sensitivity measures the largest change one record can cause. For a vector query with d outputs, such as a histogram, the change needs a norm. Laplace noise uses the ℓ_1 norm, the sum of absolute changes; Gaussian noise uses the ℓ_2 norm, the square root of the sum of squared changes. For neighbouring datasets $D \sim D'$, the query's ℓ_p **sensitivity** is the largest change between their answers:

$$\Delta_p f = \max_{D \sim D'} \left\| f(D) - f(D') \right\|_p, \qquad p \in \{1, 2\}. \tag{15.2}$$

The two are equal when one record moves a single coordinate. When it moves all d of them, $\Delta_1 f$ can be $\sqrt{d}$ times $\Delta_2 f$.

The sensitivity also depends on the neighbouring relation. Under add/remove, one dataset has one record more than the other. Under replace, one record changes into another, so the size n is public. With B for the largest value of a record after clipping, Table 15.1 gives the standard queries.

Table 15.1.: Sensitivities of the standard queries under the two neighbouring relations.

Query	Add/remove Δ_1	Add/remove Δ_2	Replace Δ_1	Replace Δ_2
Count $\lvert D\rvert$	1	1	0	0
Sum, values in $[0, B]$	B	B	B	B
Mean, values in $[0, B]$, n public	none (Section 15.6)	none	B/n	B/n

Query	Add/remove Δ_1	Add/remove Δ_2	Replace Δ_1	Replace Δ_2
Histogram with k bins	1	1	2	$\sqrt{2}$
d threshold counts $\#\{x \geq j\}$, $j = 1..d$	d	$\sqrt{d}$	d	$\sqrt{d}$

A replaced record leaves one histogram bin and goes into another, so it changes two bins. The threshold counts, the numbers of records at or above each of d values, are a **dense** query, in which one record can move every coordinate.

Calculate sensitivity from the domain before examining the data. A data-dependent bound, such as the observed maximum, can leak information (Beyond global sensitivity). Unbounded data have infinite sensitivity, so clip each value to a fixed range. If ages are clipped at 100, the mean age of 1,000 patients has sensitivity $B/n = 0.1$ under replace.

15.3. The Laplace mechanism

The Laplace mechanism adds independent noise from the **Laplace distribution**, a two-sided exponential distribution centred at 0, to each coordinate. Write $\mathrm{Lap}(b)$ for Laplace noise with scale b. For an ε-DP guarantee, set the scale to sensitivity divided by ε:

$$b = \frac{\Delta_1 f}{\varepsilon}. \tag{15.3}$$

The mechanism's error is its noise. Each coordinate's mean absolute error is b, and its probability of exceeding distance t is $e^{-t/b}$. To keep the failure probability across all d coordinates at most β, allocate β/d to each coordinate. The simultaneous error bound is

$$\Pr\left[\max_i |Z_i| > b \ln \frac{d}{\beta}\right] \leq \beta. \tag{15.4}$$

Publish this width with the release. In reverse, the width gives an error budget, because for a single number you can fix a target width t and solve for the privacy level,

$$\varepsilon = \frac{\Delta_1 f \, \ln(1/\beta)}{t}. \tag{15.5}$$

A count required to be within 10 of its true value with probability 0.95 needs $\varepsilon = \ln(20)/10 = 0.2996$. For d outputs, use failure probability β/d for each and add their privacy levels by composition.

At the hospital, 1,234 of the 10,000 patients have diabetes. A count has sensitivity 1 under add/remove, so at $\varepsilon = 0.1$ the noise is $\mathrm{Lap}(10)$. Its 95% error bar is $10 \ln 20 = 29.96$, or 2.43% of the count.

The noise scale does not depend on dataset size, so relative error for a count of size n is about $1/(n\varepsilon)$. Sensitivity must include every output a record can change. If one record affects all 10 released counts, calibrating each count separately makes the release 10ε-DP, not ε-DP.

15.4. The Gaussian mechanism

The Gaussian mechanism adds independent normal noise with mean 0 and standard deviation σ to each coordinate. Its privacy loss is unbounded in the tail, so its guarantee requires δ. Gaussian noise uses ℓ_2 sensitivity, which may be much smaller than ℓ_1 sensitivity. Let $\mu = \Delta_2 f/\sigma$ be the neighbour-output distance in noise standard deviations, and let Φ be the standard normal cumulative distribution function (CDF). The mechanism is (ε, δ)-DP exactly when δ is at least

$$\delta_\mu(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^\varepsilon\, \Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big). \tag{15.6}$$

Calibrate the mechanism using this formula. As μ increases, exact δ increases from 0 to 1, so a unique value μ^* gives the target. The minimum noise scale is

$$\sigma^* = \frac{\Delta_2 f}{\mu^*}, \qquad \text{where } \delta_{\mu^*}(\varepsilon) = \delta, \tag{15.7}$$

and libraries solve for μ^* (Section 15.7). For 10,000 patients, the book's default slack is $\delta = 10^{-5}$. At $\varepsilon = 1$, this **analytic calibration** then gives $\sigma^* = 3.7306\,\Delta_2 f$.

Many papers and libraries still use the classic calibration of Dwork and Roth (2014), proved only for ε and δ between 0 and 1:

$$\sigma = \frac{\Delta_2 f \sqrt{2\ln(1.25/\delta)}}{\varepsilon}. \tag{15.8}$$

At $\varepsilon = 1$ and $\delta = 10^{-5}$, it gives $4.8448\,\Delta_2 f$, 30% more noise than the analytic $3.7306\,\Delta_2 f$. Above $\varepsilon = 8.42$, it gives insufficient noise for its claimed guarantee. Use analytic calibration instead.

A Gaussian mechanism has a privacy curve, with a value of δ for each ε. Accountants in Part 4 compose these curves. Report $\sigma/\Delta_2 f$ and the selected (ε, δ) for each Gaussian release.

At the hospital, Gaussian noise for the diabetes count at $\varepsilon = 0.1$ and $\delta = 10^{-5}$ gives a 95% error bar of 60.27, which is 1.96 standard deviations. Laplace noise gave 29.96 with no δ, so for a single count Laplace noise is better in error and in guarantee.

15.5. Laplace or Gaussian?

Choose a mechanism based on how one record's change is distributed across coordinates. Compare the root-mean-square error (RMSE) per released value at the same ε. Laplace noise with scale $b = \Delta_1 f/\varepsilon$ has standard deviation $\sqrt{2}\,b$. Let σ_1 be the analytic Gaussian noise scale at unit ℓ_2 sensitivity. The per-coordinate errors are

$$\text{Laplace: } \frac{\sqrt{2}\,\Delta_1 f}{\varepsilon}, \qquad \text{Gaussian: } \Delta_2 f\, \sigma_1. \tag{15.9}$$

In a sparse query, such as a histogram under add/remove, one record moves one coordinate, so the two sensitivities are equal. At $\varepsilon = 1$ and the default δ, $\sigma_1 = 3.7306$. So Laplace noise is better at every d, because $\sqrt{2} = 1.4142$ is smaller. In a dense query, such as the threshold counts, one

record moves every coordinate. Then the Laplace error grows like d and the Gaussian error only like $\sqrt{d}$. So Laplace noise is more accurate only while

$$d < \frac{(\varepsilon\sigma_1)^2}{2}, \tag{15.10}$$

so with $\sigma_1 = 3.7306$, Gaussian noise is better for 7 or more threshold counts.

Gaussian noise is also better for many releases, because several Gaussian releases compose exactly into one Gaussian mechanism. Private training has dense queries and many releases, because one record's gradient moves every coordinate of a model at every step. So it clips the ℓ_2 norm of each gradient and calibrates Gaussian noise to that clip.

15.6. Calibrating a release

Every noisy release has five steps.

1. Fix the neighbouring relation and the unit of privacy, usually one person under add/remove.
2. Clip every value to a range fixed in advance. Then take the sensitivity from Table 15.1, or calculate it from the worst record.
3. Use Laplace noise for one number or a sparse vector, and Gaussian noise for a dense vector.
4. Set the scale with Equation 15.3 or Equation 15.6. Charge it to your budget (Post-processing, group privacy and composition).
5. Draw the noise with a library that experts have checked. Publish the error bar and the noise parameters with the result.

A mean under add/remove has no simple sensitivity because its denominator changes. Release a noisy sum and a noisy count, splitting the budget between them, then divide; this division is post-processing. To obtain a 95% interval for the mean, use 97.5% intervals for both quantities. The mean is at least the sum's lower bound divided by the count's upper bound, and at most the sum's upper bound divided by the count's lower bound when both lower bounds are positive.

OpenDP's `.dp.mean` uses the same split, and `summarize(alpha=0.025)` reports both 97.5% intervals. If the clipping range is unknown, Private mean estimation describes how to estimate it privately. Use `numpy.random.laplace` only for prototypes: floating-point samplers can leak through low-order output bits (Discrete and floating-point-safe mechanisms). For a complete release with clipping, grouping and one budget, see Release statistics with a library.

15.7. Code

The snippet checks the hospital's values and analytic σ^* at $\varepsilon = 1$. In OpenDP, a mechanism's `map` converts sensitivity to privacy cost. Accuracy helpers convert between noise scale and a 95% error bound. For a count with sensitivity 1, the reciprocal of the Laplace scale is the privacy level. IBM's diffprivlib calibrates the analytic Gaussian mechanism.

```python
import math

import opendp.prelude as dp
from diffprivlib.mechanisms import GaussianAnalytic

dp.enable_features("contrib")
laplace = dp.m.make_laplace(dp.atom_domain(T=float, nan=False),
 ↪  dp.absolute_distance(T=float), scale=10.0)
print(laplace.map(1.0))  # 0.1
print(f"{dp.laplacian_scale_to_accuracy(10.0, alpha=0.05):.2f}")  # 29.96
print(f"{1 / dp.accuracy_to_laplacian_scale(10.0, alpha=0.05):.4f}")  # 0.2996
gauss = GaussianAnalytic(epsilon=1.0, delta=1e-5, sensitivity=1.0)
print(f"{math.sqrt(gauss.variance(0.0)):.4f}")  # 3.7306
```

Apply the mechanism to the value to add noise, as in `laplace(1234.0)`.

15.8. Cheat sheet

Table 15.2.: The results of this chapter.

Result	Formula	Notes
Laplace mechanism	$\mathrm{Lap}(\Delta_1 f/\varepsilon)$ per coordinate	Pure DP; error does not depend on n
Laplace error bar	$b\ln(d/\beta)$ for all d coordinates, scale b	Solve it for ε to budget error
Exact Gaussian δ	$\Phi(\frac{\mu}{2}-\frac{\varepsilon}{\mu})-e^{\varepsilon}\Phi(-\frac{\mu}{2}-\frac{\varepsilon}{\mu})$, $\mu=\Delta_2 f/\sigma$	Solve for σ with `GaussianAnalytic`
Classic Gaussian	$\sigma=\Delta_2 f\sqrt{2\ln(1.25/\delta)}/\varepsilon$	Too much noise below 1, unsafe above 8.42
Dense query	Laplace wins only for $d<(\varepsilon\sigma_1)^2/2$	Otherwise use Gaussian noise

Rules of thumb

- Calibrate Laplace noise to $\Delta_1 f$ and Gaussian noise to $\Delta_2 f$, never the other way round.
- When n is not public, never use B/n as the sensitivity of a mean.

15.9. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.3 and Appendix A: the Laplace mechanism and the classic Gaussian calibration.
- Balle & Wang, "Improving the Gaussian Mechanism for Differential Privacy" (ICML 2018): the exact privacy curve and the bisection calibration.

Derivations. Every result here is derived step by step in the full edition's chapter.

16. Advanced and optimal composition

16.1. What it is

A dashboard may refresh a hundred counts, and a training run may take thousands of noisy steps. Both require one differential privacy (DP) guarantee for the full release. Each step has privacy level ε and slack δ (The definition of differential privacy). **Basic composition** adds these values, so k steps cost $k\varepsilon$ and $k\delta$ (Post-processing, group privacy and composition).

This total is valid but can be loose. Each step's privacy loss depends on the noise and may be negative, so the sum rarely reaches the worst-case value $k\varepsilon$.

Advanced composition (Dwork, Rothblum & Vadhan 2010) accounts for these cancellations. With an additional slack δ' for the run, the steps together satisfy $(\varepsilon_{\text{total}}, k\delta + \delta')$-DP, with

$$\varepsilon_{\text{total}} = \varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon\tanh(\varepsilon/2). \tag{16.1}$$

The first term is the spread of total privacy loss, which grows like $\sqrt{k}$. The second term is its mean, or drift.

Optimal composition (Kairouz, Oh & Viswanath 2015) gives the tightest total guarantee when each step is known only by its pair (ε, δ). For 100 counts on 10,000 people, each with $\varepsilon = 0.1$ and no slack, basic composition gives total $\varepsilon = 10$. Advanced and optimal composition use slack $\delta' = 10^{-5}$, the book's default for 10,000 people, and give totals 5.30 and 4.31.

16.2. When you meet it

These rules appear in papers, privacy reviews and libraries that report one total ε for multiple steps (Table 16.1).

Table 16.1.: Which composition rule to use, at a slack of 10^{-5} (none for basic composition).

What you know about the steps	Rule	Total ε for 100 steps at 0.1
Each step's ε, and δ must stay 0	Basic composition	10
Each step's (ε, δ), to plan by hand	Advanced composition	5.30
Each step's (ε, δ), for the number you report	Optimal composition	4.31

What you know about the steps	Rule	Total ε for 100 steps at 0.1
The mechanism itself, such as Gaussian noise	An accountant (Part 4), which composes whole privacy curves	Often far smaller
Budgets chosen during the run	A privacy filter, which enforces a budget fixed in advance	—

Optimal composition is tight only when each step is represented by one privacy pair. A mechanism's full privacy curve contains a δ for each ε, and reducing it to one pair discards information. For example, 1,000 Gaussian steps with noise standard deviation 20 on a query of sensitivity 1 have an exact curve of $(7.51, 10^{-5})$-DP. Composing the best single pair for each step gives 62.7, about eight times larger. When the mechanism is known, compose its curve with an accountant (Privacy accounting).

16.3. How to use it

To allocate total budget $\varepsilon_{\text{total}}$ across k steps, invert advanced composition. When $\varepsilon\sqrt{k}$ is small, the drift term in Equation 16.1 is negligible, so each step can use approximately

$$\varepsilon \approx \frac{\varepsilon_{\text{total}}}{\sqrt{2k \ln(1/\delta')}}. \tag{16.2}$$

When $\varepsilon\sqrt{k}$ approaches 1, the drift is no longer negligible and this approximation overestimates the per-step budget.

For a known mechanism, report the accountant's result at the run's total δ. Otherwise, report optimal composition. The snippet allocates a total budget of $(1, 10^{-5})$ across 100 counts, placing all slack in δ'. It then accounts for 1,000 Gaussian steps with Google's `dp_accounting` using privacy loss distribution (PLD) and Rényi differential privacy (RDP) accountants.

```
import math

import dp_accounting

print(f"{1.0 / math.sqrt(2 * 100 * math.log(1e5)):.4f}")  # 0.0208
step = dp_accounting.GaussianDpEvent(noise_multiplier=20.0)
run = dp_accounting.SelfComposedDpEvent(step, 1000)
pld = dp_accounting.pld.PLDAccountant()
pld.compose(run)
print(f"{pld.get_epsilon(target_delta=1e-5):.2f}")  # 7.51
rdp = dp_accounting.rdp.RdpAccountant()
rdp.compose(run)
print(f"{rdp.get_epsilon(target_delta=1e-5):.2f}")  # 8.08
```

Each count receives 0.0208, twice the 1/100 under basic composition. The PLD accountant returns 7.51; the RDP accountant returns the valid but larger value 8.08 (Choosing an accountant).

16.4. What goes wrong

For a small number of steps, advanced composition may give a larger bound than basic composition; use the smaller bound. Every rule also composes the per-step slacks, so include each δ.

These rules allow each query to depend on earlier answers, provided each query's budget is fixed before the run. If budgets depend on the answers, use a privacy filter (Adaptive composition). Stopping rules also matter. A rule based only on noisy answers is post-processing, so the run costs no more than its planned length. A rule that stops when a true group count exceeds a threshold reveals information through the run length. In that case, each answer may be ε-DP while the full run has no finite DP guarantee.

In a review, a total below the optimal-composition bound is a warning unless an accountant derived it from the mechanism's full privacy curve.

Derivations. Every result here is derived step by step in the full edition's chapter.

17. Selection: exponential mechanism, report-noisy-max, sparse vector

17.1. Why this matters

Many analyses select an item, such as the best model on a validation set, a median or the first statistic above a threshold. Publishing every candidate score with Laplace noise spends budget on every score. The mechanisms in this chapter **release the selected position, not the scores**. Under differential privacy (DP), selecting one item costs one query regardless of the number of candidates; the candidate count affects accuracy through its logarithm.

17.2. Which mechanism to use

Table 17.1 maps common tasks to mechanisms. In each case, **sensitivity** Δu is the maximum change one person's record can cause to a candidate's score.

Table 17.1.: Which selection mechanism to use.

Task	Mechanism	Privacy cost
Pick the best of a fixed list	Exponential mechanism or report-noisy-max	ε
Pick the top c, such as the most frequent items	Gumbel noise, keep the c largest, or noisy max c times	$c\varepsilon$
Pick a number from a range, such as a quantile or a clipping norm	Exponential mechanism over the range	ε
Find the first query in a stream above a threshold	AboveThreshold	ε, however long the stream

17.3. The exponential mechanism

Let $\mathcal{R}$ be a fixed candidate set. Each candidate r has a **score** $u(D, r)$ on dataset D, such as the number of validation records a model classifies correctly. The **exponential mechanism** of

McSherry and Talwar (2007) samples candidates with probability proportional to an exponential of their scores. With privacy budget ε, it outputs candidate r with probability

$$\Pr[M_E(D) = r] = \frac{\exp\left(\varepsilon\, u(D, r)/(2\Delta u)\right)}{\sum_s \exp\left(\varepsilon\, u(D, s)/(2\Delta u)\right)}. \tag{17.1}$$

It is ε-DP because one record changes each weight, and also their sum, by at most a factor $e^{\varepsilon/2}$, which makes e^{ε} for their ratio.

Two conditions determine whether the mechanism is valid. First, fix the candidate list independently of the private data; a data-derived list may itself reveal records, such as a word used by one person. Second, remove the factor 2 in the exponent only for **monotone** scores, where one record changes all scores in the same direction. Counts under add/remove are monotone. A histogram under replace is not: replacing a record lowers one count and raises another, doubling the privacy cost if the factor 2 is omitted.

17.4. Accuracy, and the cost of many candidates

The number of candidates $|\mathcal{R}|$ does not affect privacy but can reduce accuracy by increasing the number of low-scoring choices. Define the **gap** as the best score minus the selected score. Its expected value is at most

$$\frac{2\Delta u}{\varepsilon}(\ln|\mathcal{R}| + 1), \tag{17.2}$$

so doubling the candidate count adds only $\ln 2$ to the bracket.

Suppose 50 models are evaluated on a private validation set of 1,000 records. Model r, numbered from 1, correctly classifies $900 - r$ records, so model 1 is best. A single record changes this count by at most 1, giving $\Delta u = 1$. At $\varepsilon = 1$, the mechanism selects the best model 39% of the time but loses only 1.54 correct predictions on average, or 0.15% accuracy, because errors are mostly near-ties. Hyperparameter tuning uses this method.

17.5. A private quantile or clipping norm

A median or clipping norm computed directly from private data can leak information. Use the exponential mechanism and fix the candidate range before accessing the data, then clip records to that range. Score each candidate by how closely the fraction of records below it matches the target quantile; for the median, the target is one half. Adding or removing one record changes each score by at most 1. Scores are constant between adjacent sorted records, so the mechanism selects one such interval, called a **piece**, and returns a uniform point within it. Longer pieces near the target quantile receive higher probability.

diffprivlib's `tools.quantile` and `tools.median` accept `epsilon` and a `bounds` range. OpenDP's `.dp.quantile` and `.dp.median` operate on Polars data frames and select from a finite candidate list. The script practitioner_adult_statistics.py estimates the median age in Adult, a census dataset with 48,842 people and one row per person, at $\varepsilon = 1$. The survey documentation supplies the range 17 to 90.

```
age = adult["age"].to_numpy()
private = tools.median(age, epsilon=1.0, bounds=(17, 90), random_state=seed)
print(f"true median age: {np.median(age):.1f}  diffprivlib median: {private:.2f}")
```

With `random_state=0`, it prints a true median of 37.0 and a private median of 37.72. Ages are integers, so points between 37 and 38 have the same score; the mechanism samples a point from that piece. OpenDP requires a public upper bound on row count, specified by a `Margin`. Choose a bound from public information that is at least the true row count; a larger bound has little cost.

```
context = dp.Context.compositor(
    data=adult.lazy(),
    privacy_unit=dp.unit_of(contributions=1),  # one row per person
    privacy_loss=dp.loss_of(epsilon=1.0),
    split_evenly_over=1,
    margins=[dp.polars.Margin(max_length=100_000)],  # a public bound on the number of
 ↪  rows
)
age_col: Any = pl.col("age")  # OpenDP adds .dp to Polars expressions when it is
 ↪  imported
query = context.query().select(age_col.dp.median(candidates=list(range(17, 91))))
print(query.summarize(alpha=0.05))
released = query.release().collect()  # changes on every run: OpenDP's noise has no
 ↪  seed
```

The summary reports `ExponentialMin` with scale 2.0 because OpenDP 0.16 adds exponential noise to each candidate score and selects the best. This is **permute-and-flip** (McKenna & Sheldon 2020), which has the exponential mechanism's guarantee and no larger expected error. Accuracy is `null` because OpenDP does not provide a quantile error bar. The noise is unseeded, so the released value changes each time `uv run python code/part03-dp-foundations/practitioner_adult_statistics.py` runs.

The error bound concerns rank, not value. With probability $1 - \beta$, the exponential-mechanism gap is at most $(2\Delta u/\varepsilon)(\ln|\mathcal{R}| + \ln(1/\beta))$, the tail bound corresponding to Equation 17.2. For a median with $\Delta u = 1$, the gap is the number of records between the output and the middle rank. At $\beta = 0.05$, this gives a 95% rank interval. A value interval requires the number of records near the median, which the release does not reveal. A dashboard needing value accuracy can spend budget on noisy counts below selected candidates. A wide range adds little cost because the gap depends on the logarithm of the candidate count.

Two limitations matter in practice. If data are sparse near the quantile, the best piece is long, so a small rank error may produce a large value error. The range and candidates must also be independent of private data. Setting the upper bound to the observed maximum reintroduces leakage. diffprivlib raises a `PrivacyLeakWarning` when `bounds` is omitted (Sensitivity).

17.6. Report-noisy-max and Gumbel noise

Report-noisy-max adds independent Laplace noise to the score of each candidate and outputs the candidate with the largest noisy score. It is ε-DP for any number of candidates when the

noise has scale $2\Delta u/\varepsilon$, or $\Delta u/\varepsilon$ when one record pushes all scores the same way. Release only the index of the winner, and pay for a fresh noisy answer if you need its score.

Gumbel noise of scale $2\Delta u/\varepsilon$ in place of the Laplace noise gives exactly the exponential mechanism. Libraries sample it with this **Gumbel-max trick**, because the exponential of a score in the thousands overflows, but noisy scores do not. The c largest scores with Gumbel noise give the top c at once (Durfee & Rogers 2019).

17.7. The sparse vector technique

Sometimes queries come one at a time, and you want the first one that is high enough. The **sparse vector technique** (Dwork et al. 2009) compares each query with one noisy threshold and pays only for the query that fires. Its basic form, **AboveThreshold**, takes a threshold τ and a stream of queries f, all with the same sensitivity Δf. It divides the budget into ε_1 for the threshold and ε_2 for the queries.

1. Draw one threshold noise, written ν, from $\mathrm{Lap}(\Delta f/\varepsilon_1)$ for the whole run.
2. For each query, draw a fresh noise from $\mathrm{Lap}(2\Delta f/\varepsilon_2)$ and add it to the answer of the query. If the result is at least $\tau + \nu$, output "above" and stop. If not, output "below" and continue.

AboveThreshold is $(\varepsilon_1 + \varepsilon_2)$-DP for any number of "below" answers, even when each query is chosen after the earlier answers. All the "below" answers together pay ε_1, through the one threshold noise, and the single "above" pays ε_2.

But the accuracy depends on the stream length k, which you must fix before the run. Write β for the chance that the accuracy guarantee fails. With an equal split, $\varepsilon_1 = \varepsilon_2 = \varepsilon/2$, every query gets the correct answer unless its true value is within a margin

$$\alpha = \frac{8\Delta f(\ln k + \ln(2/\beta))}{\varepsilon} \tag{17.3}$$

of the threshold. Monitor 1,000 counts, each with $\Delta f = 1$, against a threshold of 500 at $\varepsilon = 1$. At $\beta = 0.05$, the margin is 84.77. So with probability 0.95, only a count that near the threshold can get the wrong answer.

To find the first c queries above the threshold, start AboveThreshold again after each "above", with fresh threshold noise and ε/c per run. Stop after the c-th. The margin then grows with c, so ask for as few alarms as you can. Lyu, Su and Li keep one threshold for all c alarms and multiply the query noise by c.

17.8. When the sparse vector technique goes wrong

Several published variants do not have the privacy they claim (Table 17.2).

Table 17.2.: Variants of the sparse vector technique and their actual privacy, from Lyu, Su & Li (2017).

Variant (as numbered by Lyu, Su & Li)	What it changes	Actual privacy
Algorithm 3	Outputs the noisy value of each "above"	∞
Algorithm 4	Query noise does not grow with c	$\frac{1+6c}{4}\varepsilon$
Algorithm 5	No noise on the queries	∞
Algorithm 6	Never halts	∞

The variant that never stops is the one to remember. It adds exactly as much noise per query as AboveThreshold, so a reviewer who checks noise scales does not find it. But at $\varepsilon = 1$, its privacy loss gets to 18.1246 on 80 queries.

So review the accounting, not only the noise. Draw one threshold noise per run. If you draw the threshold again for each query, every "below" answer costs budget that nobody counted. Draw fresh noise for every query, because one noise draw for all of them is the same as Algorithm 5. Stop after the permitted number of "above" answers. Never publish the noisy values that you compared.

17.9. Cheat sheet

Table 17.3.: The selection mechanisms of this chapter, with $|\mathcal{R}|$ candidates or k queries.

Mechanism	Privacy	Accuracy
Exponential mechanism, Equation 17.1	ε; drop the 2 only for monotone scores	Mean gap $\leq \frac{2\Delta u}{\varepsilon}(\ln \lvert\mathcal{R}\rvert + 1)$
Report-noisy-max, noise $\mathrm{Lap}(2\Delta u/\varepsilon)$	ε, index only	Similar to the exponential mechanism
AboveThreshold	$\varepsilon_1 + \varepsilon_2$	Margin $8\Delta f(\ln k + \ln \frac{2}{\beta})/\varepsilon$, failure probability β

Rules of thumb

- Release positions, not values.
- Fix the candidate list, and the range of any number you pick, before you look at the data.

17.10. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.3, §3.4 and §3.6: the three mechanisms in their standard form.
- Lyu, Su & Li, "Understanding the Sparse Vector Technique for Differential Privacy" (VLDB 2017): the budget split and the survey of broken variants.

Derivations. Every result here is derived step by step in the full edition's chapter.

18. Query release and private multiplicative weights

18.1. Why this matters

Analysts may request every two-way table over several columns or a synthetic dataset that supports repeated queries. Both require differential privacy (DP). These tables are **marginal tables**: each counts records for combinations of attribute values, and each count is a **cell**.

Add noise to each table and calibrate it to the maximum change one person can make to that table. With one record per person, one person affects one cell, so a table costs about as much as one count. Subsequent computation on noisy tables is post-processing. If there are too many queries to noise every cell, fit synthetic data to a selected set of noisy tables.

18.2. Which method to use

Choose a method based on query count, record count and the number of attributes per record (Table 18.1).

Table 18.1.: Which query-release method to use.

Situation	Use
A few marginal tables, with a public list of cells	Noisy counts on every cell, one budget share per table
Hundreds of tables or thousands of cells	Gaussian noise on every cell, scaled to the whole set of tables
A table whose list of values is not public	Noisy counts, keeping only cells that clear a noisy threshold (Release statistics with a library)
Far more queries than records, or queries not known in advance	Synthetic data from AIM or MST
Queries that arrive one at a time, perhaps adaptively	Online private multiplicative weights

Each person can affect every table, so the combined query is dense. The dense-query rule in the Laplace and Gaussian mechanisms identifies when Gaussian noise becomes more accurate. At privacy level $\varepsilon = 1$ and slack $\delta = 10^{-5}$, Gaussian noise is better for seven or more tables.

18.3. Noise on every cell

Under add/remove neighbours, one record changes one cell of a marginal table by 1, so the table's sensitivity is 1. Laplace noise of scale $1/\varepsilon$ on each cell makes the table ε-DP by parallel composition. Under replace neighbours, a record can move between cells, changing two cells and doubling sensitivity.

Releasing several tables of the same records composes their costs. With Laplace noise, splitting the budget across tables makes per-cell error grow in proportion to table count. Gaussian noise scales with the square root of the sum of squared sensitivities. If each person affects m tables, Gaussian error grows as $\sqrt{m}$.

For example, a table of 10,000 records has eight yes/no attributes, so $2^8 = 256$ possible records. The release is every cell of the marginal table of every set of attributes, 6,560 cells in all. Each cell is a fraction of the records, under replace neighbours. A uniform guess, which ignores the data, has a largest error of 0.232 on these cells. At $\varepsilon = 1$, Laplace noise on every cell is worse, with 0.497. Gaussian noise at $\delta = 10^{-6}$ is much better, with 0.037.

Two errors are common. Calibrating each cell as a separate query spends ε once per table. Applying basic composition to thousands of cells also wastes budget. Calibrate noise to the complete set of tables.

18.4. Releasing a marginal with OpenDP

The script practitioner_adult_statistics.py releases an education-by-income table from Adult at $\varepsilon = 1$. Adult contains 48,842 people with one row each. The survey documentation lists 16 education levels and two income classes, making the 32 table keys public. Release empty cells as well; omitting one would reveal that its count is zero. Map missing values to a listed key such as "Unknown", because OpenDP drops rows with keys outside the list. In OpenDP, `Margin(invariant="keys")` declares the cell list public, and `with_keys` restricts the release to that list.

```
keys = pl.LazyFrame({"education": EDUCATION}).join(pl.LazyFrame({"income": INCOME}),
 ↪  how="cross")
context = dp.Context.compositor(
    data=adult.lazy(),
    privacy_unit=dp.unit_of(contributions=1),
    privacy_loss=dp.loss_of(epsilon=1.0),
    split_evenly_over=1,
    margins=[dp.polars.Margin(by=["education", "income"], invariant="keys")],  # the
 ↪  cells are public
)
query = context.query().group_by("education",
 ↪  "income").agg(dp.len(signed=True)).with_keys(keys)
print(query.summarize(alpha=0.05))
released = query.release().collect()  # all 32 cells, zeros included; changes on every
 ↪  run
```

Run `uv run python code/part03-dp-foundations/practitioner_adult_statistics.py` to print the deterministic summary.

```
shape: (1, 5)
┌────────┬──────────────┬──────────────┬───────┬──────────┐
│ column ┆ aggregate    ┆ distribution ┆ scale ┆ accuracy │
│ ---    ┆ ---          ┆ ---          ┆ ---   ┆ ---      │
│ str    ┆ str          ┆ str          ┆ f64   ┆ f64      │
╞════════╪══════════════╪══════════════╪═══════╪══════════╡
│ len    ┆ Frame Length ┆ Integer Laplace ┆ 1.0 ┆ 3.375618 │
└────────┴──────────────┴──────────────┴───────┴──────────┘
```

Each cell receives integer Laplace noise with scale 1 (Discrete and floating-point-safe mechanisms). The 95% accuracy of 3.38 means each noisy count is within 3.38 of the true count with 95% probability. Report this error bar with every cell; it covers noise but not sampling error or data errors. OpenDP noise is unseeded, so the following values are from one run, alongside the true counts.

```
  Doctorate >50K: 432  true 431  accuracy 3.38
  HS-grad <=50K: 13281  true 13281  accuracy 3.38
  Preschool <=50K: 82  true 82  accuracy 3.38
  Preschool >50K: 0  true 1  accuracy 3.38
```

Noise has little effect on large cells. The smallest true cell—one person with preschool education and high income—is smaller than the noise. Merge rare categories in advance if small counts matter. With `signed=True`, a noisy count may be negative, preserving unbiasedness. Clamping at zero, rounding and scaling cells to a released total are post-processing and cost no additional privacy.

Three implementation details matter. First, deriving cell keys from private data—for example, calling `unique()` on a private column—may reveal rare values. Use partition selection when no public codebook exists (Release statistics with a library). Second, each additional table over the same people needs a budget share; set `split_evenly_over` to the table count. Third, for many tables, specify `dp.loss_of(rho=...)` with a ρ that converts to the desired (ε, δ) (Release statistics with a library). OpenDP then uses Gaussian noise and accounts in zero-concentrated differential privacy (zCDP), which composes more tightly (Concentrated differential privacy).

18.5. Synthetic data

When the query set is too large to noise every cell, or queries are not known in advance, release synthetic data using four steps.

1. **Select** a set of marginal tables to measure, from domain knowledge or privately with the exponential mechanism, scoring each table by how badly the current model fits it.
2. **Measure** each whole table with Gaussian noise, scaled to the whole set.
3. **Generate** a compact graphical model that fits the noisy tables.
4. **Sample** synthetic rows from the model.

All steps after measurement are post-processing, so synthetic rows can be queried and shared without further privacy cost. MST and AIM are standard methods developed by McKenna and coauthors; AIM selects tables adaptively. SmartNoise Synth, the `snsynth` package, provides both methods through one interface. It is not installed in this book's environment, so the example below was not run.

```
# smartnoise-synth 1.0.8: not installed in this book's environment, not run
from snsynth import Synthesizer

synth = Synthesizer.create("aim", epsilon=1.0, delta=1e-6)
synth.fit(adult_pandas, categorical_columns=["education", "sex", "race", "occupation",
 ↪  "income"])
rows = synth.sample(48_842)
```

A synthetic table fits the measured tables by construction. Evaluate it on additional queries, but account for privacy when evaluation uses private data. Selecting the best of several runs based on private data is also a private query, and each run spends the generator's budget again. Evaluate settings on public data with the same structure or charge comparisons to the budget. Synthetic data from a generator without DP are not anonymized.

18.6. Private multiplicative weights

The synthetic-data tools use the select-measure-update loop of the **multiplicative weights exponential mechanism (MWEM)** (Hardt, Ligett and McSherry 2012). MWEM maintains a guessed histogram with one weight per possible record—256 in the eight-attribute example. In a fixed number of rounds, it selects a poorly answered query using the exponential mechanism, measures it with fresh Laplace noise and updates the histogram toward that measurement. The correction count is bounded independently of the query count, so error grows logarithmically with the number of queries.

MWEM is useful for large query sets. In the eight-attribute example at privacy level 1, 38 rounds give a maximum error of 0.078. This is better than Laplace noise but about twice the Gaussian-noise error. Runtime grows with the number of possible records, limiting MWEM to a few attributes.

Fix the round count from the error bound before accessing private data; the bound uses only public quantities. Measure each query with fresh noise because exact selection scores could leak information. Release the average of the guesses, not the last one, because the accuracy guarantee applies only to the average. For sequential queries, an online variant pays only for its corrections (Adaptive data analysis).

18.7. Cheat sheet

Table 18.2.: The query-release methods of this chapter.

Method	Privacy	Error grows with
Laplace noise on every cell	ε; one table costs ε under add/remove	The number of tables
Gaussian noise on every cell	(ε, δ) or zCDP	The square root of the number of tables
Private multiplicative weights	ε or (ε, δ)	The logarithms of the numbers of queries and possible records
Select, measure, generate (MST, AIM)	(ε, δ)	No closed form; the practical choice for many attributes

Rules of thumb

- Take the list of cells from a codebook, never from the private data.
- Add noise to whole tables, scaled to the set of tables, and post-process freely afterwards.
- Fit synthetic data once, and never pick the best of several runs on the private data.

18.8. References

- McKenna, Mullins, Sheldon & Miklau, "AIM: An Adaptive and Iterative Mechanism for Differentially Private Synthetic Data" (VLDB 2022): the select, measure and generate loop as used in practice.
- Hardt, Ligett & McSherry, "A Simple and Practical Algorithm for Differentially Private Data Release" (NeurIPS 2012): MWEM, with experiments on real tables.

Derivations. Every result here is derived step by step in the full edition's chapter.

19. Beyond global sensitivity

19.1. What it is

The Laplace and Gaussian mechanisms calibrate noise to **global sensitivity**, the maximum change one record can cause across all neighbouring datasets. For ages bounded between 0 and 100, the median's global sensitivity is 100. Under differential privacy (DP) at $\varepsilon = 1$, the Laplace mechanism's median error is $100 \ln 2 = 69.31$ years. On a simulated sample of 1,001 ages, however, one person changes the median by only 0.1386 years. This smaller dataset-specific value is **local sensitivity**, the largest change one record can cause to answer f on dataset D:

$$\mathrm{LS}_f(D) = \max_{D' \text{ a neighbour of } D} |f(D) - f(D')|. \tag{19.1}$$

These techniques use the data's local scale under one condition: **a noise scale, bound or threshold computed from private data is itself a statistic and must also be private.** This chapter assumes fixed-size datasets and replace-one-record neighbours.

19.2. When you meet it

Use these techniques when a statistic has no useful worst-case sensitivity (Table 19.1).

Table 19.1.: Which technique releases what, and where you meet it.

You release	Technique	Where you meet it
A sum, mean or gradient of unbounded values	Clipping to a bound fixed in advance	Every library's `bounds`, and the clipping norm of private training
A median or quantile	Inverse sensitivity	`diffprivlib.tools.quantile`, OpenDP's `dp.quantile`
A discrete answer with a clear winner	Stability-based release	The threshold of private partition selection
A black-box model or estimator	Subsample-and-aggregate	Teacher ensembles for private prediction

A **stability-based release** publishes a discrete answer, such as the most common value, when a noisy test shows that it exceeds the runner-up by about $2\ln(1/\delta)/\varepsilon$ records for slack δ. Otherwise,

it returns no answer; downstream steps must handle refusals. The same test selects keys for a group-by (Release statistics with a library).

Subsample-and-aggregate runs an estimator on disjoint blocks, clips their answers and releases a noisy average. Private aggregation of teacher ensembles (PATE) uses this method for voting (Private prediction).

Propose-test-release proposes a local-sensitivity bound and refuses to release unless a noisy test shows that the data are sufficiently far from the bound. **Smooth sensitivity** scales heavy-tailed noise to a bound that changes gradually between neighbours. PATE uses smooth sensitivity to calibrate its voting mechanism, but on the 1,001-age example it is about twelve times less accurate than inverse sensitivity.

19.3. How to use it

For unbounded inputs, use **clipping**. Replace each value x by $\min(x, C)$, where clip bound C is fixed independently of the data. If values are nonnegative, one record changes the mean of n records by at most C/n. Laplace noise with scale $C/(n\varepsilon)$ therefore gives ε-DP for the clipped mean. A larger C reduces clipping bias but increases noise.

Choose C from public domain information, such as a legal maximum or sensor range. Otherwise, spend part of the budget on a private high quantile, account for it by composition, and clip at that quantile. Without a bound, diffprivlib uses the data to calculate one and raises `PrivacyLeakWarning`. Convert the warning to an error with `warnings.simplefilter("error", PrivacyLeakWarning)`, as in practitioner_adult_statistics.py.

For a median or quantile, use **inverse sensitivity**. This exponential mechanism favours candidate outputs that require fewer record changes to become the median. Set bounds or a candidate grid in advance. On the 1,001-age sample at $\varepsilon = 1$, median error is 0.0783 years, below the local sensitivity of 0.1386.

19.4. What goes wrong

Noise calibrated to local sensitivity alone is not private. The five ages $(0, 0, 0, 0, 100)$ have median 0, and no single change moves it, so the mechanism adds no noise. A neighbouring dataset $(0, 0, 0, 100, 100)$ receives continuous noise and never outputs exactly 0; therefore, output 0 identifies the first dataset. Similar leaks arise from a clip bound based on the observed maximum, a candidate grid derived from private data or keys read from private data. During review, search for `max()`, `quantile()` and `unique()` calls on private data that set parameters.

Upper clipping biases a mean downward. Two-sided clipping removes that bias only when the range is centred on the data (Private mean estimation). If neighbouring datasets differ by adding or removing a record, n is not public. Release a noisy clipped sum and noisy count, then divide them.

Derivations. Every result here is derived step by step in the full edition's chapter.

20. Discrete and floating-point-safe mechanisms

20.1. What it is

The proof of the Laplace mechanism assumes densities on the real line, but computers sample from a finite set of floating-point numbers with uneven spacing. The textbook sampler adds $\pm b \ln(1/U)$ to the true answer, where b is the noise scale and U is uniform. Mironov (2012) showed that this sampler leaks the true answer through low-order bits in double precision. Floating-point values are dense near 0 and sparse at large magnitudes. Some outputs can therefore occur for a true answer of 0 or 1 but not for intermediate values, identifying the input exactly.

Differential privacy (DP) does not hide this implementation leak. DP limits how much one record can change an output probability. The privacy level ε bounds the multiplicative change, and the slack δ allows an additive failure probability (The definition of differential privacy).

Let dataset D denote the data and let S denote a set of possible outputs. Suppose D can produce outputs in S, but a neighbouring dataset cannot. The neighbour assigns probability zero to S, so multiplying that probability by e^{ε} still gives zero. The mechanism therefore requires slack at least

$$\delta \geq \Pr[M(D) \in S]. \tag{20.1}$$

In a toy sampler with 12-bit floats, these outputs have probability 0.4791. Using more bits does not reduce this probability.

DP applies to the output distribution that the code actually produces. One repair is the **snapping mechanism**: clamp the answer, add noise, round to a coarse power-of-two grid, then clamp again. Another repair is to make the query integer-valued and add exact discrete Laplace or discrete Gaussian noise.

20.2. When you meet it

This issue arises when reviewing code that samples noise directly, such as `x + np.random.laplace(0, b)`, or when choosing a library. OpenDP provides exact samplers, including discrete Laplace and discrete Gaussian noise for integer data. diffprivlib provides `mechanisms.Geometric`, `mechanisms.GaussianDiscrete` and `mechanisms.Snapping`. Its continuous `Laplace` mechanism, used by its `tools` functions, reduces this risk but is not exact. It draws four uniform variates (Holohan & Braghin 2021). Secure aggregation also requires integer noise (Local, shuffle and distributed trust models).

Opacus adds continuous Gaussian noise in floating point during private training, so the proof of Private stochastic gradient descent applies to idealized code. Its `secure_mode=True` uses a cryptographic generator, making these attacks harder. Verify that released models use this mode and state which idealized mechanism their guarantee assumes.

20.3. How to use it

Use a library to add noise to integer data. **Discrete Laplace** noise assigns a noise value z probability proportional to $e^{-|z|/b}$ and uses the same calibration as continuous Laplace noise. Let Δf denote the sensitivity of the integer-valued query f. Its privacy level is $\varepsilon = \Delta f/b$. At $b = 1$, a count is exact with probability 0.4621.

Discrete Gaussian noise assigns each integer x probability proportional to $e^{-x^2/(2\sigma^2)}$, where σ is the noise width. Its exact privacy curve is slightly weaker than the continuous Gaussian's. For sensitivity 1 and noise width 1, the discrete mechanism has privacy level $\varepsilon = 1$ with slack $\delta = 0.1414$. The corresponding slack for continuous noise is 0.1269. Report the discrete curve or use OpenDP's accounting in zero-concentrated DP. Under that accounting, both Gaussian mechanisms have the same budget ρ, where $\rho = (\Delta f)^2/(2\sigma^2)$. In the snippet, `map(1)` returns each guarantee at sensitivity 1.

```
import opendp.prelude as dp

dp.enable_features("contrib")
integers, distance = dp.atom_domain(T=int), dp.absolute_distance(T=int)
laplace = dp.m.make_laplace(integers, distance, scale=1.0)
print(laplace.map(1))  # 1.0
gauss = dp.m.make_gaussian(integers, distance, scale=2.0)
print(gauss.map(1))  # 0.125
print(gauss.output_measure)  # ZeroConcentratedDivergence
```

To integerize a real-valued query, such as a sum of clipped values, divide by a step much smaller than the noise scale. Round up with probability equal to the fractional part and down otherwise; this preserves the expectation. Recompute sensitivity in these units because rounding can increase the distance between neighbours. Then add discrete noise and multiply by the step.

20.4. What goes wrong

Check each implementation against the following conditions.

- Is every sensitivity computed from bounds fixed in advance, with no `max()`, `quantile()` or `unique()` on the private data (Beyond global sensitivity)?
- Does a vetted library draw the noise, or an exact sampler with no floating-point `exp`, `log` or `random()` inside? Search for `np.random`, `random.`, `scipy.stats`, `default_rng`, `.laplace(` and `.normal(`.
- Does integer data get integer noise, and is every floating-point output snapped and clamped to a fixed range?
- Does each release draw fresh noise from a cryptographic source, such as `secrets.SystemRandom()`, rather than a fixed seed that makes it removable?
- Is the running time fixed or padded? The rounds of a rejection sampler depend on the value it draws, and Jin et al. (2022) use that time as a leak.
- Does the stated guarantee match the mechanism, including the discrete Gaussian's own curve and the sensitivity after rounding?

Derivations. Every result here is derived step by step in the full edition's chapter.

21. Local, shuffle and distributed trust models

21.1. Why this matters

Earlier mechanisms in this part assume a **trusted curator** who holds raw data and releases only the mechanism's output. In this **central model**, differential privacy (DP) protects released outputs, not data exposed by a breach, employee or court order. A browser vendor collecting usage statistics may not be able to rely on a trusted curator.

Three models reduce reliance on a curator, with different protections. In the **local model**, each person randomizes their record on-device, so no central party sees the raw record. In the **shuffle model**, a shuffler breaks the link between each report and its sender. In the **distributed model**, cryptography and enough honest participants allow a server to reveal only a sum. Each model reduces trust in intermediaries but increases noise.

21.2. Choosing a trust model

Reduced trust generally increases noise. For a private count, central-model noise remains fixed as the population grows, while local-model noise grows with the square root of population size. Shuffle and distributed models narrow this gap.

Use the central model when a curator may hold raw data, as in Release statistics with a library. Use the local model when no party may see raw records and the population is large enough, based on the standard deviation in Section 21.3. Report the trust model with the privacy parameters: the same ε protects against different adversaries in different models.

21.3. Central and local differential privacy

In the local model, each person sends a **report**, the output of a **local randomizer** R applied to their record. Let ε_0 denote the privacy level of one report. The randomizer is ε_0-DP if changing a record increases the probability of any report set by at most e^{ε_0}. Write x and x' for two possible records. Let S denote a set of reports. The guarantee is

$$\Pr[R(x) \in S] \leq e^{\varepsilon_0} \Pr[R(x') \in S]. \tag{21.1}$$

Changing one person's record changes only that person's report, so the report collection is ε_0-DP under replace-one-record neighbours. This guarantee protects the randomized value, not identifying metadata. For example, a network address stored with a report may identify its sender.

The basic local randomizer applies randomized response to one bit. Let p be the probability that a report is truthful: $p = e^{\varepsilon_0}/(1 + e^{\varepsilon_0})$. Each person reports the true bit with probability p and its complement otherwise. To correct for flips, the analyst subtracts $1 - p$ from the fraction of reported ones and divides by $2p - 1$. If n is the number of people, the estimate's standard deviation is

$$\mathrm{sd}(\hat{\pi}) = \frac{e^{\varepsilon_0/2}}{(e^{\varepsilon_0} - 1)\sqrt{n}}. \tag{21.2}$$

The script practitioner_adult_statistics.py compares the models on Adult, a census dataset of 48,842 people. It uses $\varepsilon_0 = \ln 3$ for local randomization. Each bit indicates whether a person's income exceeds $50K. The local model uses diffprivlib's `Binary` mechanism, so the analyst receives only randomized reports.

```
answers = (adult["income"] == ">50K").cast(pl.Int8).cast(pl.String).to_list()  # "1" or
 ↪  "0"
n, eps = len(answers), log(3)
coin = Binary(epsilon=eps, value0="0", value1="1", random_state=seed)
reports = [coin.randomise(a) for a in answers]  # each person randomizes their own
 ↪  answer
p = exp(eps) / (1 + exp(eps))  # chance a report is truthful: 0.75
reported = reports.count("1") / n
local = (reported - (1 - p)) / (2 * p - 1)  # undo the expected flips
local_sd = math.sqrt(p * (1 - p) / n) / (2 * p - 1)  # the randomization noise alone
print(f"true share over 50K: {answers.count('1') / n:.4f}")
print(f"local estimate: {local:.4f}  sd {local_sd:.4f}")
```

For the central-model estimate, a trusted curator applies diffprivlib's `tools.count_nonzero` to the true bits at the same privacy level. The call adds Laplace noise with standard deviation $\sqrt{2}/\varepsilon$ to the count.

```
rich = (adult["income"] == ">50K").to_numpy()
central = tools.count_nonzero(rich, epsilon=log(3), random_state=seed) / len(rich)
central_sd = math.sqrt(Laplace(epsilon=log(3), sensitivity=1).variance(0)) / len(rich)
 ↪  # the count's noise
print(f"central estimate: {central:.4f}  sd {central_sd:.6f}")
```

Running `uv run python code/part03-dp-foundations/practitioner_adult_statistics.py` with seed 0 gives:

```
true share over 50K: 0.2393
local estimate: 0.2359  sd 0.0039
central estimate: 0.2393  sd 0.000026
local sd / central sd: 149
```

At the same guarantee, local noise is 149 times larger, and the ratio grows with the square root of population size. This cost is inherent to local DP, not specific to randomized response: no local protocol estimates a fraction with error below order $1/((e^{\varepsilon_0} - 1)\sqrt{n})$, and randomized response achieves that order. Apple, Google and Microsoft therefore use local DP mainly to estimate frequent items across very large populations.

21.4. Frequency estimation: k-ary randomized response and unary encoding

Most deployed local-DP systems perform **frequency estimation**. Each person has a category, such as an emoji, from a known list; the analyst estimates each category's share. Let v denote one category. A report votes for v with probability p_1 when the person has that category and with smaller probability p_0 otherwise. Let c_v count the reports for v among the n reports. Correcting for expected false votes gives the unbiased estimate

$$\hat{f}_v = \frac{c_v/n - p_0}{p_1 - p_0}. \tag{21.3}$$

Two designs set p_1 and p_0 for a list of k values. In **k-ary randomized response**, a person reports the true value with probability $e^{\varepsilon_0}/(e^{\varepsilon_0} + k - 1)$ and otherwise reports another value. **Optimized unary encoding** sends one bit per value, retains the bit for the person's value about half the time and rarely flips a 0 to 1. Compare the designs' variances for rare values, which determine the smallest detectable share. Use k-ary randomized response when $k < 3e^{\varepsilon_0} + 2$; otherwise use unary encoding, which requires a k-bit report. The pure-ldp Python package implements both.

Consider a keyboard app estimating 128 emoji frequencies among one million users, at $\varepsilon_0 = 1$. For a rare emoji, unary encoding has standard deviation 0.0019; k-ary randomized response has 0.0066. A common detection rule requires an estimate to exceed zero by three standard deviations. Even the better design cannot reliably detect emoji used by fewer than about 3×0.0019, or 0.6%, of users. Check the population size before collection. For larger lists, local hashing and heavy-hitter protocols reduce report length to a few bits (Wang et al. 2017; Bassily et al. 2017).

21.5. Numbers in one bit: local mean estimation

One bit per person can estimate a mean when each value lies in a known interval, such as app-use minutes (Ding, Kulkarni & Yekhanin 2017). The probability of reporting 1 increases linearly with the value. The analyst inverts this relation to estimate the mean without bias. With one million users and a bound of 1,440 minutes, the standard deviation is 1.420 minutes at $\varepsilon_0 = 1$. Set the bound before collection and clip values on-device. Otherwise, one person's value could affect every user's randomizer through the estimated bound.

21.6. Collecting every day: memoization

Telemetry asks the same question every day. If a value does not change, an analyst can average its daily reports to remove their fresh noise. Composition counts that leak as the sum of the ε_0 of each report. **Memoization** prevents this averaging attack: the device draws one permanent randomized response per value and sends it each day. An unchanged value then costs only the level of that response, written ε_∞. RAPPOR, Google's collection of the homepages of Chrome users, has $\varepsilon_\infty = 2.043$.

Memoization fails when the value changes, because each new value costs another ε_∞. A user who changes their homepage every month gets to $12 \times 2.043 = 24.5$ in a year.

21.7. The shuffle model and amplification by shuffling

In the **shuffle model** (Cheu et al. 2019), each person sends a local report to a **shuffler**. The shuffler removes identifiers and timing, then sends the reports in random order. Because each report is hidden among the others, the collection can satisfy central DP with privacy level ε far below the local level ε_0. The guarantee also has slack δ (Amplification by shuffling and by iteration). Feldman, McMillan & Talwar (2021) give a closed-form bound that requires enough people:

$$\varepsilon_0 \leq \ln \frac{n}{16 \ln(2/\delta)}. \tag{21.4}$$

Fix the central target, then solve for the largest allowed ε_0. The target here is privacy level 1 and slack 10^{-6}. At a population of 10,000, the bound allows $\varepsilon_0 = 3.504$. At 1,000 people, it allows $\varepsilon_0 = 1.460$. These targets use a δ below the book's default (How small δ must be). For 10,000 people, the closed-form bound overestimates the privacy level by a factor between 2.6 and 4.6. Use it for planning, but use the paper's numerical bound for deployment.

Three conditions can remove amplification. If the shuffler colludes with the analyst, each report retains privacy level ε_0. Choose a local level that is acceptable on its own under the definition. Fake senders provide no protection; therefore, count only honest participants in n. Timing and message sizes may also leak information, so practical shufflers batch and pad traffic.

21.8. Distributed DP: splitting the noise

In the **distributed model**, each client adds a share of the central noise, and **secure aggregation** reveals only the sum to the server (Cryptography basics in the full edition). Independent Gaussian variances add. If each of n clients adds normal noise with variance σ^2/n, the sum has standard deviation σ, matching the central Gaussian mechanism.

Small shares create a vulnerability: colluding clients can reveal their noise, and dropped clients add none. Let t be the number of clients that are unavailable or colluding. The remaining noise has standard deviation

$$\sigma \sqrt{\frac{n-t}{n}}. \tag{21.5}$$

Set in advance the largest fraction ρ of clients that may collude or drop out. Have each client add variance $\sigma^2/(n(1-\rho))$ so the remaining honest clients provide the required total noise.

For example, a server aggregates a histogram for about 100,000 users through 100 clients. The target is $(1, 10^{-6})$. Changing one user's value decrements one count and increments another, giving ℓ_2 sensitivity $\sqrt{2}$. Analytic Gaussian calibration in the Laplace and Gaussian mechanisms gives total standard deviation $\sigma = 5.975$. This is $5.975/\sqrt{100} = 0.597$ per client. If a fifth of the clients collude and noise is not over-provisioned, the guarantee weakens to $\varepsilon = 1.128$ at the same δ.

Secure aggregation operates on integers, so clients add integer-valued noise, such as Skellam noise or the distributed discrete Gaussian in Discrete mechanisms. TensorFlow Federated provides `tff.aggregators.DistributedDpSumFactory` for this task. However, secure aggregation does not validate inputs, so a malicious client can still send an arbitrary vector.

21.9. Cheat sheet

Table 21.1.: Which trust model and protocol to use, by situation.

Situation	Use	Rule
A curator can be trusted	Central Laplace or Gaussian	The default
Nobody may see raw records	Local randomized response	Frequent items, very large populations
Frequencies over k values	k-ary randomized response or unary encoding	k-ary if $k < 3e^{\varepsilon_0} + 2$
The same question daily	Memoization	Each new value costs ε_∞
An anonymizing relay	Shuffle model	Deploy with a numerical bound
Federated sums	Distributed Gaussian or Skellam	Over-provision for colluders

Rules of thumb

- Local noise exceeds central noise by a factor that grows like $\sqrt{n}$, 149 on Adult at $\ln 3$.

21.10. References

- Wang, Blocki, Li & Jha, "Locally Differentially Private Protocols for Frequency Estimation" (USENIX Security 2017): frequency oracles and local hashing, compared.
- Feldman, McMillan & Talwar, "Hiding Among the Clones" (FOCS 2021): the shuffling bounds, closed form and numerical.

Derivations. Every result here is derived step by step in the full edition's chapter.

22. Lower bounds: what privacy costs

22.1. What it is

A **lower bound** states the best accuracy any mechanism satisfying differential privacy (DP) can achieve. It applies to all such mechanisms and provides a feasibility check. Accuracy has two parameters: **error** α, the maximum distance between output and true answer, and **failure probability** β, the probability that the error exceeds α.

One common lower bound applies to a histogram with N bins under pure DP, at privacy level ε and with no slack. If every count is within α records of its true value with probability at least $1 - \beta$, then

$$\alpha \geq \frac{1}{2}\left(\frac{\ln\left(N(1-\beta)\right)}{\varepsilon} - 1\right). \tag{22.1}$$

The bound grows logarithmically with the number of bins and remains positive under pure DP. Under approximate DP with slack δ, it is about $\ln(1/\delta)/\varepsilon$ records, independent of the number of bins.

22.2. When you meet it

Use a lower bound to check whether a proposed accuracy is feasible. For example, consider a histogram of one million locations at $\varepsilon = 1$, with no slack, that promises every bin within five records of its true count with 95% probability.

Determine whether one person affects one statistic, as in disjoint histogram cells, or many statistics, as in means of multiple attributes. The histogram bound applies to the first case; fingerprinting and reconstruction bounds apply to the second. If a target is below the lower bound, increase the data, coarsen the answer or weaken the guarantee. A mechanism near the bound is close to the best possible accuracy.

22.3. How to use it

For one million locations, every pure-DP histogram has error of at least 6.38 records in some bin, so the five-record target is impossible. The Laplace histogram in the Laplace and Gaussian mechanisms has error 16.79, about 2.6 times the lower bound. At $\delta = 10^{-6}$, the bound is below about $\ln(10^6) = 13.82$ records. A stability-based histogram that releases only non-empty bins has error independent of N (Beyond global sensitivity). For a large, mostly empty universe, approximate DP may therefore provide better utility.

The same bound gives a minimum dataset size: dividing count error by the record count n gives frequency error. A pure-DP histogram with all frequencies within α requires

$$n \geq \frac{\ln\left(N(1-\beta)\right) - \varepsilon}{2\alpha\varepsilon}. \tag{22.2}$$

If the data are sampled from a population, sampling error also requires records: about $\ln(2N/\beta)/(2\alpha^2)$ by Hoeffding's inequality in the full edition. Use the larger of the privacy and sampling requirements. At $\varepsilon = 1$ and 95% probability, a million cells with error 0.01 require at least 638 records for privacy but 87,522 for sampling. Sampling, not privacy, therefore limits the accuracy; no mechanism can remove that limit.

When one person affects many statistics, the cost increases. Under pure DP, estimating means of d yes/no attributes requires a number of records proportional to d. Under approximate DP, the **fingerprinting** bound requires about $\sqrt{d\ln(1/\delta)}/(\alpha\varepsilon)$ records. The Gaussian mechanism is within a factor $\sqrt{\ln d}$ of this bound. Use approximate DP for high-dimensional outputs such as gradients. Once Gaussian noise is calibrated, adding records is more effective than seeking a substantially better mechanism. For about n counting queries on n records, the reconstruction attack of Dinur and Nissim gives a lower bound of order $\sqrt{n}$ error per answer (Reconstruction attacks).

22.4. What goes wrong

Aggregates do not prevent membership inference. The attack underlying the fingerprinting bound (Homer et al. 2008) identified participants in a genome study from published frequencies of genetic variants. It scores how much released means shift toward a target's record and away from population means, attribute by attribute. Members tend to score higher because their records affect the means. For the means of 50 people over 2,000 attributes, the attack reaches true positive rate (TPR) 0.242 at false positive rate (FPR) 1%. A $(1, 10^{-5})$-DP Gaussian mechanism reduces TPR to 0.014.

Use this score to audit releases of many averages, estimating population means from a separate sample. Compare TPR at low FPR with the cap $\text{TPR} \leq e^{\varepsilon}\,\text{FPR} + \delta$ from the definition. An audit that exceeds the cap indicates a mechanism or accounting error. Passing does not establish privacy because an attack provides only a lower bound on ε (Auditing private training).

Each approximate-DP bound assumes a small δ, such as the book's default (How small δ must be). A mechanism that publishes each record with probability δ is $(0, \delta)$-DP; if δ is near $1/n$, it can publish about one record in full.

Derivations. Every result here is derived step by step in the full edition's chapter.

23. Private mean estimation

23.1. Why this matters

Private machine learning uses means extensively: each private gradient-descent step averages clipped per-example gradients, and a survey share is the mean of binary answers. Before releasing a mean, determine the required sample size and choose a clipping range without inspecting private data.

Differential privacy (DP) must hold for every dataset, including datasets excluded by modelling assumptions (The definition of differential privacy). An assumption such as normally distributed records may improve accuracy, but does not enter the privacy proof. The mechanisms below therefore clip each record to a range. Noise increases with range width, but the range cannot be estimated directly from private data. Spend part of the budget to estimate the range privately, then clip to it.

23.2. Bounded data: privacy adds a lower-order term

This chapter assumes fixed-size datasets of size n, with neighbours differing in one replaced record. It measures estimation error by **root-mean-square error** (RMSE), the typical distance between the estimate and the population mean.

Suppose records lie in a known interval of width w, such as binary answers coded 0 and 1, and let σ^2 be their variance. Release the sample mean plus Laplace noise with scale $w/(n\varepsilon)$. Since one record changes the mean by at most w/n, this gives ε-DP (The Laplace and Gaussian mechanisms). Laplace noise has variance twice the squared scale and is independent of sampling error, so the variances add:

$$\mathrm{RMSE} = \sqrt{\frac{\sigma^2}{n} + \frac{2w^2}{n^2\varepsilon^2}}. \tag{23.1}$$

To plan a study, set RMSE to target error α and solve for n, as in the Code section. The required record count is at most

$$\frac{\sigma^2}{\alpha^2} + \frac{\sqrt{2}\,w}{\alpha\varepsilon}. \tag{23.2}$$

Sampling alone requires about σ^2/α^2 records; privacy adds at most about $w/(\alpha\varepsilon)$. For a binary survey with half the population answering yes, an RMSE of 0.05 requires 100 people, increasing to 108 at $\varepsilon = 1$. The privacy term grows as $1/\alpha$, compared with $1/\alpha^2$ for sampling, so it matters most for coarse targets and small ε. It also increases with interval width w; subsequent sections reduce this width.

Libraries require a range. In diffprivlib, `tools.mean(x, epsilon=..., bounds=(lower, upper))` clips values to the bounds and adds noise. Without `bounds`, it uses the observed minimum and maximum, which leaks them, and raises `PrivacyLeakWarning`.

Bounds that are too narrow clip valid records. The script practitioner_adult_statistics.py estimates mean weekly work hours in Adult, a census dataset with 48,842 people, at $\varepsilon = 1$ using both loose and tight bounds.

```
hours = adult["hours-per-week"].to_numpy()
loose = tools.mean(hours, epsilon=1.0, bounds=(1, 99), random_state=seed)
tight = tools.mean(hours, epsilon=1.0, bounds=(20, 60), random_state=seed)
clipped = np.clip(hours, 20, 60).mean()  # what the tight bounds can at best recover
print(f"true mean hours: {hours.mean():.3f}")
print(f"bounds (1, 99): {loose:.3f}  bounds (20, 60): {tight:.3f}")
print(f"bias of (20, 60): {clipped - hours.mean():+.3f}")
```

The seed-0 run, `uv run python code/part03-dp-foundations/practitioner_adult_statistics.py`, prints the following; the final two values come from code adjacent to the excerpt.

```
true mean hours: 40.422
bounds (1, 99): 40.421  bounds (20, 60): 40.361
bias of (20, 60): -0.061
people outside (20, 60): 8.7%
noise scale (1, 99): 0.0020  (20, 60): 0.0008
```

Tight bounds reduce the noise scale from 0.0020 to 0.0008 hours but clip 8.7% of people, shifting the mean by 0.061 hours—more than the noise reduction. Privacy budget cannot remove this bias. Report sampling error, clipping bias and noise separately. Choose bounds from the schema or public data, and measure the clipped share only on data available for inspection.

These formulas assume public n, which holds for replace-one-record neighbours. If dataset size is private, release a noisy sum and noisy count, split the budget between them, then divide.

23.3. The cost of an unknown range

Now suppose records are normal with variance 1 and their mean is known only to lie within distance R of zero. Let τ be a radius that contains all records except with failure probability β. For 1,000 records and $\beta = 0.05$, τ is 4.604.

The **naive estimator** clips records to the full possible range, from $-(R + \tau)$ to $R + \tau$, then adds Laplace noise calibrated to that width. Its error therefore grows linearly with R. With 1,000 records at $\varepsilon = 1$, RMSE is 2828.44 when R is one million. Lower bounds shows that an unknown range should require only about $\ln R/\varepsilon$ additional records.

Coarse then fine (Karwa & Vadhan 2018) approaches this bound in two steps. The coarse step partitions the range into bins of width 2τ and uses budget ε_1 with the exponential mechanism to select the fullest bin. Its record requirement grows logarithmically with the bin count. Because the data may span two bins, the fine step clips records to the selected bin and its two neighbours,

then releases the mean with Laplace noise using the remaining budget. By composition, the combined procedure is ε-DP.

Allocating one quarter of the budget to the coarse step gives RMSE 0.0609 for the same 1,000 records across R from 10 to one million. The fine step uses the remaining three quarters and a three-bin window, so the naive estimator performs better for small ranges, up to $R = 13.81$. For larger ranges, use coarse then fine or estimate the range with a private quantile such as diffprivlib's `tools.quantile`. Under approximate DP with slack δ, a stability-based histogram performs the coarse step with about $\ln(1/\delta)/\varepsilon$ records, eliminating dependence on R (Beyond global sensitivity).

Two failures are possible. Choosing the budget split, bin width or β after inspecting private data leaks information. With too few records, the coarse step may select an empty bin and clip every record incorrectly.

23.4. Many dimensions: clip the norm

For d-dimensional vectors, such as embeddings or gradients, measure size with the ℓ_2 norm. Clip each record to radius C, so one replacement changes the mean by at most $2C/n$. Add Gaussian noise calibrated to this sensitivity (The Laplace and Gaussian mechanisms). Private gradient descent uses this procedure; private training explains how to select its clipping norm.

The clipping radius determines the cost. With unit variance per coordinate, records lie about $\sqrt{d}$ from their mean, so the radius must scale as $\sqrt{d}$. Total noise across coordinates then grows as d, compared with sampling error of order $\sqrt{d}$. Privacy therefore dominates in high dimensions, as the fingerprinting lower bound confirms.

If the mean's location is unknown, a grid of bins incurs a range cost in every dimension. CoinPress's **shrinking balls** method (Biswas, Dong, Kamath & Ullman 2020) avoids a grid. Each round clips to a ball centred on a noisy mean estimate and reduces the radius. Fix the number of rounds and budget allocation before using private data.

23.5. Heavy tails: clip into the data

Incomes, session lengths and per-example gradient norms can have heavy-tailed distributions. If only the variance bound σ^2 is known, a range that clips no records may be too wide for useful privacy. Clipping can reduce noise, but introduces bias.

Let τ be the clipping half-width. A coarse step estimates a centre near the mean, then each record is clipped to within τ of it. If the centre equals the mean, the bias is at most σ^2/τ, since only records beyond τ change. Noise grows in proportion to τ. The resulting error bound is minimized at

$$\tau_b = \sigma \sqrt{\frac{n\varepsilon}{2\sqrt{2}}}, \tag{23.3}$$

This width depends only on n, ε and the variance bound. At this width, error α requires about $\sigma^2/\alpha^2 + 8\sqrt{2}\,\sigma^2/(\alpha^2\varepsilon)$ records. Both terms scale as $1/\alpha^2$, so privacy can cost as much as sampling.

Table 23.1 shows this trade-off for Pareto records scaled to variance 1. At the first width, 0.05 of records are clipped and noise dominates the error. The worst-case width τ_b reduces error to 0.0607; a width tuned to this distribution reduces it further.

Table 23.1.: Clipped private means of Pareto records with variance 1, 1,000 records, $\varepsilon = 1$, for three clipping half-widths.

Clipping half-width τ	Bias	Sampling sd	Noise sd	RMSE
141.42 (no record clipped)	0.0000	0.0313	0.4000	0.4012
18.80 (worst-case best, τ_b)	−0.0018	0.0293	0.0532	0.0607
5.36 (best for this distribution)	−0.0153	0.0249	0.0152	0.0329

Use τ_b as a conservative default rather than an optimal width. A narrower range may improve accuracy, but choose it from domain knowledge or a private quantile, not by inspecting the records (Beyond global sensitivity).

23.6. Code

The snippet computes the records that a bounded mean needs, checks its noise with OpenDP, and computes the default clipping width for heavy tails. The RMSE equal to α, times n^2, is a quadratic in n, and the first function rounds its positive root up.

```
from math import ceil, sqrt

import opendp.prelude as dp


def records_for_bounded_mean(alpha: float, sd: float, width: float, eps: float) -> int:
    """Smallest n with sd^2/n + 2 (width/(n eps))^2 <= alpha^2."""
    a, c = sd**2, 2 * (width / eps) ** 2
    return ceil((a + sqrt(a * a + 4 * c * alpha**2)) / (2 * alpha**2))


def heavy_tail_width(n: int, eps: float, sd: float) -> float:
    """The clipping half-width that balances the bias bound sd^2/tau against the
     ↪ noise."""
    return sd * sqrt(n * eps / (2 * sqrt(2)))


n = records_for_bounded_mean(0.05, 0.5, 1.0, 1.0)
print(n)  # 108
print(f"{dp.laplacian_scale_to_accuracy(1.0 / (n * 1.0), alpha=0.05):.4f}")  # 0.0277
print(f"{heavy_tail_width(1000, 1.0, 1.0):.2f}")  # 18.80
```

OpenDP gives accuracy as a half-width that holds the noise with a chosen probability. So `laplacian_scale_to_accuracy` changes the noise scale $w/(n\varepsilon)$ of a range of width w at the planned 108 records into the half-width at 95%.

23.7. Cheat sheet

Table 23.2.: Private mean estimation at a glance, with $\tau_b = \sigma\sqrt{n\varepsilon/(2\sqrt{2})}$.

What you know	Mechanism	Records for error α (up to log factors)
Bounded, width w, variance σ^2	Mean plus Lap$(w/(n\varepsilon))$	$\sigma^2/\alpha^2 + \sqrt{2}\,w/(\alpha\varepsilon)$
Normal, mean within R of zero	Coarse histogram, then clip to three bins	$1/\alpha^2 + 1/(\alpha\varepsilon) + \ln R/\varepsilon$
Same, approximate DP	Stability-based coarse step	$1/\alpha^2 + 1/(\alpha\varepsilon) + \ln(1/\delta)/\varepsilon$, no R
d dimensions, mean length at most R	Clip the norm, Gaussian noise; shrinking balls to find the mean	$d/\alpha^2 + d\sqrt{\ln(1/\delta)}/(\alpha\varepsilon) + \sqrt{d}\ln R/\varepsilon$
Variance at most σ^2 only	Clip to within τ_b of the centre	$\sigma^2/\alpha^2 + 8\sqrt{2}\,\sigma^2/(\alpha^2\varepsilon)$

Rules of thumb

- Never clip to the observed minimum and maximum. Take bounds from the schema or public data, or spend a quarter of the budget or less, a simple default, finding the data.
- Bounds that clip real records bias the mean whatever ε is, so put the clipping bias into every reported interval.

23.8. References

- Karwa & Vadhan, "Finite Sample Differentially Private Confidence Intervals" (ITCS 2018): coarse then fine for normal means and variances, under pure and approximate DP.
- Biswas, Dong, Kamath & Ullman, "CoinPress: Practical Private Mean and Covariance Estimation" (NeurIPS 2020): shrinking balls in practice, with code.

Derivations. Every result here is derived step by step in the full edition's chapter.

24. Release statistics with a library

24.1. Why this matters

An online shop wants a dashboard of purchase lines, revenue and typical sale, overall and by country, plus customer counts by product. Because these statistics describe customers, their release needs a differential privacy (DP) guarantee. A customer may contribute many rows, so a row-level guarantee would protect a purchase rather than the customer. This chapter uses OpenDP to build the release, define its privacy unit and state its guarantee.

The data are Online Retail from the University of California, Irvine (UCI) Machine Learning Repository: 541,909 invoice lines from a British online shop. Each line records a product, quantity, unit price, customer and country. The script practitioner_release_statistics.py runs every step in less than a minute with OpenDP 0.16.0 and Polars 1.36.1.

```
uv run python code/part03-dp-foundations/practitioner_release_statistics.py
```

OpenDP uses an unseeded cryptographic generator, so released values vary between runs. This page quotes deterministic scales, error bars and thresholds, and marks one-run releases explicitly.

24.2. Choose the library

Choose a library that can bound the contribution of a person with many rows (Table 24.1).

Table 24.1.: Libraries for releasing statistics, and what each can express.

Library	Runs on	Person with many rows	Groups not known in advance	Use it for
OpenDP	Polars, one machine	Yes, with an identifier column	Yes, by a noisy threshold	Statistics on one machine; the default here
diffprivlib	NumPy, scikit-learn	No, one row per person	No	Quick experiments and simple models
PipelineDP	Apache Beam, Spark	Yes, built in	Yes, built in	Pipelines that already run on Beam or Spark

Tumult Analytics	Spark	Yes, with an identifier column	Yes, in a separate query	Large releases on Spark

PipelineDP and Tumult were not run; their entries are based on their documentation. Do not implement a mechanism yourself. Use one accountant to track the total budget for the release.

24.3. Decide the unit and the budget

The **unit of privacy** is a customer. Neighbouring datasets differ by one customer and all their lines, added or removed (The definition of differential privacy). The script removes returns, zero-price lines and lines without a customer identifier. Each filter uses only the line itself, so adding or removing a customer affects only that customer's lines.

```
raw = practitioner_datasets.online_retail()
lines, stats = clean(raw)
n_customers = stats.customers
delta = delta_for(n_customers)  # 1/n^1.1, rounded down to a power of ten
per_line = per_customer(lines)
print(f"customers: {n_customers:,}  purchase lines: {lines.height:,}")
print(f"delta: {delta:.0e}")
p50, p90, p99, top = quantiles(per_line, [0.5, 0.9, 0.99, 1.0])
print(f"lines per customer: p50={p50} p90={p90} p99={p99} max={top}")
```

```
customers: 4,372  purchase lines: 397,884
delta: 1e-05
lines per customer: p50=41 p90=208 p99=662 max=7847
```

The slack δ follows the book's default for n people (How small δ must be), giving 10^{-5}. OpenDP warns that δ should be at most 10^{-6}, while this book sets it from the population size. The entire release has privacy level $\varepsilon = 1$: all tables describe the same customers, and their costs compose (Post-processing, group privacy and composition).

Under a line-level guarantee, the largest customer represents 7,847 records, and group privacy multiplies ε by the group size. This maximum is reported only because the dataset is public.

24.4. Bound each customer's contribution

Let k be the maximum number of lines retained per customer in each query. A smaller k reduces noise but discards more data; a larger k retains more data but requires more noise. The script measures this trade-off on the public dataset.

```
k=10: lines kept 10.2%  customers truncated 84.8%
k=25: lines kept 22.4%  customers truncated 64.6%
k=50: lines kept 36.9%  customers truncated 44.0%
k=100: lines kept 55.1%  customers truncated 24.9%
k=200: lines kept 72.7%  customers truncated 10.4%
k=500: lines kept 88.2%  customers truncated 1.8%
```

Choose k from a high percentile in public or historical data, never from the release data. If neither is available, spend a small part of the budget on a private high quantile over candidates fixed in advance (Selection). This chapter uses $k = 100$, below the public-data 90th percentile of 208 because noise grows with k. It retains 55.1% of lines and truncates about a quarter of customers.

Keeping the first k lines by date would bias statistics toward early purchases. The script instead samples k lines per customer. The filter below matches OpenDP's `truncate_per_group(k, keep="sample")` while using a seed to keep validation values reproducible.

```
def sample_per_customer(k: int, seed: int, *by: str) -> pl.Expr:
    """True for at most k randomly chosen lines of each customer (within each value of
     ↪  `by`)."""
    return pl.int_range(pl.len()).shuffle(seed=seed).over("CustomerID", *by) < k
```

Bound each line value, defined as quantity times unit price. The script clips values to £0–£200. The lower bound follows from removing returns; the business sets the upper bound, which clips 1.02% of lines. Thus one customer can affect revenue by at most 100 lines of £200 each. Set value bounds from the domain, not the observed minimum or maximum (Beyond global sensitivity).

A grouped query may require two further bounds: L_0, the number of groups one customer may affect, and L_∞, the number of lines they may contribute to one group. Country counts need neither because a customer's k lines change the total counts by at most k, already covered by the noise. Product partition selection requires both (Section 24.8).

24.5. Set up the context

OpenDP's **context** stores the data, privacy unit, total budget and allocation across queries. Contexts require `dp.enable_features("contrib")`, which enables components that are not formally vetted. Reviewers who require vetted code must check each call against OpenDP's documentation.

```
context = dp.Context.compositor(
    data=lines.lazy(),
    privacy_unit=dp.unit_of(contributions=1, identifier="CustomerID"),
    privacy_loss=dp.loss_of(rho=rho, delta=delta / 2),  # Gaussian noise; delta/2 for
 ↪  thresholds
    split_by_weights=WEIGHTS,  # scalars, countries, products
    margins=[dp.polars.Margin(by=[], max_length=MAX_LINES)],
)
```

The identifier column defines each customer as one privacy unit. Thus `contributions=1` represents adding or removing one customer with all their lines; truncation bounds those lines. For replace-one neighbours, OpenDP instead takes `changes=`. The budget uses **zero-concentrated differential privacy (zCDP)** (Concentrated differential privacy), represented by ρ. Add ρ across queries, then convert the total to (ε, δ).

With a zCDP budget, the context adds Gaussian noise to each count and sum. This supports product counts, where one customer may affect many groups (The Laplace and Gaussian mechanisms). The script's `rho_for` selects $\rho = 0.0284$, the largest value OpenDP converts to $\varepsilon \leq 1$ using half of δ. The remaining half pays for the threshold in Section 24.8. The margin supplies a public upper bound on line counts, which OpenDP needs before summing floating-point values.

The weights allocate one quarter of the budget to overall statistics, one quarter to countries and one half to products, which need the most. The script prints this allocation and converts the total budget back to the (ε, δ) stated in the guarantee.

```
budget scalars: rho=0.0071 delta=1.25e-06
budget countries: rho=0.0071 delta=1.25e-06
budget products: rho=0.0142 delta=2.50e-06
whole release as (epsilon, delta): (1.0000, 1e-05)
```

OpenDP assigns a query its budget share when the query is built and spends that share on release. Each new query receives the next unspent share. Building all queries before releasing them can therefore assign each the first share; the product query would receive one quarter's noise while costing half the budget. Build, inspect and release each query in sequence, as the script does.

24.6. Release overall statistics with error bars

The first query releases line count, revenue and median line value. Because the context stores the raw lines, each query applies its own truncation.

```
value = clipped_value()  # the line value, clipped to [low, high]
scalars = (
    context.query()
    .filter(sample_per_customer(K, seed))  # at most K lines per customer
    .select(
        dp.len(signed=True),
        value.dp.sum((low, high)).alias("revenue"),
        value.dp.median(candidates=MEDIAN_CANDIDATES).alias("median"),
    )
)
scalar_summary = scalars.summarize(alpha=0.05)
scalars_hat = scalars.release().collect()
```

`signed=True` retains negative noisy counts. Before release, `summarize` reports each statistic's noise scale and 95% error bar.

```
scalars len: Frame Length, Integer Gaussian, scale=1,453.38 95% accuracy=2,850.0
scalars revenue: Sum, Float Gaussian, scale=290,804.33 95% accuracy=569,966.0
scalars median: 0.5-Quantile, GumbelMin, scale=1,453.38 95% accuracy=none
```

With probability 0.95, the released line count is within 2,850 of its true value and revenue is within £569,966. These error bars depend only on bounds and budget, so inspect them before release to assess whether the budget is sufficient.

The output also shows how OpenDP divides a query's budget among its statistics. Each statistic receives the same noise scale relative to its sensitivity. The count and median therefore have scale 1,453.38, while revenue, with 200 times the count's sensitivity, has about 200 times that scale. To allocate more budget to one statistic, place it in a separate weighted query. `GumbelMin`, an implementation of the exponential mechanism, selects the median from candidates spaced £0.50 apart. Its error is measured in ranks, so `summarize` provides no error bar (Selection).

The error bars cover noise, not bias from truncation and clipping, which together retain only 48.5% of revenue. The private statistic therefore estimates a truncated total, not the shop's full takings. State this in the release. If coverage matters more than noise, increase k.

24.7. Group by a public key list

The next query reports lines and revenue by country. Its key list must be fixed in advance because the distinct `Country` values reveal which countries occur in private data, and noise does not protect that list. The script uses 38 keys: the United Kingdom, 27 EU member states, four European Free Trade Association states, the United States, Canada, Australia, Japan, China and "Other". A mapping standardizes three spellings, including `EIRE` to Ireland, and maps all remaining values to Other.

```
by_country = (
    context.query()
    .filter(sample_per_customer(K, seed))
    .group_by("Market")
    .agg(dp.len(signed=True), value.dp.sum((low, high)).alias("revenue"))
    .with_keys(pl.LazyFrame({"Market": MARKETS}))  # every key is released, zeros
 ↪  included
)
country_summary = by_country.summarize(alpha=0.05)
countries_hat = by_country.release().collect()
```

```
per country len: Frame Length, Integer Gaussian, scale=1,185.25 95% accuracy=2,324.0
per country revenue: Sum, Float Gaussian, scale=237,698.48 95% accuracy=465,880.5
```

The query releases every key, including 11 countries with no purchase lines; their counts contain only noise. Thus, the key list does not disclose which markets have customers. Each count has a 95% error bar of 2,324, and each revenue an error bar of about £466,000. Germany's true count of 5,379 is informative, but its revenue of £131,902 is smaller than the error bar. Publish each cell's error bar or combine small markets into Other before release.

24.8. Group by unknown keys with partition selection

The product list is private: a row for a product would reveal that at least one customer bought it. The release therefore uses **partition selection** to select products privately. It publishes only products whose noisy counts exceed a threshold, making products with few customers unlikely to appear. The threshold incurs δ (Beyond global sensitivity).

To count customers rather than lines, the query first retains one line per customer-product pair, setting L_∞, the lines per customer in one group, to 1. It then retains at most m products per customer, setting L_0, the groups per customer, to m. The script evaluates this trade-off on public data.

```
m=1: pairs kept 1.6%  customers truncated 97.9%  scale=5.93  threshold=29  products
 ↪  above it: 1
m=5: pairs kept 7.9%  customers truncated 92.4%  scale=13.27  threshold=66  products
 ↪  above it: 9
m=10: pairs kept 15.1%  customers truncated 83.6%  scale=18.77  threshold=96  products
 ↪  above it: 19
m=20: pairs kept 27.5%  customers truncated 68.3%  scale=26.54  threshold=138  products
 ↪  above it: 30
m=50: pairs kept 52.7%  customers truncated 38.6%  scale=41.96  threshold=225  products
 ↪  above it: 50
```

The threshold rises from 29 to 225 as m increases because each customer can affect more groups. Product counts rise faster: the number of products whose true count exceeds the threshold grows from 1 to 50. The script uses $m = 50$ (`M` in the code), selected using public data.

```
by_product = (
    context.query()
    .filter(sample_per_customer(1, seed, "StockCode"))  # one line per customer and
 ↪  product
    .filter(sample_per_customer(M, seed))  # at most M products per customer
    .group_by("StockCode")
    .agg(dp.len(signed=True).alias("customers"))
)
product_summary = by_product.summarize(alpha=0.05)
products_hat = by_product.release().collect()
```

```
per product customers: Frame Length, Integer Gaussian, scale=41.96 95% accuracy=83.0
 ↪  threshold=225
```

The release shows a product only when its noisy count is at least 225; released counts have a 95% error bar of 83. Do not publish the number or identity of omitted products because both depend on private data. Positive noise can include a product just below the threshold. Treat counts near 225 as evidence of possible popularity, not as precise measurements.

In OpenDP 0.16.0, `truncate_num_groups` is intended to bound groups per customer but does not work. With default `keep="sample"`, `summarize` accepts the query, but `release` fails after consuming its budget share. Test each query end to end on public data first. With `keep="first"`,

the function retains one fewer group than requested and biases counts. The filters above avoid both failures only because $L_\infty = 1$ makes groups equivalent to lines. This book knows of no unbiased group bound for $L_\infty > 1$ in this version.

24.9. Check the release before publishing

To make sure that the code does what the plan says, the script compares the private values with the true values after the same truncation and clipping.

```
true lines after truncation: 219,291 (all purchase lines: 397,884)
true revenue after truncation and clipping: 4,318,947 (all lines, unclipped: 8,911,408)
true median line value after truncation: 15.00 (all lines: 11.80)
```

The release is random, and one run printed the following.

```
released (one run):
  lines 218,972  revenue 4,397,410  median 14.50
  United Kingdom: lines 196,311  revenue 3,927,959
  Germany: lines 7,151  revenue 643,094
  Spain: lines -546  revenue -37,048
  country line counts within their 95% accuracy: 34 of 38
  products released: 63
```

Compare it with the error bars, which must hold about 95% of the market counts. Negative values are noise on small markets, and you can round them up to zero, because post-processing costs no budget. Expect more than the 50 products whose true count is above the threshold, for the reason above. Compare the private median with the truncated £15.00, not with the £11.80 of all lines.

The check should fail when a value lies far outside its error bar, indicating incorrect sensitivity, or when substantially fewer or more than 95% of cells fall inside. It should also fail if a released product's true count is far below the threshold. In production, do not use true values to decide what to publish; that decision would itself release private information. Run these checks during development on historical or public data, then release real data once with the validated code.

24.10. Write the guarantee down

The script constructs the privacy statement from the variables that define bounds and budget, keeping the statement aligned with the implementation.

```
This release satisfies (epsilon = 1, delta = 1e-05)-differential privacy for adding or
removing one customer and all their invoice lines, with at most 100 lines per customer
(sampled at random), at most 50 random products per customer in the product counts, and
line values clipped to [0, 200] pounds, under the central model, using OpenDP 0.16.0
 ↪  with
Gaussian noise (accounted as 0.0284-zCDP plus 5e-06 for thresholds, converted to
 ↪  epsilon
```

```
and delta). The market list was fixed in advance; the product list is not public.
```

The statement gives reviewers the information needed to recompute the guarantee (Writing the guarantee down). Also state what it excludes. Any further release about the same customers adds to the total budget. For a pure ε-DP release, add the two (ε, δ) pairs. Alternatively, add its $\rho = \varepsilon^2/2$ to this release's ρ, then convert the total; use the smaller result (Concentrated differential privacy). The dashboard must also state that its figures use truncated data. Release and governance connects this guarantee to data-protection law.

24.11. Checklist

Table 24.2.: The release checklist: each step, its default and its check.

Step	Default	Check
Unit and budget	One person, add/remove; one ε; $\delta \leq 1/n^{1.1}$	n counts people, not rows
Contribution bound	k from a high percentile of public or historical data; random sample	No bound read from the release data
Value bounds	From the domain or the business	Share clipped and bias stated
Context	zCDP with Gaussian noise; weights by need	ρ converted to (ε, δ) equals the claim
Queries	Build, summarize and release one at a time	Scales match the plan; each query run end to end on public data
Public keys	Fixed in advance, values mapped by a rule; every key released	No key list built from the data
Unknown keys	Partition selection with L_0 and L_∞	Number dropped never published
Check	On historical or public data	About 95% inside their error bars
Statement	Printed by the releasing code	Unit, bounds, version and public information named

24.12. References

- The OpenDP documentation, "Polars" and "Grouping" in its user guide: contexts, margins and partition selection.
- Wilson et al., "Differentially Private SQL with Bounded User Contribution" (PETS 2020): contribution bounding in a production SQL engine.

- Desfontaines, Voss, Gipson & Mandayam, "Differentially private partition selection" (PETS 2022): the threshold and why it needs δ.

Derivations. This workflow has no counterpart in the full edition. The results it uses are derived in the full edition's chapters on the definition in the full edition, its properties in the full edition, the Laplace and Gaussian mechanisms in the full edition, selection in the full edition and sensitivity in the full edition.

Part IV.

Privacy accounting

Stochastic gradient descent (SGD) trains a model through repeated updates. Differential privacy (DP) adds noise to each update; this method is DP-SGD. An **accountant** tracks each step's privacy cost and converts the total to a guarantee (ε, δ) for the full run. Part 3's composition theorems can give loose bounds.

In the running example, each of 10,000 DP-SGD steps samples records with probability 0.01 and adds noise with multiplier 1. At $\delta = 10^{-5}$, advanced composition gives $\varepsilon = 24{,}379.55$. Google's privacy-loss-distribution accountant in `dp_accounting` gives 6.19. Report the accountant, sampler and neighbouring relation with each result; all three affect the bound.

Start with choosing an accountant, which matches accounting assumptions to an implementation, and calibrating a DP-SGD run, which selects noise for Part 5's Adult-data training run. The full chapters on amplification by subsampling and Rényi differential privacy explain the two main tools: random sampling can reduce privacy cost, and Rényi differential privacy (RDP) provides a curve that many libraries track.

The reference pages cover RDP conversion, numerical accounting and Gaussian differential privacy, which interprets guarantees as limits on attacks. They also cover adaptive composition for budgets chosen during training, concentrated differential privacy for analytics ledgers and shuffling and iteration for other amplification methods. Code using Opacus and `dp_accounting` produces every number. By the end, you can calibrate a DP-SGD run, account for it with two libraries and report its privacy parameters.

25. Choosing an accountant

25.1. Why this matters

Report a model protected by differential privacy (DP), with privacy level ε and slack δ. The code computing ε is an **accountant**. Different accountants can give very different results for the same run: one reports 6.24, another 24,379.55. Table 25.1 compares advanced composition, Rényi differential privacy (RDP) and privacy loss distribution (PLD) accounting for one stochastic gradient descent (SGD) run with DP. Each record is sampled independently with probability $q = 0.01$, the sampling rate. The noise multiplier σ is the noise standard deviation relative to the clipping norm; here it is 1.

Table 25.1.: One DP-SGD run, $q = 0.01$ and $\sigma = 1$, under three accountants: ε at $\delta = 10^{-5}$. The PLD columns round the privacy loss up and down to a grid of width 10^{-5}, so they bracket the true value.

Steps	Advanced composition	RDP, improved	PLD upper	PLD lower
1,000	1258.64	2.1014	1.8332	1.8232
10,000	24379.55	6.7127	6.2377	6.1377

The true ε lies between the PLD bounds. Advanced composition gives a bound too loose to use here. RDP is tighter but still overstates ε by 14.6% for the shorter run and 7.6% for the longer one. **Report the tightest upper bound whose assumptions match the implementation, with enough detail for independent recomputation.**

25.2. Which accountant

An accountant is tight only when its per-step model matches the mechanism. Choose it accordingly (Table 25.2).

Table 25.2.: Which accountant to report for which mechanism.

Mechanism	Report with	Avoid
Gaussian noise, no subsampling	Gaussian differential privacy (GDP), converted exactly (f-DP and Gaussian DP)	Per-step composition

Mechanism	Report with	Avoid
DP-SGD with Poisson sampling	PLD, or the privacy loss random variable (PRV) accountant (Numerical accounting), with the PLD's lower value as the error bar (its code)	Zero-concentrated DP (zCDP); the GDP central limit theorem (CLT)
Pure-DP steps (Laplace, randomized response)	PLD, exact when the privacy loss takes few values; Laplace with scale b is `LaplaceDpEvent(b/Δ)`, Δ its ℓ_1 sensitivity	Advanced composition
Many statistical releases	A zCDP ledger (Concentrated DP) or PLD	Adding up each release's ε
Noise or budget chosen during the run	A Rényi filter (Adaptive composition)	Plugging realized values into a fixed theorem
Correlated noise across steps: matrix factorization, or the follow the regularized leader (FTRL) optimizer on tree sums, DP-FTRL	The mechanism's own accountant (Matrix mechanisms)	The Poisson DP-SGD accountant
A shuffler of locally randomized reports, or only the last model released	Shuffling and iteration, with its conditions; a loader that shuffles is a sampler (Table 25.3)	The Poisson accountant for a shuffler; a last-model claim for non-convex training

The central-limit approximation gives 1.6177 for 1,000 steps, but it is not a bound. This falls below the PLD's certified lower bound of 1.8232, so it claims a guarantee the run does not satisfy. Opacus's `"gdp"` accountant uses this approximation; do not report it. Opacus's default PRV accountant is safe.

25.3. The inverse problem: noise for a target ε

Usually, fix target privacy level ε, batch size and epoch count, then solve for the smallest acceptable noise multiplier σ. A looser accountant requires more noise to meet the same target. For a target $\varepsilon = 3$, RDP requires noise multiplier 1.662 on the 10,000-step run. PLD requires 1.584. Calibrate with the accountant used for reporting. Opacus's `get_noise_multiplier` defaults to `"rdp"`, so specify `accountant="prv"` when needed (Calibrate and account a DP-SGD run).

25.4. Choosing δ

Let n denote the number of people in the protected unit. For user-level privacy, count users rather than records. Set δ using the book's rule: at most $1/n^{1.1}$, rounded down to a power of ten (How small δ must be). The run in Table 25.1 has 10,000 steps. The sampling rate is 0.01, and the dataset has 50,000 records. Together, these settings give 100 epochs. The rule sets $\delta = 10^{-6}$, so the table's 10^{-5} is too large. The US National Institute of Standards and Technology (NIST) recommends the stricter $1/n^2$; also report ε at that value.

Publish the named accountant's upper bound, rounded up; it does not certify the lower bound. Write the statement gives a complete privacy statement.

25.5. Matching the accountant to the code

An accountant's result is valid only if the code satisfies its assumptions. Many papers reported Poisson-sampling bounds but trained with fixed-size shuffled batches, a different algorithm (Ponomareva et al. 2023). Check the implementation against Table 25.3 before reporting.

Table 25.3.: A checklist for matching the accountant to the training code.

Item	The accountant assumes	Check in the code	If it does not match
Sampler	Poisson sampling at rate q	Do batch sizes vary from step to step?	The sampler's row of Subsampling: fixed-size batches at $\sigma/2$; shuffle and partition unamplified, for zero-out neighbours
Neighbours	Add or remove one record	Which relation does the claim state?	Replace-one costs more per step; for Poisson sampling, compose the same steps in `PLDAccountant(NeighboringRelati`
Unit	One record	Can one person give several records?	Clip and sample per user, or pay for the group (User-level DP)

Item	The accountant assumes	Check in the code	If it does not match
Tuning, adaptivity and steps	Settings and steps fixed in advance	Tuning on private data, early stopping, restarts?	Account for the tuning runs (Hyperparameter tuning); report the planned maximum; keep the accountant's state across restarts
Other data	Only the training rows are private	Pretraining, fitted preprocessing, validation rows?	Name them; preprocessing fitted on private rows needs its own budget

The sampler has the largest effect. At $\delta = 10^{-5}$, `dp_accounting`'s PLD gives 6.19 for Poisson sampling and 43.37 for fixed-size batches. Shuffle and partition give 91.82. The last bound applies only to zero-out neighbours (Subsampling).

Use the same checklist when reviewing another claim; Review someone else's claim gives the review order.

25.6. Checking an accountant

Test accountants against a known exact case. The sampling rate q is the probability that a record enters a batch; at $q = 1$, every record enters. Each step is then a full Gaussian mechanism, whose composition is exact (f-DP and Gaussian DP). Ten steps at noise multiplier 1 are $(17.8566, 10^{-5})$-DP. An upper bound must be at least 17.8566, and a PLD bracket must contain it. Also verify that ε does not decrease with more steps or increase with more noise.

Then compare two maintained libraries at pinned versions. This snippet recomputes Table 25.1 with Google's `dp_accounting`.

```
import dp_accounting
from dp_accounting.pld import PLDAccountant
from dp_accounting.rdp import RdpAccountant

step = dp_accounting.PoissonSampledDpEvent(0.01, dp_accounting.GaussianDpEvent(1.0))
run = dp_accounting.SelfComposedDpEvent(step, 10_000)  # q = 0.01, sigma = 1, 10,000
 ↪  steps
pld = PLDAccountant(value_discretization_interval=1e-4).compose(run)
rdp = RdpAccountant(orders=list(range(2, 65))).compose(run)  # integer orders only
print(f"{pld.get_epsilon(1e-5):.4f} {rdp.get_epsilon(1e-5):.4f}")  # 6.1877 6.7194
print(f"{pld.get_epsilon(1e-6):.4f} {rdp.get_epsilon(1e-6):.4f}")  # 6.9074 7.4869
```

Each PLD result falls inside the table's bracket. RDP results are slightly higher because the table also evaluates fractional orders. When libraries disagree, compare their assumptions and calculations. RDP differences can result from using different orders; at order 10, that accounts

for 0.3612 (RDP conversions). Non-overlapping PLD brackets may indicate a library bug. A factor-of-two difference often indicates different neighbouring relations.

25.7. Cheat sheet

Table 25.4.: Choosing and checking an accountant.

Question	Default	Check
Accountant for DP-SGD	PLD or PRV, upper value	RDP lies above it; never report the CLT
δ	The rule of How small δ must be, n counted in the claim's unit	Also report ε at $1/n^2$
Fixed-size batches	Poisson accountant at $\sigma/2$ (add/remove); replace-one: see Subsampling	Poisson batch sizes vary from step to step
Shuffle and partition	One Gaussian step per epoch, zero-out neighbours	Better, switch to Opacus's Poisson loader

25.8. References

- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): matching the accounting to the pipeline, and what to report.
- Near, Darais, Lefkovitz & Howarth, "Guidelines for Evaluating Differential Privacy Guarantees" (NIST SP 800-226, 2025): what an evaluator will ask of your claim.

Derivations. Every result here is derived step by step in the full edition's chapter.

26. Calibrate and account a DP-SGD run

26.1. Why this matters

An **accountant** maps settings for stochastic gradient descent (SGD) with differential privacy (DP) to privacy level ε. Before training, it finds the noise needed for a target; afterward, it reports the run's ε. Both calculations can be wrong without an obvious training signal. Calibration and reporting may use different accountants, the loader may use a different sampler than assumed, or a restart may reset the step count. Any mismatch invalidates the claim without changing the loss curve.

This chapter accounts for DP-SGD on Adult from the UCI Machine Learning Repository; it does not train a model. The planned run uses 30,000 people, batches of 1,000, ten epochs and target $\varepsilon = 4$. Train a model with DP-SGD performs the training.

The script practitioner_calibrate.py runs these steps in one to two minutes with Opacus 1.6.0 and `dp-accounting` 0.6.0. Defaults match Part 5. Options `--n`, `--batch`, `--epochs` and `--epsilon` configure another run, as shown for MNIST in the second command.

```
uv run python code/part04-privacy-accounting/practitioner_calibrate.py
uv run python code/part04-privacy-accounting/practitioner_calibrate.py --n 60000
 ↪  --batch 256 --epochs 10 --epsilon 3 --dataset MNIST --unit image
```

26.2. Fix the privacy unit, n and δ

The guarantee protects one record, so define the record first. Each Adult row represents one person. If a person could have multiple rows, n must count people, as in User-level differential privacy.

Slack δ bounds the probability of the guarantee's exception. Set it using the book's rule: at most $1/n^{1.1}$, rounded down to a power of ten (How small δ must be).

```
def delta_for(n: int) -> float:
    """The book's default delta: at most 1/n^1.1, rounded down to a power of ten."""
    return 10.0 ** math.floor(math.log10(n**-1.1))
```

```
Adult rows: 48,842; Part 5's training split: n=30,000
delta_for(30,000) = 1e-05
delta_for(60,000) = 1e-06
delta_for(569) = 1e-04
```

Here, n counts people in the training split, not all Adult rows, so δ is 10^{-5}. Set validation rows aside because the run neither trains on nor protects them. The other examples show that δ must be recomputed for each dataset.

26.3. Choose the sampler

The accountants assume **Poisson sampling**: each record independently joins a batch with probability q, so batch sizes vary. Other samplers can cost much more privacy (Section 26.6). For an expected batch size B, the sampling rate is $q = B/n$. A run of E epochs takes E/q steps, rounded down; the output calls this T.

```
Poisson: B=1000, E=10: q=0.033333, T=300
Opacus DPDataLoader.from_data_loader: q=1/30=0.033333, 30 steps an epoch, 300 in 10
 ↪  epochs
chosen run uses the loader's q and steps; get_noise_multiplier at that q counts
 ↪  int(E/q)=300: equal True
```

Training uses the Poisson `DPDataLoader` that Opacus creates from the input loader. Its rate is the reciprocal of the input loader's batch count, $\lceil n/B \rceil$, and it takes that many steps per epoch. Here, B divides n, so both plans agree. Otherwise, the script reads q and T from the loader and checks that Opacus calibration counts the same steps.

This assumption fails if training uses a custom loader with `shuffle=True` and fixed `batch_size`; the logged batches will all have the same size (Amplification by subsampling).

26.4. Plan the noise

Choose batch size, epochs and noise multiplier σ. Training clips each record's gradient to length at most C, the clipping norm, and σ is the standard deviation of the noise as a multiple of C. For a given target and sampler, an accountant solves for the smallest acceptable σ. Calibrate with the accountant you will report (The inverse problem). The script cross-checks two tight accountants (Numerical accounting): Opacus's privacy loss random variable (PRV), accurate to 0.01 in ε, and `dp_accounting`'s privacy loss distribution (PLD), accurate to 0.01 in σ.

```
def sigma_opacus(epsilon: float, delta: float, q: float, epochs: int) -> float:
    """Opacus: the sigma whose PRV epsilon after `epochs` is at most the target (to
     ↪  0.01 in epsilon)."""
    return get_noise_multiplier(
        target_epsilon=epsilon, target_delta=delta, sample_rate=q, epochs=epochs,
 ↪  accountant="prv"
    )


def sigma_dp_accounting(epsilon: float, delta: float, q: float, steps: int) -> float:
    """dp_accounting: the same search with a PLD accountant (to 0.01 in sigma)."""

    def training_run(sigma: float) -> DpEvent:
```

```
        return SelfComposedDpEvent(PoissonSampledDpEvent(q, GaussianDpEvent(sigma)),
        ↪  steps)

    def accountant() -> PLDAccountant:
        return PLDAccountant(value_discretization_interval=PLD_GRID)

    return calibrate_dp_mechanism(accountant, training_run, epsilon, delta,
    ↪  LowerEndpointAndGuess(0.3, 1.0), tol=0.01)
```

Specify `accountant="prv"` because `get_noise_multiplier` otherwise uses the looser Rényi differential privacy (RDP) accountant and returns more noise. If `dp_accounting` raises an error, reduce 0.3 in `LowerEndpointAndGuess(0.3, 1.0)`, the initial lower search bound.

The table uses a PLD grid of 10^{-3}, the step to which the accountant rounds the privacy loss, so each search takes about a second. The average over a batch of B records divides the noise on the sum by B. The final column, σ/B times 1,000, gives average-gradient noise in units of the clipping norm.

dataset	n	delta	B	E	eps	T	sigma PRV	sigma PLD	sigma/B x1000
Adult	30000	1e-05	100	10	4	3000	0.62	0.62	6.16
Adult	30000	1e-05	300	10	4	1000	0.74	0.74	2.45
Adult	30000	1e-05	1000	10	4	300	0.97	0.98	0.97
Adult	30000	1e-05	1000	10	8	300	0.72	0.73	0.72
Adult	30000	1e-05	1000	20	4	600	1.16	1.16	1.16

The block shows five of the script's 18 Adult cases. At $\varepsilon = 4$ and ten epochs, batch size 1,000 requires more noise than batch size 100: 0.97 rather than 0.62. Averaging across ten times as many records reduces noise on the average from 6.16 to 0.97. Twenty epochs increase σ to 1.16.

The table does not determine batch size: larger batches reduce noise on the average but also reduce the number of steps. Estimate the required step count using a non-private run or public data, then choose the largest batch that preserves those steps and fits memory. Part 5 uses 1,000 (Batch size at a fixed budget).

A row from the end of the table shows that small datasets are different.

breast cancer	569	1e-04	64	20	1	177	4.96	4.91	77.51

largest gap between PRV and PLD sigma: 1.6% (Adult, B=300, E=10, eps=8)

With 569 records, batch size 64 exceeds one tenth of the dataset, so subsampling provides little amplification. At $\varepsilon = 1$, noise on the average is 77.51. Small datasets may require a larger ε, public pretraining or another method.

The final line compares the two libraries. Across all 24 cases, their σ values differ by at most 1.6%. A gap above about 3% suggests mismatched inputs, such as batch rate, epoch-to-step conversion or slack δ.

26.5. Check ε with a second accountant

Train with the calibrated, unrounded σ as Part 5 does with Opacus's 0.9729. If rounding is necessary, round up to the three decimals in the statement; lower noise increases ε. Before training, compute the planned run's ε with each available accountant. Include Gaussian differential privacy (GDP) in the comparison, but do not report it as a bound.

```
def crosscheck_epsilon(sigma: float, q: float, steps: int, delta: float) -> dict[str,
 ↪  float]:
    """Epsilon of one planned run by five accountants (GDP is the trap: it is not an
     ↪  upper bound)."""
    epsilons: dict[str, float] = {}
    for name, opacus_accountant in [("Opacus RDP", RDPAccountant()), ("Opacus PRV",
     ↪  PRVAccountant())]:
        opacus_accountant.history = [(sigma, q, steps)]  # what `steps` calls of
 ↪  .step() would record
        epsilons[name] = opacus_accountant.get_epsilon(delta)
    run = SelfComposedDpEvent(PoissonSampledDpEvent(q, GaussianDpEvent(sigma)), steps)
    epsilons["dp_accounting RDP"] = RdpAccountant().compose(run).get_epsilon(delta)
    epsilons["dp_accounting PLD"] = PLDAccountant().compose(run).get_epsilon(delta)
    gdp = GaussianAccountant()
    gdp.history = [(sigma, q, steps)]
    epsilons["Opacus GDP"] = gdp.get_epsilon(delta)
    return epsilons
```

```
chosen cell: Adult, B=1000, E=10, eps=4: sigma=0.972900 (Opacus PRV, as Part 5)
Opacus RDP         eps=4.50
Opacus PRV         eps=3.99
dp_accounting RDP  eps=4.50
dp_accounting PLD  eps=3.98
Opacus GDP         eps=3.34
PLD <= target: True; RDP >= PLD: True; |PRV - PLD|/PLD < 3%: True
crosscheck: pass
PLD bracket (grid 1e-4): lower 3.9656 (optimistic), upper 3.9806 (pessimistic); width
 ↪  0.0150
```

The tight PRV and PLD accountants agree at 3.99 and 3.98, both meeting the target. The two RDP accountants agree at 4.50, a looser bound above target. GDP's 3.34 is unsafe: it uses a central limit theorem (CLT) approximation rather than a bound and falls below certified results. Do not report it.

The pass line checks three conditions: PLD is at most the target, RDP is at least PLD within grid error, and PRV agrees with PLD within 3%. The PLD check fails if σ was rounded down or the step count differs from the plan. Any other failed comparison indicates mismatched accountant inputs.

The final line gives the PLD error interval. Rounding each step's privacy loss up gives 3.9806; rounding down gives 3.9656. The true ε lies between them. Report the upper value and the lower value as its error bar (Which accountant). Longer runs widen the interval, so use a finer grid.

26.6. If the loader is not Poisson

Some training loops use non-Poisson samplers with fixed batch sizes. Calculate their costs before training; the script evaluates four samplers from Amplification by subsampling.

```
Poisson (PLD)                                eps=3.98
fixed size, add/remove, sigma/2 (PLD)        eps=23.03
fixed size, replace-one (RDP)                eps=52.60
shuffle, zero-out, no amplification (PLD)    eps=18.51
PLDAccountant supports sampling without replacement: add/remove False, replace-one
 ↪  False
```

Fixed-size random batches cost more because an included record replaces another, allowing the clipped sum to move by twice the clipping norm C. Under add/remove neighbours, account with a Poisson accountant at $\sigma/2$; the cost rises from 3.98 to 23.03. For replace-one neighbours, only RDP supports the sampler, also at $\sigma/2$ (last line). Its value 52.60 protects record values, not record presence, and is not comparable with add/remove values.

Shuffle-and-partition sampling has no amplification. The valid bound is one Gaussian step per epoch, 18.51 here, under zero-out neighbours, where one gradient is set to zero. State this relation explicitly. The sum changes by at most C, as under add/remove, so the values are comparable. Shuffle-and-partition is cheaper than fixed-size random batches here, but the comparison reverses in the 100-epoch run of Matching the accountant to the code. Prefer Opacus's Poisson loader when possible.

26.7. Train, stop early and resume

During training, Opacus's optimizer calls `accountant.step` once per noisy update, or logical batch. This remains true when `BatchMemoryManager` divides it into smaller physical batches. The script simulates 300 calls and computes ε after each epoch.

```
def train_epochs(
    accountant: PRVAccountant, sigma: float, q: float, steps_per_epoch: int, epochs:
 ↪  int, delta: float
) -> list[float]:
    """Step the accountant once per logical batch for `epochs` epochs; epsilon at the
     ↪  end of each epoch."""
    epsilons: list[float] = []
    for _ in range(epochs):
        for _ in range(steps_per_epoch):
            accountant.step(noise_multiplier=sigma, sample_rate=q)  # what the
 ↪  optimizer's step does
        epsilons.append(float(accountant.get_epsilon(delta)))
    return epsilons
```

If validation loss stalls and training stops after six of ten epochs, the planned guarantee still covers this prefix. Report the planned maximum because stopping depends on released outputs; the

realized six-epoch ε is not a valid claim. A privacy filter initialized at the first step could certify adaptive stopping (Adaptive composition). Log ε each epoch; it must not decrease. A decrease indicates an accountant reset.

Do not train beyond the planned maximum. Adding epochs after inspecting the model makes step count data-dependent. Create a new plan and account from the first step, or plan the longest run in advance and treat shorter runs as early stops.

When a job resumes, preserve accountant state. Save the accountant's `state_dict` with the model and restore it before the next step. Opacus's `PrivacyEngine.save_checkpoint` and `load_checkpoint` handle this.

```
state = accountant.state_dict()  # saved with the model; PrivacyEngine.save_checkpoint
 ↪  stores it too
# ... the job stops, and later restarts ...
restarted = PRVAccountant()
restarted.load_state_dict(state)  # without this, epsilon would restart from zero
resumed = train_epochs(restarted, sigma, q, steps_per_epoch, epochs - stop, delta)
steps_recorded = sum(steps for _, _, steps in restarted.history)
```

```
steps per epoch: 30; planned: 10 epochs = 300 steps
epoch  1: eps=1.70
epoch  2: eps=2.09
epoch  4: eps=2.69
epoch  6: eps=3.18
epsilon never decreases over 10 epochs: True
stopped early at epoch 6: eps=3.18; report the planned maximum: eps=3.99
after resuming at epoch 6: eps at epoch 10=3.99; uninterrupted run: eps=3.99; equal:
 ↪  True
steps recorded: 300; planned (the loader's 10 x 30): 300; equal: True
```

The resumed run ends at 3.99, like a run that never stopped, with all 300 steps in its history. A fresh accountant would have counted only the last four epochs and reported 2.69.

26.8. Write the statement

The last step writes the claim with every input that a reviewer needs to recompute it. The script fills the template from the run. It rounds up the PLD's ε at two values of the slack, the rule's δ and $1/n^2$, where n is still the number of training records.

```
def privacy_statement(
    unit: str, dataset: str, n: int, batch: int, epochs: int, sigma: float, clip:
 ↪  float, delta: float, calibration: str
) -> str:
    """The filled template of the reporting page: every input a reviewer needs to
     ↪  recompute epsilon."""
    q, _, steps = opacus_loader_plan(n, batch, epochs)  # the steps the Poisson loader
 ↪  runs
    run = SelfComposedDpEvent(PoissonSampledDpEvent(q, GaussianDpEvent(sigma)), steps)
```

```
    epsilon = round_up(PLDAccountant().compose(run).get_epsilon(delta))
    epsilon_tiny_delta = round_up(PLDAccountant().compose(run).get_epsilon(1 / n**2))
    return f"""\
Unit: one {unit} (a row of {dataset}); neighbouring datasets differ by adding or
 ↪  removing one {unit}
Data: n = {n:,}; Poisson sampling with q = {q:.6f} (B = {batch}); T = {steps:,} steps
 ↪  ({epochs} epochs)
Mechanism: DP-SGD, clipping norm C = {clip}, noise multiplier sigma = {sigma:.3f}
Accountant: PLD, dp-accounting {version("dp-accounting")}; {calibration}
Guarantee: (epsilon = {epsilon:.2f}, delta = {delta:.0e}); epsilon =
 ↪  {epsilon_tiny_delta:.2f} at delta = 1/n^2 = {1 / n**2:.1e}
Not covered: hyperparameter tuning runs (see Part 5), and any model released before
 ↪  this one"""
```

```
Unit: one person (a row of Adult); neighbouring datasets differ by adding or removing
 ↪  one person
Data: n = 30,000; Poisson sampling with q = 0.033333 (B = 1000); T = 300 steps (10
 ↪  epochs)
Mechanism: DP-SGD, clipping norm C = 1.0, noise multiplier sigma = 0.973
Accountant: PLD, dp-accounting 0.6.0; calibrated with Opacus 1.6.0 (PRV)
Guarantee: (epsilon = 3.99, delta = 1e-05); epsilon = 6.30 at delta = 1/n^2 = 1.1e-09
Not covered: hyperparameter tuning runs (see Part 5), and any model released before
 ↪  this one
```

The guarantee reads 3.99 because the script rounds the PLD value up, while the cross-check above printed it to the nearest value, 3.98. The unit, the dataset and the clipping norm come from options, because the accountant never sees them. The last line matters most to a reviewer, because it names privacy spent outside this ε. Examples are tuning on the private data (Hyperparameter tuning) and earlier models trained on the same people.

A published statement also gives what the script cannot know (Write the statement). That is the setting, the grid and bracket of the PLD, and the data outside the run, such as pretrained weights and the unprotected validation rows.

26.9. Checklist

Table 26.1.: The steps of calibrating and accounting a DP-SGD run.

Step	Default	Check
Unit, n and δ	One person; n from the training split, validation rows excluded; δ by the book's rule	n counts people, not rows; δ recomputed for each dataset
Sampler	Poisson; q and T from Opacus's loader, $\lceil n/B \rceil$ steps an epoch	Calibration counts the same steps; batch sizes vary

Step	Default	Check
Plan the noise	Calibrate with PRV or PLD, the accountant you report	Two libraries agree within about 3%
Cross-check	Train at the calibrated σ, or round it up	PLD at most the target; RDP above PLD; GDP never reported; PLD bracket reported
Other samplers	Keep Opacus's Poisson loader	No `shuffle=True` loader with fixed batches in the training code
Train and resume	One accountant step per logical batch; save the accountant's state; any extension inside the plan	ε never decreases; steps recorded equal the loader's steps
Statement	ε at the planned maximum, rounded up, at δ and $1/n^2$	Every input to recompute it; tuning, earlier models, pretraining, preprocessing and validation named

26.10. References

- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): planning a run's privacy budget, and what to report.
- Lebeda, Regehr, Kamath & Steinke, "Avoiding Pitfalls for Privacy Accounting of Subsampled Mechanisms under Composition" (IEEE SaTML 2025): why fixed-size and shuffled batches need their own accounting.

Derivations. The results this workflow relies on are derived in the full edition's chapters Amplification by subsampling in the full edition, Numerical accounting in the full edition and Choosing an accountant in the full edition.

27. Amplification by subsampling

27.1. Why this matters

Private training often uses stochastic gradient descent (SGD) adapted to differential privacy (DP), called DP-SGD. Each step clips record gradients to a maximum length, the **clipping norm**, sums them and adds Gaussian noise. The noise standard deviation equals the clipping norm times the **noise multiplier** σ. If every step used the full dataset, 10,000 steps at $\sigma = 1$ would cost $\varepsilon = 5425.5$ at $\delta = 10^{-5}$, providing no useful guarantee (Rényi DP).

DP-SGD reduces this cost by using a random batch at each step. Let q be the **sampling rate**, the probability that a record is included. Uncertainty about whether the record was sampled provides **amplification by subsampling**. The amplification depends on the sampling method, so the accountant must match the training sampler. A sampler mismatch causes larger accounting errors than the choice of accountant (Section 27.4).

27.2. One step under Poisson sampling

Under **Poisson sampling**, each record independently enters the batch with probability q. This sampler pairs with **add/remove** neighbours, which differ by one extra record. Balle, Barthe and Gaboardi (2018) give the tightest one-step bound for this relation. If a full-dataset step is (ε, δ)-DP, where δ is the slack defined in Part 3, then the same step on a Poisson sample is (ε', δ')-DP, with

$$\varepsilon' = \ln\left(1 + q(e^{\varepsilon} - 1)\right), \qquad \delta' = q\delta. \tag{27.1}$$

The amplification depends on the step's ε. For small ε, the cost is about $q\varepsilon$: sampling 1% makes a 1-DP step cost 0.0170, about one hundred times less. For large ε, the cost is approximately

$$\varepsilon' \approx \varepsilon - \ln(1/q), \tag{27.2}$$

so subsampling subtracts only a fixed amount; a 10-DP step on the same sample still costs 5.3993. The transition occurs near $\varepsilon = \ln(1/q)$. Subsampling therefore helps most when each step is already fairly private, as in DP-SGD's many small steps.

This result requires two conditions. Each record must be sampled independently with probability exactly q, regardless of other batch members. The batch must also remain secret; an attacker who sees the sampled records, such as in a log, knows whether a step included the target.

27.3. Match the sampler to the accountant

Most training code uses another sampler. Some implementations draw a new fixed-size batch of B from n records at every step. Others shuffle once per epoch and divide the data into consecutive batches, a method called **shuffle and partition**.

Each sampler requires accounting matched to its neighbouring relation. Under **replace-one**, datasets have the same size and differ in one record's value. Under **zero-out**, the target's gradient is set to zero. Table 27.1 lists the corresponding DP-SGD events in Google's `dp_accounting`; a Rényi differential privacy (RDP) or privacy loss distribution (PLD) accountant composes them.

Table 27.1.: What to account for under each sampler and neighbouring relation in DP-SGD.

Sampler	Neighbours	What to account for	`dp_accounting` step
Poisson, rate q	add/remove	A Poisson accountant with q and σ	`PoissonSampledDpEvent(q, GaussianDpEvent(σ))`
Poisson, rate q	replace-one	The same event, at σ, in a PLD accountant built with `NeighboringRelation.REPLACE_ONE`, which models the swap itself	`PoissonSampledDpEvent(q, GaussianDpEvent(σ))`
Fixed size B of n	add/remove	A Poisson accountant with rate B/n and noise multiplier $\sigma/2$	`PoissonSampledDpEvent(B/n, GaussianDpEvent(σ/2))`
Fixed size B of n	replace-one	Sampling without replacement at noise multiplier $\sigma/2$, in the RDP accountant only	`SampledWithoutReplacementDpEver B, GaussianDpEvent(σ/2))`
Shuffle and partition	zero-out	No amplification: each epoch is one full Gaussian release	`GaussianDpEvent(σ)` once per epoch

The factor of two for fixed-size batches follows from the gradient sum. Under Poisson sampling, adding or removing one record changes the clipped sum by at most one clipping norm. A fixed-size batch has no extra slot, so a sampled record replaces another and the sum can change by twice as much (Lebeda et al. 2025). This is equivalent to halving the noise multiplier. Replace-one also swaps records and doubles the change; for fixed-size batches, only `dp_accounting`'s RDP accountant supports this relation.

Shuffle and partition provides no amplification because each record appears in exactly one batch per epoch. An attacker only needs to identify that batch. Chua et al. (2024) construct a dataset where the target gradient points against all others, making its batch distinguishable. At $q = 0.01$ and $\sigma = 1$, one shuffled epoch has ε at least 4.06, while a Poisson accountant reports at most 1.21.

The valid bound is one full Gaussian release per epoch, as shown in the table, because each record is used once.

Inspect the data loader before accepting a reported ε. A PyTorch `DataLoader` with `shuffle=True` and `batch_size` uses shuffle and partition. By default, Opacus replaces it with Poisson sampling: `PrivacyEngine.make_private` sets `poisson_sampling=True`. Its `batch_sampler` is a `UniformWithReplacementSampler` that independently samples each record, despite its name. Batch sizes therefore vary, which you can verify in the training log. If using another sampler, account for the matching row of Table 27.1 and report the sampler with ε.

27.4. Accounting each sampler

Do not account for DP-SGD by applying Equation 27.1 separately to every step. This gives each step a very small share of δ, which raises its per-step ε and limits amplification. For 10,000 steps, advanced composition then gives 24379.55, worse than the 5425.5 bound without subsampling. An RDP accountant instead composes the Rényi curves and converts once, giving 6.71 (Rényi DP). A PLD accountant is tighter (Numerical accounting).

The snippet accounts for the run under four rows of Table 27.1, using 50,000 records and batches of 500 ($q = 0.01$). It uses $\delta = 10^{-5}$ for comparison within this part, not the smaller reporting value (How small δ must be).

```
from dp_accounting import GaussianDpEvent, NeighboringRelation, PoissonSampledDpEvent
from dp_accounting import SampledWithoutReplacementDpEvent, SelfComposedDpEvent
from dp_accounting.pld import PLDAccountant
from dp_accounting.rdp import RdpAccountant

n, batch, sigma, steps, delta = 50_000, 500, 1.0, 10_000, 1e-5  # q = 0.01, 100 epochs
poisson = SelfComposedDpEvent(PoissonSampledDpEvent(batch / n, GaussianDpEvent(sigma)),
 ↪  steps)
fixed_size = SelfComposedDpEvent(PoissonSampledDpEvent(batch / n, GaussianDpEvent(sigma
 ↪  / 2)), steps)  # add/remove
shuffled = SelfComposedDpEvent(GaussianDpEvent(sigma), 100)  # one Gaussian release per
 ↪  epoch, no amplification
pld = [PLDAccountant(value_discretization_interval=1e-3).compose(e) for e in (poisson,
 ↪  fixed_size, shuffled)]  # 1e-3: the PLD's grid spacing
print(" ".join(f"{accountant.get_epsilon(delta):.2f}" for accountant in pld))  # 6.19
 ↪  43.37 91.82
replace_one = RdpAccountant(neighboring_relation=NeighboringRelation.REPLACE_ONE)
swap = SampledWithoutReplacementDpEvent(n, batch, GaussianDpEvent(sigma / 2))  #
 ↪  replace-one: the sum moves by up to 2C
replace_one.compose(SelfComposedDpEvent(swap, steps))
print(f"{replace_one.get_epsilon(delta):.2f}")  # 118.73
```

Poisson sampling costs 6.19. With the same noise, fixed-size batches cost 43.37 under add/remove neighbours, about seven times more; shuffle and partition costs 91.82 under zero-out. The replace-one value, 118.73, protects a record's value rather than its presence, so compare it only with other replace-one results. The sampler changes ε by at least sevenfold, while the accountant changes it by less than 10%: RDP gives 6.71 and PLD gives 6.19.

Before composing an unfamiliar event, check `PLDAccountant().supports(event)`. For example, it returns `False` for `SampledWithoutReplacementDpEvent`.

27.5. Batch size, epochs and noise

Subsampling also affects run planning. Let E be the number of epochs; at sampling rate q, the run takes E/q steps. For small q and a large noise multiplier, its Rényi cost at order α is approximately

$$\varepsilon(\alpha) \approx \frac{E\, q\, \alpha}{2\sigma^2}. \tag{27.3}$$

At a fixed epoch count, a smaller sampling rate costs less privacy: each step costs about q^2, while the step count grows as $1/q$. Use this approximation only for planning. At high orders, the true curve rises substantially above it, beginning around order 9 for $\sigma = 1$ and $q = 0.01$.

Privacy is not the only cost: noise also affects the averaged gradient. If C is the clipping norm, noise on an average over B records has standard deviation $\sigma C/B$. Compare plans using σ/B, not σ alone. At the same ε, a batch ten times larger needs a larger σ, but its σ/B is six times smaller (Calibrate and account a DP-SGD run).

Plan in this order: choose batch size B and epochs E based on model accuracy. They determine $q = B/n$ and the step count, where n is the filtered training-set size. Then use an accountant to find σ. Specify the numerical accountant in Opacus's `get_noise_multiplier`, whose default is RDP (Calibrate and account a DP-SGD run).

27.6. Cheat sheet

Table 27.2.: Which accounting to use for each sampler.

Situation	What to use	Notes
One step, Poisson, add/remove	$\varepsilon' = \ln(1 + q(e^{\varepsilon} - 1))$, $\delta' = q\delta$	About $q\varepsilon$ for small ε
DP-SGD, Poisson loader	PLD or RDP accountant with q and σ	6.19 by PLD for the running example
DP-SGD, fixed-size batches, add/remove	Poisson accountant with B/n and $\sigma/2$	43.37, about seven times the Poisson value
DP-SGD, fixed-size batches, replace-one	`SampledWithoutReplacementDpEvent` at $\sigma/2$ in the RDP accountant	Not comparable with add/remove numbers
DP-SGD, shuffle and partition	One Gaussian release per epoch, under zero-out	A Poisson number does not apply; better, switch the loader to Poisson sampling

Rules of thumb

- Account for the sampler you run, check that its batches vary in size, and state the neighbouring relation next to ε.

27.7. References

- Balle, Barthe & Gaboardi, "Privacy Amplification by Subsampling: Tight Analyses via Couplings and Divergences" (NeurIPS 2018): amplification for each sampler and neighbouring relation.
- Lebeda, Regehr, Kamath & Steinke, "Avoiding Pitfalls for Privacy Accounting of Subsampled Mechanisms under Composition" (IEEE SaTML 2025): fixed-size batches and other accounting pitfalls.

Derivations. Every result here is derived step by step in the full edition's chapter.

28. Rényi differential privacy

28.1. Why this matters

Model training may use thousands of private stochastic gradient descent (SGD) steps, but its report gives one differential privacy (DP) guarantee, (ε, δ). Part 3's advanced composition can be too pessimistic (Composition). For example, 1,000 full-batch noisy gradient-descent steps, each with Gaussian noise 20 times the maximum per-record gradient change, give $\varepsilon = 68.01$ at $\delta = 10^{-5}$ under advanced composition, nine times the exact value 7.51. Calibration to this loose bound adds unnecessary noise.

Rényi differential privacy (RDP) tracks a curve of privacy costs for each step. The curves compose without spending δ; convert the total to (ε, δ) once, after composition. Even Mironov's original conversion (Section 28.6) gives 8.84 for this run, 18% above the exact value. Opacus and Google's `dp_accounting` provide RDP accountants.

28.2. The RDP curve

A DP guarantee compares mechanism outputs on neighbouring datasets, which differ in one person's record. RDP measures the distance between the output distributions using the **Rényi divergence** from Divergences. Its **order** $\alpha > 1$ controls sensitivity to rare outputs: low orders emphasize typical outputs, while high orders give more weight to outputs much likelier on one dataset.

A mechanism is (α, ε)**-RDP** if its divergence is at most ε at order α for every neighbouring pair. The smallest such value at each order defines its **RDP curve** $\varepsilon(\alpha)$; higher curves mean weaker privacy. The curve is nondecreasing. At infinite order it equals the largest output privacy loss, bounded by pure DP, where $\delta = 0$. Accountants use known curves for standard mechanisms on a grid of orders; you do not need to compute divergences directly.

28.3. RDP curves of the basic mechanisms

Accounting requires the mechanism's RDP curve. The basic mechanisms have closed-form curves (Mironov 2017). For the **Gaussian mechanism**, let σ be its noise multiplier: the noise standard deviation divided by the query's ℓ_2 sensitivity. Its curve is

$$\varepsilon(\alpha) = \frac{\alpha}{2\sigma^2}. \tag{28.1}$$

Table 28.1 gives the curves of the Gaussian, Laplace and randomized-response mechanisms at a few orders, each at sensitivity 1.

Table 28.1.: RDP curves of the three basic mechanisms with sensitivity 1.

Mechanism	$\varepsilon(1.5)$	$\varepsilon(2)$	$\varepsilon(10)$	$\varepsilon(64)$	$\varepsilon(\infty)$
Gaussian, $\sigma = 1$	0.7500	1.0000	5.0000	32.0000	∞
Laplace, scale $b = 1$	0.5129	0.6191	0.9287	0.9891	1
Randomized response, which reports a bit truthfully with probability 0.75	0.7340	0.8473	1.0666	1.0940	1.0986

The Laplace and randomized-response curves approach their pure-DP levels. The Gaussian curve has no finite limit, so the Gaussian mechanism is not pure ε-DP for any finite ε. Its curve remains finite at each finite order, where accountants perform conversions. These curves describe full-dataset steps. For DP-SGD, each step uses a random batch, and libraries compute the smaller subsampled curves (Amplification by subsampling).

28.4. Composition adds curves

Training composes mechanisms sequentially, often adaptively: each gradient step depends on the current model. RDP adds their curves order by order. Let ε_i be the curve for step i. A run of k steps has curve ε_M satisfying

$$\varepsilon_M(\alpha) \leq \sum_{i=1}^{k} \varepsilon_i(\alpha) \qquad \text{for every order } \alpha. \tag{28.2}$$

Different mechanisms can be mixed because composition adds their values at each common order.

The sum requires two conditions. All curves must use the same neighbouring relation, such as add/remove or replace-one, and the same privacy unit, such as one record or one user. Also, each step's curve must be fixed before the run. Steps may depend on previous outputs, but data-dependent noise or stopping makes the curve data-dependent. Early stopping based on validation performance therefore requires Adaptive composition.

For Gaussian steps, each curve is the line $\alpha/(2\sigma^2)$. Adding k such curves gives the cost of one step with noise multiplier $\sigma/\sqrt{k}$. Thus, k full-dataset steps cost as much as one step with $\sqrt{k}$ times less noise. For 10,000 steps at noise multiplier 1, this is equivalent to one step at multiplier $1/\sqrt{10{,}000} = 0.01$, whose ε at $\delta = 10^{-5}$ is 5425.5. DP-SGD reduces this cost by using small random batches. The running example uses 10,000 steps at multiplier 1, each sampling 1% of the data.

28.5. Post-processing and group privacy

Post-processing is free in RDP: a computation on the output that does not access the data again cannot increase the curve. Releasing the trained model or its predictions therefore adds no cost beyond training.

The generic RDP group rule is loose. For Gaussian noise and a user with m records, use the group's sensitivity, at most m times the per-record sensitivity. The curve is

$$\varepsilon_{\text{group}}(\alpha) = \frac{m^2\alpha}{2\sigma^2}, \tag{28.3}$$

This is m^2 times the per-record curve; ten records per user therefore cost 100 times as much. Clip each user's total contribution to one norm and account per user (User-level privacy).

28.6. Compose, then convert once

After composition, convert the curve to the reported (ε, δ) pair. Both libraries use the **improved conversion** (Balle et al. 2020; independently, Canonne, Kamath and Steinke 2020). Given curve value $\varepsilon(\alpha)$ at order α and target δ, the conversion gives

$$\varepsilon' = \varepsilon(\alpha) + \ln\frac{\alpha-1}{\alpha} - \frac{\ln\delta + \ln\alpha}{\alpha-1}. \tag{28.4}$$

Report the smallest ε' on the order grid, or 0 if it is negative. The minimizing order is the best order.

Older papers and code may use Mironov's original conversion, which gives 7.43 rather than 6.71 for the DP-SGD example. The original paper's **moments accountant** (Abadi et al. 2016) uses an RDP curve with that conversion. Its ε is looser but valid (From Rényi DP to (ε, δ) and to tests).

28.7. Check the order grid

A conversion can use only the orders on the grid of the accountant, so the grid must contain the best order. That order moves with the run, towards 1 for long runs with little noise and up for short runs with much noise. For the noisy gradient descent at the start of the chapter, it falls from 128 after one step to 4 after 1000.

The default grid of Opacus, `RDPAccountant.DEFAULT_ALPHAS`, stops at order 63, and that of `dp_accounting`, `DEFAULT_RDP_ORDERS`, at 1024. A best order at either end of the grid means that the true optimum is outside it, so give a longer list, with `alphas=[...]` in Opacus or `orders=[...]` in `dp_accounting`. Opacus warns you, but `dp_accounting` does not, so compare the returned order with the ends of the list yourself.

28.8. Accounting with a library

In practice, you describe each step to a library, and it composes and converts. In `dp_accounting`, a DP-SGD step is a `GaussianDpEvent` inside a `PoissonSampledDpEvent`, which samples each record independently, with probability 0.01 in the running example. This snippet composes the 10,000 steps and prints ε with the best order.

```
import dp_accounting

step = dp_accounting.PoissonSampledDpEvent(0.01, dp_accounting.GaussianDpEvent(1.0))
accountant = dp_accounting.rdp.RdpAccountant()
accountant.compose(step, count=10_000)
eps, order = accountant.get_epsilon_and_optimal_order(1e-5)
print(f"{eps:.4f} {order:g}")  # 6.7128 4.1
```

In Opacus, the accountant is attached to the optimizer, which calls `step` after every noisy update. The same calls by hand give the same ε to the fourth decimal, at the same order, 4.1, which is well inside both grids.

```
from opacus.accountants import RDPAccountant

accountant = RDPAccountant()
for _ in range(10_000):  # what the optimizer does at each step of training
    accountant.step(noise_multiplier=1.0, sample_rate=0.01)
eps, order = accountant.get_privacy_spent(delta=1e-5)
print(f"{eps:.4f} {order:g}")  # 6.7127 4.1
```

Opacus's defaults mix two accountants, so name the one that you want. In Opacus 1.6.0, `PrivacyEngine` uses `"prv"`, a numerical accountant, by default, and `accountant="rdp"` selects this class. But `get_noise_multiplier`, which solves for the noise multiplier σ, uses RDP by default. Both give valid upper bounds, but different ones, and the numerical accountant gives 6.19 for the running example.

Two more checks catch most mistakes. The event must match the sampler of your data loader, because shuffled batches of fixed size get none of the amplification of a Poisson event (Amplification by subsampling). And a manual call to `step` on top of the optimizer's own call charges each update twice.

28.9. Cheat sheet

Table 28.2.: The results of this chapter.

Result	Formula	Notes
RDP curve	$\varepsilon(\alpha)$ over orders $\alpha > 1$	Never decreases; $\varepsilon(\infty)$ is pure DP

Result	Formula	Notes
Gaussian, noise multiplier σ	$\varepsilon(\alpha) = \alpha/(2\sigma^2)$	σ in units of the ℓ_2 sensitivity
Composition	$\varepsilon_M(\alpha) \le \sum_i \varepsilon_i(\alpha)$	Same relation and unit; curves fixed in advance
Group of m, Gaussian	$m^2\alpha/(2\sigma^2)$	The generic group rule is loose
Improved conversion	$\varepsilon(\alpha) + \ln\frac{\alpha-1}{\alpha} - \frac{\ln\delta+\ln\alpha}{\alpha-1}$, best α	What Opacus and `dp_accounting` use

28.10. References

- Mironov, "Rényi Differential Privacy" (IEEE CSF 2017): the definition, composition, the curves of the basic mechanisms and group privacy.
- Abadi et al., "Deep Learning with Differential Privacy" (CCS 2016): DP-SGD and the moments accountant.

Derivations. Every result here is derived step by step in the full edition's chapter.

29. From Rényi DP to (ε, δ) and to tests

29.1. What it is

An accountant that tracks Rényi differential privacy (RDP) returns a curve (Rényi differential privacy). At each order $\alpha > 1$, it gives a privacy level $\varepsilon(\alpha)$. A report needs a differential privacy (DP) guarantee (ε', δ). Conversion maps each point on the curve to such a pair; report the smallest resulting ε'. These conversions are used to report the privacy cost of stochastic gradient descent (SGD).

Two conversions are in use. The original one, **Mironov's conversion** (Mironov 2017), gives at each order

$$\varepsilon' = \varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1}. \tag{29.1}$$

The **improved conversion** (Balle et al. 2020) gives

$$\varepsilon' = \varepsilon(\alpha) + \ln \frac{\alpha - 1}{\alpha} - \frac{\ln \delta + \ln \alpha}{\alpha - 1}. \tag{29.2}$$

Both conversions are valid. At a fixed order, the improved conversion is always lower: by 0.3612 at order 10, and by more at smaller orders. Report 0 if the improved value is negative.

For example, one Gaussian release with noise multiplier 1 has the curve $\alpha/2$. At $\delta = 10^{-5}$ its exact ε is 4.377, Mironov's conversion gives 5.299 and the improved one 4.729.

29.2. When you meet it

Use these conversions when comparing RDP results from different sources. Opacus 1.6.0 and `dp_accounting` 0.6.0 use the improved conversion; older papers and code often use Mironov's. In this part's running example, 10,000 steps of private stochastic gradient descent (DP-SGD) each sample 1% of the data and use noise multiplier 1. Mironov's conversion gives 7.43, compared with 6.71 for the improved conversion.

The difference depends only on the order. Two RDP accountants reporting values about 0.36 apart for the same run, with a best order near 10, likely use different conversions. Recompute both with one library before comparing them.

29.3. How to use it

In `dp_accounting`, `get_epsilon_and_optimal_order` converts at the best grid order and returns both values. The same curve supports multiple δ values, so `get_epsilon` can report several pairs without additional privacy cost. The snippet converts the Gaussian release above.

```
from dp_accounting import GaussianDpEvent
from dp_accounting.rdp import RdpAccountant

accountant = RdpAccountant()
accountant.compose(GaussianDpEvent(1.0))  # one release, noise multiplier 1
eps, order = accountant.get_epsilon_and_optimal_order(1e-5)
print(f"{eps:.4f} {order:g}")  # 4.7285 5.4
print(f"{accountant.get_epsilon(1e-8):.4f}")  # 6.0864
```

Reducing δ by a factor of 1,000 raises ε from 4.7285 to 6.0864 because the conversion depends logarithmically on δ. The best order is selected from a fixed grid. Short, high-noise runs may require orders above 63, beyond Opacus's default grid. If the returned order is at a grid endpoint, extend the grid and convert again (Rényi differential privacy).

29.4. What goes wrong

No conversion reaches the exact value because the curve is only a summary. Other mechanisms with the same curve may have a larger exact ε, so the conversion must cover them too. For the plain Gaussian mechanism, the improved conversion is 5–10% above the exact value, and the gap is larger for subsampled DP-SGD steps. A privacy loss distribution (PLD) accountant tracks the full distribution and returns the exact 4.377 for the single release (Numerical accounting).

Conversion is one-way: an (ε, δ) guarantee with $\delta > 0$ does not determine an RDP curve. For example, a mechanism that publishes a record with probability δ is $(0, \delta)$-DP but has infinite Rényi divergence. Do not enter an (ε, δ) pair into an RDP accountant; obtain the curve from the mechanism.

A conversion also loses what the curve says about attacks. Take a membership attack on the single release above, at a false-positive rate of 10^{-3}. The curve limits the attack's true-positive rate to 2.4910×10^{-2}. But the pair at $\varepsilon = 4.729$ limits it only to 0.1131 (f-DP and Gaussian differential privacy).

29.5. References

- Balle, Barthe, Gaboardi, Hsu & Sato, "Hypothesis Testing Interpretations and Rényi Differential Privacy" (AISTATS 2020): the improved conversion, and what an RDP curve says about tests.

Derivations. Every result here is derived step by step in the full edition's chapter.

30. Numerical accounting with privacy loss distributions

30.1. What it is

A privacy loss distribution (PLD) accountant numerically computes the privacy level ε of a complete differential privacy (DP) run. Compare a dataset with a neighbour differing in one record. The **privacy loss** L is the logarithm of the output likelihood ratio between these datasets. Because the output is random, L has a distribution called the PLD. Outputs with loss above ε contribute to slack δ; the minimum slack is given below, where $(\cdot)_+$ sets negative values to 0:

$$\delta(\varepsilon) = \mathbb{E}[(1 - e^{\varepsilon - L})_+]. \tag{30.1}$$

With independent noise, step losses add, so the run's PLD is the convolution of the step PLDs. Numerical error comes from representing losses on a grid with spacing h. Rounding losses up can only increase δ, so the resulting ε is an upper bound, the **pessimistic** value. Rounding down gives the optimistic lower bound. Each step's loss shifts by at most h, so after T steps the two bounds differ by at most Th.

30.2. When you meet it

Opacus uses a PLD accountant by default for training: `"prv"` is the privacy loss random variable (PRV) accountant of Gopi, Lee and Wutschitz (2021). It rounds each loss to a grid point, adds a proven error bound and reports the upper value. Google's `dp_accounting` provides `PLDAccountant`; its default **connect the dots** method gives the tightest pessimistic rounding on a grid.

Table 30.1 compares PLD accounting with Rényi differential privacy (RDP) and the central limit theorem (CLT) estimate of Gaussian differential privacy (GDP). The example is stochastic gradient descent (SGD) with differential privacy, per-step clipping and noise, using sampling rate 0.01 and noise multiplier 1.

Table 30.1.: ε at $\delta = 10^{-5}$, for the running example. The PLD columns round up and down on the grid $h = 10^{-5}$, without connect the dots.

Steps	PLD upper	PLD lower	RDP	GDP CLT
1	0.1995	0.1994	0.9555	0.0368
1000	1.8332	1.8232	2.1014	1.6177
10000	6.2377	6.1377	6.7127	6.0071

RDP gives a valid but loose bound (Rényi DP). At every run length, the CLT estimate is below the PLD lower bound and therefore overstates privacy (f-DP and Gaussian DP).

30.3. How to use it

In `dp_accounting`, compose the run as an event and request ε at the desired δ. The `get_delta` method instead returns a bound on δ for a chosen ε.

```
from dp_accounting import GaussianDpEvent, PoissonSampledDpEvent, SelfComposedDpEvent
from dp_accounting.pld import PLDAccountant
from opacus.accountants import PRVAccountant

run = SelfComposedDpEvent(PoissonSampledDpEvent(0.01, GaussianDpEvent(1.0)), 10_000)
pld = PLDAccountant(value_discretization_interval=1e-4).compose(run)  # h, 10x the
 ↪  table's
print(f"{pld.get_epsilon(1e-5):.4f}")  # 6.1877
print(f"{pld.get_delta(6.24):.1e}")  # 8.5e-06

prv = PRVAccountant()
prv.history = [(1.0, 0.01, 10_000)]  # (noise multiplier, sample rate, steps)
print(f"{prv.get_epsilon(delta=1e-5):.4f}")  # 6.1980
```

Both answers are upper bounds within Table 30.1's bracket. A reviewer can use `get_delta` to verify the $(6.24, 10^{-5})$ claim: it bounds the run's slack at that ε by 8.5×10^{-6}.

Each library exposes one accuracy setting. In `dp_accounting`, `value_discretization_interval` sets grid spacing; making the grid ten times finer narrows the bracket tenfold and uses about ten times more memory. In Opacus, `eps_error` sets the allowed error in ε. Since error grows with the number of steps, configure either setting for the longest planned run.

30.4. What goes wrong

The main reporting error is to use the wrong side of the bracket. Only the pessimistic value, or the PRV accountant's upper value, is a guarantee; the optimistic value and midpoint are estimates.

Under add/remove adjacency, a subsampled step has separate PLDs for adding and removing a record; neither direction is always larger. An accountant must compose both and report the larger, as `dp_accounting` does by default. Opacus's PRV accountant models removal only, which is the larger direction in this example. Both libraries assume Poisson sampling; shuffled or fixed-size batches violate that assumption (Amplification by subsampling).

For a run that adapts its steps or noise to earlier outputs, use a filter (Adaptive composition).

30.5. References

- Gopi, Lee & Wutschitz, “Numerical Composition of Differential Privacy” (NeurIPS 2021): the PRV accountant, with explicit error bounds.

Derivations. Every result here is derived step by step in the full edition’s chapter.

31. f-DP and Gaussian differential privacy

31.1. What it is

Every differential privacy (DP) guarantee limits an attacker's ability to determine whether a target's record was in the data. The attacker performs a hypothesis test, so the most complete guarantee is a **trade-off curve**: the minimum false negative rate at each false positive rate. A mechanism is f**-DP** if its curve is nowhere below a specified curve f (Dong, Roth and Su 2022). A privacy level ε and slack δ summarize this curve.

For Gaussian noise, the curve has one parameter. Let σ be the noise multiplier, the noise standard deviation in units of the query's ℓ_2 sensitivity. The attacker must distinguish a standard normal draw from one shifted by $\mu = 1/\sigma$. The mechanism therefore satisfies Gaussian differential privacy (GDP) with parameter μ, or μ**-GDP**. Let Φ be the standard normal distribution function. At false positive rate α, no membership attack achieves a true positive rate above

$$1 - \Phi(\Phi^{-1}(1 - \alpha) - \mu). \tag{31.1}$$

31.2. When you meet it

Use GDP to assess whether a guarantee is sufficient; Opacus also exposes it as a potentially unsafe option (Section 31.4). Table 31.1 gives the maximum true positive rate (TPR) at each false positive rate (FPR), regardless of the attacker's other information. The same representation can account for stochastic gradient descent (SGD) runs without subsampling.

Table 31.1.: Reading μ-GDP as an attack and as (ε, δ).

μ	Best TPR at 0.1% FPR	At 1%	At 5%	ε at $\delta = 10^{-5}$
0.5	0.0048	0.0339	0.1261	1.9931
1	0.0183	0.0924	0.2595	4.3772
2	0.1378	0.3721	0.6388	9.9973

At 1% FPR, no attacker against a 1-GDP release can identify the record in more than 9.24% of trials where it is present. This bounds any membership inference attack and is the limit tested by a privacy audit. Dong et al. suggest $\mu \leq 0.5$ as a reasonable guarantee and note that $\mu \geq 6$ offers little protection. Report μ alongside (ε, δ) for Gaussian pipelines.

31.3. How to use it

Gaussian releases compose exactly in GDP, including adaptive releases. Let μ_i be the parameter of release i. The full run is μ-GDP, where

$$\mu = \sqrt{\mu_1^2 + \mu_2^2 + \cdots + \mu_k^2}, \tag{31.2}$$

Thus, μ^2 composes additively. For whole-dataset releases, such as analytics queries and full-batch noisy gradient descent, no accountant gives a tighter bound. Track μ^2 and convert once after composition. Four 1-GDP releases compose to 2-GDP, or $(9.9973, 10^{-5})$-DP. Converting each release separately using one quarter of the slack gives a summed ε of 18.7607, almost twice as large.

The snippet computes the attack ceiling and converts between GDP and (ε, δ). The `dp_accounting` functions accept a noise multiplier, so pass $1/\mu$ for μ-GDP.

```
from statistics import NormalDist

import dp_accounting

N = NormalDist()


def attack_power(mu: float, fpr: float) -> float:
    """Best true positive rate of any membership attack on mu-GDP at a false positive
     ↪  rate."""
    return 1 - N.cdf(N.inv_cdf(1 - fpr) - mu)


print(" ".join(f"{attack_power(mu, 0.001):.4f}" for mu in (0.5, 1.0, 2.0)))  # 0.0048
 ↪  0.0183 0.1378
print(f"{dp_accounting.get_epsilon_gaussian(1.0, 1e-5):.4f}")  # 4.3772
print(f"{dp_accounting.get_sigma_gaussian(1.0, 1e-5):.4f}")  # 3.7306
```

The final line returns the noise multiplier for one $(1, 10^{-5})$-DP release. It uses the exact curve, which requires less noise than the classical formula in the Gaussian mechanism.

31.4. What goes wrong

Opacus supports `accountant="gdp"` for DP-SGD. It estimates μ using the central limit theorem (CLT) of Bu et al. (2020), which approximates many small privacy losses by a Gaussian curve. This is an approximation, not a bound, and its error can be unsafe. Small δ depends on rare, large losses that the CLT models poorly. For this part's 10,000-step example, the CLT gives 6.0071, while a numerical accountant establishes that true ε is at least 6.1377. Use Opacus's default numerical accountant, `"prv"`, for reporting.

Mixed Laplace and Gaussian releases require a numerical accountant because their composition is not Gaussian. For Gaussian pipelines without subsampling, GDP is the tightest accountant. Even the best conversion from a Rényi DP curve is looser: for 1-GDP at $\varepsilon = 1$, it gives $\delta = 0.2149$ instead of the exact 0.1269.

31.5. References

- Dong, Roth & Su, "Gaussian Differential Privacy" (Journal of the Royal Statistical Society Series B, 2022).

Derivations. Every result here is derived step by step in the full edition's chapter.

32. Adaptive composition: filters, odometers and individual accounting

32.1. What it is

Differential privacy (DP) composition theorems assume that each step's privacy parameter and the total step count are fixed before the run (Composition). Adaptive runs violate this assumption when later choices depend on earlier outputs; for example, training may stop based on validation loss. The fixed-parameter theorems do not cover the realized values (Rogers et al. 2016). This **plug-in mistake** can make the bound fail about three times as often as its stated δ.

A **privacy filter** fixes a total budget and rejects any step that would exceed it, preserving the guarantee under any stopping rule. A **privacy odometer** reports cumulative spend for open-ended analyses. Its anytime guarantee has a higher ε.

32.2. When you meet it

Early stopping with a fixed maximum needs neither tool. Set noise, sampling rate and maximum steps in advance. You can then stop when validation loss stops improving, because the executed steps are a prefix of the plan. **Report the planned maximum's ε, not the value for the steps actually run.** The plan's guarantee is $(5.299, 10^{-5})$-DP. It covers 100 full-batch Gaussian steps at noise multiplier 10. Training may stop at step 60 and still use that guarantee.

The stop signal must use released models or held-out data, so the decision is post-processing. To exceed the maximum, create a new plan and account for the run from its first step. When resuming, restore the accountant state (the calibration workflow).

Use filters when training changes noise in response to outputs. Analytics services also provide them, including OpenDP's `make_privacy_filter` and Tumult Analytics' `Session`.

32.3. How to use it

Feldman and Zrnic's (2021) Rényi filter uses one Rényi divergence order α, the order parameter fixed in advance. It charges each step's Rényi differential privacy (RDP) spend at that order (Rényi DP). A step's spend may depend on earlier outputs, but must be known before that step runs. The filter rejects any step that would exceed budget B. Let ε_g denote the target privacy level and δ the slack. Invert the RDP conversion to set

$$B = \varepsilon_g - \frac{\ln(1/\delta)}{\alpha - 1}. \tag{32.1}$$

Let σ denote the Gaussian noise multiplier, the standard deviation relative to sensitivity. A Gaussian step then spends $\alpha/(2\sigma^2)$ at order α. For this target, the best order is 5.8 and the budget is $B = 2.900$. This allows 100 full-batch steps at noise multiplier 10. If validation loss stops improving at step 60, halving the noise makes each later step cost four times as much. The remaining budget covers 10 more steps and preserves the target.

OpenDP's filter uses zero-concentrated DP (zCDP). Its budget ρ bounds RDP spend at every order (Concentrated DP).

```
import opendp.prelude as dp

dp.enable_features("contrib")
domain, metric = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
odometer = dp.c.make_fully_adaptive_composition(domain, metric,
 ↪  dp.zero_concentrated_divergence())
queryable = dp.c.make_privacy_filter(odometer, d_in=1.0, d_out=0.5)(0.0)  # rho = 0.5,
 ↪  i.e. 2.9 at order 5.8

answered = 0
try:
    for _ in range(1000):  # sigma = 10: rho = 1 / (2 sigma^2) = 0.005 each
        queryable(dp.m.make_gaussian(domain, metric, scale=10.0))
        answered += 1
except dp.OpenDPException:  # the release that would exceed the budget
    pass
print(answered)  # 99
```

OpenDP permits 99 releases rather than 100 because it rounds each charge up for floating-point safety. For pure-DP queries, the **Whitehouse filter** (Whitehouse et al. 2023) allows more small queries than basic summation of ε.

32.4. What goes wrong

An odometer gives a looser bound than a filter. The **Ville odometer** converts RDP spend at the best order on a grid fixed in advance. It accounts for this later choice by dividing δ by the grid size. For the 100-step run above, this raises ε from 5.299 to 6.253.

Individual accounting charges each record according to its clipped gradient norm, since smaller gradients move the model less. It excludes a record when its spend would exceed the budget. In the full edition's simulation, median individual ε is 3.17, compared with worst-case 5.299. Because this accounting uses observed norms, it is an estimate rather than a guarantee. It also reveals which records the model fits well. High-gradient records are excluded first and may be the records the model serves worst (Disparate impact).

32.5. References

- Whitehouse, Ramdas, Rogers & Wu, "Fully-Adaptive Composition in Differential Privacy" (ICML 2023).

- Feldman & Zrnic, "Individual Privacy Accounting via a Rényi Filter" (NeurIPS 2021).

Derivations. Every result here is derived step by step in the full edition's chapter.

33. Concentrated differential privacy

33.1. What it is

Zero-concentrated differential privacy (zCDP) represents a differential privacy (DP) budget by one composable number. It uses the curve of Rényi differential privacy (RDP), which bounds divergence between outputs on neighbouring datasets at each order α, where α denotes the order parameter. A mechanism is **ρ-zCDP** when its curve lies below the line $\rho\alpha$. Composition adds these slopes, including for adaptive steps.

The Gaussian mechanism has a linear RDP curve. Let f denote the query. Let $\Delta_2 f$ denote its ℓ_2 sensitivity, the largest Euclidean change caused by replacing one record. Let σ denote the standard deviation of the added noise. A release costs

$$\rho = \frac{(\Delta_2 f)^2}{2\sigma^2}. \tag{33.1}$$

With noise multiplier 1, $\sigma = \Delta_2 f$ and $\rho = 0.5$. Doubling the noise reduces ρ to one quarter.

33.2. When you meet it

Use zCDP for whole-dataset releases with Gaussian noise, such as noisy counts or the marginals used to generate synthetic data. The 2020 United States Census used zCDP with the discrete Gaussian mechanism (Discrete and floating-point-safe mechanisms). Tumult Analytics and OpenDP also represent budgets with ρ. For stochastic gradient descent (SGD) with differential privacy, use RDP or numerical accounting instead.

33.3. How to use it

Track releases in a ledger and charge each according to Table 33.1.

Table 33.1.: What each kind of step costs in a zCDP ledger.

Step	Charge in ρ	Notes
Gaussian, sensitivity $\Delta_2 f$, noise σ	$(\Delta_2 f)^2/(2\sigma^2)$	Exact
Any pure ε-DP step	$\varepsilon^2/2$, or $\varepsilon^2/8$ for an exponential mechanism	Above the plain ε once $\varepsilon \geq 2$: keep such a step in RDP

Step	Charge in ρ	Notes
An (ε, δ)-DP step	$\varepsilon^2/2$, plus δ in a second column that also adds	**Approximate zCDP**
Poisson-subsampled Gaussian	$(\Delta_2 f)^2/(2\sigma^2)$, as without sampling	No amplification

Add the charges, then convert the total once. Let ε denote the resulting privacy level, and let δ denote the slack. For an all-Gaussian ledger, total ρ is equivalent to one Gaussian release with noise multiplier $1/\sqrt{2\rho}$, so `get_epsilon_gaussian` converts it exactly. The snippet performs this conversion and calibrates a $(1, 10^{-5})$-DP release.

```
from math import sqrt

import dp_accounting

rho = 1 / (2 * 1.0**2)  # one Gaussian release, noise multiplier 1
print(f"{rho:.1f} {dp_accounting.get_epsilon_gaussian(1 / sqrt(2 * rho), 1e-5):.4f}")
 ↪  # 0.5 4.3772
sigma = dp_accounting.get_sigma_gaussian(1.0, 1e-5)  # one release that must be (1,
 ↪  1e-5)-DP
budget = 1 / (2 * sigma**2)
quarter = budget / 4  # four releases share it
print(f"{budget:.5f} {sigma:.3f} {1 / sqrt(2 * quarter):.3f}")  # 0.03593 3.731 7.461
```

For a target privacy level of 1 and slack 10^{-5}, the required budget is $\rho = 0.03593$. One query meets it with noise multiplier 3.731. If four queries share this budget equally, each receives one quarter and requires twice the noise. Allocate in ρ, which composes exactly, not in ε. A histogram with each person in one cell incurs one charge, not one per cell (Parallel composition).

For a ledger containing non-Gaussian steps, conversion is not exact. At $\rho = 0.5$, Bun and Steinke's (2016) simple conversion is 21% above the exact Gaussian value. Canonne, Kamath and Steinke's (2020) improved conversion is 8% above. The RDP accountant in `dp_accounting` uses this improved conversion. Use it, or the exact conversion for an all-Gaussian ledger, and identify the method in the report.

33.4. What goes wrong

Do not confuse ρ with ε. A Gaussian release at $\rho = 0.5$ converts to privacy level $\varepsilon = 4.3772$. This conversion uses slack 10^{-5}; it is not privacy level $\varepsilon = 0.5$. Also, ρ covers only data processed through the private mechanism. The Census Bureau released some counts exactly; list these **invariants** separately because they are outside the guarantee.

Do not account for subsampled steps with zCDP because it does not capture subsampling amplification. In this part's running example, zCDP gives privacy level $\varepsilon = 5475.8$ at slack $\delta = 10^{-5}$ for DP-SGD. RDP gives 6.719 for the same run. Use RDP or a numerical accountant for DP-SGD pipelines. Add a ρ-zCDP release without loss as a `GaussianDpEvent` with noise multiplier $1/\sqrt{2\rho}$.

Multiple records from one user increase cost: a group of m records costs $m^2\rho$. Four records in a 0.1-zCDP release therefore cost $\rho = 1.6$.

Budgets do not renew automatically: releases about the same people compose across years. Set the accounting period and total in advance (Release and governance). Track long-term budgets in ρ, not diffprivlib's `BudgetAccountant`, which adds ε by basic composition (Keeping a budget).

33.5. References

- Bun & Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (TCC 2016): the definition.
- Canonne, Kamath & Steinke, "The Discrete Gaussian for Differential Privacy" (NeurIPS 2020): the improved conversion.

Derivations. Every result here is derived step by step in the full edition's chapter.

34. Amplification by shuffling and by iteration

34.1. What it is

In the shuffle model, each of n users randomizes their data, producing a report that satisfies local differential privacy (DP) at level ε_0. A **shuffler** removes identities and randomizes report order. The combined reports then satisfy central DP at a smaller ε, which scales as $\sqrt{\ln(1/\delta)/n}$ for $\delta > 0$. Results here use replace-one neighbours.

For binary randomized response, the analyst observes only the count of 1-reports, allowing an exact shuffled guarantee. For other local randomizers, use the clone bound of Feldman, McMillan and Talwar (2021). Another user's report matches a possible target report with probability $e^{-\varepsilon_0}$; these **clones** conceal the target's report. The **numeric clone bound** computes the guarantee directly. The paper's closed form is looser and requires enough users.

34.2. When you meet it

Shuffling is used in telemetry systems such as Prochlo (Bittau et al. 2017), where you must choose ε_0. No maintained library supports this accounting, so use shuffling_iteration.py from the full edition.

Two other uses of "shuffle" do not provide this amplification. A data loader that shuffles once per epoch is a sampler (Match the sampler to the accountant). Federated clients that join when available do not pass through a shuffler; account for them without amplification or use matrix mechanisms.

34.3. How to use it

Start with the central target. Evaluate the numeric clone bound over candidate ε_0 values and select the largest value that meets the target.

Table 34.1.: Central ε at $\delta = 10^{-6}$ after shuffling the reports of n users. "n/a" marks too few users for the closed form.

n	ε_0	Exact, randomized response	Clones, numeric	Clones, closed form
1,000	4	1.6910	3.9900	n/a
10,000	2	0.0870	0.1550	0.5009

n	ε_0	Exact, randomized response	Clones, numeric	Clones, closed form
100,000	2	0.0247	0.0450	0.1862
100,000	4	0.0847	0.1698	0.5346

The bound can determine whether a design is feasible. For 100,000 users and a target of 0.2, the closed form permits ε_0 up to 2, while the numeric bound permits 4. Use the tighter numeric bound.

Verify these assumptions before reporting shuffled ε. If any fail—for example, if report order leaks—only the local guarantee ε_0 remains.

- The order is uniformly random and hidden, with no metadata such as arrival time or network address.
- The shuffler does not collude with the analyst.
- Only honest users count in the number of users n, because colluders reveal their reports to the analyst.
- The count of users is the number of reports shuffled, so with several batches it is the size of the smallest batch.
- Each user sends one report per batch, or the reports compose like any other releases.

34.4. What goes wrong

Approximations become unreliable with few users or large ε_0. For 1,000 users at $\varepsilon_0 = 4$, a normal approximation to the count of 1-reports gives 0.9667, compared with the exact value 1.6910.

In amplification by iteration, later training noise also protects records used earlier. The result applies to one pass of noisy stochastic gradient descent (SGD) whose weights remain in a fixed convex set (Feldman et al. 2018). It requires releasing only the final model and making every step a contraction, meaning it cannot increase the distance between weight vectors. This requires a convex, smooth loss and a sufficiently small step size.

The privacy gain varies by record. For 10,000 records and noise multiplier 1, the middle record gets $\varepsilon = 0.0707$ at $\delta = 10^{-5}$, while the last record gets 5.2985, the cost of one Gaussian step. Report the worst-case value.

Deep learning typically violates both conditions. Checkpoints, logged models and selecting the best of several checkpoints reveal more than the final model alone. Steps on non-convex losses need not be contractions; Annamalai (2024) constructs a loss where the final model leaks as much as all intermediate models. Claim iteration amplification only for convex models, such as logistic regression, when releasing only final weights.

34.5. References

- Feldman, McMillan & Talwar, “Hiding Among the Clones” (FOCS 2021): the clone bound and how to compute it.

Derivations. Every result here is derived step by step in the full edition’s chapter.

Part V.

Differential privacy in machine learning

This part covers model training, deployment and review under differential privacy (DP). It builds on the Gaussian mechanism and composition in Part 3, the accountants in Part 4 and the membership-inference audits in Part 2.

Private training produces a privacy guarantee and a model. An accountant computes ε; evaluation measures utility, such as accuracy. The main algorithm is stochastic gradient descent (SGD) with clipped per-example gradients and Gaussian noise (DP-SGD). Its accountant depends on the noise multiplier, sampling rate and step count. The chapters assess both the validity of ε and model utility at that level.

Three pages present end-to-end workflows using real libraries and public data. Train a model with DP-SGD trains an Adult classifier with Opacus. Fine-tune a language model privately with LoRA adapts a small public model to private text and tests for memorized information. Release and governance recomputes and documents the guarantee, relates it to law, and gives pre-release checks.

The other chapters explain workflow decisions. Private empirical risk minimization covers convex models, followed by DP-SGD, hyperparameter tuning, user-level differential privacy and choosing ε. Public data and private fine-tuning covers public starting models. Disparate impact and Auditing private training address pre-release checks. Reference pages cover private prediction, correlated noise across steps and adaptive data analysis.

Scripts check every reported number.

35. Private empirical risk minimization

35.1. What it is

Empirical risk minimization (ERM) selects weights θ that minimize average loss on a training set. Let D be the training set and n its record count. Let λ be the strength of the ridge penalty. Training minimizes

$$\mathcal{L}(\theta; D) = \frac{1}{n} \sum_{i=1}^{n} \ell(\theta; z_i) + \frac{\lambda}{2} \|\theta\|^2, \tag{35.1}$$

where $\ell(\theta; z)$ is the loss of the weights on one record z. Differential privacy (DP) requires that replacing one record has limited effect on released weights. Add noise to the final weights, the objective or each gradient step.

Output perturbation trains normally, then adds Gaussian noise to the final weights, with slack δ. Calibrate noise to the maximum change in trained weights θ^* between neighbouring datasets D and D'. For a convex loss, ridge regularization bounds that change:

$$\|\theta^*(D) - \theta^*(D')\| \leq \frac{2L}{\lambda n}. \tag{35.2}$$

Here, L bounds the maximum norm of one record's loss gradient. For logistic regression, labels are ± 1 and feature norms are at most 1. These conditions give L of 1.

Objective perturbation adds a random linear term to the objective and minimizes it exactly. Part of privacy budget ε accounts for loss curvature, bounded by constant c (one quarter for logistic loss). The remaining budget is

$$\varepsilon' = \varepsilon - 2 \ln \left(1 + \frac{c}{n\lambda}\right). \tag{35.3}$$

As $n\lambda$ decreases, the subtracted term increases. With little data, increase the ridge strength to keep ε' positive.

Gradient perturbation adds noise at every gradient step. With random batches and clipped per-record gradients, this is noisy stochastic gradient descent (SGD), or DP-SGD, which also applies to non-convex models.

35.2. When you meet it

Use these methods for linear models, such as a linear probe on features from a public pretrained network (Public data and private fine-tuning). Objective perturbation often gives higher accuracy because the objective's full curvature reduces its noise; output perturbation relies only on ridge curvature. Use output perturbation when training is a black box.

Required data depends on n and ε. In the training objective, they enter through their product, so halving ε has the same effect as halving the sample size. On new data with d features, privacy adds less error than sampling once n is much larger than $d \ln(1/\delta)/\varepsilon^2$.

35.3. How to use it

IBM's diffprivlib offers objective perturbation as `LogisticRegression(epsilon=..., data_norm=...)`, with `data_norm` bounding each row's length. Its version 0.6.6 fails with scikit-learn 1.8, so the snippet calls the mechanism it wraps, `mechanisms.Vector`, with the curvature bound c as `function_sensitivity`. The breast-cancer data has no public data dictionary for feature bounds. So the snippet treats 500 rows as private and 69 as a public stand-in. It clips the private rows to the stand-in's log-transformed ranges and maps them to $[-1, 1]$. A constant 1 replaces the intercept, and dividing by $\sqrt{31}$ keeps each row's length at most 1.

```
import numpy as np
from diffprivlib.mechanisms import Vector
from scipy.optimize import minimize
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
public, private = np.log1p(data.data[:69]), np.log1p(data.data[69:])  # 69 stand-in
 ↪  rows fix the bounds
lo, hi = public.min(axis=0), public.max(axis=0)
X = np.hstack([2 * (np.clip(private, lo, hi) - lo) / (hi - lo) - 1, np.ones((500, 1))])
 ↪  / np.sqrt(31)
y = 2 * data.target[69:] - 1
Xt, yt, n = X[::2], y[::2], 250  # even rows train, odd rows test
lam = 1 / n  # diffprivlib's default ridge, fixed in advance

def objective(w):  # J(w) and its gradient
    m = yt * (Xt @ w)
    return np.logaddexp(0, -m).mean() + lam / 2 * w @ w, lam * w - Xt.T @ (yt / (1 +
     ↪  np.exp(m))) / n

def accuracy(eps, seed):  # eps = inf adds no noise
    vector = Vector(epsilon=eps, function_sensitivity=0.25, dimension=31, alpha=n *
 ↪  lam, n=n, random_state=seed)
    w = minimize(vector.randomise(objective), np.zeros(31), jac=True, options={"gtol":
 ↪  1e-9}).x
    return np.mean(np.sign(X[1::2] @ w) == y[1::2])

print(" ".join(f"{np.mean([accuracy(e, s) for s in range(20)]):.3f}" for e in [1, 4,
 ↪  np.inf]))  # 0.783 0.946 0.964
```

Across 20 noise draws, test accuracy is 0.783 at $\varepsilon = 1$ and 0.946 at $\varepsilon = 4$. Without noise, it is 0.964. With 250 training records, accuracy is close to the non-private result only at $\varepsilon = 4$. No parameter was tuned on the private rows, so the guarantee is pure DP for the 250 training rows under replace-one adjacency. The 69 stand-in rows receive no protection.

35.4. What goes wrong

The common failure is deriving a bound from private data. If omitted, `data_norm` is calculated from training rows and raises `PrivacyLeakWarning`; the guarantee is then invalid. A range table computed from those rows also leaks through its minima and maxima. Set bounds from public knowledge or public rows, then clip private rows to them.

The solver can also violate assumptions: output and objective perturbation require the exact minimizer. If optimization stops with gradient norm g, sensitivity increases by $2g/\lambda$. This increase is negligible relative to exact sensitivity $2L/(\lambda n)$ only when g is much less than L/n. Set a corresponding tolerance and inspect the final gradient norm, not just the iteration count.

The sensitivity bound also requires ridge curvature in every direction. Scikit-learn's loss, reused by diffprivlib, leaves the intercept unpenalized, so the snippet adds a constant feature. Set λ by a fixed rule or public data; private-data tuning must be accounted for (Hyperparameter tuning). These guarantees use replace-one neighbours, unlike later add/remove accounting, and the bounds are not directly interchangeable. State the relation with every ε.

Derivations. Every result here is derived step by step in the full edition's chapter.

36. DP-SGD

36.1. Why this matters

Nearly every deep-learning system with differential privacy (DP) uses the differentially private version of stochastic gradient descent (SGD), called DP-SGD: it clips per-record gradients and adds Gaussian noise at each step (Abadi et al. 2016). Clipping bounds each record's effect on the model; noise obscures that effect. An **accountant** composes step costs into privacy level ε and slack δ.

The accountant uses only the noise, sampling rate and number of steps; it does not inspect the model. These parameters determine ε, while clipping norm and learning rate can be tuned for accuracy. Most failures arise when code violates a proof assumption, so implementation checks are as important as parameter choices. Train a model with DP-SGD demonstrates the full Opacus workflow.

36.2. The algorithm and its guarantee

A training set holds n records. The gradient of one record's loss is its **per-example gradient**, written g_i for record i. Each step of DP-SGD does five things.

1. Sample a batch. Each record enters independently with probability q, the **sampling rate**, which is called Poisson sampling.
2. Compute the per-example gradient of each record in the batch.
3. Clip each gradient longer than the **clipping norm** C to length C.
4. Sum the clipped gradients, and add Gaussian noise with standard deviation σC to each coordinate, where the **noise multiplier** σ measures the noise in units of C.
5. Divide the noisy sum by the expected batch size $B = qn$, a fixed public number, and step against it, scaled by the **learning rate** η.

Write θ_t for the weights at step t and S_t for its batch. With Gaussian noise ξ_t drawn afresh on each coordinate, the update is

$$\theta_{t+1} = \theta_t - \frac{\eta}{B}\Big(\sum_{i \in S_t} g_i \min\Big(1, \frac{C}{\|g_i\|}\Big) + \xi_t \Big), \qquad \xi_t \sim \mathcal{N}(0, \sigma^2 C^2 I). \tag{36.1}$$

Releasing every intermediate set of weights then satisfies (ε, δ)-DP for **add/remove neighbours**, datasets that differ by one record (Amplification by subsampling).

This part's running example (Section 36.7) is the run of Train a model with DP-SGD. A network with one hidden layer of 64 units learns from 30,000 people of the Adult census survey whether a person's income exceeds \$50K. It runs 300 steps with $\sigma = 0.973$, and each person joins a batch

with probability $q = 1/30$, so a batch holds about 1,000 people. At the $\delta = 10^{-5}$ of the book's rule (How small δ must be), the accountant Opacus uses gives $\varepsilon = 3.99$ for this run. The Rényi differential privacy (RDP) accountant, a standard and slightly loose upper bound, gives 4.50. To pick σ for a target ε, let the accountant you will report solve for it (Calibrate and account a DP-SGD run).

36.3. The implementation checklist

The guarantee requires four implementation properties. Violating one does not necessarily stop training or reporting.

First, one record may change the clipped sum by at most C. Batch normalization violates this condition because its statistics let one record affect other records' gradients. Use group or layer normalization instead; these operate on each record separately. With `strict=False`, Opacus's validator lists incompatible layers rather than raising an error.

```
from opacus.validators import ModuleValidator
from torchvision.models import resnet18

model = resnet18(weights=None)
print(len(ModuleValidator.validate(model, strict=False)))  # 20
model = ModuleValidator.fix(model)
print(len(ModuleValidator.validate(model, strict=False)))  # 0
```

Set C in advance using public data or a private estimate. Estimating C from the current batch makes the noise data-dependent.

Second, match the accountant to the implemented sampler (Amplification by subsampling). Poisson accounting benefits from the chance that each record is absent from a batch. A loader that shuffles once per epoch and divides data into fixed batches includes every record exactly once per epoch, eliminating that amplification. Its true ε may greatly exceed the Poisson estimate.

Fresh fixed-size batches can be accounted with a Poisson accountant using the batch's sampling rate and half the noise multiplier. A sampled record replaces another, so the sum can change by $2C$. In the running example, `dp_accounting`'s numerical accountant gives $\varepsilon = 23.03$, compared with 3.98 for Poisson sampling; this bound uses add/remove neighbours.

Third, divide the noisy sum by the public expected batch size B, not the realized size. The realized size is one larger when the protected record is included, so dividing by it reveals that inclusion.

Fourth, account for every step that accessed the data, including discarded runs during tuning.

State the neighbouring relation with each ε, and make data-dependent preprocessing private. Use user-level privacy when one person contributes multiple records. For released models, enable Opacus's `secure_mode=True`; floating-point sampling does not produce the exact Gaussian noise assumed by the proof (Discrete and floating-point-safe mechanisms).

36.4. Clipping and the learning rate

Clip each record's gradient separately. PyTorch's `torch.nn.utils.clip_grad_norm_` clips the batch gradient, which does not bound any single record's contribution and provides no privacy guarantee.

The clipping norm C does not change ε, but it affects accuracy. Small C clips more gradients, reducing the contribution of high-gradient records, which may be rare (Disparate impact). Large C increases noise because its standard deviation is σC.

When all gradients are clipped to length C, the noise also scales with C. The update therefore depends on the product ηC, where η is the learning rate. If clipping is aggressive, tune this product. In the running example at $\varepsilon = 4$, fixing $\eta C = 2$ keeps test accuracy between 0.8446 and 0.8464 for C from 0.1 to 10. Fixing $\eta = 2$ instead gives accuracy 0.7869 at $C = 10$.

Noise and clipping both affect the optimal learning rate. Noise lowers it (Optimization in the full edition); clipping raises it because gradients become shorter. Their effects do not cancel predictably. Retune the learning rate when σ or C changes, and include all search runs in the privacy cost.

Gradient norms change during training. In the running example, the median per-example norm on public rows falls from 1.444 to 0.044 over 300 steps. Thus, $C = 1$ clips every row initially but only one quarter at the end. Adaptive clipping adjusts C toward a gradient-norm quantile using a private count; Opacus enables it with `clipping="adaptive"`. This count consumes budget, increasing gradient noise at the same σ.

36.5. Batch size at a fixed budget

Averaging the noisy sum over B records divides the added noise by B, so larger batches dilute it. At a fixed budget and number of epochs, a larger batch also means fewer steps. So the noise multiplier must rise to keep ε, but more slowly than B does. For a small sampling rate q and a σ well above 1, the accountant's cost over a fixed number of epochs depends on q/σ^2. So σ only has to grow like $\sqrt{q}$. Since $q = B/n$, with n the number of records, the noise on the averaged gradient falls with the batch size,

$$\sigma \propto \sqrt{\frac{B}{n}}, \qquad \frac{\sigma C}{B} \propto \frac{C}{\sqrt{Bn}}. \tag{36.2}$$

So choose the largest batch your hardware allows, with a larger learning rate for the fewer steps. When it does not fit in memory, Opacus's `BatchMemoryManager` splits the sampled batch into physical batches that fit, and still steps once per sampled batch.

36.6. Making it fast

Computed one record at a time, DP-SGD can run tens or hundreds of times slower than ordinary training. Two fixes cost no accuracy. Vectorized per-example gradients, from `torch.func.vmap` or Opacus's hooks, compute the same clipped sum in parallel. **Ghost clipping** also saves memory,

because it gets each record's gradient norm from each layer's inputs and errors without the gradient itself. Opacus turns it on with `grad_sample_mode="ghost"`.

The third fix, microbatching, costs accuracy. It clips the average gradient of each group of m records. One record can change its group's clipped average by up to $2C$, so the noise doubles. The update then averages over B/m groups rather than B records, so the noise on each coordinate of the averaged update becomes

$$2m \cdot \frac{\sigma C}{B}. \tag{36.3}$$

On the running example at $\sigma = 0.973$, groups of 4 cut test accuracy from 0.8444 to 0.8206, more than a quarter of the way down to always predicting the larger class. So use microbatching only as a last resort.

36.7. What privacy costs in accuracy

Table 36.1 trains the running example at five budgets with $C = 1$ and $\eta = 2$, ten times the best non-private learning rate on public data, and scores it on the 16,842 test rows. Always predicting the larger class, an income of at most \$50K, scores 0.759.

Table 36.1.: Test accuracy of the running example under DP-SGD at five budgets, with $\delta = 10^{-5}$ and Opacus's accountant.

	Noise multiplier	Test accuracy (mean ± standard deviation over 3 runs)
$\varepsilon = 0.25$	8.125	0.7933 ± 0.0200
$\varepsilon = 1$	2.383	0.8406 ± 0.0023
$\varepsilon = 2$	1.431	0.8440 ± 0.0014
$\varepsilon = 4$	0.973	0.8447 ± 0.0010
$\varepsilon = 8$	0.722	0.8452 ± 0.0012
Clipped, no noise	0	0.8454 ± 0.0005
Non-private SGD	—	0.8449 ± 0.0029

Clipping without noise scores as well as non-private SGD here, so the whole loss comes from the noise, and it grows steeply as ε shrinks. Going from $\varepsilon = 8$ to 1 costs half a point, while going from 1 to 0.25 costs almost 5, and the spread between runs grows eightfold. The noise grows in total length with the square root of the number of trained weights, while the clipped signal stays at most C, so a smaller model or public pretraining often helps.

36.8. Cheat sheet

Table 36.2.: DP-SGD starting settings and checks.

Setting	Start with	Check
δ	The book's rule	n counts people
Batch size	256 to 4,096, as large as fits	Noise on the average is $\sigma C/B$
Epochs	Fixed before calibrating; Adult uses 10	σ recomputed if they change
Noise multiplier	From the accountant you report	A second accountant gives the same ε
Clipping norm	1, or the median norm on public data	Per record, never `clip_grad_norm_`
Optimizer, learning rate	SGD with momentum 0.9; several times the non-private learning rate (ten on Adult), chosen on public data	Retuned when σ or C changes
Sampler	Poisson	Fixed-size batches: account at $\sigma/2$ (add/remove only)
Report	ε, δ, neighbours, accountant	Every run counted

Rules of thumb

- Fix ε, δ and the number of epochs first, since they are the contract. Then choose the batch, let the accountant set σ, and tune the learning rate last.
- Find the ε below which accuracy collapses on your task before you promise a budget.

36.9. References

- Abadi et al., "Deep Learning with Differential Privacy" (CCS 2016): DP-SGD and the moments accountant.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): what to clip, how to tune and what to report.

Derivations. Every result here is derived step by step in the full edition's chapter.

37. Train a model with DP-SGD

37.1. Why this matters

You have a PyTorch training loop on sensitive data, and it must become private. The algorithm is stochastic gradient descent (SGD) with differential privacy (DP), called DP-SGD, which clips each record's gradient and adds noise at every step (DP-SGD). Opacus does it with one call that wraps the model. The hard part is the decisions around that call. A mistake in one of them leaves code that trains as before but reports a false privacy level ε.

This chapter trains on Adult, a census survey of 48,842 people from the UCI Machine Learning Repository, to predict whether a person earns more than $50,000 a year. Calibrate and account a DP-SGD run plans the privacy side of this run, and this chapter keeps training, tuning and utility. The script practitioner_dp_sgd.py runs every step except tuning, which practitioner_tuning.py does. Each runs in one to two minutes without a GPU.

```
uv run python code/part05-dp-ml/practitioner_dp_sgd.py
uv run python code/part05-dp-ml/practitioner_tuning.py
```

37.2. Fix the privacy unit, n and δ

The guarantee protects exactly one record, so first decide what a record is (User-level differential privacy). Adult has no person identifier, so one row counts as one person because of how the survey was collected. When your data has an identifier, count distinct identifiers, not rows.

The script splits the rows with seed 0 into 30,000 private training rows and 16,842 test rows. The other 2,000, the stand-in rows from here on, play the part of public data. The script chooses settings on them but never trains on them. They come from the same survey, so this is an assumption for the demonstration. In production the role belongs to public data, and without any, tuning costs privacy (Section 37.10).

The slack δ depends on the number of people n in the training set, not in the whole dataset. The book's rule (How small δ must be) gives $\delta = 10^{-5}$ for $n = 30{,}000$.

37.3. Preprocess from public facts

Every statistic computed from the private rows is a release that the accountant does not count. A scaler fitted on the training set leaks each column's mean and spread. An encoder fitted on it leaks which categories occur, even a country that only one person comes from. So the script takes its category lists from the dataset's documentation, and scales each number by bounds the

survey publishes, such as ages from 17 to 90. Capital gains and losses go on a log scale first, and a missing value becomes its own category.

```
def features(rows: pl.DataFrame) -> Array:
    """One-hot categories from the documented lists, numbers scaled by public
     ↪  bounds."""
    rows = rows.with_columns(pl.col("capital-gain", "capital-loss").log1p())
    numeric = [
        np.clip((rows[name].to_numpy() - low) / (high - low), 0.0, 1.0) for name, (low,
 ↪  high) in NUMERIC_BOUNDS.items()
    ]
    encoder = OneHotEncoder(categories=list(CATEGORIES.values()), sparse_output=False)
    text = rows.select(pl.col(list(CATEGORIES)).fill_null("?")).to_numpy()
    encoder.fit(text[:1])  # the categories are given, so fitting learns nothing from
 ↪  the rows
    return np.hstack([np.column_stack(numeric), encoder.transform(text)])
```

This gives 91 features for a network with one hidden layer of 64 units and 6,018 weights. To check this step in other code, find every `fit` call and ask what data it sees.

37.4. Train the baselines

A private accuracy means little on its own, so train three baselines with the same model, batch size, epochs and optimizer. The non-private model shows the best you can expect. A model with clipping and no noise separates the cost of clipping from the cost of noise. The majority class shows the floor, where a collapsed private model lands.

```
non-private: test accuracy 0.847
clip only (C=1, no noise): test accuracy 0.845
always '<=50K': test accuracy 0.759
```

Clipping alone costs little here. If clip-only accuracy were far below non-private, no budget would fix it, because the problem would be the clipping norm or the learning rate.

37.5. Check the model

Opacus computes one gradient per record, and some layers break that. Batch normalization is the common case, because its statistics let one record change every other record's gradient. The validator lists such layers before you train.

```
errors = ModuleValidator.validate(model, strict=False)
print(f"validator errors: {len(errors)}")
```

Here it finds none. For a model with such layers, `ModuleValidator.fix` replaces batch normalization with group normalization (DP-SGD shows it on a ResNet).

37.6. Choose the clipping norm, batch and epochs

Fix these settings before you touch the private data, because choosing them on it costs privacy. The usual guide for the clipping norm C, the length each record's gradient is cut to, is the typical gradient norm on public data. So the script measures it on the stand-in rows.

```
wrapped = GradSampleModule(model)  # adds hooks that keep one gradient per row
wrapped.train()
logits: torch.Tensor = wrapped(split.x)
nn.CrossEntropyLoss()(logits, split.y).backward()  # fills p.grad_sample for every
 ↪  weight p
norms = stacked_norms(wrapped)
```

```
at initialisation: median norm 1.444  90th percentile 1.538
fraction clipped at C=1: 1.000
```

The median rule would give a C of about 1.4. The script keeps the usual default $C = 1$, just below the median, so every gradient is clipped at the start. That costs nothing, because when every gradient is clipped, only the learning rate times C matters (DP-SGD).

Make the batch as large as you can afford. The reason is that the noise on the averaged gradient is $\sigma C/B$, where B is the expected batch size and the noise multiplier σ is the noise's standard deviation in units of C. With $B = 1{,}000$, Opacus sets the sampling rate q, the chance that a row joins a batch, to $1/30 = 0.0333$. Ten epochs then make 300 steps. Fix the epochs now, because the noise is calibrated to them.

The optimizer is SGD with momentum 0.9. The script picks its learning rate on the stand-in rows too. It trains a non-private model on 1,500 of them at six rates and scores each on the other 500.

```
  lr=0.05: 0.814  lr=0.1: 0.816  lr=0.2: 0.826  lr=0.5: 0.792  lr=1: 0.812  lr=2: 0.768
non-private learning rate 0.2; DP-SGD learning rate 2 (x10)
```

DP-SGD then uses ten times the winning rate, a rule of thumb rather than a measurement. Private runs usually need a larger step, because clipped gradients are shorter (DP-SGD). No private row was used, so this choice costs no privacy.

37.7. Calibrate the noise

The target is a privacy level of $\varepsilon = 4$ at the slack $\delta = 10^{-5}$ of Section 37.2. For this q and these epochs, Opacus's `get_noise_multiplier` finds the smallest σ that meets it. With its privacy loss random variable (PRV) accountant it gives $\sigma = 0.973$, and Calibrate and account a DP-SGD run shows the call. That σ holds only for those epochs and that q, so recompute it if you change either.

37.8. Train

The privacy engine wraps the model, optimizer and data loader. Its optimizer clips per-record gradients and adds noise. Its loader draws Poisson batches, independently including each row with probability q, matching the accountant. Because the script initially uses fixed batches, `make_private` must replace its loader.

```
engine = PrivacyEngine(accountant="prv")
optimizer = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9)
model, optimizer, data_loader = engine.make_private(
    module=model,
    optimizer=optimizer,
    data_loader=data_loader,
    noise_multiplier=sigma,
    max_grad_norm=clip,
)  # Poisson sampling is the default: each row joins each batch with probability q
```

The training loop runs inside `BatchMemoryManager`, which splits each logical batch of about 1,000 rows into physical batches of at most 500. The optimizer still steps once per logical batch, so the accounting does not change.

```
with BatchMemoryManager(
    data_loader=data_loader, max_physical_batch_size=PHYSICAL_BATCH,
 ↪  optimizer=optimizer
) as batches:
    for x, y in batches:  # Opacus splits each logical batch; it steps once per logical
     ↪  batch
        optimizer.zero_grad()
        loss = criterion(model(x), y)
        loss.backward()
        optimizer.step()
```

```
steps=300  q=0.0333
epsilon (Opacus PRV) = 3.99
test accuracy at epsilon=4: 0.846
```

The script disables `secure_mode` to seed noise and reproduce results; the run record documents this. For production releases, set `secure_mode=True` in `PrivacyEngine`. It uses a secure generator and addresses floating-point weaknesses in ordinary Gaussian sampling (Discrete and floating-point-safe mechanisms).

37.9. Cross-check ε

An ε depends on an accountant's implementation. The script independently recomputes it with Google's `dp_accounting`: the privacy loss distribution (PLD) accountant gives 3.98, compared with Opacus's 3.99. The looser Rényi differential privacy (RDP) accountant gives 4.50. Name the accountant in every statement. Calibrate and account a DP-SGD run explains this check and potential discrepancies.

37.10. Tune honestly

This run selected all hyperparameters using public stand-in data, so tuning incurred no privacy cost. Without public data, every private run that affects the release consumes privacy, including discarded runs (Hyperparameter tuning).

The tuning script compares six learning rates under total $\varepsilon = 4$. A grid evaluates all six and composes their costs. Random search draws a Poisson number of models with mean 4, selects each rate from the six candidates, and releases only the best. Since only RDP supports a random run count, the script uses it throughout; its single-run σ exceeds the PRV value above.

```
def random_search(sigma: float) -> dp_accounting.DpEvent:
    """A Poisson number of runs with mean 4 (shape=inf selects the Poisson case); only
     ↪  the best is released."""
    return dp_accounting.dp_event.RepeatAndSelectDpEvent(one_run(sigma),
     ↪  mean=MEAN_RUNS, shape=math.inf)


def sigma_for(strategy: Callable[[float], dp_accounting.DpEvent], epsilon: float,
 ↪  delta: float) -> float:
    """The smallest noise multiplier per run that keeps the whole strategy within
     ↪  (epsilon, delta).

    The search starts at 0.1 and tries 1 first.
    """
    return dp_accounting.calibrate_dp_mechanism(
        dp_accounting.rdp.RdpAccountant, strategy, epsilon, delta,
 ↪  dp_accounting.LowerEndpointAndGuess(0.1, 1.0)
    )
```

```
one run, no tuning: sigma=1.029
grid of 6, all runs counted: sigma=1.816
random search, Poisson mean 4 runs: sigma=1.256
naive grid of 6 at sigma=1.029: reports epsilon=4.00
naive grid, true epsilon of all 6 runs = 9.84
```

The naive grid illustrates a common error: it trains six models at single-run noise, reports one model's ε, but spends 9.84. Random search requires only a moderate increase in σ, from 1.029 to 1.256.

With seed 0, the search draws two runs, both at learning rate 0.05, and reaches test accuracy 0.828, compared with 0.846 for the publicly tuned run. Do not rerun the search until a preferred result appears; the guarantee depends on the draw. Use private-data tuning only when public data cannot provide a suitable substitute.

37.11. Measure utility and groups

Train at several privacy budgets while holding other settings fixed, then compare with the baselines. Set the release budget in advance, as the script does at $\varepsilon = 4$. The script evaluates on private rows, and that evaluation is not included in the training ε (Choosing ε).

Table 37.1.: Test accuracy on Adult against the privacy budget, with $\delta = 10^{-5}$ and one seed.

Run	Noise multiplier	Test accuracy
Non-private	—	0.847
Clipped, no noise	0	0.845
$\varepsilon = 8$	0.722	0.846
$\varepsilon = 4$	0.973	0.846
$\varepsilon = 2$	1.431	0.845
$\varepsilon = 1$	2.383	0.843
$\varepsilon = 0.25$	8.125	0.793
Always "≤50K"	—	0.759

Privacy costs almost nothing on Adult down to $\varepsilon = 1$. Only at $\varepsilon = 0.25$ does accuracy drop, by about five points, and the other differences are too small to read on one seed.

Adult is easy. Its training set is large, so a batch of 1,000 still has a small sampling rate and divides the noise by 1,000. Its model is small, and the noise's total length grows with the square root of the number of trained weights. So expect a steeper curve with millions of weights trained from scratch, a few thousand records, a rare class or many more epochs. Even here, five points are lost at $\varepsilon = 0.25$ (Table 37.1), and public pretraining is the usual remedy: on the 1,500 public stand-in rows it wins most of them back (Public data and private fine-tuning).

An average can hide a group that pays. So check accuracy by group with exact (Clopper–Pearson) intervals, which stay valid for small groups (Disparate impact).

```
in_group = sex == group
correct, total = int(right[in_group].sum()), int(in_group.sum())
interval = binomtest(correct, total).proportion_ci(confidence_level=0.95,
 ↪  method="exact")
```

```
non-private Female: 0.924 [0.917, 0.931]  (5,186/5,613)
non-private Male: 0.808 [0.800, 0.815]  (9,071/11,229)
DP Female: 0.922 [0.915, 0.929]  (5,177/5,613)
DP Male: 0.807 [0.800, 0.815]  (9,065/11,229)
```

At $\varepsilon = 4$ neither group pays a measurable price, since each private interval almost covers its non-private one. The gap between the groups exists without privacy too, so it is the baseline for the private model.

37.12. When it goes wrong

Table 37.2.: Common DP-SGD failures and their fixes.

Symptom	Likely cause	What to do
The validator reports errors	Batch normalization or an unsupported custom layer	`ModuleValidator.fix`, then build the optimizer on the fixed model's weights and retrain the baseline
"Model needs to be in training mode"	A model last used for evaluation	`model.train()` before `make_private`
Out of memory	Per-example gradients for a large physical batch	Lower `max_physical_batch_size`, or use ghost clipping: pass `grad_sample_mode="ghost"` and `criterion=` to `make_private`, which then returns the model, optimizer, loss function and loader, and train with that loss
Ten times slower than usual	Per-example gradients for every weight	Train fewer weights: freeze a public backbone, use adapters
Accuracy near the majority class	Noise too large for the signal	Check $\sigma C/B$; raise the batch, or train fewer epochs so that the accountant needs a smaller σ; start from a public model
ε looks too good	A shuffling loader with `poisson_sampling=False`, uncounted tuning runs, or epochs changed after calibration	Check that the loader is Opacus's `DPDataLoader`, as the record's sampler does; `engine.get_epsilon` and the cross-check report the same ε either way. Recount runs and epochs

Accuracy and privacy fail independently. A run whose ε is wrong usually trains normally, so only the cross-check and the audit of Auditing private training catch it.

37.13. Record the run

A reviewer can only check a claim whose inputs are on record. So the script ends by writing a record of the run. Release and governance lists its fields and turns it into a statement. The script reads the sampler from the loader Opacus returned and `secure_mode` from the engine, so the record cannot claim Poisson sampling that the code did not do.

37.14. Checklist

Table 37.3.: The steps of training with DP-SGD, their defaults and their checks.

Step	Default	Check
Unit, n, δ	Person; δ by the book's rule	n counts people in the training set
Preprocessing	Public categories and bounds	No `fit` on private rows
Baselines	Non-private, clip-only, majority class	Clip-only close to non-private
Model	No batch normalization	`ModuleValidator.validate` returns nothing
Clipping, batch, epochs	C near or below the median norm on public data; large B; epochs fixed	Chosen without private data
Noise and cross-check	As in Calibrate and account a DP-SGD run	Epochs and q match the run
Training	Poisson sampling, `BatchMemoryManager`, `secure_mode=True` for release	The loader is Opacus's
Tuning	On public data; else random search with a Poisson number of runs	Every private run counted
Utility and groups	Several budgets; per-group exact intervals	Compared with the baselines
Record	Every parameter and version written	ε recomputable from it

37.15. References

- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): the decisions of this chapter, with what to report.
- The Opacus documentation and tutorials (opacus.ai): the privacy engine, `BatchMemoryManager`, ghost clipping and the validator.

Derivations. This workflow draws on the full edition's chapters on DP-SGD in the full edition, hyperparameter tuning in the full edition, disparate impact in the full edition and choosing ε in the full edition.

38. Hyperparameter tuning and private selection

38.1. Why this matters

Stochastic gradient descent (SGD) with clipped gradients and Gaussian noise satisfies differential privacy (DP) at the accountant's reported level for one run; this is DP-SGD. Hyperparameter search trains and evaluates multiple models on private data, then selects one. The reported privacy level ε must account for the entire search.

Selecting only the best of a fixed number of runs can cost as much as releasing every model. On Adult, six runs each at $\varepsilon = 4$ cost 9.84 in total. If the run count is drawn from a distribution fixed in advance, the search can cost only slightly more than one run.

38.2. The procedure and the report

The least costly tuning uses no private data. Runs on public or proxy data and published settings for similar tasks incur no privacy cost. Fix a large batch, epoch count and small clipping norm in advance. When the norm is below typical gradient size, accuracy often depends mainly on the product of clipping norm and learning rate, so search only that product (DP-SGD).

If private-data tuning is necessary, specify the complete search in advance: candidate set, distribution for the number of runs (Section 38.4), private score (Section 38.5) and validation set. Extending or restarting the search after seeing results makes it data-dependent and invalidates the stated accounting.

Account for all runs. For two or three candidates, compose the cost of each run (Section 38.3). For larger searches, use random search with a random run count (Section 38.4). Include runs that crashed or were used for debugging, and evaluations used for early stopping.

Log the candidate set, sampled run count, and each run's settings and score. Report both the released run's ε, which assumes public hyperparameters, and the whole procedure's ε, which is the release guarantee. An audit tests only the released model and cannot verify the search cost. At $\delta = 10^{-5}$, the book's rule for 30,000 records (How small δ must be), the Adult search in Section 38.6 reports:

> DP-SGD ran for 300 steps with Poisson sampling at rate 1/30. It clipped gradients to norm 1 and added noise with multiplier 1.256, so one run satisfies $(2.73, 10^{-5})$-DP. The learning rate was chosen by random search over $\{0.05, 0.1, 0.2, 0.5, 1, 2\}$. The number of runs was drawn from a Poisson distribution with mean 4, and each run was scored on 2,000 validation records treated as public. The whole procedure satisfies $(4.00, 10^{-5})$-DP for add/remove neighbours, datasets that differ by one record. It was computed with `dp_accounting` from the Rényi bound of Papernot & Steinke (2022).

38.3. Paying for every run

Choosing among eight learning rates does not limit leakage to three bits. The selected model depends on all run outcomes. Even if each run is labelled only "good" or "bad", a bad selected run implies that all candidates were bad, revealing information about every run. Tuning without privacy and then training once privately can also leak: outliers may change the selected regularization weight, revealing their presence (Membership inference).

The safe approach charges for every run because selection uses all run outputs (Composition). For DP-SGD, k runs of T steps cost the same as one run of kT steps. One run of Part 4's example—10,000 steps, sampling rate 0.01 and noise multiplier 1—costs $\varepsilon = 6.71$ at $\delta = 10^{-5}$. Ten runs cost 27.23, about four times the single-run cost rather than ten times, because conversion to ε incurs the δ cost once.

38.4. A random number of runs

The way out is to draw the number of runs K at random, independently of the data. A search whose runs all came out bad then most likely drew few runs, which says little about the data. So the whole search costs about one run, plus a term that grows with the logarithm of the expected number of runs. For DP-SGD, use the version of Papernot & Steinke (2022), which works with the Rényi differential privacy (RDP) that DP-SGD's accountants already compute (Rényi DP).

To use it, fix the candidate set and the distribution of K, either Poisson or a truncated negative binomial with a shape parameter ν. That family includes the geometric distribution, $\nu = 1$, which is the **random stopping** of Liu & Talwar, and the logarithmic one, $\nu = 0$. Then draw K, run that many candidates drawn at random from the set, and release the best one by a private score. To get the cost from Google's `dp_accounting`, wrap the event of one run in a `RepeatAndSelectDpEvent`. Give it ν as its `shape` (infinite for Poisson), and compose it in an `RdpAccountant`.

NumPy draws K with `rng.poisson(mean)`, `rng.geometric(1 / mean)` or `rng.logseries(p)`. The last takes p rather than a mean, so solve its mean, $-p/((1-p)\ln(1-p))$, for p.

Table 38.1.: The ε at $\delta = 10^{-5}$ of a search over runs of the running example (one run: 6.71), from the full edition's chapter.

Expected runs	10	100	1,000
Composition	27.23	145.17	1035.93
Poisson	13.67	67.77	302.57
Geometric ($\nu = 1$)	11.31	14.32	16.76
Logarithmic ($\nu = 0$)	10.08	11.95	13.46

In Table 38.1, the geometric and logarithmic searches barely grow from 10 to 1,000 runs, while composition grows almost linearly. A smaller ν gives a cheaper bound, but the search also stops after one run more often. The Poisson bound needs runs that are already very private, which a run at $\varepsilon = 6.71$ is not, so here it grows almost like composition. A random number of runs may

also miss the best candidate. With eight candidates and a geometric number of runs with mean eight, it never tries the best one with probability 0.4667.

The theorems have three conditions, and each is easy to break. First, each run must draw its candidate independently from a fixed distribution, as random search does. Bayesian optimization picks each candidate from earlier scores, so it is not covered and needs adaptive composition. Second, the number of runs must be drawn independently of the data. Third, the score that ranks the runs must be private and counted in each run's cost.

38.5. Selecting by a noisy validation score

Sometimes the candidates are fixed in advance, such as public pretrained checkpoints, and only the choice among them uses private data. That is a **selection** problem, which Selection solves with the exponential mechanism, a random choice weighted toward high scores. Score each candidate by its number of correct predictions, which one record changes by at most 1, not by an accuracy, which divides by a private set size. The mechanism's shortfall from the best count then grows only with the logarithm of the number of candidates.

For the random-number bounds, add Gaussian noise to each run's count of correct predictions, since an exact validation accuracy leaves the validation records unprotected. With noise of standard deviation 20, one noisy count costs $\varepsilon = 0.182$. If the validation set is disjoint from the training set, each record pays only one of the two costs. So the event for one run is the costlier of its training and its count, not their sum. Otherwise compose both into one event. On the running example the counts are the cheaper, so they add nothing to the total.

38.6. Tuning on Adult

The script practitioner_tuning.py tunes the learning rate of a small network on 30,000 Adult census records, with 300 steps per run. It asks the accountant for the noise multiplier per run that each strategy can afford.

Table 38.2.: Tuning the learning rate on Adult at a total of $\varepsilon = 4$, $\delta = 10^{-5}$, with `dp_accounting`'s RDP accountant.

Strategy	Noise multiplier per run	Total ε
One run, no tuning	1.029	4.00
Grid of 6 at the noise of one run, one run's ε reported	1.029	9.84 (reported as 4.00)
Grid of 6, every run counted	1.816	4.00
Random search, Poisson number of runs with mean 4	1.256	4.00

The second row is the reporting bug of Section 38.3. This snippet recomputes it, together with the numbers of the report in Section 38.2.

```
import dp_accounting


def one_run(sigma: float) -> dp_accounting.DpEvent:
    step = dp_accounting.PoissonSampledDpEvent(1 / 30,
 ↪  dp_accounting.GaussianDpEvent(sigma))
    return dp_accounting.SelfComposedDpEvent(step, 300)


def epsilon(event: dp_accounting.DpEvent) -> float:
    return dp_accounting.rdp.RdpAccountant().compose(event).get_epsilon(1e-5)


search = dp_accounting.dp_event.RepeatAndSelectDpEvent(one_run(1.256), mean=4,
 ↪  shape=float("inf"))
naive_grid = dp_accounting.SelfComposedDpEvent(one_run(1.029), 6)
print(f"{epsilon(one_run(1.256)):.2f}")  # 2.73
print(f"{epsilon(search):.2f}")  # 4.00
print(f"{epsilon(naive_grid):.2f}")  # 9.84
```

With seed 0, the search drew only two runs, both at the smallest learning rate. So its model reaches only 0.828 on the test set, against 0.846 for the run of Train a model with DP-SGD, tuned for free on public stand-in rows.

38.7. Cheat sheet

Table 38.3.: Which tuning strategy for which search.

Situation	Strategy	Cost
Two or three candidates	Fixed grid, composition	k runs of T steps = one run of kT steps
Many candidates	Random search; compute Poisson, geometric and logarithmic for your per-run noise and mean, and take the cheapest	Poisson wins only for very private runs; at one run of 6.71, logarithmic is cheapest at every size in Table 38.1, but it stops after one run more often
Fixed candidates	Exponential mechanism on correct counts	Shortfall grows with the log of the candidate count
Trained candidates, disjoint validation set	Exponential mechanism or noisy counts	The larger of training and selection costs

Rules of thumb

- Tune on public or proxy data when you can. Otherwise fix the search before touching the private data, and never extend it.

- Report the whole search's ε, never the selected run's alone.

38.8. References

- Papernot & Steinke, "Hyperparameter Tuning with Renyi Differential Privacy" (ICLR 2022): the Poisson and truncated negative binomial bounds for DP-SGD.
- Liu & Talwar, "Private Selection from Private Candidates" (STOC 2019): random stopping and its bound of three times one run.

Derivations. Every result here is derived step by step in the full edition's chapter.

39. User-level differential privacy

39.1. Why this matters

The differential privacy (DP) guarantees described so far protect one record. Stochastic gradient descent (SGD) with DP is called DP-SGD. In this part's DP-SGD Adult run, record-level privacy loss is $\varepsilon = 3.98$. A messaging user may contribute hundreds of messages, whose combined information can exceed the protection for any one message.

A DP guarantee protects the changes defined by neighbouring datasets, so **the neighbouring relation determines the privacy unit**. Adult has no household identifier. To illustrate larger units, this chapter assigns its 30,000 training rows to 3,000 synthetic households, each containing similar rows. The median household has 4 rows and the largest has 359. Here a household is the user and a row is a record.

39.2. The privacy unit and δ

Take a mechanism M, and call two datasets neighbours, D and D', when they differ by adding or removing all the records of one user. The mechanism is **user-level** (ε, δ)-DP, with a small slack δ, if every such pair and every set S of outputs satisfy

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{39.1}$$

Example-level DP is the same inequality for datasets that differ in one record. So every property and accountant of Part 4 holds for user-level DP unchanged. What changes is the sensitivity, because one user can move the computation with all of their records at once.

Choose the privacy unit—record, user, household or device—to specify whose participation is protected. Report it with each ε (Release and governance). It also determines δ: the book's rule (How small δ must be) counts users. For 3,000 households it gives 10^{-4} (Section 39.8), so this chapter's 10^{-5} is stricter.

The general guarantee for a user with k records is **group privacy** (Post-processing, group privacy and composition). An example-level (ε, δ)-DP mechanism also protects users with at most k records at privacy level $k\varepsilon$ and slack

$$\delta_k = \delta \, \frac{e^{k\varepsilon} - 1}{e^{\varepsilon} - 1}. \tag{39.2}$$

The slack grows roughly as $e^{(k-1)\varepsilon}$; a record-level δ is not a user-level δ. To keep user-level slack at 10^{-5}, per-record δ must be much smaller, which increases per-record ε. The full edition's Rényi accountant gives the Adult run $\varepsilon = 4.50$ per row. A two-row household therefore has $\varepsilon = 12.43$, not twice 4.50; for ten rows, no sufficiently small per-row δ is available.

39.3. What an example-level model gives a user

Group privacy treats DP-SGD as a black box, but its sampling process is known. Each record is independently included with probability q, so a user with k records is represented in a step with probability about kq. When kq is small, multiple records from that user rarely share a batch. Thus, **a user with k records can be approximated as one record sampled at rate kq.** Screen exposure by rerunning the record-level accountant at rate kq (Section 39.8). Charles et al. (2024) give an exact user accountant.

Table 39.1.: The household-level ε at $\delta = 10^{-5}$ of the Adult run's example-level DP-SGD ($q = 1/30$, $\sigma = 0.973$, 300 steps) for a household with k rows, from the full edition's Rényi accountant (4.59 at one record comes from its coarser grid of orders).

Rows of the household k	1	2	5	10	20
Exact user accountant	4.59	10.02	29.96	74.82	202.89
One record at rate kq	4.50	9.25	25.37	57.57	138.67

Ten rows cost 74.82, offering little practical protection. For every user with multiple records, the exact accountant is tighter than group privacy; do not use the generic group bound for DP-SGD.

The rate-kq screen underestimates the exact cost by 8% for two rows and 23% for ten. It ignores steps that sample multiple household rows, which are common at $q = 1/30$. Report an exact value when an implementation is available. The libraries used here provide only the screen; report it with this limitation and keep kq small. It is not a conservative bound because similar records can move gradients in the same direction.

39.4. User-level DP-SGD

The better route makes the user the unit that DP-SGD samples, clips and accounts for (McMahan et al. 2018). Starting from weights chosen without looking at the data, each step does five things.

1. **Sample users.** Put each user in the batch independently with probability q, now the user sampling rate.
2. **Compute one vector per user.** From that user's records alone, compute their average gradient, or the change in weights after a few training steps on their records.
3. **Clip per user.** Rescale that vector to length at most the clipping norm C.
4. **Add noise.** Sum the clipped vectors and add Gaussian noise of standard deviation σC, for a noise multiplier σ.
5. **Step.** Divide by the expected number of sampled users and take a gradient step.

One user moves the sum by at most C, however many records they have. So the guarantee is user-level DP with exactly the accountant of example-level DP-SGD, run at the user sampling rate.

That has two consequences. First, **you pay for having fewer units**. With 300 steps and about 100 units per batch, sampling rows needs $\sigma = 0.581$ for $\varepsilon = 4$. The same batch drawn from ten

times fewer households needs a ten times higher rate, and so $\sigma = 1.03$. Second, **records inside a user are free**, since you can average them, train on them for several steps or subsample them without changing ε.

Carry a user identifier through the pipeline, and split training and test data by user. Opacus clips each row of a batch, so write a data loader that yields users and clip each user's vector yourself. Three things break the guarantee:

1. A user's vector depends on other users' data, for example through normalization statistics computed over all users.
2. One person is spread over several accounts, so the unit is the account and the person is again a group.
3. Users are iterated in a fixed shuffled order instead of Poisson-sampled, which is a different mechanism that this accountant does not cover (Matrix factorization).

39.5. Contribution bounding

A pipeline built around records can still protect users. **Contribution bounding** keeps at most k records of each user, the cap, and runs example-level DP-SGD on what remains. The result is user-level DP with the ε the user accountant gives for k records. The condition is that each user's kept records are chosen without looking at other users' data, for example at random. Otherwise adding one user could change which records other users keep, and neighbours would differ by more than one user's records. To calibrate the noise for a cap, solve for σ with `dp_accounting.calibrate_dp_mechanism` at rate kq (Train a model with DP-SGD).

```
import polars as pl

rows = pl.DataFrame({"user": ["a", "a", "a", "a", "b", "c", "c"], "x": [1, 2, 3, 4, 5,
 ↪  6, 7]})
k = 2  # the cap, fixed from public knowledge of the population
capped = rows.sample(fraction=1.0, shuffle=True, seed=0).group_by("user").head(k)
print(capped.height)  # 5
print(capped.group_by("user").len()["len"].max())  # 2
```

Choosing the cap trades noise against bias (Amin et al. 2019). A larger cap keeps more data but needs more noise, while a smaller cap discards records and reweights the data toward light users. On Adult's households at a household-level $\varepsilon = 4$, a cap of fifty rows instead of one multiplies the noise by 4.2, and accuracy falls from 0.8362 to 0.7943.

So take the cap from public knowledge of the population, or choose it privately and count the cost, for example with a private quantile of records per user (Beyond global sensitivity). Trying caps on test accuracy spends privacy that the accountant does not see.

39.6. Heterogeneous users

User-level training also changes the objective. Averaging per user weights users equally, while example-level training weights them by their number of records (Disparate impact). If heavy

users should count for more, sum a user's gradients before clipping. Their influence then rises with their records until clipping caps the sum at C, and ε does not change.

39.7. Which strategy

Table 39.2 compares three routes on Adult's households, each with 300 steps, about 100 units per batch and the learning rate 0.2. That rate is the best of 0.1, 0.2, 0.5 and 1 for route (b) at $\varepsilon = 4$, and also the workflow's non-private rate. Strategy (a) runs example-level DP-SGD on all rows and accounts for the largest household.

Table 39.2.: Test accuracy at the same household-level budget, $\delta = 10^{-5}$, one run each. The non-private model reaches 0.847, and always predicting the larger class 0.759.

Household-level ε	(a) All rows, $k = 359$	(b) Household-level DP-SGD	(c) At most 2 rows per household
1	0.5730	0.8212	0.7929
2	0.6179	0.8374	0.8207
4	0.6431	0.8432	0.8309
8	0.6838	0.8445	0.8348

Keeping the row as the unit fails. Strategy (a) stays below always predicting the larger class, because the largest household forces a noise multiplier between 13.4 and 84.6 on everyone. Household-level DP-SGD is the best at every budget, and its cost is compute, since each step processes all the rows of about 100 households. The cap is the fallback for a pipeline that must stay row-based. How far it trails depends on how rows are spread over households, and here it keeps only 5,394 of the 30,000.

39.8. Code

The rate-kq screen needs only a record-level accountant, such as Google's `dp_accounting`. This snippet screens households with one and two rows, and computes the book's δ for 3,000 households.

```
import math

import dp_accounting


def screen(k: int, q: float = 1 / 30, sigma: float = 0.973, steps: int = 300) -> float:
    """The epsilon of a user with k records, screened as one record sampled at rate k *
     ↪  q."""
    step = dp_accounting.PoissonSampledDpEvent(k * q,
 ↪  dp_accounting.GaussianDpEvent(sigma))
    accountant = dp_accounting.rdp.RdpAccountant()
    accountant.compose(dp_accounting.SelfComposedDpEvent(step, steps))
```

```
    return accountant.get_epsilon(1e-5)


print(f"{screen(1):.2f}")  # 4.50
print(f"{screen(2):.2f}")  # 9.25
print(10 ** math.floor(math.log10(3_000**-1.1)))  # 0.0001
```

Its 9.25 matches the table's screen, but two Rényi accountants with different orders can differ in the last digits, so quote the value you ran and name its accountant. With $k = 1$ and q the user sampling rate, the same code accounts for user-level DP-SGD.

39.9. Cheat sheet

Table 39.3.: Which route to a user-level guarantee.

Situation	Use	What you get
Example-level model already trained	User accountant, or the rate-kq screen	A user's ε grows about linearly in k, then faster for large kq
Records can be grouped by user	User-level DP-SGD	DP-SGD's accountant over users
Pipeline must stay record-based	Contribution bounding with a public cap	User accountant for k records; loses data
Heavy users should count more	Sum before clipping instead of averaging	Same ε, different objective

Rules of thumb

- Name the privacy unit with every ε, and set δ from the number of users.
- Never use group privacy for DP-SGD; screen a user at rate kq instead.

39.10. References

- Charles et al., "Fine-Tuning Large Language Models with User-Level Differential Privacy" (arXiv 2407.07737, 2024): the tight user accountant, and sampling records against sampling users.
- Chua et al., "Mind the Privacy Unit! User-Level Differential Privacy for Language Model Fine-Tuning" (Conference on Language Modeling 2024): contribution bounding against user-level DP-SGD.

Derivations. Every result here is derived step by step in the full edition's chapter.

40. Choosing ε

40.1. Why this matters

Every model protected by differential privacy (DP) has a privacy budget ε. The differentially private version of stochastic gradient descent (SGD), DP-SGD, shows how this budget affects accuracy. This chapter relates ε to risks for people in the data, providing a basis for choosing it rather than following convention.

Membership tests assess whether a record appeared in training. Their true positive rate (TPR) measures correctly identified members; their false positive rate (FPR) measures non-members incorrectly flagged. The guarantee (ε, δ) bounds TPR in terms of FPR (The definition of differential privacy). Each run also has a tighter trade-off curve of achievable rates. Compute that curve, evaluate it at the threat's FPR and compare it with accuracy at each budget.

40.2. Justifying a choice

A defensible choice specifies what the budget protects before selecting its value.

1. **Unit and period.** Say whether one unit is a record, a user or a device, and whether neighbouring datasets differ by adding or removing one unit or by replacing one. Say which runs the budget covers, since every tuning run spends budget too (Hyperparameter tuning).
2. **Threat.** Say whether the concern is membership, reconstruction or attribute inference, and what the attacker knows and sees. This sets the false positive rate at which you read the curve, a low one for an attacker who must rarely accuse a non-member.
3. **Whole guarantee.** Choose δ by the book's rule (Section 40.5), then compute the trade-off curve (Section 40.3) and read it there.
4. **Evidence.** The curve is a theorem about an algorithm, not about your code, so run an audit that could refute it (Auditing private training).
5. **Utility.** Put accuracy against ε next to the risk, and pick the smallest budget past the bend, where accuracy stops improving. Draw it on public or proxy data, because private runs that choose ε are a search whose cost the reported ε leaves out.
6. **Comparison.** Only now compare with deployments (Section 40.6) and with the tiers in the rules of thumb.

The Adult public census survey illustrates step 5. Train a model with DP-SGD trains a small network on 30,000 people at five privacy budgets, each with $\delta = 10^{-5}$. Table 40.1 compares test accuracy with the maximum membership-test TPR at FPR 10^{-3}, computed from Section 40.3.

Table 40.1.: Accuracy and worst-case membership risk on Adult, one seed.

ε	Noise multiplier	Test accuracy	Largest TPR at FPR 10^{-3}
0.25	8.125	0.793	0.0013
1	2.383	0.843	0.0023
2	1.431	0.845	0.0046
4	0.973	0.846	0.0123
8	0.722	0.846	0.0445
Non-private	—	0.847	—

Accuracy increases five points from $\varepsilon = 0.25$ to 1, then remains within a few thousandths of the non-private result, comparable to one-seed variation. Risk increases from 0.0023 at $\varepsilon = 1$ to 0.0445 at $\varepsilon = 8$. For Adult, the utility curve bends near $\varepsilon = 1$, making 1 or 2 defensible choices. Because these runs were scored on private rows, the table is illustrative only; choose budgets using public stand-in data. Harder tasks, larger models or smaller datasets may shift the bend to higher ε.

40.3. The pair and the whole curve

A membership test says "member" or "non-member" about one target record. Its TPR is the chance that it says "member" when the target was in the training set, and its FPR the chance when it was not. Every test against an (ε, δ)-DP mechanism satisfies

$$\mathrm{TPR} \le \min\left\{1,\ e^{\varepsilon}\,\mathrm{FPR} + \delta,\ 1 - e^{-\varepsilon}(1 - \delta - \mathrm{FPR})\right\}. \tag{40.1}$$

The middle term is the one that binds at low FPR, and the last applies the same cap to the members a test misses. Attacks are judged at a low FPR, because an attack that rarely flags a non-member identifies people with confidence (Membership inference). Yet at $\varepsilon = 8$ and an FPR of 10^{-3}, the pair caps the TPR at 0.9997, which constrains almost nothing.

A real training run satisfies DP for a whole range of pairs, one $\delta(\varepsilon)$ for each ε, because its accountant can trade ε for δ. Each pair gives a line, the middle term of Equation 40.1, and every test must stay under all of these lines at once. The best test reaches the lowest line exactly, which is called the **trade-off curve**:

$$\mathrm{TPR}^{*}(\mathrm{FPR}) = \min_{\varepsilon}\left(e^{\varepsilon}\,\mathrm{FPR} + \delta(\varepsilon)\right). \tag{40.2}$$

For DP-SGD, a privacy loss distribution (PLD) accountant computes this curve. It can round the privacy loss up or down, and the two curves it gets bracket the true one (Numerical accounting).

Table 40.2 compares the pair with the curve for two of the Adult runs of Table 40.1, each sampled at rate 1/30 for 300 steps.

Table 40.2.: Largest TPR of a membership test against two Adult runs at $\delta = 10^{-5}$, from the pair and from the curve.

ε	Noise multiplier	Pair, FPR 10^{-3}	Curve, FPR 10^{-3}	Curve, FPR 10^{-2}
4	0.973	0.05461	[0.01221, 0.01234]	[0.06447, 0.06515]
8	0.722	0.9997	[0.04414, 0.04452]	[0.155, 0.1563]

So a budget of 8 means little as a bare pair, while its curve caps the TPR at 0.0445 at an FPR of 10^{-3}.

To compute the curve with Google's `dp_accounting`, build the run's PLD, ask it for δ on a grid of ε, and take the lowest line. The snippet's first print is the upper end of a bracket in Table 40.2, and its second gives the last column of Table 40.1.

```
import numpy as np
from dp_accounting.pld import privacy_loss_distribution as pld

EPS_GRID = np.linspace(0.0, 20.0, 2001)


def tpr_at(sigma: float, q: float, steps: int, fpr: float) -> float:
    run = pld.from_gaussian_mechanism(sigma, sampling_prob=q).self_compose(steps)
    deltas = np.array(run.get_delta_for_epsilon(EPS_GRID))
    return float(np.min(np.exp(EPS_GRID) * fpr + deltas))


print(f"{tpr_at(0.722, 1 / 30, 300, 1e-2):.4f}")  # 0.1563
adult = [tpr_at(sigma, 1 / 30, 300, 1e-3) for sigma in (8.125, 2.383, 1.431, 0.973,
 ↪  0.722)]
print(" ".join(f"{tpr:.4f}" for tpr in adult))  # 0.0013 0.0023 0.0046 0.0123 0.0445
```

The library's default PLD rounds the loss up, so its curve is a valid upper bound. A Rényi differential privacy (RDP) accountant gives a valid but looser curve. The Gaussian differential privacy (GDP) approximation is quick but not a bound, so use it to explain and the PLD to certify.

The curve is a worst case, for an attacker who sees every intermediate model, so a typical record attacked through the final model alone is much safer. The curve bounds reconstruction too, since a guess among m equally likely records is a membership test at an FPR of $1/m$ (Reconstruction attacks).

40.4. What an attacker can believe

Most people ask how sure an attacker can become that a person was in the data. Write p for the attacker's **prior**, its probability of "member" before it sees the model. A privacy loss of at most ε

means that no output is more than e^{ε} times as likely with the person's record as without it. So the attacker's belief afterwards is at most

$$\Pr[\text{member} \mid \text{output}] \le \frac{p\, e^{\varepsilon}}{p\, e^{\varepsilon} + 1 - p}. \tag{40.3}$$

So DP bounds the change of belief, not the belief itself.

For example, an attacker who starts at 1% can end at 35.55% when $\varepsilon = 4$. That is for the worst output, and in DP-SGD the privacy loss of the released model is usually far below ε.

Base rates matter as much as the loss. An attacker who accuses every person its test flags as a member is right with probability

$$\Pr[\text{member} \mid \text{flag}] = \frac{p\,\mathrm{TPR}}{p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR}}. \tag{40.4}$$

Take the Adult run at $\varepsilon = 8$ and an attacker with a 1% prior who flags at an FPR of 10^{-2}. The curve caps its TPR there at 0.1563 (Table 40.2), so at most 0.1363 of its flags are right, and more than six out of seven are false. An insider with a prior of 0.5 would be right far more often, so a statement about beliefs must name the attacker.

40.5. How small δ must be

For n privacy units, the book's rule sets $\delta \le 1/n^{1.1}$, rounded down to a power of ten (How small δ must be). The units are people when one person has many records (User-level differential privacy). For Adult's 30,000 people it gives 10^{-5}. That is not far below $1/n$, as always when n is just above a power of ten. NIST SP 800-226, the US guidelines for evaluating DP claims, calls a δ above 10^{-5} suspicious unless justified.

For DP-SGD a smaller δ is cheap. Take the Adult run at $\varepsilon = 4$. When δ falls to 10^{-8}, the PLD accountant puts it at only 5.763. So report ε at a second, smaller δ too, and never quote a δ as large as one over the number of units.

40.6. Comparing with deployed values

"Industry uses ε between 1 and 10" hides what every ε needs, its **privacy unit** and the **period** over which it is spent. The Census Bureau's 2020 redistricting data used zero-concentrated differential privacy (zCDP), which states the guarantee as one number, written ρ, that converts to (ε, δ) pairs. Its unit was one person's record under the replace-one relation, and persons got $\rho = 2.56$, reported as $\varepsilon = 17.14$ at $\delta = 10^{-10}$.

So before you compare, convert both budgets to the same unit, neighbouring relation and δ. A zCDP parameter ρ converts to a pair by

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}. \tag{40.5}$$

For the Census figures this formula gives 17.92, and the Bureau's 17.14 comes from a tighter conversion. Since a replacement is a removal followed by an addition, an add/remove guarantee

(ε, δ) implies only the replace-one guarantee $(2\varepsilon, (1 + e^{\varepsilon})\delta)$ (The definition of differential privacy). And a budget stated per day adds up over time by composition, to 365 a year at $\varepsilon = 1$ per day.

So it would be wrong to call the Adult run at $\varepsilon = 8$ "less than half the Census budget". At the Census's $\delta = 10^{-10}$ it is at 13.812, and under the Census's replace-one relation that can double.

40.7. Cheat sheet

Table 40.3.: Which tool answers which question when choosing ε.

Question	Tool	Notes
What does DP-SGD allow?	The trade-off curve, Equation 40.2	Certify with the PLD, not with GDP
How sure can an attacker become?	Equation 40.3 and Equation 40.4	Name the prior; base rates dominate
How small a δ?	The book's rule, Section 40.5	n counts privacy units
Comparing with a deployment?	Equation 40.5; replace-one doubles ε	Same unit and δ

Rules of thumb

- A budget of at most 1 is strong for any mechanism. A budget between 1 and 10 needs DP-SGD's curve as justification, and a budget above 10 needs the curve and an audit (Ponomareva et al. 2023).
- Never quote ε without its unit, δ and period.

40.8. References

- Near, Darais, Lefkovitz & Howarth, *Guidelines for Evaluating Differential Privacy Guarantees* (NIST SP 800-226, 2025): privacy units, δ, and the hazards of large budgets.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): tiers and reporting.

Derivations. Every result here is derived step by step in the full edition's chapter.

41. Public data and private fine-tuning

41.1. Why this matters

Differential privacy (DP) applied to stochastic gradient descent (SGD), or DP-SGD (DP-SGD), can cause substantial accuracy loss when training from random initialization, especially at privacy loss $\varepsilon \leq 1$. More accurate private models often start from weights **pretrained** without privacy on a large public dataset, then **fine-tune** them on sensitive data (Tramèr & Boneh 2021).

Fine-tuning fewer weights can further improve accuracy. DP-SGD adds noise to every trained weight, while the useful signal does not grow with the number of weights. Training fewer weights therefore reduces noise relative to signal. Fine-tune a language model privately with LoRA demonstrates this workflow.

41.2. How many weights to train

The trainable weights do not change ε: DP-SGD requires only that one record changes the clipped sum by at most clipping norm C. The choice affects the noise added to the model. Other computations based only on public data are also free, provided they do not depend on private data. Examples include initial weights, tokenizers and normalization statistics.

Noise magnitude grows with the number d of trainable weights. Let σ be the noise multiplier and B the expected batch size; Poisson sampling gives variable batch sizes. The averaged-gradient noise has length about

$$\text{noise length} \approx \frac{\sigma C \sqrt{d}}{B}. \tag{41.1}$$

The average clipped-gradient signal has length at most C, independent of d. Training fewer weights omits gradient components for frozen weights, although clipping may then reduce the remaining components less.

Consider the Adult network in DP-SGD, with 6,018 weights. Pretrain it without privacy on the 1,500 public stand-in rows used by Train a model with DP-SGD, then fine-tune on 30,000 private rows. At $\varepsilon = 1$, noise is 1.6 times the signal when all weights train, but 0.4 times the signal when only the 130 last-layer weights train. This difference is smaller than the formula's factor of 6.8 because the last-layer signal is also shorter (0.0726 rather than 0.1137).

Parameter count overstates the cost of full fine-tuning. Noise affects loss mainly along directions with curvature; steps along flat directions have little effect. Near a pretrained model, few directions are curved (Li, Liu et al. 2022). In the running example, full fine-tuning uses 46 times as many weights as last-layer training, but its noise should increase loss only 16 times as much. Larger learning rates expose more nearly flat directions to noise, so private fine-tuning often needs smaller steps than training from scratch.

Training fewer weights has its own cost, a **bias**, because no private data can move the model along a frozen direction that the task needs.

41.3. Linear probing first

The simplest option freezes all but the last layer and trains a linear classifier on the fixed features, a method called **linear probing**. Its loss is convex, so it reduces private deep learning to Private empirical risk minimization and provides a useful baseline.

A private probe has two error sources. Privacy error for convex empirical risk minimization (ERM) grows with feature count d. The **feature gap** is the difference between the best linear model on the features and the best model of any type; more data or a larger ε cannot remove it. In this example, the non-private probe reaches test accuracy 0.8431, while a non-private network trained from scratch reaches 0.8498. Across budgets in Section 41.5, private-probe accuracy stays within 0.002 of its ceiling, comparable to run-to-run variation.

Measure the probe's ceiling by fitting it without privacy on labelled public data similar to the task. If it meets the target, use the probe. For private training, pass frozen features to diffprivlib's `LogisticRegression` (Private empirical risk minimization) or run DP-SGD on the final layer.

41.4. Low-rank adapters and full fine-tuning

When the probe's ceiling is below your target, only changing the earlier layers can lift it. **Low-rank adaptation** (LoRA, Hu et al. 2022) freezes each pretrained matrix and learns a small correction to it. The correction is the product of two thin matrices that each have r columns, where r is the rank. A matrix with d_{in} inputs and d_{out} outputs has $d_{\text{in}}d_{\text{out}}$ weights, while its adapter has only

$$d = r(d_{\text{in}} + d_{\text{out}}). \tag{41.2}$$

The adapter comes from Hugging Face's `peft` library. This snippet puts a rank-4 adapter on the first layer of the running example's network, and trains its last layer in full.

```
import torch
from peft import LoraConfig, get_peft_model

model = torch.nn.Sequential(torch.nn.Linear(91, 64), torch.nn.ReLU(),
 ↪  torch.nn.Linear(64, 2))
print(sum(p.numel() for p in model.parameters()))  # 6018
config = LoraConfig(r=4, lora_alpha=8, target_modules=["0"], modules_to_save=["2"],
 ↪  lora_dropout=0.0)
adapted = get_peft_model(model, config)  # freezes every pretrained weight
print(sum(p.numel() for p in adapted.parameters() if p.requires_grad))  # 750
```

In the snippet, `target_modules` and `modules_to_save` name layers by their index in the `Sequential`, and `lora_alpha` scales the correction. It trains 750 weights, the adapter's $4 \times (91 + 64) = 620$ and the last layer's 130. Opacus's `PrivacyEngine` computes per-example gradients only for the parameters that require them, so the frozen weights cost nothing. Call

`model.train()` before `make_private`, because Opacus rejects a model in evaluation mode, as a pretrained model often is after you measure it.

The adapter's second factor starts at zero, so training begins exactly at the pretrained model. The first factor starts random, and that draw must not depend on the private data.

Yu et al. (2022) showed that private fine-tuning of language models with LoRA matches or beats private full fine-tuning, at a fraction of the compute and memory. Each adapter factor is an ordinary linear layer, so the ghost clipping of DP-SGD applies to it. Still, Li, Tramèr et al. (2022) matched the adapters with full fine-tuning, using large batches and a learning rate tuned for private training.

41.5. Which method at which budget

The script practitioner_adult_studies.py pretrains the running example on the public stand-in rows, then fine-tunes it with Opacus on the private rows, for the workflow's 300 steps. DP-SGD fine-tunes the last layer, a rank-4 adapter plus the last layer, or all weights, against a model trained privately from scratch. Each method's learning rate was chosen at $\varepsilon = 1$ from 0.1, 0.3, 1 and 3, on half of the test rows, and accuracy comes from the other half. That choice looked at private data without paying, so the table shows each method at its best, not what a real run could claim (Hyperparameter tuning).

Table 41.1.: Accuracy on half of the Adult test rows (mean of two runs), $\delta = 10^{-5}$. The pretrained model without fine-tuning scores 0.8383.

	Scratch	Full	Last layer	Adapter, rank 4
Trainable weights d	6,018	6,018	130	750
Learning rate	1	1	1	0.3
$\varepsilon = 0.25$	0.8260	0.8233	0.8416	0.8396
$\varepsilon = 0.5$	0.8402	0.8404	0.8432	0.8456
$\varepsilon = 1$	0.8434	0.8459	0.8440	0.8469
$\varepsilon = 4$	0.8450	0.8491	0.8435	0.8468

Adult is an easy task, so every gap here is under two points, but the pattern is the one larger tasks show. Pretraining is worth about a factor of eight in ε, since the adapter at $\varepsilon = 0.5$ matches training from scratch at $\varepsilon = 4$.

At $\varepsilon = 0.25$ the last layer is best, because its noise is 6.8 times shorter than full fine-tuning's, and its ceiling does not yet bind. From $\varepsilon = 0.5$ on it stays near its ceiling, while first the adapter and then full fine-tuning pass it. The adapter is never more than 0.0023 below the best method, so it is the robust middle choice here. But a public task further from the private one can reverse the ranking even when the probe's ceiling looks fine, so rerun the comparison on a public stand-in for your task.

The learning rate matters as much as the method, since a step three times too large costs full fine-tuning 1.1 percentage points of accuracy at $\varepsilon = 1$. The adapter's rank matters little, since at $\varepsilon = 1$ ranks 1, 4 and 16 differ by about the run-to-run spread.

41.6. What public data does not buy

Whatever the public data are, training on the private data is (ε, δ)-DP, where δ is the small additive slack the guarantee allows. The public data get no guarantee of any kind. Non-private pretraining can memorize individual records (Memorization and training-data extraction), and the fine-tuned model inherits whatever the pretrained weights encode (Tramèr, Kamath & Carlini 2024).

So a person in both datasets has protection only in the private copy. A membership attack against the pretrained model (Membership inference) has no bound at all. Web-scale "public" datasets contain posts, profiles and leaked documents, so private notes or emails may well overlap with them. For text, also mind the privacy unit, because one person's documents span many training sequences (User-level differential privacy).

Check three things before you train and when you report.

1. Check that the public data resemble the private task, and deduplicate the private set against them. Deduplication protects copies, not people, since exact matching misses paraphrases.
2. Choose the public model and the hyperparameters on public data. Otherwise every private run costs privacy (Hyperparameter tuning).
3. Name the public data and the overlap check, and state that the guarantee covers the private data given the pretrained model.

41.7. Cheat sheet

Table 41.2.: Where to start when fine-tuning privately; the ranking depends on the task.

Situation	Train	Why
Tiny budget, or the probe meets the target	Last layer (linear probe)	Fewest weights; convex
Moderate budget, features not quite right	Low-rank adapter plus last layer	Never far from the best
Larger budget, large batches, tuned learning rate	All weights	Noise costs only along curved directions

Rule of thumb. Measure the linear probe's ceiling before anything else. Then use large batches and a smaller learning rate than for training from scratch. Tune the learning rate with DP-SGD runs at the target noise on public data, because a non-private sweep does not show how the noise moves the best step.

41.8. References

- Tramèr & Boneh, "Differentially Private Learning Needs Better Features (or Much More Data)" (ICLR 2021): linear probing on good features.

- Li, Tramèr, Liang & Hashimoto, "Large Language Models Can Be Strong Differentially Private Learners" (ICLR 2022), and Yu et al., "Differentially Private Fine-tuning of Language Models" (ICLR 2022): full and adapter-based private fine-tuning.

Derivations. Every result here is derived step by step in the full edition's chapter.

42. Fine-tune a language model privately with LoRA

42.1. Why this matters

Fine-tune a pretrained language model on private text, such as support tickets or clinical notes, without exposing training examples. Non-private fine-tuning can reproduce frequent phone numbers or passwords (Memorization and training-data extraction). Differential privacy (DP) applied through stochastic gradient descent (SGD), or DP-SGD, limits this risk. Training a small adapter on frozen weights can retain accuracy while reducing noise cost (Public data and private fine-tuning). This workflow uses Hugging Face's low-rank adaptation (LoRA) in `peft` and Opacus.

The model is EleutherAI's public Pythia-14m, with 14 million parameters. The private text is Tiny Shakespeare, about one million characters of plays. The script fine-tunes the model with and without privacy. The private run has privacy loss ε of 8 per training sequence. It inserts synthetic secrets, called canaries, to test memorization. practitioner_lora.py runs in a few minutes without a GPU.

```
uv run python code/part05-dp-ml/practitioner_lora.py
```

42.2. Fix the privacy unit, n and δ

Define the privacy unit before training. Language models process fixed-length sequences of tokens, or word pieces produced by a tokenizer. This script uses 32-token sequences, roughly one or two lines of verse, as units. It trains on the first 1,024 sequences and holds out the last 128 to limit runtime.

```
sequences of 32 tokens: 10,632  training n=1,024  held out 128
delta=1e-04 (privacy unit: one training sequence)
```

For $n = 1{,}024$ sequences, the book's rule for the slack δ gives 10^{-4} (How small δ must be).

A sequence is suitable for this demonstration but rarely for production. A person's document may span many sequences, and one user may contribute several documents. Sequence-level privacy therefore provides weaker person-level protection. For production, use a document or user as the unit, bound each contribution, and set δ from the user count (User-level differential privacy).

42.3. Plant canaries

The script writes **canaries** into the training text so their memorization can be measured. Each canary is a sentence such as "the pin of Alice is 8 6 5 2 3"; its last five digits are the secret. The script inserts three canaries 1, 4, 16 and 64 times each. It draws three controls in the same way but never inserts them. The controls show how an unseen secret scores in the audit of Section 42.9.

```
def insert_canaries(
    train: torch.Tensor, canaries: list[Canary], tokenizer: PreTrainedTokenizerBase,
 ↪  rng: np.random.Generator
) -> torch.Tensor:
    """Write each copy of each canary over the start of a different training
     ↪  sequence."""
    train = train.clone()
    total = sum(c.copies for c in canaries)
    rows = iter(rng.choice(len(train), size=total, replace=False).tolist())
    for canary in canaries:
        tokens = torch.tensor(tokenizer(canary.text)["input_ids"])
        for _ in range(canary.copies):
            train[next(rows), : len(tokens)] = tokens
    return train
```

The 255 copies take up a quarter of the training set, far more than secrets occupy in real data. This deliberate density lets a small model memorize them during a short run and provides a positive control for the audit.

```
canary sequences: 255 (3 canaries each at (1, 4, 16, 64) copies)
tokens per secret: [5]
candidates: 2,000  exposure cap 10.97 bits  (full space 16.61)
```

Spaces separate the digits so every possible secret has exactly five tokens. The second line confirms this for 1,000 random secrets.

42.4. Load a public model and tokenizer

All components not trained privately must come from outside the private data. This includes the tokenizer: a vocabulary built from private text could expose rare words. The script loads the weights and tokenizer from the same pinned public checkpoint, Pythia-14m. It loads the model in float32 because `transformers` loads some checkpoints in float16, which runs far more slowly on a CPU.

The guarantee then covers the private text given the pretrained model, and says nothing about the model's own training data. Pythia was pretrained on the Pile, which includes Project Gutenberg's books and so very likely Shakespeare's plays. That is harmless here, but with real data, check for overlap with the public model's training data (Public data and private fine-tuning).

42.5. Choose what to train

DP-SGD adds noise to every trained weight. The noise on the averaged gradient grows with the square root of the number of trained weights, but the clipped signal does not (Public data and private fine-tuning). The number of trainable weights and the learning rate therefore affect accuracy. LoRA freezes each pretrained matrix and learns a correction formed by multiplying two thin factors; their width is the rank.

```
attention_and_mlp = ["query_key_value", "dense", "dense_h_to_4h", "dense_4h_to_h"]  #
 ↪  every linear layer
config = LoraConfig(r=rank, lora_alpha=2 * rank, target_modules=attention_and_mlp,
 ↪  lora_dropout=0.0)
lora_model = get_peft_model(model, config)  # freezes the pretrained weights
trainable_count = sum(p.numel() for p in lora_model.parameters() if p.requires_grad)
total_count = sum(p.numel() for p in lora_model.parameters())
print(f"trainable parameters: LoRA {trainable_count:,}  full fine-tuning {total_count
 ↪  - trainable_count:,}")
```

```
full model parameters: 14,067,712
trainable parameters: LoRA 98,304  full fine-tuning 14,067,712
```

By default, the adapters have rank 8 and cover every linear layer in attention and the feed-forward blocks. They train 98,304 weights, compared with about 14 million for full fine-tuning. `lora_alpha` scales the correction and is set to twice the rank. For a real task, choose the rank and layers using public data. Choices based on private runs cost privacy (Hyperparameter tuning).

42.6. Make the model Opacus-compatible

Opacus needs one gradient per sequence. Layers that mix records within a batch, such as batch normalization, prevent this calculation. Opacus's `ModuleValidator.validate` lists such layers before training.

```
validator errors: 0
```

Pythia has none, and the script trains only the adapters. These are ordinary linear layers handled by Opacus's default hooks, so the script sets `grad_sample_mode="hooks"`. Opacus computes per-sequence gradients only for trainable weights; the 14 million frozen weights add no cost. Call `model.train()` before wrapping a model that has been evaluated, because Opacus rejects evaluation mode. After training, the hooks remain attached and slow later operations. The script calls `remove_hooks()` before evaluating the private model.

42.7. Calibrate and train

The accountant finds the noise for a fixed target, here $\varepsilon = 8$ over 3 epochs with an expected batch of 32 sequences. The sampling rate q is the share of the data in an average batch, or 32/1,024. The run therefore takes 96 steps. The accountant is the privacy loss random variable (PRV) accountant, which computes ε almost exactly (Choosing an accountant).

```
privacy_engine = PrivacyEngine(accountant="prv")
private_model, optimizer, loader = privacy_engine.make_private_with_epsilon(
    module=model,
    optimizer=optimizer,
    data_loader=loader,  # replaced by Poisson sampling with rate BATCH / n
    target_epsilon=epsilon,
    target_delta=delta,
    epochs=EPOCHS,
    max_grad_norm=MAX_GRAD_NORM,
    grad_sample_mode="hooks",
)
print(f"noise multiplier: {optimizer.noise_multiplier:.3f}")
```

The training loop is an ordinary one, with Poisson batches, in which each sequence joins each batch independently with probability q.

```
for (batch,) in loader:  # each batch holds a random number of sequences
    optimizer.zero_grad()
    loss = private_model(input_ids=batch, labels=batch).loss
    loss.backward()  # Opacus's hooks keep one gradient per sequence
    optimizer.step()  # clips each one to MAX_GRAD_NORM, sums, adds noise
    steps += 1
```

```
noise multiplier: 0.553
sample rate q=0.03125  steps=96  noise multiplier=0.553
epsilon (Opacus PRV)=7.99  epsilon (dp_accounting PLD)=7.98  at delta=1e-04
```

The noise multiplier gives the noise's standard deviation in units of the clipping norm, `MAX_GRAD_NORM`, which the script sets to 1. The last line cross-checks ε with the privacy loss distribution (PLD) accountant in Google's `dp_accounting`, as in Calibrate and account a DP-SGD run. The accountants agree.

Both runs use AdamW at a learning rate of 10^{-3}. The private run differs only in clipping, noise and Poisson sampling. The learning rate, rank and epochs were chosen in non-private trials on the same text. A real run must choose them using similar public text, such as other public-domain plays, or account for the trials in ε (Hyperparameter tuning).

For a real run, use batches in the thousands when the dataset is larger. Opacus's `BatchMemoryManager` can split them into physical batches that fit in memory (Train a model with DP-SGD). Set `secure_mode=True` in `PrivacyEngine` for a released model. Secure noise cannot be seeded, so the script leaves this option off.

42.8. Evaluate

Language-model quality can be measured by predictions on held-out text. The script measures mean cross-entropy per token on the 128 held-out sequences and reports its exponential, the **perplexity**. A perplexity of 159.3 corresponds to uncertainty similar to choosing among about 159 equally likely tokens. Lower perplexity is better.

```
held-out loss pretrained: 5.306 nats/token  perplexity 201.6
held-out loss LoRA non-private: 5.071 nats/token  perplexity 159.3
held-out loss LoRA DP eps=8: 5.044 nats/token  perplexity 155.1
```

Both fine-tuned models improve on the pretrained model. On this seed, the private model's slightly lower perplexity shows only that DP has no measurable cost in this run. Both gains are modest because canaries, rather than Shakespeare, begin a quarter of the training sequences.

This result does not imply that privacy has little cost on other tasks. This run is short, trains few weights, and uses text likely to appear in the pretraining data. For a harder task or smaller ε, compare the private model with both baselines at several budgets, as in Train a model with DP-SGD.

42.9. Check memorization

The canaries show what perplexity cannot, whether the model will repeat a secret. The script measures each canary in two ways. The first is extraction. It gives the model the prefix "the pin of Alice is", lets it write the likeliest next token five times, and checks whether the result is the secret.

```
@torch.no_grad()
def greedy_completion(model: PeftModel, tokenizer: PreTrainedTokenizerBase, prefix:
 ↪  str) -> str:
    """The five tokens the model writes after the prefix when it always picks the
     ↪  likeliest one."""
    ids = tokenizer(prefix, return_tensors="pt")["input_ids"]
    for _ in range(5):
        next_token = model(input_ids=ids).logits[:, -1].argmax(dim=-1, keepdim=True)
        ids = torch.cat([ids, next_token], dim=1)
    return tokenizer.batch_decode(ids[:, -5:])[0].strip()
```

The second is the **exposure** of Memorization and training-data extraction. It ranks the canary's secret among random secrets of the same format by how likely the model finds each, and turns the rank into bits. The script's likelihood is the summed log-probability of the five digits. Each bit halves the number of guesses an attacker needs.

```
@torch.no_grad()
def secret_scores(model: PeftModel, tokenizer: PreTrainedTokenizerBase, prefix: str,
 ↪  secrets: list[str]) -> Scores:
    """Summed log-probability of each secret's five digit tokens after the prefix."""
```

```
    prefix_ids = tokenizer(prefix)["input_ids"]
    rows = [prefix_ids + tokenizer(" " + s)["input_ids"] for s in secrets]
    ids = torch.tensor(rows)  # every secret is five tokens, so the rows have equal
↪   length
    logits = model(input_ids=ids[:, :-1], logits_to_keep=5).logits  # the predictions
↪   of the five digits
    logprobs = logits.log_softmax(dim=-1).gather(-1, ids[:, -5:, None])
    return logprobs.sum(dim=(1, 2)).double().numpy()


def canary_exposure(
    model: PeftModel, tokenizer: PreTrainedTokenizerBase, canary: Canary, candidates:
↪   list[str]
) -> tuple[float, float]:
    """The exposure of the canary's secret among the candidates, measured and
     ↪  extrapolated past the cap."""
    candidate_scores = scores_batched(model, tokenizer, canary.prefix, candidates)
    score = float(scores_batched(model, tokenizer, canary.prefix, [canary.secret])[0])
    return exposure(score, candidate_scores), extrapolated_exposure(score,
     ↪  candidate_scores, SECRET_SPACE)
```

No library installed with this book computes exposure, so `exposure` and `extrapolated_exposure` come from the book's helper module practitioner_privacy_tests.py, after TensorFlow Privacy's versions. With 2,000 random candidates, a canary that beats them all reaches a cap of 10.97 bits. A secret the model has never seen has a random rank and averages about 1.44 bits ($\log_2 e$). Each line below gives a group's mean exposure, the same past the cap, and how many of its three secrets greedy decoding wrote out.

```
non-private 1 copies: mean exposure 2.0 bits  extrapolated 2.0  extracted 0/3
non-private 4 copies: mean exposure 5.9 bits  extrapolated 5.7  extracted 0/3
non-private 16 copies: mean exposure 10.0 bits  extrapolated 13.0  extracted 0/3
non-private 64 copies: mean exposure 11.0 bits  extrapolated 16.3  extracted 3/3
non-private controls: mean exposure 1.4 bits  extrapolated 1.5  extracted 0/3
DP 1 copies: mean exposure 0.9 bits  extrapolated 0.9  extracted 0/3
DP 4 copies: mean exposure 0.6 bits  extrapolated 0.6  extracted 0/3
DP 16 copies: mean exposure 4.2 bits  extrapolated 4.3  extracted 0/3
DP 64 copies: mean exposure 4.0 bits  extrapolated 3.9  extracted 0/3
DP controls: mean exposure 2.2 bits  extrapolated 2.2  extracted 0/3
```

The non-private model, the positive control, leaks. Exposure rises with the number of copies, from 2.0 bits for one copy to the cap for 64, and the model writes out all three 64-copy secrets. The extrapolated column sees past the cap by fitting a skew-normal distribution, a normal with one heavier tail, to the candidates' scores. It puts the 64-copy secrets near the 16.61 bits of the whole space.

The 4-copy canaries are the instructive case. Two of the three reach 6.9 and 8.2 bits, yet none is extracted, so exposure detects memorization that extraction alone would miss.

Greedy decoding extracts nothing from the private model, and its canaries with 1 or 4 copies score no higher than its controls. The controls' 2.2 bits are within the scatter of a mean over three

secrets. Still, DP limits the effect without erasing it, since the canaries with 16 and 64 copies still average about 4 bits.

The guarantee allows that, because $\varepsilon = 8$ protects one sequence, while a canary with many copies is many sequences. Group privacy bounds a group of units at about their number times ε. So 16 copies get 16×8, far too much to promise anything (User-level differential privacy). A secret that a user repeats across many documents is the same case, one more reason to make the unit the user.

Two cautions apply. Because the canaries are so dense, a real non-private model may leak less than this one, and a real audit should plant fewer copies. And three canaries per group is a small sample. For a fuller assessment before release, run the attacks of Pre-release privacy risk assessment. For a statistical lower bound on ε, run the one-run audit of Auditing private training.

42.10. Variants

Three variants matter for real models. The block below shows ghost clipping, LoRA on GPT-2, and packages that wrap the whole loop, with pinned versions. It was not run.

```
# opacus==1.6.0: ghost clipping, when per-sequence gradients do not fit in memory
model, optimizer, criterion, loader = privacy_engine.make_private_with_epsilon(
    module=model, optimizer=optimizer, data_loader=loader,
 ↪  criterion=torch.nn.CrossEntropyLoss(),
    target_epsilon=8.0, target_delta=1e-4, epochs=3, max_grad_norm=1.0,
 ↪  grad_sample_mode="ghost",
)
logits = model(input_ids=batch).logits[:, :-1]                # predict each next
 ↪  token
loss = criterion(logits.flatten(0, 1), batch[:, 1:].flatten(), shape=logits.shape)
loss.backward()                                               # norms first, then the
 ↪  clipped sum

# peft==0.21.2: GPT-2 stores its attention as Conv1D, with the weight transposed
LoraConfig(r=8, lora_alpha=16, target_modules=["c_attn"], fan_in_fan_out=True)

# dp-transformers==1.0.0 (Microsoft): Opacus inside the Hugging Face Trainer
# private-transformers (Li et al.): install from its GitHub repository, pinned to a
 ↪  commit
```

Ghost clipping gets each sequence's gradient norm without storing the gradient, so memory no longer holds one gradient per sequence (DP-SGD).

42.11. Report

Release and governance builds the statement from the run's record. For this run, it adds three things. The guarantee covers the private text given the pretrained model, named with its revision. The hyperparameters came from non-private trials on the private text, which a real release could not claim for free. And the canary audit is attached as evidence, with its caveats.

What ships is the adapter. With `--save-adapter FOLDER`, the script saves it after `remove_hooks()` and loads it back onto the same pinned checkpoint.

```
private.save_pretrained(path)  # only the adapter weights and config, after Opacus's
 ↪  hooks are removed
base_model = AutoModelForCausalLM.from_pretrained(folder, dtype=torch.float32)  # the
 ↪  same pinned checkpoint
reloaded = PeftModel.from_pretrained(base_model, path)
```

The folder holds only the adapter, so the statement names the base checkpoint and revision it fits. The guarantee covers the adapter and anything computed from it.

42.12. Checklist

Table 42.1.: The steps of fine-tuning a language model privately, their defaults and their checks.

Step	Default	Check
Unit, n, δ	Document or user in production; δ by the book's rule	n counts units, not tokens or lines
Canaries	A few per group, inserted 1 to many times, plus controls	Controls near 1.44 bits, allowing for the scatter of a few
Public model and tokenizer	One public checkpoint, pinned revision, float32	Nothing fitted on private text; overlap checked
What to train	LoRA, rank 8, on every linear layer	Trainable count printed; chosen on public data
Opacus	Validator; hooks; `remove_hooks()` after training	No validator errors
Calibrate and train	PRV accountant; Poisson sampling; large batches	PLD agrees with PRV
Evaluate	Held-out perplexity against the pretrained and non-private models	Private model beats the pretrained one
Memorization	Exposure and greedy extraction, non-private and private	Nothing extracted; one-copy canaries no higher than the controls
Report and save	Unit, ε, δ, accountant, public model, tuning; adapter saved after `remove_hooks()`	Statement names what is not covered and the base revision

42.13. References

- Yu et al., "Differentially Private Fine-tuning of Language Models" (ICLR 2022): LoRA and other adapters with DP-SGD, and their accuracy.
- Li, Tramèr, Liang & Hashimoto, "Large Language Models Can Be Strong Differentially Private Learners" (ICLR 2022): full fine-tuning with ghost clipping, and the hyperparameters that make it work.
- Carlini et al., "The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks" (USENIX Security 2019): canaries and exposure.

Derivations. This workflow draws on the full edition's chapters on public data and private fine-tuning in the full edition, DP-SGD in the full edition and memorization and training-data extraction in the full edition.

43. Private prediction: PATE and in-context learning

43.1. What it is

Private prediction applies differential privacy (DP) to each answer rather than to stochastic gradient descent (SGD) training. Split private records into disjoint shards and train an ordinary **teacher** model on each. Keep teachers private. For each query, add noise to vote counts and release the winning label. The privacy level is ε, and δ is the failure-probability allowance. Each record belongs to one shard and can change one vote from one label to another, changing the count vector by at most $\sqrt{2}$ in Euclidean norm.

Let σ denote the standard deviation of the added Gaussian noise. **GNMax** adds this noise to each count and releases the label with the largest noisy count. In Rényi differential privacy (RDP), each answer costs a line with slope $1/\sigma^2$ at order α, the Rényi order parameter. Slopes compose additively (Rényi DP). Let $\rho(\varepsilon, \delta)$ be the largest slope that converts to the target guarantee. For a fixed number Q of answers, including adaptively chosen queries, the required noise is

$$\sigma = \sqrt{\frac{Q}{\rho(\varepsilon, \delta)}}. \tag{43.1}$$

For example, set target privacy level $\varepsilon = 1$ and slack $\delta = 10^{-5}$. The allowed slope is 0.0306. Then 1,000 answers require $\sigma = 181$.

43.2. When you meet it

Use private prediction when private model training is impractical. One example is a large language model (LLM) interface whose answers could expose records in prompts. Split records into disjoint groups, query the model once per group, and release a GNMax label (Duan et al. 2023). Wu et al. (2024) answered 10,000 sentiment queries with accuracy matching one non-private prompt. Their privacy level was $\varepsilon = 1$.

The alternative is a private model with fixed accuracy. Vote noise grows as $\sqrt{Q}$, so private answers are preferable only up to a **break-even** query count. On the full edition's synthetic task, the data contains 10,000 records. Voting outperforms private SGD (DP-SGD) only for the first 18 queries at privacy level $\varepsilon = 1$. Compare both approaches at the expected query count and budget, using Train a model with DP-SGD as the baseline.

PATE (private aggregation of teacher ensembles) labels public, unlabelled inputs with GNMax and trains a **student** model on those labels. Only the student is released, at the privacy cost of its labels. PATE therefore requires public inputs from the private data distribution. Papernot et al.'s MNIST student comes within one percentage point of non-private accuracy, but its ε is

data-dependent (Section 43.4). On the synthetic task, DP-SGD outperforms the best student at every budget.

43.3. How to use it

`dp_accounting` accounts for the vote. One GNMax answer is a `GaussianDpEvent` whose noise multiplier, the noise over the sensitivity, is $\sigma/\sqrt{2}$.

```
import numpy as np
from dp_accounting import GaussianDpEvent, SelfComposedDpEvent
from dp_accounting.pld import PLDAccountant
from dp_accounting.rdp import RdpAccountant

sigma, queries = 181.0, 1000  # noise per count, answers to release
event = SelfComposedDpEvent(GaussianDpEvent(sigma / np.sqrt(2)), queries)  #
 ↪  sensitivity sqrt(2)
rdp = RdpAccountant().compose(event).get_epsilon(1e-5)
pld = PLDAccountant().compose(event).get_epsilon(1e-5)
print(f"{rdp:.3f} {pld:.3f}")  # 0.999 0.914
```

The RDP accountant confirms $\varepsilon = 0.999$. The privacy loss distribution (PLD) accountant, exact for Gaussian noise, gives 0.914, so solving for σ with it gives less noise. To solve, pass a function from σ to this event to `dp_accounting.calibrate_dp_mechanism`. For many answers the PLD is much slower, so RDP is the fallback. When each query draws a fresh Poisson sample of records, as in Wu et al., wrap each event in a `PoissonSampledDpEvent`.

43.4. What goes wrong

The guarantee needs each record in exactly one shard. So assign shards by a hash of the record's or the user's identifier, never by position in a file, where one inserted record shifts every later shard. Release the label only, and fix Q in advance.

Noise this large wipes out close votes. With two labels and a gap g between the counts, the vote returns the plurality with probability $\Phi(g/(\sqrt{2}\sigma))$, where Φ is the standard normal distribution function.

For example, 1,000 answers at privacy level $\varepsilon = 4$ require noise $\sigma = 51.8$. With that noise, a vote of 98 to 2 is correct with probability 0.9051. For a close vote of 52 to 48, accuracy is only about one half. Measure the noiseless vote first, because no privacy budget can improve it. Adding teachers widens vote gaps because σ does not depend on the teacher count. This helps only until the shards become too small to train on.

Finally, beware of a data-dependent ε. When teachers agree, a sharper analysis charges less, but that smaller ε is computed from the votes and leaks them. Until it is released with noise, it is a measurement, not a guarantee.

Confident-GNMax answers only queries on which a noisy check finds agreement, the cheap and accurate ones.

43.5. References

- Papernot et al., "Scalable Private Learning with PATE" (ICLR 2018): GNMax, its data-dependent analysis and Confident-GNMax.
- Wu et al., "Privacy-Preserving In-Context Learning for Large Language Models" (ICLR 2024): subsampled voting over prompts.

Derivations. Every result here is derived step by step in the full edition's chapter.

44. DP-FTRL and matrix factorization

44.1. What it is

Gradient descent without momentum accumulates clipped gradients, so weights after each step depend on their **prefix sum**. Differential privacy (DP) applied to stochastic gradient descent (SGD) yields DP-SGD, which adds fresh noise at every step. Prefix-sum noise then grows like a random walk. Matrix mechanisms correlate noise across steps so some noise cancels in later sums.

Place the step gradients in the rows of matrix x. Their prefix sums are Ax, where A has ones on and below its diagonal. The **matrix factorization mechanism** factors A into decoder L and encoder R, adds Gaussian noise z to the encoded gradients, then decodes:

$$\hat{S} = L(Rx + z) = Ax + Lz. \tag{44.1}$$

Each record affects only the encoder column for its step, so privacy depends on the encoder's largest column norm. Accuracy depends on the decoder: row t determines how noise affects prefix sum t.

The first method applied follow the regularized leader (FTRL) to binary-tree sums and is called DP-FTRL (Kairouz et al. 2021). The common **square-root factorization** uses the lower-triangular square root of A for both factors. At equal privacy over 100 steps, its final-sum noise standard deviation is about one quarter that of independent noise.

44.2. When you meet it

Use matrix mechanisms when the pipeline cannot use the Poisson sampler assumed by DP-SGD accounting. Examples include fixed batches from a shuffled dataset, data processed in arrival order and federated devices that report when available. Treating these as Poisson understates ε (Amplification by subsampling); matrix mechanisms do not require subsampling.

Without sampling, independent noise can sharply reduce accuracy; correlated noise can recover much of it. On the full edition's synthetic task, ten fixed-order epochs at $\varepsilon = 2$ give test accuracy 0.8641 with independent noise, 0.9491 with the banded square root of Section 44.4 and 0.9493 with Poisson-sampled DP-SGD. Use Opacus with Poisson sampling when possible; otherwise consider a banded matrix mechanism, as Google did for Gboard models (Xu et al. 2023).

44.3. How to use it

Google's JAX Privacy implements banded matrix factorization. The block below was not run, because JAX is not in this book's environment. It cycles through 100 fixed groups of records, one per step, so each record takes part once every 100 steps.

```
# jax-privacy==2.0.0 (not run)
from jax_privacy.experimental.execution_plan import BandMFExecutionPlanConfig

config = BandMFExecutionPlanConfig.default(
    num_bands=100,       # at most the 100 steps between a record's turns
    iterations=1000,     # ten epochs of 100 steps
    epsilon=2.0,
    delta=1e-5,
    sampling_prob=1.0,   # all of the step's group joins: no amplification claimed
    l2_clip_norm=1.0,
)
plan = config.plan       # calibrates the noise multiplier with dp_accounting
```

The training loop samples batches with `plan.batch_selection_strategy`, clips and sums gradients with `plan.clipped_grad`, and adds noise with `plan.noise_addition_transform`. Use `dp_accounting` on `plan.dp_event` to calculate reported ε. The module is experimental; pin its version.

44.4. What goes wrong

Repeated participation creates the main risk. Across E epochs, one record affects E encoder columns. Nearby square-root columns overlap, increasing sensitivity faster than $\sqrt{E}$. **Banding** retains b encoder diagonals: the main diagonal and $b-1$ below it. It requires each pair of a record's participations to be separated by at least b steps (b**-min-separation**). The columns then share no rows, bounding sensitivity by $\sqrt{E}$ times the longest column, as with independent noise.

Data order is part of the mechanism. Fix it independently of private data and reuse the same permutation every epoch. Reshuffling can place a record at the end of one epoch and the start of the next, violating the separation condition.

The neighbouring relation also affects sensitivity. This analysis leaves a neighbour's slot empty, so one step changes by at most the clipping norm. Replacing a record with another doubles sensitivity; report the relation with each ε.

`dp_accounting` 0.6.0 enforces this restriction. Its tree-aggregation event is supported only by a Rényi differential privacy (RDP) accountant using `NeighboringRelation.REPLACE_SPECIAL`, where a record is replaced by an empty slot. Under DP-SGD's default add/remove relation, it raises `UnsupportedEventError`; the privacy loss distribution (PLD) accountant also rejects it. Therefore, DP-FTRL and DP-SGD runs cannot share an add/remove ledger.

44.5. References

- Choquette-Choo et al., "(Amplified) Banded Matrix Factorizations: A Unified Approach to Private Training" (NeurIPS 2023): banding, min-separation and amplification.

Derivations. Every result here is derived step by step in the full edition's chapter.

45. Disparate impact

45.1. Why this matters

Average test accuracy can hide groups that bear more of privacy's accuracy cost. Differential privacy (DP) applied to stochastic gradient descent (SGD), called DP-SGD (DP-SGD), can disproportionately reduce accuracy for under-represented groups. Bagdasaryan, Poursaeed & Shmatikov (2019) called this the **disparate impact** of DP.

A one-point average loss can conceal a ten-point loss for a small group. Before release, compare each group's private-model accuracy with a non-private baseline and report confidence intervals.

45.2. Evaluating a private model per group

Specify relevant groups before examining results to avoid selecting groups based on outcomes. Train a non-private baseline with the same data and procedure; group differences may exist without privacy. Measure DP's cost as each group's accuracy change under private training.

Let acc_g be model accuracy on test records from group g. Since private training is random, average accuracy over multiple runs, denoted $\mathbb{E}$. The group's **privacy cost** π_g is its accuracy loss under private rather than non-private training with the same data and hyperparameters:

$$\pi_g = \text{acc}_g(\text{non-private}) - \mathbb{E}[\text{acc}_g(\text{private})]. \tag{45.1}$$

DP has disparate impact when groups have different privacy costs.

Report an interval for each accuracy. Correct classifications are binomial outcomes, so the exact Clopper–Pearson interval covers the true group accuracy in at least 95% of samples (Hypothesis testing).

Compute a separate interval for differences between groups. Let $\hat{p}_g$ be measured accuracy for group g, and let m_g be the number of its test records. For disjoint test sets, the variances add. This gives the following approximate 95% interval when counts are sufficiently large:

$$(\hat{p}_a - \hat{p}_b) \pm 1.96\sqrt{\frac{\hat{p}_a(1-\hat{p}_a)}{m_a} + \frac{\hat{p}_b(1-\hat{p}_b)}{m_b}}. \tag{45.2}$$

Small test groups produce wide intervals. For example, suppose 88 of 100 minority-group records are classified correctly. The majority group has 1,812 correct predictions among 1,900 records, or accuracy 0.954; the measured gap is 0.074. SciPy gives the minority interval, and Equation 45.2 gives the gap interval.

```
from math import sqrt

from scipy.stats import binomtest

minority = binomtest(88, 100).proportion_ci(confidence_level=0.95, method="exact")
print(f"{minority.low:.3f} {minority.high:.3f}")  # 0.800 0.936

gap, se = 0.954 - 0.880, sqrt(0.954 * 0.046 / 1_900 + 0.880 * 0.120 / 100)
print(f"{gap - 1.96 * se:.3f} {gap + 1.96 * se:.3f}")  # 0.010 0.138
```

The gap's interval excludes zero, but its value could range from one to fourteen points. Distinguishing a five-point gap from zero requires hundreds of group test records; a one-point gap requires thousands.

Privacy cost needs a paired interval because both models use the same test records, so their accuracies are dependent and Equation 45.2 does not apply. For group g, let u count records only the non-private model classifies correctly. Let v count records only the private model gets right, and let m_g be the test-record count. One run's cost is $(u - v)/m_g$. For large counts, an approximate 95% interval is

$$\frac{u - v}{m_g} \pm \frac{1.96}{m_g} \sqrt{u + v - \frac{(u - v)^2}{m_g}}. \tag{45.3}$$

Training introduces additional variation that a larger test set cannot remove. Small-group accuracy varies more across training runs, so a one-run comparison can reverse. Train each procedure with multiple seeds and report mean cost and its spread.

Per-group accuracy on a private test set is itself a statistic of private data. To publish it, add noise to each group's correct-prediction count and allocate a separate privacy budget (Release statistics with a library).

45.3. Disparate impact, measured

The script practitioner_dp_sgd.py of Train a model with DP-SGD trains a small network on the Adult census data to predict whether a person's income exceeds $50K. It trains once without privacy and once with DP-SGD at a privacy level ε of 4 and a slack δ of 10^{-5}. Then it scores both on the test rows of each `sex`, with exact intervals from SciPy, which gives Table 45.1 for seed 0.

Table 45.1.: Accuracy on Adult by sex, with exact 95% intervals, for one training run of each model.

Group	Test records	Non-private accuracy	Accuracy at $\varepsilon = 4$	Privacy cost
Female	5,613	0.924 [0.917, 0.931]	0.922 [0.915, 0.929]	+0.002
Male	11,229	0.808 [0.800, 0.815]	0.807 [0.800, 0.815]	+0.001

Here DP has no measurable disparate impact. Each group loses at most 0.002 in one private run, well inside its intervals. The large gap between the groups is there without privacy too, so DP is not responsible for it. Both groups are large, and the whole model loses almost nothing at this budget.

Small groups tell a different story at a smaller budget. Adult's `race` column has three groups with under 1,000 of the 30,000 training rows. The script practitioner_adult_studies.py trains the same model three times each without privacy, with clipping only, and at $\varepsilon = 4$ and 1, and scores every group (Table 45.2).

Table 45.2.: Test accuracy on Adult by race, averaged over 3 runs ($\delta = 10^{-5}$, noise multiplier 0.973 at $\varepsilon = 4$ and 2.383 at $\varepsilon = 1$). Costs are fractions of accuracy.

Group	Training rows	Test rows	Non-private	Clipping only	$\varepsilon = 4$	$\varepsilon = 1$	Privacy cost at $\varepsilon = 1$
White	25,670	14,352	0.8368	0.8376	0.8363	0.8331	0.0037
Black	2,876	1,650	0.9042	0.9046	0.9079	0.9038	0.0004
Asian-Pac-Islander	926	532	0.8346	0.8289	0.8315	0.8045	0.0301
Amer-Indian-Eskimo	283	167	0.9281	0.9361	0.9361	0.9281	0.0000
Other	245	141	0.9149	0.9031	0.9031	0.8936	0.0213

At $\varepsilon = 4$, no group loses more than 1.2 points. At $\varepsilon = 1$, the White group loses 0.37 points. The Asian-Pac-Islander group loses 3.01 points, and the Other group loses 2.13. Black and Amer-Indian-Eskimo people lose nothing, although the latter is one of the smallest groups. The share of high incomes does not explain the pattern: about 12% of the Black, Amer-Indian-Eskimo and Other groups earn above $50K. Only the Other group has a measured loss. No simple group feature predicts privacy cost, so check every small group regardless of overall accuracy.

These results are suggestive, not conclusive. At accuracy 0.85, the exact interval for 532 test rows is $[0.816, 0.879]$, or about three points on each side. With 141 rows, the interval is about six points on each side. Three training runs do not remove this test-sampling uncertainty. A real check needs the paired interval of Equation 45.3 and more runs.

45.4. Why small groups pay: clipping and noise

DP-SGD can lose accuracy through clipping and noise, and both steps treat every record alike. The clipping-only column of Table 45.2 adds no noise and gives no privacy. Clipping alone costs the Asian-Pac-Islander group 0.57 points and the Other group 1.18 points. The White group gains 0.08 points, so clipping can affect groups unequally.

Clipping caps every record's pull on the update at the clipping norm C, so it cuts back the groups with long gradients hardest. A group's gradients are long where the model fits it badly, as it does

a rare pattern the rest of the data does not teach. So training gives such a group less weight than its share of the records.

The noise is the same for every group, while a group's signal grows with its size. A small group therefore has a weaker signal relative to the noise. When that group needs a rule of its own, its privacy cost can grow inversely with its share of the training records.

On Adult, reducing the Asian-Pac-Islander share from 3.1% to 1% leaves its cost at $\varepsilon = 1$ within the spread of three runs. The measured cost is 3.13 points. Measure the cost rather than predicting it from group size. Clipping and noise compound because noise acts on the update after clipping has reduced it. If clipping halves a group's gradients, its signal is like that of a group half its size.

To see which cause hurts your model, measure per-example gradient norms by group. Opacus's `GradSampleModule`, the wrapper DP-SGD training already uses, keeps one gradient per row in each parameter's `grad_sample`, so one backward pass gives the norms.

```
wrapped = GradSampleModule(model)  # adds hooks that keep one gradient per row
wrapped.train()
logits: torch.Tensor = wrapped(split.x)
nn.CrossEntropyLoss()(logits, split.y).backward()  # fills p.grad_sample for every
 ↪  weight p
norms = stacked_norms(wrapped)
```

Group the norms by each row's group label, kept beside the batch but not fed to the model. Log each group's median norm and fraction clipped as training goes on, because the groups can separate late. At initialization on Adult, the median norm is 1.444, so $C = 1$ clips every row, whatever its group.

45.5. Mitigations and what they cost

At a fixed budget, a change to the training recipe only moves the cost between the groups or between clipping and noise (Table 45.3). Only the last row, which changes the data, the starting model or the budget, reduces it.

Table 45.3.: Mitigations for disparate impact, and when to use them.

Fix	What happens	Use it?
Upweight the group's loss, or use a smaller clipping norm	Clipping cuts an upweighted gradient back to C, and a smaller norm clips more of the group	No
Larger clipping norm, with learning rate times C held fixed	Less clipping, but more noise relative to short gradients	Rarely, since it trades one cause for the other

Fix	What happens	Use it?
Clipping norm set to the largest gradient seen in the data	Breaks the privacy proof, which assumes C is fixed in advance	Only if chosen privately and charged
Oversample the group	Its records' ε is set by their higher sampling rate, so σ must rise	Only with that accounting
Global scaling with an adaptive bound (Esipova et al. 2023), one shared scale for every gradient	Keeps the update's direction and needs no group labels	Worth a try
More data from the group, a public pretrained model (Public data and private fine-tuning), or a larger ε	Raises the group's signal or lowers the noise	The fixes that work

Oversampling costs the group's privacy because a record's guarantee depends on its chance of being sampled (Amplification by subsampling). Run the accountant at the oversampled rate. On Adult, sampling the Asian-Pac-Islander rows four times as often at $\varepsilon = 1$ raises the noise multiplier from 2.383 to 8.828.

Every group then loses accuracy, with the largest loss in the oversampled group. Its accuracy falls from 0.8145 to 0.7657; the White group's falls from 0.8310 to 0.7860. These comparisons use the same DP-SGD at the ordinary sampling rate. This is **levelling down** with no gain for anyone.

No algorithm avoids some privacy cost. A model that learns a small group's rule must estimate it privately from that group's records. With few records, private estimation has an error floor that no mechanism can beat (Lower bounds).

45.6. Cheat sheet

Table 45.4.: The decisions of this chapter.

Question	Answer	Conditions
Does DP hurt this group?	Its privacy cost, Equation 45.1, against the non-private baseline	Same data and recipe, averaged over runs
How big will the cost be?	Negligible for large groups at moderate ε, roughly one over the group's share for small ones	Grows as ε falls

Rules of thumb

- Report accuracy and privacy cost per group, with intervals, next to the non-private model's.
- Never report the gap alone, because levelling down closes it too.

45.7. References

- Bagdasaryan, Poursaeed & Shmatikov, "Differential Privacy Has Disparate Impact on Model Accuracy" (NeurIPS 2019): the observation.
- Esipova, Ghomi, Luo & Cresswell, "Disparate Impact in Differential Privacy from Gradient Misalignment" (ICLR 2023): global scaling.

Derivations. Every result here is derived step by step in the full edition's chapter.

46. Auditing private training

46.1. Why this matters

For differential privacy (DP), an accountant maps the training configuration to privacy level ε and slack δ. This proves a property of the algorithm, but deployed systems run code. Bugs in noise, clipping or sampling can weaken privacy without noticeably changing accuracy, which often remains stable across a wide range of noise levels (the training workflow). Standard accuracy tests may not detect these bugs.

An **audit** tests implementation privacy by attack. It inserts **canaries**, trains with and without them, then measures membership-test success. DP limits every test's success rate, so a rate above the limit disproves the claimed guarantee. A low rate proves nothing: a weak attack may fail even against a non-private model.

46.2. Known bugs, and which audit catches them

The following implementation errors can invalidate stochastic gradient descent (SGD) with differential privacy, DP-SGD.

- **The variance passed where the standard deviation is expected,** or **noise scaled for the mean but added to the sum,** which adds the batch size times too little noise. A unit test in continuous integration (CI) catches both (Section 46.3).
- **Clipping applied after averaging, or noise from a weak seed.** Nasr et al. (2023) found both in real code with per-step audits like the CI test.
- **The accountant told the wrong numbers,** such as a record count that includes augmented copies, which makes ε look smaller than it is. Only an end-to-end audit can catch it, and only if the error is large.
- **Batches shuffled but accounted as Poisson sampling,** which can make the true ε several times the reported one (Amplification by subsampling). No audit here exercises the sampling, so check the loader itself.

Audits detect only privacy loss beyond the claim; code that adds excess noise will pass.

46.3. A CI test for the noise and the clipping

An efficient strong audit repeatedly tests one optimizer step. A **Dirac canary** has a nonzero per-example gradient only in one coordinate; all other records have zero gradient there. Include it at every step in one run and omit it in another. Let x_t be the noisy gradient sum in that

coordinate at step t, divided by clipping norm C. A canary longer than C is clipped to length C, contributing exactly 1. The optimizer adds noise with standard deviation σC, where σ is the noise multiplier, so each reading follows

$$x_t \sim \mathcal{N}(0, \sigma^2) \text{ without the canary}, \qquad x_t \sim \mathcal{N}(1, \sigma^2) \text{ with it.} \tag{46.1}$$

A threshold test reports "canary in" when a reading exceeds a threshold. Its **true positive rate (TPR)** is the detection frequency with the canary present; its **false positive rate (FPR)** is the frequency without it.

For two normal distributions separated by μ, every threshold satisfies $\Phi^{-1}(\mathrm{TPR}) - \Phi^{-1}(\mathrm{FPR}) = \mu$, where Φ is the standard normal distribution function. Measured rates therefore bound the separation μ, which a correct optimizer limits to $1/\sigma$:

$$\mu \geq \Phi^{-1}(\mathrm{TPR}) - \Phi^{-1}(\mathrm{FPR}). \tag{46.2}$$

The test plugs in the low end of the TPR's Clopper–Pearson interval and the high end of the FPR's. Each end is taken at level $\beta/2$, so the bound holds with probability $1 - \beta$ (Hypothesis testing). If this bound exceeds $1/\sigma$, the code adds less noise than it claims.

The script practitioner_audit.py runs the test on Opacus's own `DPOptimizer`. It writes per-example gradients into `grad_sample` by hand, as Opacus's hooks would, and reads the noisy sum that `optimizer.step()` leaves in `grad`.

```
def canary_coordinate(sigma: float, planted_bug: str, with_canary: bool, steps: int)
 ↪  -> Array:
    """Run the optimizer for `steps` steps; return the canary coordinate of each noisy
     ↪  sum, in units of C."""
    layer = torch.nn.Linear(DIM_CI, 1, bias=False)
    optimizer = make_optimizer(layer, sigma, planted_bug)
    observations = np.empty(steps)
    for t in range(steps):
        setattr(layer.weight, "grad_sample", batch_grad_samples(with_canary))  # as
         ↪  Opacus's hooks would
        optimizer.step()  # clip each row to C, sum, add N(0, σ²C²)
        assert layer.weight.grad is not None
        observations[t] = layer.weight.grad[0, CANARY_COORD].item() / CLIP_CI
        optimizer.zero_grad()
    return observations
```

The canary's gradient is ten times C, so the test also exercises clipping. The script chooses the threshold on the first half of the steps and counts the rates on the second half. It also checks that the canary-free readings have a standard deviation within 10% of σ, since with no noise every reading is 0. The command `uv run python code/part05-dp-ml/practitioner_audit.py` tests three optimizers at $\sigma = 0.7$ and a batch of $B = 64$ records, the correct one, one with the variance bug and one with no noise. The variance bug removes noise only because $\sigma^2 < \sigma$ when σ is below 1, and above 1 it would pass.

```
sigma=0.7  C=1.0  B=64  steps per run=2000  claim mu=1/sigma=1.429
correct: noise std 0.687 (claim 0.7)  FP 206/1000  TP 710/1000
correct: mu_lo=1.201  pass
variance for std: noise std 0.481 (claim 0.7)  FP 206/1000  TP 862/1000
variance for std: mu_lo=1.722  FAIL
no noise: noise std 0.000 (claim 0.7)  FP 0/1000  TP 1000/1000
no noise: mu_lo=5.360  FAIL
```

The correct optimizer's bound on μ is 1.201, below its claimed 1.429, so it passes. The variance bug's bound is 1.722 and the missing noise's is 5.360, so both fail. The test takes seconds, so run it on every commit.

Three things go wrong in practice.

First, a correct optimizer still fails with probability β, here 5%, so an unseeded test fails one build in twenty. Fix the seed, or use a β of 10^{-3} with more steps. Second, the script builds its own optimizer, so it tests Opacus, not your call to it. To test your code, have your setup function return the optimizer it makes, and drive that one. Third, the test never draws a batch or calls the accountant. So the same suite should assert that the loader is Opacus's `DPDataLoader` and that a second library's ε matches yours (Calibrate and account a DP-SGD run).

46.4. From an attack to a lower bound

For a whole training run, the attack is a membership test on a planted canary. A **trial** is one training run, with or without the canary. Under (ε, δ)-DP, every such test satisfies

$$\mathrm{TPR} \le e^{\varepsilon}\,\mathrm{FPR} + \delta, \qquad 1 - \mathrm{FPR} \le e^{\varepsilon}\,(1 - \mathrm{TPR}) + \delta. \tag{46.3}$$

Solving these for ε turns measured rates into a lower bound on it. Write $\overline{\mathrm{FPR}}$ for the one-sided Clopper–Pearson upper bound on the FPR at level $\beta/2$, and $\underline{\mathrm{TPR}}$ for the TPR's lower bound. With probability at least $1 - \beta$, the code is then not (ε, δ)-DP, at the claimed δ, for any ε below

$$\varepsilon_{\mathrm{lo}} = \max\left\{ \ln \frac{\underline{\mathrm{TPR}} - \delta}{\overline{\mathrm{FPR}}},\ \ln \frac{1 - \overline{\mathrm{FPR}} - \delta}{1 - \underline{\mathrm{TPR}}} \right\}. \tag{46.4}$$

The bound needs independent trials, and a threshold chosen on other trials than those it counts, because a threshold tuned on them inflates it.

No library installed with this book computes the bound, so the script imports `eps_lower_bound` from the book's helper module, practitioner_privacy_tests.py. Suppose an attack catches the canary in 500 of 1,000 runs that include it, and raises one false alarm in 1,000 runs without it.

```
for false_pos, true_pos in ((1, 500), (0, 1_000)):  # out of 1,000 runs without and
 ↪  with the canary
    eps_lo = eps_lower_bound(false_pos, true_pos, 1_000, DELTA)
    print(f"FP {false_pos}/1000  TP {true_pos}/1000: certified eps lower bound
     ↪  {eps_lo:.2f}")
```

```
FP 1/1000  TP 500/1000: certified eps lower bound 4.43
FP 0/1000  TP 1000/1000: certified eps lower bound 5.60
```

The raw rates alone would suggest 6.21, but the certified bound is only 4.43, and even a perfect attack certifies only 5.60 from 1,000 trials per hypothesis. So do not expect an end-to-end audit to certify all of a large claim.

46.5. A one-run audit of the trained model

Thousands of runs are out of reach for a real model, so Steinke, Nasr & Jagielski (2023) get many trials from one run. Plant a number m of canaries, include each one by its own fair coin, train once and score every canary. Then fix a list length r. Guess "in" for the $r/2$ highest scores and "out" for the $r/2$ lowest, and make no guess about the rest. Write W for the number of correct guesses.

Under ε-DP, no attack guesses one canary's coin right with a probability above the limit p_ε given below. So the count W is no more likely to reach any value v than a binomial count with that success rate is:

$$\Pr[W \geq v] \leq \Pr\left[\mathrm{Bin}(r, p_\varepsilon) \geq v\right], \qquad p_\varepsilon = \frac{e^\varepsilon}{1 + e^\varepsilon}. \tag{46.5}$$

An observed count that this binomial reaches with probability below β rules out that ε, and the certified bound is the largest ε ruled out. Under (ε, δ)-DP, the paper adds at most $2m\delta$ to that probability. The coins must be independent and fair, and several list lengths need a Bonferroni correction, which runs each test at β divided by their number.

The script runs it on the released pipeline of Train a model with DP-SGD on the Adult census data. It adds only the canaries, 1,000 held-out rows with flipped labels. The coins put 537 of them in the training set, and the release's noise multiplier gives $\varepsilon = 3.91$ for this larger set at $\delta = 10^{-5}$. A non-private model uses the release's baseline recipe, and a memorizing control runs non-private Adam for 60 epochs with small batches. Each canary is scored by minus its loss, and Table 46.1 gives each model's best of six list lengths from 20 to 1,000.

Table 46.1.: One-run audit of the released pipeline on Adult with 1,000 mislabeled canaries, at 95% overall confidence and $\delta = 10^{-5}$.

Training	Mean canary loss, in / out	Correct guesses	Certified ε
DP-SGD, release recipe	4.105 / 4.006	10 of 20	0.00
Non-private, release recipe	2.240 / 2.222	8 of 20	0.00
Memorizing control	1.994 / 2.992	75 of 100	0.53

Both release-recipe models certify nothing, and the reason is the recipe, not privacy. Ten epochs with large batches barely fit the flipped labels, so included canaries have the same loss as excluded

ones, with or without noise. Only the memorizing control separates its canaries. Even it certifies just 0.53 for a model with no finite ε, because each canary is a single trial. A perfect list of all 1,000 guesses would certify at most 5.27.

So use the one-run audit to expose a bug, when a DP model certifies more than its claim, never as evidence of privacy. Report it with its threat model, a black-box attacker who sees only the final model, and with a control that shows the attack can detect leakage on your data.

Stronger black-box attacks, such as the likelihood ratio attack (LiRA) of Pre-release privacy risk assessment, train **shadow models**. These are copies of the model, trained with its exact recipe on similar data. Run them before release and keep their results with the evidence (Release and governance). LiRA's TPR at a low FPR shows what a realistic attacker learns. But its (model, record) pairs share models, so they are not independent trials and give no certified bound.

46.6. Cheat sheet

Table 46.2.: Which audit answers which question, and its rule.

Question	Audit and cost	Rule	Blind to
Does the code clip and add the noise it claims?	CI test, seconds	Fail if the bound on μ, Equation 46.2, exceeds $1/\sigma$	Sampling and accounting bugs
Is the claimed ε right, accounting included?	End-to-end audit, thousands of runs	Equation 46.4, Clopper–Pearson at $\beta/2$	Bugs smaller than the bound's shortfall
Does a trained model leak planted records?	One-run audit, one extra run	Equation 46.5 plus $2m\delta$, Bonferroni	Most bugs, since its bounds are low
What can a realistic attacker learn?	Shadow-model attack (LiRA)	TPR at a low FPR; no certified bound	The most the code could leak

46.7. References

- Nasr et al., "Tight Auditing of Differentially Private Machine Learning" (USENIX Security 2023): Dirac canaries, per-step audits and real bugs.
- Steinke, Nasr & Jagielski, "Privacy Auditing with One (1) Training Run" (NeurIPS 2023): the one-run audit.

Derivations. Every result here is derived step by step in the full edition's chapter.

47. Adaptive data analysis

47.1. What it is

Tuning on a held-out validation set uses scores to choose later settings. This **adaptive** reuse overfits the holdout, making its scores overstate performance on new data.

Each question is a **statistical query**: the average of a per-record value in $[0, 1]$ over n held-out records, where n is the holdout size. Classifier accuracy is one example; its population value is the corresponding average on new data. If records are independent and k denotes the number of queries fixed before any answers, the expected largest gap between empirical and population values is at most

$$\sqrt{\frac{\ln(2k)}{2n}}. \tag{47.1}$$

The query count affects this bound only logarithmically.

Adaptive queries invalidate this fixed-query bound because later queries can fit the sample's noise. Blum & Hardt's boosting attack demonstrates this on a leaderboard with coin-flip labels, where population accuracy cannot exceed 0.5. After 1,000 submissions, the leaderboard reports 0.7182. This score uses 1,000 holdout records.

Differential privacy (DP) limits how much answers reveal about each record, reducing their ability to overfit the sample.

Let α be the average distance between answers and their empirical values. Let ε denote the privacy level and δ the slack. Suppose the complete exchange is differentially private under replace-one neighbours. The **transfer theorem** bounds expected maximum error against population values by

$$\alpha + e^{\varepsilon} - 1 + \delta, \tag{47.2}$$

regardless of how the analyst uses the answers. For small ε, the middle term is approximately ε, so the bound is useful mainly when $\varepsilon \ll 1$. It protects answer accuracy, not the people represented in the holdout.

47.2. When you meet it

Adaptivity arises when a validation set, test set or leaderboard answers questions in multiple rounds, as in hyperparameter search. A round is a batch fixed before receiving its answers. The bounds below depend on the round count; Table 47.1 summarizes the options.

Table 47.1.: How to reuse a holdout set, by situation.

Situation	What to do
A final evaluation	Keep a test set and use it once
Few adaptive rounds, plenty of data	Give each round its own slice of data
Many adaptive rounds, scarce data	Answer through noise, or use Thresholdout
A search that is private in the holdout set	Nothing more, since the transfer theorem covers the winner
A public leaderboard	Show a new score only when it beats the best by a margin (Blum & Hardt's Ladder)

47.3. How to use it

The simplest method is **sample splitting**: assign each of r rounds its own n/r records. For k total queries, expected maximum error is at most $\sqrt{r \ln(2k)/(2n)}$, so required data grow proportionally with r. Suppose 1,000 rounds evaluate 1,000 adaptively selected configurations. The holdout has 10,000 validation records, and the bound is 0.6165. Accuracy scores lie in $[0, 1]$, so that bound is too large. To reduce error to 0.05, sample splitting would require 1,520,181 records. Fixed queries require 1,521 records for the same error.

The transfer theorem implies that Gaussian-noisy answers require data growing only as $\sqrt{k}$. Its constants are large, however, so sample splitting remains cheaper for up to a few hundred adaptive queries.

Thresholdout (Dwork et al. 2015) answers using training-set values while they agree with holdout values; these answers reveal nothing about the holdout. When the difference exceeds a noisy threshold, it records a **detection** and returns a noisy holdout value. Privacy cost depends on detections rather than total queries. However, the full edition's experiment gives $\varepsilon \approx 29$. No mainstream library implements the method, so treat it as a practical limit on reuse rather than a strong guarantee.

47.4. What goes wrong

The common failure is a long search that selects each setting from earlier validation scores. A fixed grid evaluated once, as in scikit-learn's `GridSearchCV`, is one round, so Equation 47.1 bounds the selected score's overstatement. Count and report the rounds of validation reuse. Typical overfitting may be smaller than the worst-case bound, but risk remains high with few records and many rounds.

47.5. References

- Dwork et al., "The Reusable Holdout" (*Science* 2015): Thresholdout.
- Blum & Hardt, "The Ladder" (ICML 2015): the boosting attack and a leaderboard that resists it.

Derivations. Every result here is derived step by step in the full edition's chapter.

48. Release and governance

48.1. Why this matters

A model trained with differential privacy (DP) is not ready to ship when training ends. A privacy reviewer or data protection officer must determine what the guarantee covers, whether its value is correct and how it relates to law. They may not be able to rerun training, so provide the evidence needed to verify the claim. This chapter defines the privacy statement and review process.

The example model is a small stochastic gradient descent (SGD) network trained under this guarantee on 30,000 Adult records, with target privacy level $\varepsilon = 4$. It is produced by Train a model with DP-SGD. The script practitioner_release.py runs in seconds and reads the training run record supplied with `--record PATH`, or a built-in example record.

```
uv run python code/part05-dp-ml/practitioner_release.py
```

48.2. Assemble the run record

The run record must include the inputs to the accountant, which converts settings to ε (Choosing an accountant): noise multiplier σ, sampling rate q, step count and slack δ. It must also document assumptions: privacy unit (one person here), neighbouring relation, batch sampler and whether any other use of private data was excluded.

Also record n, the number of units, the random seed, Opacus's `secure_mode`, library versions, hyperparameter selection and any preprocessing on private data. In the excerpt, `data` is the training record and `RELEASE_CONTEXT` contains release-owner additions.

```
merged = RELEASE_CONTEXT | data  # the training record, plus what the release owner
 ↪  adds
rec = load_record(merged)
for key, value in asdict(rec).items():
    print(f"{key}: {value}")
problems = record_problems(merged)  # missing fields, sampler not Poisson, delta above
 ↪  1/n^1.1, tuning
limitations = record_limitations(merged)  # stated, so not a problem, but the statement
 ↪  must disclose them
print(f"record problems: {len(problems)}")
for problem in problems:
    print(f"  - {problem}")
print(f"record limitations: {len(limitations)}")
for limitation in limitations:
    print(f"  - {limitation}")
```

```
unit: person (one row of Adult)
neighbours: add/remove
n: 30000
delta: 1e-05
sigma: 0.972900390625
sample_rate: 0.03333333333333333
sampler: poisson
secure_mode: False
seed: 0
steps: 300
...
record problems: 0
record limitations: 1
  - secure_mode=False: Opacus drew the noise from a seeded, non-cryptographic generator
 ↪  so the numbers reproduce; floating-point attacks on the noise are not excluded.
 ↪  Retrain with secure_mode=True to ship
```

The record's $\delta = 10^{-5}$ follows the book's rule (How small δ must be). Its ε values come from the independent check in Calibrate and account a DP-SGD run.

The tuning record is essential because the final run's ε excludes runs used to select hyperparameters. Here, non-private runs selected the learning rate on 2,000 held-out Adult rows treated as public, and $\varepsilon = 4$ was fixed before private training. The training run's ε therefore equals the procedure's cost. The 2,000 people receive no guarantee and must be named in the statement. For private-data tuning, report both costs (Hyperparameter tuning).

Stop release for a record problem such as missing fields, a non-Poisson sampler, excessive δ or undocumented tuning. Here, `secure_mode` is disabled to keep example numbers reproducible. Disclose this limitation and enable secure mode when retraining a production model.

48.3. Recompute the guarantee

The release owner recomputes ε from σ, q and the steps alone, with a library other than the one that trained. A recomputation inside the training code would reuse any wrong number. Compare the result only with a budget from the same kind of accountant, because a looser accountant gives a larger ε for the same run. The script uses the privacy loss distribution (PLD) accountant of `dp_accounting`, as in the cross-check, and compares it with the looser Rényi differential privacy (RDP) accountant. It also reads ε at a δ a hundred times smaller, and the trade-off curve. That curve gives the largest true positive rate (TPR) that any membership test can reach at each false positive rate (FPR) (Choosing ε).

```
pld = privacy_loss_distribution.from_gaussian_mechanism(sigma,
 ↪  sampling_prob=sample_rate).self_compose(steps)
eps_pld = pld.get_epsilon_for_delta(delta)
eps_pld_small = pld.get_epsilon_for_delta(delta / 100)
eps_rdp = rdp_epsilon(sigma, sample_rate, steps, delta)
eps_rdp_small = rdp_epsilon(sigma, sample_rate, steps, delta / 100)
tpr = {fpr: tpr_bound(pld, fpr) for fpr in FPRS}  # largest TPR of any membership test
```

```
eps PLD at delta=1e-05: 3.981
eps RDP at delta=1e-05: 4.503
eps PLD at delta=1e-07: 5.191
eps RDP at delta=1e-07: 5.736
TPR bound at FPR 1e-03: 0.0123 (PLD curve)  0.0536 (pair eps, delta only)
TPR bound at FPR 1e-02: 0.0652 (PLD curve)  0.5355 (pair eps, delta only)
record's PRV eps: reported 3.99, recomputed 3.98: consistent
```

The PLD value, 3.981, agrees with the reported 3.99. The curve gives a number a non-specialist can use. At an FPR of 10^{-3}, no membership test can reach a TPR above 0.0123, while the pair $(3.98, 10^{-5})$ on its own would allow 0.0536.

The script flags a reported ε more than 0.05 below the recomputation. The subtler failure is a recomputation that agrees on wrong inputs, such as code that shuffles the data into fixed batches while the accountant assumes Poisson sampling (Amplification by subsampling). No number shows that, so the record's sampler, read from the loader, is what catches it.

48.4. Attach the evidence

An accountant proves a theorem about an algorithm, not about the code that runs it, so the release also needs evidence about the code and the model. Attach each piece in Table 48.1, and say which attacker it models. The curve covers the strongest attacker, who sees every intermediate model, while the audit and the attack cover only the attacks they run.

Table 48.1.: The evidence attached to a DP model release, and its limits.

Evidence	Source	On the Adult run	What it cannot show
Independent ε	This chapter's recomputation	3.981 against the reported 3.99	That the sampler matches the code
Clipping and noise test	Continuous integration (CI) test of Auditing private training	Passes on Opacus's optimizer; fails with the variance passed as the standard deviation, or with no noise	Sampling or accounting bugs
One-run audit	Auditing private training: the released pipeline retrained with 1,000 planted canaries	A lower bound on ε of 0.00 for DP-SGD and for the non-private baseline, against 0.53 for a memorizing control	That the model is private; a low bound proves nothing

Evidence	Source	On the Adult run	What it cannot show
Black-box attack	Likelihood ratio attack of Pre-release privacy risk assessment	Not run on this model; say so in the evidence	Whether a stronger attacker does better
Groups	Per-group accuracy with exact 95% intervals (Disparate impact)	Women 0.922 [0.915, 0.929], men 0.807 [0.800, 0.815]	Groups too small to measure

The audits can only refute a claim. A one-run bound above the claimed 4 would prove a bug, but 0.00 shows only that this attack found nothing, as in the non-private baseline. The audit also tests the released training code, not the released weights.

The group accuracies come from the 16,842 Adult test rows, which are private too. To publish them, add noise to each group's count of correct predictions and charge it to its own budget (Release statistics with a library). Or measure them on a test set that is not private.

48.5. Write the statement

The statement goes in the model card's privacy section. The script's `format_statement` builds it from the record, so no number is typed by hand. The block omits lines that repeat the record.

```
Guarantee: (3.98, 1e-05)-DP per person (one row of Adult), add/remove neighbours,
  for the whole training run on Adult (UCI 2), 30,000 training rows (n = 30,000).
  Also (5.19, 1e-07)-DP. Any membership test has TPR at most 0.0123 at FPR 1e-3.
Mechanism: DP-SGD, poisson sampling q = 0.0333, noise multiplier 0.9729,
  clipping norm 1, 300 steps (10 epochs).
Accounting: prv, Opacus 1.6.0; recomputed with dp_accounting PLD (RDP gives 4.50).
...
Noise generator: Opacus secure_mode=False, seed 0.
...
Not covered: data outside the training set, API outputs that use other private data,
 ↪  other releases.
```

Below the statement, add the evidence of Table 48.1. The last line needs the most care. The guarantee covers the weights and anything computed from them alone, such as the logits that the deployed application programming interface (API) returns, because post-processing cannot weaken DP. An output that also uses other private data, such as a retrieval index, is a new release, and so is a second model trained on the same people.

48.6. Map the claim to regulation

This section is not legal advice, since every answer depends on facts beyond the model. In the European Union, Recital 26 of the General Data Protection Regulation (GDPR) exempts

anonymous information. It asks about the means of identification "reasonably likely to be used". Pseudonymised data, which only information kept separately links to a person, remain personal data under Article 4(5).

For AI models, Opinion 28/2024 of the European Data Protection Board (EDPB) sets a test, assessed case by case. A model is anonymous only if the likelihood of extracting personal data about its training set is insignificant. The opinion names DP and attack tests, such as membership inference, as measures to weigh, and warns that a test shows resistance only to the attacks tested. The trade-off curve speaks to extraction as well as membership, because it caps reconstruction too (Choosing ε).

Table 48.2.: What DP evidence answers, and leaves open, for the legal questions a release meets.

Legal question	What DP evidence supports	What DP does not answer
Is the model anonymous (GDPR Recital 26; EDPB Opinion 28/2024)?	A bound on what any attacker can learn about one person's presence or record: the TPR cap and the audits	Whether the bound is "insignificant"; the training data stay personal data
Is this only pseudonymisation (GDPR Art. 4(5))?	DP relies on no separately kept key that could re-link the data to people	Removing identifiers before training is pseudonymisation at most
What goes in the risk assessment (GDPR Art. 35)?	The statement, the curve, the audits and the group table	Necessity and proportionality (Art. 35(7)(b)), and the lawful basis (Art. 6)
Is the risk of identification very small (HIPAA §164.514(b)(1))?	A quantitative bound that an expert can cite, with documented methods	Whether it is "very small"; the expert's determination itself
Is the information deidentified (California Consumer Privacy Act, Cal. Civ. Code §1798.140(m))?	A bound on how much one person's record can change what is inferred	Whether that meets the standard, since DP does not stop inferences from population patterns; the business's own duties
Does the data governance meet the EU AI Act (Art. 10)?	Documented data handling and per-group evaluation	Data quality, representativeness and bias, which DP can worsen

Training still processes personal data, so the GDPR applies to it, lawful basis included. Article 35 requires a data protection impact assessment (DPIA) before processing likely to result in a high risk. The DPIA is the natural home for the statement. In the United States, the safe harbor route of the Health Insurance Portability and Accountability Act (HIPAA) removes listed identifiers (§164.514(b)(2)), which DP does not do. Its expert determination route can cite DP evidence but not be replaced by it. California's law, as amended by the California Privacy Rights Act, adds duties against re-identification for the business.

None of these texts says that DP makes a model anonymous by itself. A DPIA entry built from this chapter's evidence can read as follows, and the controller and counsel fill in the bracketed parts.

> The model was trained on personal data under [lawful basis, Art. 6]. Training used DP-SGD with a guarantee of $(3.98, 10^{-5})$-DP per person, add/remove neighbours, for the whole training run. No membership test against the released weights can exceed a TPR of 0.0123 at an FPR of 10^{-3}, and the same curve caps reconstruction. The code passed a CI test of its noise and clipping. A one-run audit of the released pipeline found nothing, while a memorizing control certified 0.53. [A shadow-model attack was / was not run.] These tests show resistance only to the attacks run. DP does not remove identifiers and is not pseudonymisation. Whether the bound makes the model anonymous is for [counsel], and we make no claim of automatic anonymity. Not covered: the 2,000 stand-in rows, the test rows, outputs that use other private data, and later model versions.

48.7. Review someone else's claim

When someone else hands you a model, run the same steps in reverse. Ask for the record of Section 48.2, and refuse a claim that cannot be recomputed. Recompute ε from the claim's own parameters, and check its δ against the number of units. Matching the accountant to the code gives the checks on the accountant and the sampler.

The script reviews a claim made up for this book, with every flaw a review should catch. It claims $\varepsilon = 1$ at $\delta = 10^{-3}$ for a model trained on 50,000 records, over 5,000 steps with noise multiplier 0.8. Its batches of 256 were shuffled and of fixed size but are accounted as Poisson sampling. The claim states no unit, no clipping norm and no tuning.

```
def review(claim: dict[str, Any]) -> list[str]:
    """Recompute a claim from its own parameters and list what does not hold up."""
    findings = record_problems(claim)  # includes a sampler that is missing or not
↪   Poisson
    sigma, q, steps, delta = claim["sigma"], claim["sample_rate"], claim["steps"],
↪   claim["delta"]
    pld = privacy_loss_distribution.from_gaussian_mechanism(sigma,
↪   sampling_prob=q).self_compose(steps)
    recomputed = {"PLD": pld.get_epsilon_for_delta(delta)}
    if "rdp" in str(claim.get("accountant")).lower():  # also the accountant the claim
↪       names
        recomputed["RDP, the accountant named,"] = rdp_epsilon(sigma, q, steps, delta)
    for name, eps in recomputed.items():
        if eps > claim["epsilon"] + 0.05:
            findings.append(f"eps={claim['epsilon']} does not follow: {name} gives
↪   {eps:.2f} at its delta")
    delta_ok = delta_for(claim["n"])
    findings.append(f"at delta={delta_ok:.0e} (1/n^1.1) PLD gives
↪   eps={pld.get_epsilon_for_delta(delta_ok):.2f}")
    return findings
```

```
claim: Our model is trained with differential privacy, epsilon = 1.
findings: 10
  - missing field: unit
  - missing field: neighbours
```

```
  - missing field: max_grad_norm
  - missing field: secure_mode
  - sampler 'shuffled fixed-size batches' is not Poisson subsampling:
↪  Poisson-subsampled accounting does not apply
  - delta=1e-03 is above 1/n^1.1 (use 1e-06)
  - tuning not reported: the cost of choosing hyperparameters is unknown
  - eps=1.0 does not follow: PLD gives 2.17 at its delta
  - eps=1.0 does not follow: RDP, the accountant named, gives 2.55 at its delta
  - at delta=1e-06 (1/n^1.1) PLD gives eps=3.75
```

By the vendor's own parameters the claimed 1 is 2.17, and the RDP accountant it names gives 2.55. At a δ the book accepts it is 3.75. Even that is too low, because shuffled fixed-size batches get no amplification by Poisson subsampling, so the true ε can be far above all these numbers.

The script takes n as given, so it cannot see a wrong unit. If the records are messages and a customer sends many, screen the per-customer ε yourself with User-level differential privacy. Send the findings back with a request for a corrected record.

48.8. After release

Every retraining on the same people is a new release that composes with the old ones. For identical runs, compose the PLD of one run with itself once per version, `pld.self_compose(steps * versions)`. State the budget per model version and in total over a fixed **period**. If the total over a schedule is too large, calibrate each version to a share of it.

Deletion requests need their own answer, because DP deletes nothing. The model is about as likely as one trained without the person, but their row is still in the data. So honour a request by removing the row, from the stand-in and test rows too, and retraining on a fixed schedule. Whether that is fast enough for the GDPR's Article 17 is a question for counsel.

Give each model version its own record and statement, and name in advance the incidents that reopen a release. A failing CI test, an audit bound above the claim, or a bug in the sampler or the accountant needs a corrected statement or a retrained model.

48.9. Checklist

Table 48.3.: The release checklist: each step, its default and its check.

Step	Default	Check
Run record	The fields of Section 48.2, with the sampler read from the loader	`record problems: 0`; `secure_mode` on for the shipped model
δ	The book's rule (How small δ must be)	n counts units, not rows
Tuning	Choices on public data, or every private run counted	Run and procedure ε both reported

Step	Default	Check
Recomputed ε	PLD in `dp_accounting`, at δ and $\delta/100$, and TPR from the curve	Reported value not below the recomputation
Audits	The evidence of Table 48.1	CI passes on the released commit; no bound above the claim; the control detects leakage; any attack not run is named
Groups	Per-group accuracy with exact intervals	Test set not private, or its release accounted
Statement	Generated from the record	Every number recomputable; "not covered" filled in
Regulation	DPIA entry mapping the evidence to each legal question	Reviewed by counsel; no claim of automatic anonymity
Third-party claim	Recompute from its own parameters	Unit, sampler, δ and tuning all stated
After release	Budget per period; retraining and deletion schedule; incident triggers	Versions composed; each has its own record

48.10. References

- Near, Darais, Lefkovitz & Howarth, *Guidelines for Evaluating Differential Privacy Guarantees* (NIST SP 800-226, 2025): what to ask of any DP claim.
- European Data Protection Board, *Opinion 28/2024 on certain data protection aspects related to the processing of personal data in the context of AI models* (December 2024): when a model can count as anonymous.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): its reporting section lists what a privacy statement contains.

Derivations. This workflow has no counterpart in the full edition.

A. Glossary

Acronyms used in the book and the pages that use them.

AI Artificial intelligence. Used in Release and governance.

API Application programming interface. Used in Threat models and privacy games, Pre-release privacy risk assessment, Release and governance.

ART Adversarial Robustness Toolbox. Used in Threat models and privacy games, Membership inference, Attribute inference and model inversion.

AUC Area under the curve. Used in Hypothesis testing, Membership inference.

CDF Cumulative distribution function. Used in The Laplace and Gaussian mechanisms.

CI Continuous integration. Used in Auditing private training, Release and governance.

CLT Central limit theorem. Used in Choosing an accountant, Calibrate and account a DP-SGD run, Numerical accounting with privacy loss distributions, f-DP and Gaussian differential privacy.

CPU Central processing unit. Used in Pre-release privacy risk assessment, Fine-tune a language model privately with LoRA.

DLG Deep Leakage from Gradients. Used in Gradient inversion.

DP Differential privacy. Used in Property inference, Gradient inversion, Pre-release privacy risk assessment, Differential privacy foundations, The definition of differential privacy, Post-processing, group privacy and composition, The Laplace and Gaussian mechanisms, Advanced and optimal composition, Selection: exponential mechanism, report-noisy-max, sparse vector, Query release and private multiplicative weights, Beyond global sensitivity, Discrete and floating-point-safe mechanisms, Local, shuffle and distributed trust models, Lower bounds: what privacy costs, Private mean estimation, Release statistics with a library, Privacy accounting, Choosing an accountant, Calibrate and account a DP-SGD run, Amplification by subsampling, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Numerical accounting with privacy loss distributions, f-DP and Gaussian differential privacy, Adaptive composition: filters, odometers and individual accounting, Concentrated differential privacy, Amplification by shuffling and by iteration, Differential privacy in machine learning, Private empirical risk minimization, DP-SGD, Train a model with DP-SGD, Hyperparameter tuning and private selection, User-level differential privacy, Choosing ε, Public data and private fine-tuning, Fine-tune a language model privately with LoRA, Private prediction: PATE and in-context learning, DP-FTRL and matrix factorization, Disparate impact, Auditing private training, Adaptive data analysis, Release and governance.

DPIA Data protection impact assessment. Used in Release and governance.

ERM Empirical risk minimization. Used in Private empirical risk minimization, Public data and private fine-tuning.

FL Federated learning. Used in Gradient inversion.

FNR False negative rate. Used in Hypothesis testing.

FPR False positive rate. Used in Hypothesis testing, Threat models and privacy games, Membership inference, Pre-release privacy risk assessment, The definition of differential privacy, Lower bounds: what privacy costs, f-DP and Gaussian differential privacy, Choosing ε, Auditing private training, Release and governance.

FTRL Follow the regularized leader. Used in Choosing an accountant, DP-FTRL and matrix factorization.

GDP Gaussian differential privacy. Used in Choosing an accountant, Calibrate and account a DP-SGD run, Numerical accounting with privacy loss distributions, f-DP and Gaussian differential privacy, Choosing ε.

GDPR General Data Protection Regulation. Used in Linkage and re-identification, Release and governance.

GPU Graphics processing unit. Used in Train a model with DP-SGD, Fine-tune a language model privately with LoRA.

HIPAA Health Insurance Portability and Accountability Act. Used in Release and governance.

HIV Human immunodeficiency virus. Used in Syntactic anonymity and how it breaks.

HMAC Hash-based message authentication code. Used in Linkage and re-identification.

KL Kullback–Leibler. Used in Divergences.

LLM Large language model. Used in Private prediction: PATE and in-context learning.

LR Likelihood ratio. Used in Hypothesis testing.

ML Machine learning. Used in Membership inference, Choosing an accountant, Calibrate and account a DP-SGD run, DP-SGD, Train a model with DP-SGD, Choosing ε, Release and governance.

MWEM Multiplicative weights exponential mechanism. Used in Query release and private multiplicative weights.

PATE Private aggregation of teacher ensembles. Used in Beyond global sensitivity, Private prediction: PATE and in-context learning.

PII Personally identifiable information. Used in Memorization and training-data extraction.

PLD Privacy loss distribution. Used in Advanced and optimal composition, Choosing an accountant, Calibrate and account a DP-SGD run, Amplification by subsampling, From Rényi DP to (ε, δ) and to tests, Numerical accounting with privacy loss distributions, Train a model with DP-SGD, Choosing ε, Fine-tune a language model privately with LoRA, Private prediction: PATE and in-context learning, DP-FTRL and matrix factorization, Release and governance.

PRV Privacy loss random variable. Used in Choosing an accountant, Calibrate and account a DP-SGD run, Numerical accounting with privacy loss distributions, Train a model with DP-SGD, Fine-tune a language model privately with LoRA.

RDP Rényi differential privacy. Used in Advanced and optimal composition, Privacy accounting, Choosing an accountant, Calibrate and account a DP-SGD run, Amplification by subsampling, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Numerical accounting with privacy loss distributions, Adaptive composition: filters, odometers and individual accounting, Concentrated differential privacy, DP-SGD, Train a model with DP-SGD, Hyperparameter tuning and private selection, Choosing ε, Private prediction: PATE and in-context learning, DP-FTRL and matrix factorization, Release and governance.

RMSE Root-mean-square error. Used in The Laplace and Gaussian mechanisms, Private mean estimation.

ROC Receiver operating characteristic. Used in Hypothesis testing, Threat models and privacy games, Membership inference.

SGD Stochastic gradient descent. Used in Pre-release privacy risk assessment, Privacy accounting,

Choosing an accountant, Calibrate and account a DP-SGD run, Amplification by subsampling, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Numerical accounting with privacy loss distributions, f-DP and Gaussian differential privacy, Concentrated differential privacy, Amplification by shuffling and by iteration, Differential privacy in machine learning, Private empirical risk minimization, DP-SGD, Train a model with DP-SGD, Hyperparameter tuning and private selection, User-level differential privacy, Choosing ε, Public data and private fine-tuning, Fine-tune a language model privately with LoRA, Private prediction: PATE and in-context learning, DP-FTRL and matrix factorization, Disparate impact, Auditing private training, Release and governance.

SQL Structured Query Language. Used in Release statistics with a library.

TPR True positive rate. Used in Hypothesis testing, Threat models and privacy games, Membership inference, Pre-release privacy risk assessment, The definition of differential privacy, Lower bounds: what privacy costs, f-DP and Gaussian differential privacy, Choosing ε, Auditing private training, Release and governance.

TV Total variation. Used in Divergences, Hypothesis testing, Syntactic anonymity and how it breaks.

ZIP Zone Improvement Plan code, the US postal code. Used in Linkage and re-identification.

B. Index

AboveThreshold, 81, 85
adaptive composition, 119, 121, 152
advanced composition, 60, 78, 119–121
amplification by subsampling, 119, 137, 219
approximate differential privacy, 108
approximate zCDP, 155
attribute inference, 15, 28, 57, 188
auxiliary knowledge, 16

basic composition, 71, 78
binomial distribution, 13, 182
black-box attack, 92, 230

canary, 17, 20, 34, 54, 57, 58, 205, 206, 222, 223, 229
central limit theorem, 121, 124, 145
clipping, 81, 92, 107, 108, 165, 169, 176, 177, 184, 187, 206, 215–217, 219, 229, 232
Clopper–Pearson interval, 11, 13, 223
composition, 49, 60, 66, 71, 75, 78, 119–121, 132, 137, 142, 147, 152, 161, 179, 181
composition attack, 49

deduplication, 196
Deep Leakage from Gradients, 39
differential privacy, 1, 15, 16, 34, 38, 60, 66, 71, 76, 86, 91, 101, 108, 116, 117, 119, 120, 124, 132, 142, 144, 147, 150, 152, 156, 161, 169, 177, 182, 187, 192, 196, 197, 206, 218, 223, 234
discrete Gaussian distribution, 95, 156
DP-SGD, 54, 57, 119–121, 124, 131, 134, 136, 142, 161, 168, 169, 176–178, 182, 184, 186, 187, 192, 206, 222, 229, 232

empirical risk minimization, 161
entropy, 39, 48
exponential mechanism, 81, 85, 89, 154, 181

f-DP, 120
frequency oracle, 101
fully adaptive composition, 152

Gaussian differential privacy, 119–121, 132, 145, 148, 150, 192
Gaussian mechanism, 60, 76, 161
generalization gap, 25
global sensitivity, 95
gradient inversion, 15, 38, 57
group privacy, 60, 71, 75, 142, 187

honest-but-curious, 16

imputation baseline, 28
individual privacy accounting, 153

k-anonymity, 48, 49

l-diversity, 48, 49
Laplace distribution, 4
Laplace mechanism, 76
likelihood ratio attack, 25, 54, 57, 58, 223, 230
loss attack, 25, 57

membership inference, 15, 20, 25, 58, 161
meta-classifier, 54
model inversion, 28, 54
moments accountant, 142, 169

Narayanan–Shmatikov attack, 44
neighbouring datasets, 60, 71, 72, 75, 119, 134, 137, 188

optimal composition, 78

parallel composition, 71
permutation invariance, 30

Poisson distribution, 121, 122, 124, 131, 132, 134, 136, 155, 165, 169, 176–182, 185, 205, 219
Poisson sampling, 121, 122, 136, 165, 177, 178, 205, 219
post-processing, 60, 70, 71, 75
privacy filter, 79
privacy game, 15, 20
privacy loss, 119–121
privacy loss distribution, 119–121, 124, 132, 134, 136, 145, 192, 205, 234
privacy loss random variable, 121
privacy profile, 76, 79
property inference, 15, 30, 57
PRV accountant, 147, 205
pure differential privacy, 65, 76, 121, 141

quasi-identifier, 44, 54, 57

randomized response, 101, 121, 139, 157, 158
RDP curve, 139, 141, 144
re-identification, 58, 65
reconstruction attack, 15, 52
report-noisy-max, 81, 85
ROC curve, 11, 13, 20
Rényi differential privacy, 119, 120, 124, 132, 134, 136, 139, 141, 142, 144, 145, 154, 180
Rényi filter, 121, 153

sampling without replacement, 122, 124, 134, 136, 137, 169
secure aggregation, 39, 54
sensitivity, 60, 75, 76, 92, 95, 121, 139, 142, 154
shadow model, 25, 54, 223, 232
shuffle model, 101, 121, 158
sparse vector technique, 75, 85, 86
stochastic gradient descent, 54, 57, 119–121, 124, 131, 134, 136, 142, 161, 168, 169, 176–178, 182, 184, 186, 187, 192, 206, 222, 229, 232
subsample-and-aggregate, 92
subsampled Gaussian mechanism, 155

t-closeness, 49
threat model, 1, 15, 16, 57
trade-off function, 4, 12, 188, 192

unary encoding, 101
unit of privacy, 60, 65, 75, 122, 184, 187

variance, 4, 25, 107, 108, 219, 229

zero-concentrated differential privacy, 91, 116, 121, 154, 155, 192